Représentations en grille et en graphe pour l'apprentissage par auto-jeu à Hive

Une comparaison pré-enregistrée sous budget de calcul limité

Mohamed Bechir Kefi

Rapport de recherche indépendant (non évalué par les pairs)

Version v2.1-draft, Octobre 2026

Résumé

Hive est un jeu de stratégie hexagonal sans plateau dont les coups sont des paires (pièce, destination) sur un ensemble de cellules en perpétuel changement, ce qui en fait, en principe, un candidat naturel pour des encodages par réseaux de neurones en graphe plutôt que pour les encodages convolutifs en grille standards des systèmes de type AlphaZero. Nous testons cette intuition sous un protocole pré-enregistré, gelé avant toute exécution comparative : un CNN en grille et un réseau relationnel à passage de messages de capacité appariée (+1.5%) partagent un moteur validé côté règles, un décodeur d'actions unique, des réglages d'entraînement identiques, une population gelée de trois adversaires et 250 ouvertures gelées, évalués sous deux lectures budgétaires (à exemples d'entraînement égaux ; à temps mural égal à un seuil pré-enregistré), avec cinq graines indépendantes par bras et la troncature rapportée comme un résultat à part entière. L'hypothèse est rejetée : le bras graphe obtient un score inférieur contre deux des trois adversaires sous les deux lectures, à un coût en temps mural double, son déficit se concentrant dans la conversion des positions gagnées. Les ablations montrent que les relations d'arêtes typées du bras graphe sont nécessaires à la stabilité de l'optimisation, tandis que son agrégation globale (global pooling) est dispensable. Une seconde étude, pré-enregistrée séparément (protocole gelé le 13 octobre 2026) et menée sur cinq nouvelles graines par bras sous une boucle à démarrage à chaud avec une fenêtre de rejeu de trois générations, a testé la principale objection à ce verdict : aucun avantage du graphe n'a été détecté sous l'une ou l'autre lecture, chaque intervalle graphe–grille inclut zéro (plus grandes bornes supérieures +0.090 et +0.055), le déficit de la première étude n'est plus visible à exemples égaux et est réduit à temps mural égal, et la pathologie de conversion persiste dans trois graines graphe sur cinq, à 2.34 fois le coût. Les deux résultats sont pré-enregistrés et reproductibles à partir des enregistrements publiés.

Mots-clés. Hive ; AlphaZero ; réseaux de neurones en graphe ; apprentissage de représentations ; pré-enregistrement ; résultat négatif.

Abstract

Hive is a boardless hexagonal strategy game whose moves are (piece, destination) pairs over an ever-changing set of cells, which makes it, in principle, a natural candidate for graph neural encodings in place of the convolutional grid encodings standard in AlphaZero-style systems. We test that intuition under a pre-registered protocol frozen before any comparison run: one grid CNN and one capacity-matched (+1.5%) relational message-passing network share a rules-validated engine, one action decoder, identical training settings, a frozen three-opponent population and 250 frozen openings, evaluated under two budget readings (equal training examples; equal wall-clock at a pre-registered cutoff) with five independent seeds per arm and truncation reported as its own outcome. The hypothesis is rejected: the graph arm scores lower against two of three opponents under both readings, at twice the wall-clock cost, with its deficit concentrated in converting won positions. Ablations show that the graph arm's typed edge relations are necessary for optimization stability, while its global pooling is dispensable. A second study, pre-registered separately (protocol frozen on 13 October 2026) and run on five new seeds per arm under a warm-started loop with a three-generation replay window, tested the main objection to that verdict: no graph advantage was detected under either reading, every graph–grid interval includes zero (largest upper bounds +0.090 and +0.055), the first study's deficit is no longer visible at equal examples and reduced at equal wall-clock, and the conversion pathology persists in three graph seeds of five at 2.34 times the cost. Both results are pre-registered and reproducible from the released records.

Keywords. Hive; AlphaZero; graph neural networks; representation learning; pre-registration; negative result.

Statut, code et déclaration d'assistance

Statut. Rapport de recherche indépendant préparé en vue d'une candidature au doctorat. Il n'est pas évalué par les pairs et n'est la publication d'aucune institution. Version 2.1-draft, octobre 2026 ; le contenu expérimental est définitif et aucun nombre ne diffère des résultats bruts gelés du 9 octobre 2026 (première étude) et du 27 octobre 2026 (seconde étude).

Code et artefacts. Le moteur, le pipeline d'entraînement, les deux encodeurs, les configurations gelées, les enregistrements bruts d'évaluation par partie et les scripts qui régénèrent chaque table et chaque figure de ce rapport forment un seul dépôt public, https://github.com/mohamedbechirkefi/hive-graph-selfplay, publié à l'étiquette v2.0-report le 9 octobre 2026 pour la première étude (l'archive des enregistrements est jointe à cette publication) ; les enregistrements de la seconde étude et la présente révision accompagnent la publication étiquetée v2.1-report. Un scénario de reproduction en environnement vierge est décrit en Annexe G ; la provenance de chaque résultat est tabulée en Annexe E.

Déclaration d'assistance par un système d'intelligence artificielle. Cette étude a été exécutée selon une méthodologie « l'humain comme chercheur principal », dans laquelle un assistant de recherche IA (Claude, Anthropic) a implémenté le code, mené les campagnes de calcul, tenu les journaux d'expériences et rédigé le texte, sous un système de six portes de décision réservant chaque engagement scientifique à l'auteur humain : le gel du protocole, des adversaires et des ouvertures, chaque dépense de calcul, tout ce qui est rendu public et la suppression de toute donnée. L'auteur humain a choisi la question, approuvé par écrit chaque décision soumise à une porte, vérifié les conclusions et assume chaque affirmation. Chapitre 10 documente cette méthodologie de travail, sa justification et son historique.

Conventions. Les scores sont des moyennes sur les parties décidées (victoire 1, nulle 0.5, défaite 0) contre une population d'adversaires fixe ; les parties arrêtées au plafond de coups expérimental sont *tronquées* et sont rapportées comme une issue distincte, jamais comme des nulles. Les intervalles entre crochets sont des intervalles bootstrap percentile à 95% ayant pour unité de rééchantillonnage l'exécution d'entraînement indépendante (la graine). Le bras entraîné sur l'encodage en grille est le *bras grille* ; le bras entraîné sur l'encodage en graphe est le *bras graphe*. Un glossaire donnant les équivalents français de tous les termes techniques figure en Annexe F. Les figures conservent leurs annotations internes en anglais ; leurs légendes sont en français. Les nombres suivent la typographie du master anglais afin que leur identité entre les deux éditions soit vérifiable mécaniquement : le point est le séparateur décimal ; les séparateurs de milliers sont rendus par une espace fine.

Table des matières

Résumé 1
Abstract 2
Statut, code et déclaration d'assistance 3
1 Introduction 12
1.1 Contexte : apprendre à jouer par auto-jeu, et le coût de la représentation 12
1.2 Le problème : une intuition qui coupe dans les deux sens 12
1.3 Questions de recherche et hypothèse 13
1.4 Ce qui a été fait 13
1.5 Ce qui a été trouvé 14
1.6 Contributions 14
1.7 Organisation de ce rapport 15
1.8 Comment lire les nombres 15
2 Contexte : Hive, apprentissage par auto-jeu et représentations d'état 16
2.1 Le jeu de Hive 16
2.2 Formalisation utilisée dans l'étude 17
2.3 Apprentissage par auto-jeu dans la famille AlphaZero 19
2.4 Représentations d'état 21
2.5 Budgets de calcul et les deux lectures 23
2.6 Évaluer des agents à petite échelle 24
3 Travaux connexes et positionnement 26
3.1 Les programmes qui jouent à Hive 26
3.2 Recherche guidée par réseau et coût de l'auto-jeu 27
3.3 Représenter les états et les ensembles d'actions variables 28
3.4 Comparaisons directes grille-contre-graphe 28
3.5 Standards de preuve pour les comparaisons à peu d'exécutions 29
3.6 Synthèse des travaux les plus proches 30
3.7 Positionnement 31
4 Comment et quand le système et l'étude ont été construits 35
4.1 Le projet de moteur (juillet 2026) 35
4.2 La boucle d'auto-jeu antérieure (juillet–août 2026) 35
4.3 Le programme de recherche (à partir du 9 septembre 2026) 36
4.4 Chronologie 38
5 Le moteur Hive : conception, implémentation et validation 40
5.1 Objectifs de conception 40
5.2 Architecture 40
5.3 Représentation du plateau et hachage 42
5.4 Génération de coups et cas limites des règles 42
5.5 Campagne de validation (septembre 2026) 43
5.6 Profil de débit 47
5.7 Deux correctifs côté moteur consignés avant l'entraînement 48
6 Le pipeline d'apprentissage par auto-jeu 49
6.1 La boucle de générations 49
6.2 Travailleurs d'auto-jeu et réglages de recherche 51
6.3 L'enregistrement d'entraînement 51

6.4 La procédure d'entraînement 52
6.5 Export et inférence 53
6.6 L'arène d'évaluation 53
6.7 Sept vérifications automatisées pré-entraînement 54
6.8 Le pilote du 10 septembre 2026 55
6.9 Un défaut de harnais détecté pendant le pilote 56
7 Les deux représentations d'état et leurs réseaux 58
7.1 Ce qui varie entre les bras, et ce qui, de manière démontrable, ne varie pas 58
7.2 L'encodage grille 59
7.3 L'encodage graphe 61
7.4 Appariement de capacité et asymétries de coût mesurées 64
7.5 Épinglage des encodeurs : tests dorés et tests de propriétés 65
7.6 Les deux variantes d'ablation 66
7.7 Augmentation par symétries : exclue par décision 67
7.8 Limites de ce que ce chapitre établit 67
8 La population d'adversaires gelée 68
8.1 Pourquoi une population fixe plutôt qu'un classement contre des cibles mouvantes 68
8.2 Les trois adversaires 68
8.3 Vérification tactique avant usage 69
8.4 Caractérisation 70
8.5 Le gel du 9 septembre 2026 71
9 Méthodologie expérimentale : un protocole pré-enregistré et gelé 73
9.1 Pourquoi pré-enregistrer, et pourquoi geler, à petit budget de calcul 73
9.2 Questions de recherche et hypothèse 74
9.3 Conception : deux bras, cinq graines, dix générations 75
9.4 Les deux lectures de budget et le seuil à temps mural égal 76
9.5 Évaluation : population gelée, ouvertures gelées, recherche épinglée 76
9.6 Statistiques et politique de troncature 77
9.7 Critères d'exclusion et faits consignés 78
9.8 Conception des ablations 78
9.9 Ressources réellement consommées 79
9.10 Ce qui a été fixé avant la première exécution de comparaison, et ce qui est venu après 79
9.11 Matrice question → expérience → résultat 80
10 Méthodologie de travail : une recherche humain–IA à portes 81
10.1 Pourquoi une méthode à portes 81
10.2 Division du travail 81
10.3 Les six portes 82
10.4 Principes scientifiques appliqués à chaque phase 84
10.5 Ce que la discipline a attrapé 85
10.6 Ce que l'assistant n'a pas fait, et la responsabilité de l'auteur 88
11 Résultats : la comparaison principale (RQ-H1) 90
11.1 Le verdict en un paragraphe 90
11.2 Lecture 1 : nombre égal d'exemples d'entraînement 90
11.3 Lecture 2 : temps mural égal sur la même machine 91
11.4 Le contraste entre bras 92
11.5 Comment l'analyse à cinq graines se rapporte à l'analyse à trois graines 93

11.6 Le plafond de coups peut-il sauver l'hypothèse ? 93
11.7 Où se loge le déficit 93
12 Résultats : coût et efficacité (RQ-H2) 95
12.1 Ce que coûte chaque bras sur la machine d'étude 95
12.2 Les deux lectures côte à côte 96
12.3 Le score en fonction du temps 96
12.4 Ressources consommées par l'étude 97
13 Résultats : ablations par composant (RQ-H3) 98
13.1 Conception des trois cellules 98
13.2 A1 : retirer les relations typées retire l'entraînabilité 98
13.3 A2 : retirer le pooling global ne change rien de mesurable 99
13.4 A1′ : la variante à adjacence naïve stabilisée (supplément à deux composants) 100
13.5 Table de synthèse H-T3 100
14 Résultats : positions d'échec et comportement d'entraînement 102
14.1 Trois positions d'échec, sélectionnées mécaniquement 102
14.2 Le canal de troncature, exécution par exécution 103
14.3 Ce que montrent les métriques d'entraînement 104
14.4 Trajectoires par adversaire 105
15 Une seconde étude : la comparaison sous une boucle à démarrage à chaud avec fenêtre de rejeu 106
15.1 Pourquoi une seconde étude 106
15.2 Ce qui change, et ce qui ne change pas 106
15.3 Implémentation et vérification avant le gel 108
15.4 Exécution 109
15.5 Résultats 110
15.6 Les deux études côte à côte 115
16 Discussion et menaces à la validité 119
16.1 Lire le résultat 119
16.2 Quelles explications sont démontrées, et lesquelles sont plausibles 119
16.3 Rapport aux travaux antérieurs 120
16.4 Validité interne : bogues et comparabilité 121
16.5 Validité de mesure : adversaires et troncature 121
16.6 Validité statistique : graines et dépendances 121
16.7 Validité externe : variante, matériel et budget 122
16.8 Une propriété du pipeline qui borne la force absolue atteinte 122
16.9 Ce que nous ferions différemment 123
17 Conclusion et perspectives 124
17.1 La réponse à la question de recherche 124
17.2 Contributions, telles que démontrées 125
17.3 Deux suites motivées par les données 125
17.4 Une méthode qui tient indépendamment du signe 125
Bibliographie 126
Webographie 128
A Corpus de positions annotées 130
A.1 Corpus de règles critiques (30 cas) 130
A.2 Jeu de vérification tactique (5 cas) 139
B Architectures, décodeur et formats de données en détail 142

B.1 HiveNet, le réseau grille 142
B.2 HiveGraphNet, le réseau graphe 143
B.3 Le décodeur d'actions partagé en détail 144
B.4 Format d'enregistrement 145
B.5 Réglages de recherche 146
C Constantes gelées, hyperparamètres, graines et configurations 148
C.1 Constantes gelées 148
C.2 Hyperparamètres d'entraînement 149
C.3 Dérivation des graines 150
C.4 Configuration de chaque bras et variante 150
C.5 Profil machine mesuré 151
C.6 La seconde étude : constantes gelées, régime et graines 152
D Procédures statistiques et tables de résultats brutes 154
D.1 Score, taux de troncature et score de sensibilité 154
D.2 Le bootstrap au niveau des graines 154
D.3 Tables brutes par graine : lecture à exemples égaux 155
D.4 Tables brutes par graine : lecture à temps mural égal 156
D.5 Moyennes au niveau des graines, intervalles et contrastes entre bras 157
D.6 Le seuil à temps mural égal et la carte des points de contrôle 158
D.7 Bornes de sensibilité au plafond 159
D.8 La seconde étude : scores de sensibilité, carte des seuils et temps mural 159
D.9 Tables brutes des ablations 161
D.10 Données d'évaluation par génération 163
E Provenance de chaque résultat 164
E.1 La chaîne de preuve 164
E.2 De chaque résultat à son artefact 165
E.3 Index des décisions 169
E.4 Registre des affirmations et des preuves 170
E.5 Données brutes, identifiants et hachages gelés 176
F Glossaire 179
G Guide de reproduction 183
G.1 Ce qui est publié 183
G.2 Le scénario minimal en environnement vierge 184
G.3 Commandes de reproduction complète 184
G.4 Durées attendues et disque 185
G.5 Identifiants qu'une reproduction doit retrouver 186

Liste des figures

Fig. 1 Une position de Hive à cinq tuiles sous les deux encodages comparés dans cette étude. À gauche, la vue du bras grille : la position plongée dans un cadre fixe 32×32 de cellules hexagonales (dessinées pour l'orientation), chaque cellule du cadre portant 77 valeurs de caractéristiques, de sorte que les tuiles apparaissent comme des activations aux coordonnées du cadre. À droite, la vue du bras graphe : le graphe de cellules sur l'ensemble candidat, dans lequel les cellules occupées (pleines, étiquetées par couleur et par tuile) et chaque cellule vide adjacente à la ruche (cercles ouverts), formant ensemble l'ensemble de toutes les destinations légales, sont reliées par des arêtes typées par les six directions hexagonales. Les deux vues dérivent du même état du moteur sans perte ; seule la structure offerte au réseau diffère. 21

Fig. 2 Chronologie du projet, juillet–octobre 2026 : projet de moteur (12 juillet), boucle d'auto-jeu antérieure classée comme démonstration (13 juillet – 14 août) et programme de recherche (9 septembre – 9 octobre) avec les gels (population le 9 septembre ; protocole et ouvertures le 10 septembre), la campagne principale (10–17 septembre), le seuil à temps mural égal (16 septembre), l'analyse à trois graines (19 septembre), les ablations (20 septembre – 6 octobre), l'extension des graines (27 septembre – 6 octobre) et l'analyse finale à cinq graines (9 octobre) ; seconde étude (13–27 octobre) : protocole v2 gelé et campagne lancée (13 octobre), dix exécutions à démarrage à chaud (13–26 octobre), seuil à temps mural égal issu des horloges grille (25 octobre), analyse (27 octobre). 39

Fig. 3 La boucle de générations partagée par les deux bras. L'auto-jeu avec recherche arborescente Monte-Carlo produit des enregistrements d'entraînement ; les enregistrements entraînent un réseau ; le réseau est exporté en ONNX et pilote l'auto-jeu de la génération suivante ; à des générations fixées, le réseau exporté est évalué contre la population d'adversaires gelée (B-RND, B-HEU, B-MCTS) sur des ouvertures gelées. Le bras grille et le bras graphe ne diffèrent que par l'encodeur d'état et le corps du réseau à l'intérieur des boîtes marquées « réseau ». 49

Fig. 4 Schémas par blocs des deux réseaux. À gauche, HiveNet (bras grille) : tronc convolutif 3 × 3, 8 blocs résiduels de 96 canaux avec un biais de pooling global dans les blocs 2 et 5, une tête de politique plate à 28 673 sorties et une tête de valeur à 3 classes, 1.44 M paramètres au total. À droite, HiveGraphNet (bras graphe) : couche linéaire d'entrée vers 152 canaux, 8 couches relationnelles à passage de messages avec six matrices typées par direction et un biais de pooling global une couche sur trois, une fonction de notation de politique par candidat sur les plongements de destination, de source et d'emplacement, et une tête de valeur moyenne‖max masquée, 1.47 M paramètres au total. Les deux réseaux alimentent le décodeur d'actions partagé ; seuls l'encodeur et le corps diffèrent. 64

Fig. 5 Score moyen contre la population gelée (moyenne des trois adversaires, troncatures exclues) en fonction du temps mural d'entraînement cumulé, pour les dix exécutions principales ; évaluations aux générations 5, 8 et 10 (20, 20 et

100 parties par adversaire). Bleu : bras grille ; rouge : bras graphe ; une ligne par graine. La ligne verticale en pointillés marque le seuil à temps mural égal de 18.77 heures. 94
Fig. 6 Trois positions d'échec commentées, reproduites et vérifiées contre leurs lignes de résultat enregistrées. F1 (à gauche) : bras graphe, graine 1, contre l'aléatoire légal, ouverture 2, blancs ; la partie atteint le plafond de 300 plis avec le bras graphe matériellement en avance mais incapable d'achever l'encerclement. F2 (au centre) : bras grille, graine 2, contre l'heuristique, ouverture 2, noirs ; une défaite décidée en 19 plis face à la tactique de ciblage de la reine de l'heuristique. F3 (à droite) : bras graphe, graine 3, contre l'adversaire à recherche, ouverture 19, blancs ; une nulle en 79 plis dans laquelle le bras graphe ne génère jamais de menace gagnante. 102
Fig. 7 Taux de troncature à l'évaluation finale contre l'adversaire aléatoire légal pour chacune des dix exécutions principales (points de contrôle finaux ; 100 parties par exécution à 400 simulations sur les 250 ouvertures gelées ; plafond de 300 plis). Bleu : bras grille, graines 1–5 ; rouge : bras graphe, graines 1–5. La troncature est rapportée séparément des nulles tout au long de l'étude. 103
Fig. 8 Métriques d'entraînement par génération pour les dix exécutions de la campagne principale (ligne de l'époque 1 de chaque génération) : à gauche, accord top-1 de la politique avec la distribution de visites MCTS sur la partition de validation d'auto-jeu ; à droite, exactitude de la valeur sur les échantillons non tronqués. Bleu : bras grille ; rouge : bras graphe ; une ligne par graine. Les exécutions d'ablation sont exclues. 104
Fig. 9 Score contre chaque adversaire gelé aux générations évaluées (5, 8 et 10 ; 20, 20 et 100 parties par adversaire respectivement) pour chaque exécution principale. Bleu : bras grille ; rouge : bras graphe ; une ligne par graine. Les scores excluent les parties tronquées. 105
Fig. 10 Seconde étude : part des 100 parties d'évaluation finale contre l'adversaire aléatoire légal (B-RND) arrêtées au plafond de 300 demi-coups, par exécution ; en bleu, les cinq graines grille (11 à 15, de gauche à droite) ; en rouge, les cinq graines graphe. Points de contrôle finaux (génération 10) de dix exécutions à démarrage à chaud, 100 parties appariées à couleurs échangées chacune sur les 250 ouvertures gelées à 400 simulations. 113
Fig. 11 Seconde étude : score moyen contre la population gelée (moyenne des trois adversaires, troncatures exclues) en fonction du temps mural d'entraînement cumulé, pour les dix exécutions à démarrage à chaud ; évaluations aux générations 5, 8 et 10 (20, 20 et 100 parties par adversaire). Bleu : bras grille ; rouge : bras graphe ; une ligne par graine, étiquetée à son extrémité. La ligne verticale en pointillés marque le seuil à temps mural égal de la seconde étude, 17.63 heures. 114
Fig. 12 Seconde étude : score contre chaque adversaire gelé aux générations 5, 8 et 10 (20, 20 et 100 parties appariées par adversaire), une ligne par exécution, cinq graines à démarrage à chaud par bras ; en bleu, le bras grille ; en rouge, le bras graphe ; troncatures exclues. Les points intermédiaires reposent sur 20 parties et ne servent qu'à la trajectoire. 114
Fig. 13 Seconde étude : exactitude top-1 de la politique en validation (à gauche) et exactitude de la valeur sur les échantillons non tronqués (à droite) après la première époque de chaque génération, telles qu'imprimées par l'entraîneur

sur la partition de validation de l'ensemble d'entraînement fenêtré de la génération ; une ligne par exécution, cinq graines à démarrage à chaud par bras ; en bleu, le bras grille ; en rouge, le bras graphe. Diagnostics d'entraînement, non mesures de force. 116

Partie I. Problème et contexte

1 Introduction

1.1 Contexte : apprendre à jouer par auto-jeu, et le coût de la représentation

Depuis AlphaZero, la recette dominante pour apprendre à jouer à un jeu de plateau sans exemples humains est restée stable : un réseau de neurones estime, pour une position, une distribution de probabilité sur les coups (la *politique*) et une issue attendue (la *valeur*) ; une recherche arborescente Monte-Carlo guidée par le réseau produit des décisions plus fortes que le réseau seul ; les parties que la recherche joue contre elle-même deviennent les données d'entraînement du réseau suivant (Silver et al., 2018). La recette est générale, mais l'un de ses ingrédients ne l'est pas : la manière dont une position est présentée au réseau. Pour les échecs, le shogi et le Go, la réponse allait de soi : le plateau est un tableau rectangulaire fixe, et une position devient une pile de plans semblables à une image, lus par un réseau convolutif. Le succès de cet encodage en *grille* repose sur une propriété que ces jeux partagent : chaque case existe à chaque instant, à une place fixe.

Hive ne possède pas cette propriété. C'est un jeu à deux joueurs et à information parfaite, joué avec des tuiles hexagonales et sans plateau : les tuiles elles-mêmes forment la surface de jeu (la *ruche*), cette surface change de forme à chaque tour, les tuiles peuvent grimper les unes sur les autres, et un coup se décrit au mieux par « cette pièce va vers cette cellule » sur un ensemble de cellules qui n'existe que relativement à la ruche courante. L'objet mathématique naturel pour une telle position est un graphe plutôt qu'une image, avec les cellules comme nœuds, l'adjacence comme arêtes et les piles comme attributs de nœud ; le réseau naturel est alors un réseau de neurones en graphe, qui lit directement un graphe de taille variable et n'a besoin d'aucun cadre. Les encodages en grille peuvent néanmoins s'appliquer à Hive en dépliant la ruche dans un grand cadre fixe, comme l'ont fait jusqu'ici tous les systèmes d'apprentissage pour ce jeu ; mais cela introduit des choix d'ancrage, de l'espace vide et un très grand espace de coups discret dont le réseau doit apprendre la structure à partir de zéro.

1.2 Le problème : une intuition qui coupe dans les deux sens

Il est tentant de conclure qu'un encodage en graphe doit nécessairement mieux apprendre Hive. Cette étude a été conçue parce que l'intuition est véritablement incertaine. D'un côté, un réseau en graphe épouse la structure native du jeu et ne porte aucun artefact d'ancrage. De l'autre, l'issue d'une partie de Hive se joue sur des tactiques d'*encerclement* à courte portée (une abeille reine perd lorsque ses six cellules voisines sont occupées), et la reconnaissance de motifs à courte portée est précisément le régime où la localité convolutive est la plus forte. De plus, un graphe sur les seules cellules *occupées* ne peut exprimer les destinations légales d'un coup, qui sont des propriétés de l'espace vide ; le graphe doit donc porter des cellules candidates vides, et l'on ne sait pas si un réseau simple à passage de messages, recevant un tel graphe, apprend les contraintes de glissement, d'escalade et de connexité des règles mieux qu'un réseau convolutif recevant le cadre déplié.

Les résultats antérieurs ne tranchent pas la question. La seule étude publiée de type AlphaZero sur Hive a comparé cinq encodages de plateau et constaté que le choix de l'encodage modifie mesurablement l'apprentissage, mais les cinq étaient des encodages en grille et les moteurs obtenus restaient plus faibles qu'une recherche classique sans apprentissage (de Goede et al., 2022). Les deux comparaisons directes grille-contre-graphe menées sur d'autres jeux pointent dans des directions opposées : à Hex, sous un apprenant fondé sur la valeur plutôt que par auto-jeu, un réseau en graphe a dominé sur les dépendances à longue portée tandis que le réseau convolutif restait plus affûté sur les motifs locaux (Keller et al., 2023) ; aux échecs, sous auto-jeu, un réseau à attention sur graphe a surpassé les lignes de base convolutives, bien qu'à partir d'une seule exécution d'entraînement par modèle et sans réplication par graines (Rigaux & Kashima, 2024). Aucune de ces études n'a fait tourner les deux bras sous un même pipeline d'auto-jeu, à capacité appariée, sous un budget apparié, avec plusieurs exécutions d'entraînement indépendantes par bras, sur Hive. Chapitre 3 développe ce positionnement.

1.3 Questions de recherche et hypothèse

L'étude pose une question principale et deux questions secondaires.

- **RQ-H1 (représentation).** À budget d'entraînement comparable, une architecture en graphe apprend-elle une meilleure politique qu'une architecture en grille pour Hive en jeu de base, au sein d'un pipeline d'auto-jeu de type AlphaZero ?
- **RQ-H2 (efficacité).** Comment les deux représentations se comparent-elles lorsque le budget est lu comme un nombre d'exemples d'entraînement, et lorsqu'il est lu comme un temps mural sur le même matériel ?
- **RQ-H3 (composants).** Quels composants de l'architecture en graphe portent son comportement, en particulier ses relations typées par direction et son pooling global ?

L'hypothèse testée, **H1**, énonce qu'un réseau en graphe simple à passage de messages, recevant la ruche sous forme de graphe, atteint un score moyen plus élevé contre une population d'adversaires fixe qu'un réseau convolutif en grille recevant un cadre déplié de 32×32, à budget d'entraînement égal. H1 n'a pas été présumée vraie : le protocole (Chapitre 9) énonce explicitement qu'un résultat négatif ou nul est une issue publiable, et il a fixé, avant toute exécution comparative, les adversaires, les ouvertures, les budgets, les réglages d'évaluation, l'unité statistique et la règle par laquelle H1 serait rejetée.

1.4 Ce qui a été fait

Les deux encodages ont été construits derrière un décodeur d'actions partagé unique, sur un moteur validé côté règles ; les deux réseaux ont été appariés en capacité à +1.5% près (1.44 M contre 1.47 M paramètres) ; cinq graines indépendantes par bras ont été entraînées sous des réglages d'auto-jeu identiques pendant dix générations de 500 parties chacune ; la comparaison a été lue de deux façons, à exemples d'entraînement égaux et à temps mural égal à un seuil pré-enregistré, contre une population gelée de trois adversaires (aléatoire légal, une heuristique documentée et un agent de recherche sans réseau à 6 400 simulations) sur 250 ouvertures gelées, la troncature étant traitée tout du long comme une issue de premier ordre. Deux ablations ont retiré chacune un composant du bras graphe à parité complète, et une troisième exécution, explicitement

à deux composants, a complété la première ablation après que celle-ci s'est révélée inentraînable.

1.5 Ce qui a été trouvé

L'hypothèse est rejetée. Le bras grille obtient un score supérieur contre deux des trois adversaires sous les deux lectures budgétaires, pour environ la moitié du coût en temps mural ; la plus grande borne supérieure de tout intervalle sur un avantage du graphe est +0.028. Le déficit du bras graphe se concentre dans la conversion tactique locale : il gagne du matériel contre l'adversaire aléatoire puis échoue à conclure, tronquant 20–57% de ces parties là où le bras grille n'en tronque presque aucune. Les ablations localisent la machinerie du bras graphe : ses relations typées par direction sont nécessaires à l'optimisation elle-même (les retirer a fait diverger l'entraînement pour chaque graine), tandis que son pooling global est dispensable.

Une seconde étude, pré-enregistrée séparément et menée après la rédaction de la première, a testé la première objection à ce verdict : que la réinitialisation du réseau à chaque génération ait maintenu les deux bras dans un régime trop précoce pour qu'un a priori de graphe paie. Sous une boucle à démarrage à chaud avec une fenêtre de rejeu de trois générations, sur cinq nouvelles graines par bras, aucun avantage du graphe n'a été détecté sous l'une ou l'autre lecture du budget ; chaque intervalle graphe−grille inclut zéro, les plus grandes bornes supérieures étant +0.090 et +0.055 ; le déficit de la première étude n'est plus visible à exemples égaux et est réduit à temps égal ; la pathologie de conversion du bras graphe persiste dans trois graines sur cinq, et son rapport de coût monte à 2.34 (Chapitre 15).

1.6 Contributions

Quatre contributions sont revendiquées, chacune vérifiable à partir du dépôt publié.

1. **La première comparaison grille-contre-graphe contrôlée, à budget apparié et multi-graines pour Hive**, les deux bras étant placés sous le même pipeline de type AlphaZero, lus sous deux égalisations de budget, avec une réponse négative pré-enregistrée (Chapitre 11, Chapitre 12 ; enregistrements bruts publiés).
2. **Un harnais de comparaison reproductible pour les jeux sans cadre et à empilement** : un moteur de règles validé indépendamment de l'apprentissage ; deux encodeurs épinglés à l'octet près entre deux langages d'implémentation ; un décodeur partagé sur un ensemble d'actions variable ; une évaluation consciente de la troncature ; et une discipline d'artefacts gelés (Chapitre 5 à Chapitre 9).
3. **Une attribution par composant pour le bras graphe**, dans laquelle les relations typées déterminent l'entraînabilité et le pooling a un effet nul, avec un supplément à deux composants honnêtement étiqueté (Chapitre 13).
4. **Une seconde comparaison pré-enregistrée sous le régime standard à démarrage à chaud**, dont le protocole, le seuil à temps égal et les graines ont été fixés avant toute exécution, et dont le résultat nul borne le résultat négatif de la première étude et est rapporté à côté de lui sans agrégation (Chapitre 15).

La construction de logiciel, si substantielle soit-elle, n'est pas comptée comme une contribution scientifique en soi ; elle est rapportée parce que le résultat ne peut être évalué sans elle.

1.7 Organisation de ce rapport

La partie I pose le problème : Chapitre 2 donne les règles de Hive, la formalisation utilisée et les éléments d'apprentissage par auto-jeu, de représentation d'état et d'évaluation sur petits échantillons dont un lecteur a besoin ; Chapitre 3 positionne l'étude par rapport aux travaux antérieurs. La partie II décrit comment le système a été construit, et quand (Chapitre 4) : le moteur et sa validation (Chapitre 5), le pipeline d'apprentissage (Chapitre 6), les deux représentations et leurs réseaux (Chapitre 7) et la population d'adversaires gelée (Chapitre 8). La partie III expose la méthodologie et les raisons de son choix : le protocole expérimental pré-enregistré (Chapitre 9) et la méthode de travail humain-IA à portes (Chapitre 10). La partie IV rapporte les résultats par question : la comparaison principale (Chapitre 11), le coût et l'efficacité (Chapitre 12), les ablations (Chapitre 13) et une analyse qualitative des positions d'échec et du comportement d'entraînement (Chapitre 14), puis la seconde étude sous le régime à démarrage à chaud (Chapitre 15). La partie V discute les mécanismes et les menaces à la validité (Chapitre 16) et conclut (Chapitre 17). La bibliographie et la webographie suivent la conclusion. Les annexes contiennent les corpus de positions annotées, les architectures et les formats de données en entier, chaque hyperparamètre et chaque graine, les procédures statistiques avec les tables brutes par graine, la provenance de chaque résultat, un glossaire avec les équivalents français et le guide de reproduction.

1.8 Comment lire les nombres

Chaque score de ce rapport est une moyenne sur les parties *décidées* contre un adversaire gelé, avec victoire = 1, nulle = 0.5 et défaite = 0 ; les parties arrêtées au plafond expérimental de 300 plis sont comptées séparément comme *troncatures* et ne sont jamais comptées comme des nulles. Chaque intervalle est un intervalle bootstrap percentile à 95% dont l'unité de rééchantillonnage est l'exécution d'entraînement indépendante (la *graine*), et non la partie : mille parties jouées par un réseau sont une observation de ce réseau, non mille observations de la méthode. Les différences entre bras s'écrivent graphe moins grille, de sorte qu'un nombre négatif favorise le bras grille. Tous les nombres remontent aux enregistrements bruts par partie par la chaîne documentée en Annexe E.

2 Contexte : Hive, apprentissage par auto-jeu et représentations d'état

Ce chapitre expose ce que le reste du rapport tient pour acquis : les règles de Hive et ce qui en fait un objet malcommode pour les machines apprenantes ; les objets formels que l'étude manipule, y compris la convention de troncature ; l'auto-jeu de type AlphaZero tel que le système l'implémente, y compris une perte qui s'écarte de la forme usuelle sur deux points délibérés ; les deux représentations d'état comparées ; les conventions budgétaires sous lesquelles elles sont comparées ; et les statistiques permettant d'évaluer des agents à partir d'une poignée d'exécutions d'entraînement indépendantes.

2.1 Le jeu de Hive

Hive, conçu par John Yianni et publié par Gen42 Games, est un jeu à deux joueurs, à information parfaite et à somme nulle, à coups alternés. Il se joue sans plateau : des tuiles hexagonales sont posées sur n'importe quelle surface plane, et les tuiles en jeu définissent la surface de jeu. Dans le jeu de base, chaque joueur possède onze tuiles : une reine, deux araignées, deux scarabées, trois sauterelles et trois fourmis soldats. Trois tuiles supplémentaires (Mosquito, Ladybug, Pillbug) existent en tant qu'extensions ; le moteur utilisé ici les implémente, mais aucun résultat de ce rapport ne fait intervenir une extension.

2.1.1 Placement

Le jeu commence par la pose d'une tuile par un joueur, l'autre accolant ensuite une tuile de sa couleur à celle-ci, bord contre bord. Par la suite, une tuile entrant en jeu ne peut toucher une tuile de la couleur adverse. La reine doit entrer en jeu au plus tard au quatrième tour de chaque joueur, et aucune tuile ne peut se déplacer avant que la reine de son propriétaire soit en jeu. L'étude adopte la règle d'ouverture de tournoi (pas de reine au premier tour de l'un ou l'autre joueur), qui est la convention de l'Universal Hive Protocol et des deux moteurs de référence contre lesquels le noyau de règles a été validé ; la seule feuille de règles de 2010 de l'éditeur autoriserait une reine au premier tour.

2.1.2 Mouvement

La reine glisse d'une cellule. Le scarabée se déplace également d'une cellule, mais peut grimper au sommet de la ruche ; une tuile recouverte par un scarabée ne peut se déplacer et, aux fins du placement, une pile prend la couleur du scarabée qui la coiffe. La sauterelle ne glisse pas du tout : elle saute en ligne droite par-dessus une ou plusieurs tuiles adjacentes jusqu'à la première cellule vide au-delà. L'araignée glisse d'exactement trois cellules le long du pourtour de la ruche, sans revenir sur ses pas. La fourmi soldat glisse vers n'importe quelle cellule atteignable autour de la ruche. Les ascensions du scarabée produisent des piles, et la hauteur d'une pile compte : une tuile qui se déplace au-dessus du niveau du sol doit elle aussi pouvoir glisser, et une *porte du scarabée* formée par deux piles voisines plus hautes la bloque.

Deux contraintes globales traversent ces mouvements. La **règle One-Hive** exige que les tuiles en jeu restent connectées à tout moment : une tuile qui constitue le seul lien entre deux parties de la ruche ne peut se déplacer, quoi que sa propre règle de mouvement autorise. La **règle de liberté de mouvement** exige qu'une tuile qui glisse passe physiquement entre ses voisines : une cellule dont les deux cellules flanquantes sont toutes deux occupées ne peut être ni gagnée ni quittée en glissant. Ensemble, elles signifient que les coups légaux d'une pièce dépendent de la configuration entière plutôt que du seul voisinage de la pièce.

2.1.3 Fin de la partie

La partie se termine dès qu'une reine est entourée sur ses six côtés par des tuiles de n'importe quelle couleur ; son propriétaire perd, même lorsque c'est son propre coup qui a achevé l'encerclement. Si un même coup achève l'encerclement des deux reines, la partie est nulle ; le moteur déclare en outre la nulle sur triple répétition. Un joueur qui ne peut ni placer ni déplacer doit passer, et l'adversaire rejoue.

2.1.4 Pourquoi Hive est difficile pour les machines

Quatre propriétés font de Hive un objet inhabituel pour les méthodes de Chapitre 2.3, toutes développées sur des jeux à plateau fixe.

Géométrie sans cadre. Il n'y a pas de tenseur de plateau à remplir. Une position est un ensemble connexe fini de cellules occupées, de forme et d'étendue arbitraires, plus les cellules vides qui l'entourent ; son emplacement absolu et son orientation ne portent aucune signification. Tout encodage en tableau de taille fixe doit donc choisir un ancrage et un cadre, une décision de modélisation lourde de conséquences (Chapitre 2.4).

Empilement. Les scarabées grimpent, de sorte qu'une cellule peut porter plusieurs tuiles, et les tuiles enfouies restent pertinentes : elles comptent dans les encerclements et réapparaissent lorsque le scarabée s'en va.

Branchement et longueur. Kampert et al. (2021) mesurent un facteur de branchement moyen qui se stabilise autour de 60, environ le double de celui des échecs, et constatent qu'une recherche en pleine largeur à profondeur quatre coûte déjà des dizaines de secondes sur un moteur hautement optimisé, de sorte que le goulot d'étranglement est la qualité de l'évaluation plutôt que la vitesse de recherche. Les parties sont longues : la plus longue partie guidée par la recherche observée lors du profilage du moteur a duré 202 plis, et l'auto-jeu à partir d'un réseau initialisé aléatoirement a atteint le plafond de 300 plis de Chapitre 2.2 dans 56.7% des parties.

La légalité est non locale. Sous la règle One-Hive et la règle de liberté de mouvement, l'existence d'un coup dépend de la configuration globale (points d'articulation de la ruche, portes formées par des piles éloignées) et de l'espace vide. Les mêmes auteurs n'ont trouvé aucune évaluation artisanale à la fois peu coûteuse et fiable : le décompte intuitif des tuiles autour de la reine adverse est franchement trompeur lorsqu'il est fortement pondéré. D'où l'intérêt d'apprendre l'évaluation, et de se demander quelle représentation permet à un réseau de l'apprendre le plus efficacement.

2.2 Formalisation utilisée dans l'étude

Tableau 1 rassemble les symboles utilisés à partir d'ici.

Un **état** s comprend les tuiles en jeu avec leurs cellules et leurs niveaux de pile, les tuiles encore en main pour chaque joueur, le camp au trait, le compte de plis et un marqueur pour la tuile déplacée au pli précédent (pertinent uniquement sous l'extension Pillbug ; conservé pour l'uniformité du noyau et faisant partie de la position hachée). Le moteur expose s via l'Universal Hive Protocol, et la correction de ses règles a été établie indépendamment de tout composant d'apprentissage (Chapitre 5).

Une **action** est une paire $a = (p, d)$ formée d'une pièce p et d'une cellule de destination d, à laquelle s'ajoute une action distinguée, *passe*, légale exactement lorsqu'aucun coup n'existe. La paire identifie un coup de Hive de manière unique, puisque les rares cas où une tuile pourrait atteindre la même cellule par deux mécanismes produisent des états successeurs identiques. Les pièces sont adressées relativement au camp au trait, de sorte qu'un indice d'action signifie la même chose pour l'une ou l'autre couleur. L'ensemble des actions légales $A(s)$ est produit par le générateur de coups du moteur ; les deux architectures apprises le reçoivent et normalisent leurs politiques exactement sur $A(s)$ au moyen d'un décodeur partagé (Chapitre 2.4). La **transition** $s' = T(s, a)$ est l'application du coup par le moteur.

Les **résultats** sont exprimés du point de vue du joueur au trait dans l'état évalué : une victoire vaut $+1$, une défaite -1, une nulle 0. Parce qu'un joueur peut perdre en encerclant sa propre reine, l'implémentation calcule le signe à partir du résultat et du camp au trait plutôt que de supposer que le dernier joueur à avoir joué a gagné.

Troncature expérimentale. Les parties d'auto-jeu et d'évaluation sont arrêtées après 300 plis. Une partie arrêtée de la sorte n'est *pas* une nulle. Elle forme une quatrième classe de résultat, *tronquée*, portée comme telle à travers le format de données, la perte d'entraînement (les enregistrements tronqués ne contribuent à aucune cible de valeur, Chapitre 2.3), le lanceur de matchs, chaque table de résultats (une colonne séparée de taux de troncature) et l'analyse statistique. Le plafond a été fixé à partir de mesures, au-dessus de la plus longue partie guidée par la recherche observée, et a été gelé avec le reste du protocole le 10 septembre 2026, avant toute exécution de comparaison. Traiter une partie plafonnée comme une nulle enseignerait silencieusement à la tête de valeur que les parties longues et non résolues sont équilibrées ; maintenir la troncature à part fait de sa fréquence une quantité rapportée et permet de tester la sensibilité de chaque conclusion au plafond (Chapitre 2.6).

Tableau 1 : Notation utilisée dans ce rapport.

Symbole	Signification
s, s'	un état de jeu et son successeur
$A(s)$	l'ensemble des actions légales dans s (les coups, plus la passe lorsqu'aucun coup n'existe)
$a = (p, d)$	une action : la pièce p vers la cellule de destination d
$T(s, a)$	la transition (l'application du coup par le moteur)
$z \in \{+1, 0, -1\}$	résultat terminal du point de vue du joueur au trait : victoire, nulle, défaite
tronquée	quatrième classe de résultat : partie arrêtée au plafond de 300 plis ; jamais une nulle
$p_\theta(a \mid s)$, $v_\theta(s)$	politique du réseau sur $A(s)$ et valeur du réseau dans $[-1, 1]$
$P(s, a)$, $N(s, a)$, $Q(s, a)$	a priori, nombre de visites et valeur moyenne d'une arête de recherche
$N(s)$	nombre de visites du nœud s
c	constante d'exploration de la règle de sélection (1.4)

Symbole	Signification
ε, α	poids de mélange du bruit à la racine (0.25 en auto-jeu, 0 en évaluation) et concentration (0.15)
$\pi(a \mid s)$	distribution de visites enregistrée à la racine ; la cible de politique
λ_v	poids de la perte de valeur (0.6)
h_i, $h_{i'}$	plongement du nœud de graphe i avant et après une couche de passage de messages
$n_i(d)$	le voisin du nœud i dans la direction hexagonale $d \in \{1, ..., 6\}$
W_{self}, W_d, b	poids propre, poids typés par direction et biais d'une couche de passage de messages
score	moyenne de victoire $= 1$, nulle $= 0.5$, défaite $= 0$ sur les parties d'évaluation non tronquées
graine	une exécution d'entraînement indépendante ; l'unité d'analyse

2.3 Apprentissage par auto-jeu dans la famille AlphaZero

2.3.1 Réseau politique-valeur et recherche

AlphaZero (Silver et al., 2018) apprend à jouer à partir des seules règles. Un unique réseau de paramètres θ associe à un état une politique $p_\theta(\cdot \mid s)$ sur les actions et une valeur $v_\theta(s)$ estimant le résultat pour le joueur au trait. Le réseau ne joue pas directement : il guide une recherche arborescente Monte-Carlo, et la recherche, plus forte que le réseau nu, engendre les parties sur lesquelles le réseau est ensuite entraîné. Silver et al. exécutaient 800 simulations par coup pendant l'auto-jeu et ont atteint une force surhumaine aux échecs, au shogi et au go avec des milliers d'accélérateurs spécialisés et une seule exécution d'entraînement par jeu ; l'étude décrite ici conserve le cœur algorithmique et change l'échelle, le jeu et les statistiques.

La recherche procède par descentes répétées depuis la racine. À un nœud s, l'implémentation sélectionne l'arête qui maximise

$$Q(s,a) + c\,P(s,a)\,\frac{\sqrt{N(s)}}{1+N(s,a)} \tag{1}$$

avec $c = 1.4$, soit la règle PUCT de Silver et al. Ici, $P(s,a)$ est l'a priori du réseau, normalisé sur $A(s)$; $N(s,a)$ le nombre de visites passant par l'arête ; $N(s)$ le nombre de visites du parent, pris au moins égal à un ; et $Q(s,a)$ la moyenne des valeurs remontées par l'arête, du point de vue du parent (les valeurs de l'enfant sont négativées, son camp au trait étant l'adversaire). Deux détails d'implémentation importent pour la reproductibilité. Les évaluations de feuilles sont traitées par lots : une descente qui atteint une feuille non évaluée attend que le lot se remplisse et, entre-temps, une *perte virtuelle* (une visite supplémentaire comptée comme une défaite à la fois dans Q et dans N) est placée sur chaque arête de son chemin, de sorte que les descentes concurrentes se dispersent. Et une arête dont l'enfant n'a jamais été visité prend $Q(s,a) = Q(s) - 0.2$, où $Q(s)$ est la valeur moyenne propre du parent : une réduction d'*urgence au premier coup* (first-play urgency) qui dissuade d'ouvrir tous les enfants avant d'approfondir les plus prometteurs. Une nouvelle feuille est évaluée par le réseau ; sa valeur est remontée le long du chemin en alternant le signe, et les nœuds terminaux remontent leur résultat exact. Lorsque le

budget de simulations est épuisé, la recherche renvoie le coup le plus visité à la racine et le vecteur des nombres de visites à la racine.

2.3.2 Exploration en auto-jeu

Deux dispositifs rendent les parties d'auto-jeu diverses. À la racine, les a priori sont mélangés à du bruit, $P(s,a) \leftarrow (1-\varepsilon)\,P(s,a) + \varepsilon\,\eta_a$, avec $\varepsilon = 0.25$ et η un vecteur aléatoire tiré une fois par recherche selon une distribution de Dirichlet approchée de concentration $\alpha = 0.15$ (approchée dans l'implémentation par des tirages uniformes transformés et normalisés plutôt que par des variables Gamma exactes). Silver et al. ajustaient α au nombre typique de coups légaux de chaque jeu (0.3 pour les échecs, 0.15 pour le shogi, 0.03 pour le go) ; la valeur utilisée ici est leur réglage pour le shogi. Pendant les 12 premiers plis d'une partie, le coup effectivement joué est échantillonné proportionnellement aux nombres de visites à la racine (température un) ; ensuite, le coup le plus visité est joué. Les parties d'évaluation n'appliquent rien de tout cela : $\varepsilon = 0$, pas de température, le coup le plus visité toujours joué, à 400 simulations par décision, imposé par test et épinglé par configuration.

2.3.3 Cibles d'entraînement et perte telle qu'implémentée

Chaque décision enregistrée fournit un exemple d'entraînement : l'état, la distribution de visites à la racine $\pi(\cdot \mid s)$ (les quinze actions les plus visitées, renormalisées), l'ensemble légal $A(s)$ et le résultat final de la partie z du point de vue du joueur au trait dans cet état, ou sinon le marqueur *tronquée*. La cible de politique est la distribution de la recherche plutôt que le coup joué, parce que la recherche améliore l'a priori du réseau et que le réseau est entraîné à prédire cette amélioration.

Les deux bras minimisent la même perte. Pour un mini-lot de B enregistrements, soit $K \subseteq \{1, \ldots, B\}$ les enregistrements dont le résultat est une victoire, une nulle ou une défaite, et soit $q_\theta(\cdot \mid s)$ la softmax à trois voies de la tête de valeur sur ces trois classes. Alors

$$\mathcal{L}(\theta) = -\frac{1}{B}\sum_{b=1}^{B}\sum_{a \in A(s_b)} \pi(a \mid s_b)\log p_\theta(a \mid s_b) - \lambda_v \frac{1}{|K|}\sum_{b \in K}\log q_\theta(z_b \mid s_b), \qquad \lambda_v = 0.6. \quad (2)$$

Le premier terme est l'entropie croisée entre la distribution de visites enregistrée et la politique du réseau, où $p_\theta(\cdot \mid s)$ est une softmax sur *exactement* l'ensemble légal (le bras grille fixe chaque logit illégal à une grande constante négative avant la softmax ; le bras graphe ne produit des logits que pour les candidats légaux), de sorte que les actions illégales portent une masse identiquement nulle dans les deux bras et que l'entraînement utilise la normalisation qu'utilise la recherche. Le second terme est l'entropie croisée du résultat à trois classes, moyennée sur les seuls $|K|$ enregistrements non tronqués (un lot qui n'en contient aucun ne contribue rien) et sous-pondérée par $\lambda_v = 0.6$. Les deux écarts par rapport à une perte conjointe politique-valeur ordinaire sont le masquage légal, qui supprime un facteur de confusion entre les bras (la tête à 28 673 sorties du bras grille dépenserait sinon de la capacité à supprimer des indices qui ne sont jamais légaux), et l'exclusion des parties tronquées de la cible de valeur, moitié côté entraînement de la convention de troncature.

2.3.4 Économies pour petits budgets

Le coût de l'auto-jeu est dominé par la recherche. Wu (2020) a montré, dans le projet KataGo, qu'un pipeline de type AlphaZero peut être rendu bien plus efficace en échantillons par une poignée de changements agnostiques à la représentation. Celui qui est adopté ici, à l'identique pour les deux bras, est la **randomisation du plafond de simulations** (playout-cap randomization) : la tête de valeur a besoin de nombreuses parties tandis que la tête de politique a besoin de recherches profondes, de sorte que la plupart des décisions sont recherchées à bas coût et non enregistrées, et qu'une minorité reçoit une recherche complète et devient un exemple d'entraînement. Wu utilisait des recherches complètes de 600–1 000 visites sur environ un quart des tours et des recherches rapides de 100–200 visites sinon ; l'étude utilise 128 et 32 simulations et enregistre une décision sur quatre, valeurs qui ont été mesurées sur la machine d'étude, comme l'explique le chapitre consacré au protocole. L'auto-jeu termine également une partie par **abandon** lorsque la valeur à la racine tombe sous -0.92, en enregistrant une défaite pour le camp qui abandonne ; dans 10% des parties, choisies avant le premier coup, l'abandon est désactivé afin de pouvoir auditer le taux auquel une position « perdue » aurait en fait été sauvée, garde-fou standard des pipelines d'auto-jeu.

2.4 Représentations d'état

Tout ce qui précède est indifférent à la manière dont un état est remis au réseau, ce qui est la variable de cette étude. Fig. 1 montre une même position sous les deux encodages comparés.

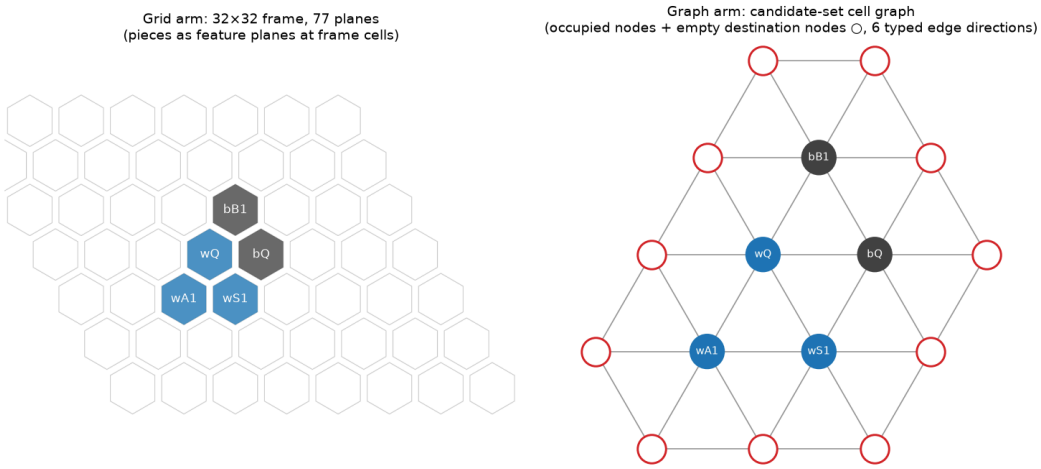


Fig. 1 : Une position de Hive à cinq tuiles sous les deux encodages comparés dans cette étude. À gauche, la vue du bras grille : la position plongée dans un cadre fixe 32×32 de cellules hexagonales (dessinées pour l'orientation), chaque cellule du cadre portant 77 valeurs de caractéristiques, de sorte que les tuiles apparaissent comme des activations aux coordonnées du cadre. À droite, la vue du bras graphe : le graphe de cellules sur l'ensemble candidat, dans lequel les cellules occupées (pleines, étiquetées par couleur et par tuile) et chaque cellule vide adjacente à la ruche (cercles ouverts), formant ensemble l'ensemble de toutes les destinations légales, sont reliées par des arêtes typées par les six directions hexagonales. Les deux vues dérivent du même état du moteur sans perte ; seule la structure offerte au réseau diffère.

2.4.1 Plans empilés pour réseaux convolutifs

L'encodage canonique de la famille AlphaZero est une pile de plans binaires ou scalaires sur le tableau du plateau, lue par un réseau convolutif résiduel, que la convolution rend équivariant par translation, avec un champ réceptif qui s'élargit avec la profondeur. Silver et al. utilisaient plus d'une centaine de plans sur le plateau d'échecs 8×8 et 17 plans pour le go, et exprimaient la politique elle-même sous forme de plans spatiaux. L'encodage présuppose un tableau fixe. Pour Hive, il faut le fabriquer : le bras grille déplie le plateau du moteur, stocké sur un tore, par un parcours en largeur depuis une cellule occupée quelconque, translate la position de sorte que le centre de sa boîte englobante tombe au centre d'un cadre 32×32, et écrit 77 plans : plans de pièces indexés par propriétaire, type d'insecte et niveau de pile ; plans pour les tuiles clouées, la dernière tuile déplacée et les régions de placement légal des deux camps ; et plans constants pour le camp au trait, les libertés des reines, le pli et les réserves. En coordonnées axiales, les six voisins hexagonaux d'une cellule forment un sous-ensemble de son voisinage carré 3×3, de sorte que des convolutions 3×3 ordinaires couvrent l'adjacence hexagonale, deux poids de coin ne rencontrant jamais de voisin. Le cadre est dimensionné pour que toute position du jeu de base et son anneau de cellules candidates y tiennent par construction, et l'encodeur le vérifie par assertion à chaque appel. Aucune canonicalisation par rotation ou réflexion n'est appliquée : le cadre fixe la translation, mais non l'orientation.

2.4.2 Graphes et passage de messages

L'alternative consiste à remettre au réseau la structure d'adjacence elle-même. Un réseau de neurones sur graphe calcule le plongement de chaque nœud à partir des caractéristiques propres du nœud et des plongements de ses voisins, avec des poids partagés entre tous les nœuds et tous les graphes. Dans le schéma d'agrégation de voisinage de Hamilton et al. (2017), la couche k calcule $h_v^k = \sigma\big(W \cdot \big[\, h_v^{k-1} \,;\, \mathrm{AGG}\big(\{h_u^{k-1} : u \in \mathcal{N}(v)\}\big)\,\big]\big)$ pour chaque nœud v, où AGG est une fonction invariante par permutation de l'ensemble des voisins. Parce que les poids appartiennent à la couche et non au nœud, un même réseau entraîné plonge un graphe de n'importe quelle taille ou forme. C'est la propriété inductive qu'appelle une position de Hive sans cadre et de taille variable, et que le cadre fixe du bras grille ne fait qu'approcher. L'agrégation de voisinage ordinaire ignore la sémantique des arêtes : un voisin est un voisin. À Hive, la *direction* d'un voisin compte (une sauterelle saute le long d'une ligne ; une porte est formée par les deux cellules flanquant une direction), de sorte que le bras graphe utilise des **relations typées par direction**, une matrice de poids par direction hexagonale. Sa couche de passage de messages met à jour le plongement de la cellule i selon

$$h_{i'} = \mathrm{ReLU}\left(h_i + W_{\mathrm{self}}\, h_i + b + \sum_{d=1}^{6} W_d\, h_{n_i(d)} \right), \tag{3}$$

où $n_i(d)$ est le voisin de i dans la direction d (le terme s'annule lorsqu'aucun voisin de ce type n'existe dans l'ensemble candidat), W_{self} et b sont la transformation propre et son biais, les six W_d sont les matrices typées par direction, et le h_i en tête est une connexion résiduelle. Les nœuds sont les cellules de l'ensemble candidat, c'est-à-dire chaque cellule occupée et chaque cellule vide adjacente à la ruche, de sorte que chaque destination légale, y compris les vides, est un objet à part entière que la politique peut noter ; un graphe sur les seules pièces n'aurait rien à quoi rattacher une destination. Les tuiles ne sont pas des nœuds séparés : les caractéristiques d'une cellule décrivent toute sa pile

niveau par niveau, de sorte que l'empilement entre comme contenu de nœud plutôt que comme structure de graphe. L'information globale (camp au trait, pli, libertés des reines, réserves par type d'insecte) entre comme un vecteur diffusé à chaque nœud en entrée et concaténé à la représentation agrégée au niveau des têtes. Parce que le passage de messages propage d'un saut par couche alors que la sécurité de la reine dépend de la ruche entière, une couche sur trois ajoute aussi un **biais de pooling global**, d'après le dispositif de pooling global de Wu (2020) : la moyenne et le maximum des plongements de nœuds, passés par une application linéaire et ajoutés à chaque nœud. La tête de valeur lit un pooling masqué moyenne-et-maximum des plongements finaux ; la tête de politique note chaque paire légale (pièce, destination) à partir du plongement du nœud de destination, du nœud courant de la pièce qui se déplace (ou d'un vecteur appris pour les placements depuis la main) et d'un plongement de l'emplacement de pièce.

2.4.3 Ce qu'un réseau de graphe offre et n'offre pas

Un encodage en graphe n'accorde aucune invariance gratuitement. L'encodage est sans coordonnées : aucune coordonnée absolue n'apparaît nulle part, de sorte que les questions d'ancrage ne se posent pas. Le passage de messages avec des arêtes typées par direction n'est néanmoins *pas* invariant par rotation (faire tourner la ruche permute les types de relation), les champs réceptifs sont limités par la profondeur, et aucune règle de Hive n'est connue du réseau. La légalité est fournie position par position par le moteur, à travers le masque légal du décodeur partagé, à l'identique pour les deux bras, et aucun des deux réseaux ne la calcule jamais. Toute affirmation d'invariance sur la fonction apprise doit être mesurée, non supposée. Inversement, le bras grille n'est pas dépourvu de contexte global : deux de ses blocs résiduels portent également un biais de pooling global, de sorte que les deux bras diffèrent par la manière dont l'adjacence est présentée plutôt que par leur capacité à voir la position entière.

2.5 Budgets de calcul et les deux lectures

« À budget comparable » est la clause qui donne son sens à la question de recherche, et elle est ambiguë. Les grands articles d'auto-jeu énoncent les budgets en termes matériels. Silver et al. rapportent des nombres d'accélérateurs et un temps mural pour une seule exécution par jeu, le matériel de génération et celui d'entraînement étant comptabilisés séparément, ce qui ne permet aucune comparaison normalisée par le coût entre architectures. Wu (2020) rapporte au contraire le type et le nombre de machines, le temps mural, les parties d'auto-jeu et les échantillons d'entraînement, et trace la force en fonction du coût cumulé plutôt que des itérations. Jones (2021), entraînant des agents de type AlphaZero à Hex à une échelle délibérément celle d'un petit laboratoire, mesure les budgets en FLOP-secondes, en GPU-heures et en échantillons, trace des *frontières de calcul* (la meilleure force atteignable par unité de calcul) et constate que le calcul à l'entraînement et le calcul au test s'arbitrent l'un contre l'autre : environ dix fois plus de calcul d'entraînement remplaçait environ quinze fois plus de recherche à force constante. Une évaluation doit donc fixer le budget de recherche à l'identique entre les bras, sous peine de confondre la qualité de la représentation avec la recherche.

L'étude comptabilise donc chaque exécution en quatre dénominations, à savoir le temps mural, le matériel (machine, cœurs, accélérateur), le nombre d'états d'entraînement consommés et les simulations par décision, et lit la comparaison sous deux égalisations.

Sous la **lecture à exemples égaux**, les deux bras s'entraînent sur le même nombre d'états d'auto-jeu, générés avec le même budget de simulations par décision, et la question est de savoir quelle représentation extrait le plus de chaque exemple. Sous la **lecture à temps mural égal**, les deux bras reçoivent le même nombre d'heures sur la même machine, et la question est de savoir quelle représentation fournit le plus de force par heure. Les deux lectures peuvent diverger : un réseau de graphe qui est meilleur par exemple mais plus lent par exemple (en inférence d'auto-jeu, en entraînement, ou les deux) peut gagner la première et perdre la seconde. Aucune n'est privilégiée ; les deux sont rapportées côte à côte, l'asymétrie de coût entre les bras étant mesurée sur la machine d'étude et imputée plutôt que neutralisée par égalisation, et les deux utilisent le même budget de recherche en évaluation.

2.6 Évaluer des agents à petite échelle

Mesurer la force d'un agent de Hive appris soulève trois questions : contre qui, avec quel score, et avec quelle notion d'incertitude.

Contre qui. Hive ne dispose d'aucune échelle publique de réseaux de référence du type de celle contre laquelle KataGo a été mesuré, ni d'aucun joueur parfait pour ancrer une échelle de cotation comme MoHex ancre les expériences de Jones à Hex ; le taux de victoire contre un joueur aléatoire sature et ne permet pas de séparer des agents convenables (Kampert et al., 2021). L'étude évalue donc contre une **population d'adversaires fixe** de trois agents, gelée le 9 septembre 2026 avant toute exécution d'entraînement et jamais réajustée, reversionnée ni étendue par la suite : B-RND, uniforme parmi les coups légaux ; B-HEU, un agent glouton sur une évaluation artisanale documentée aux poids épinglés par hachage ; et B-MCTS, la même recherche arborescente que les agents appris mais avec des a priori uniformes et l'évaluation artisanale comme valeur de feuille, à 6 400 simulations par décision. Aucun point de contrôle de l'un ou l'autre bras n'appartient à la population, de sorte qu'aucun bras n'est noté contre lui-même. Chaque match joue des parties appariées depuis les mêmes ouvertures pré-tirées, à couleurs échangées, de sorte que tous les bras, graines et adversaires font face à des calendriers d'ouvertures et de couleurs identiques. Les cotations de type Elo, là où elles sont imprimées, sont descriptives, relatives à cette population, et jamais un chiffre de premier plan.

Avec quel score. La métrique primaire est le **score** moyen contre la population, une victoire comptant 1, une nulle 0.5 et une défaite 0, calculé sur les seules parties non tronquées. Les parties tronquées sont exclues de la moyenne et rapportées comme un **taux de troncature** séparé dans chaque table ; une colonne de sensibilité note en outre les troncatures à 0.5, et des traitements encadrants (troncatures comptées comme défaites, troncatures comptées comme victoires) bornent ce que tout plafond alternatif pourrait changer. Si la direction d'une conclusion change avec le plafond, cette fragilité est elle-même un résultat rapporté.

Avec quelle incertitude. Agarwal et al. (2021) ont montré que les résultats d'apprentissage par renforcement profond issus d'une poignée d'exécutions d'entraînement sont couramment sur-interprétés : des conclusions tirées d'estimations ponctuelles s'inversent sous une analyse par intervalles, et l'incertitude est gravement sous-estimée en deçà d'une dizaine d'exécutions. Leur remède, des estimations par intervalles issues d'un bootstrap stratifié sur les exécutions plutôt que des moyennes nues, est adopté avec

l'adaptation qu'ils signalent pour une tâche unique : la stratification se réduit à un bootstrap sur les graines. La **graine**, une exécution d'entraînement indépendante, est l'unité d'analyse. Les parties d'évaluation sont d'abord agrégées en un score par cellule (graine, adversaire) ; la moyenne et l'intervalle d'un bras proviennent ensuite d'un bootstrap percentile sur les scores au niveau des graines, avec 10 000 rééchantillonnages et les quantiles 0.025 et 0.975 comme intervalle à 95%, et le contraste graphe moins grille rééchantillonne indépendamment les ensembles de graines des deux bras. Les parties ne sont jamais regroupées comme observations indépendantes : des milliers de parties issues d'un seul modèle mesurent ce modèle avec précision mais ne disent rien de la variabilité entre exécutions d'entraînement, qui est l'objet même de la comparaison.

Ces conventions (adversaires gelés, un score qui garde la troncature visible, la graine comme unité, des intervalles sur les graines) ont été fixées dans le protocole avant qu'aucune exécution de comparaison n'existe. Chapitre 9 les énonce intégralement, avec le nombre de graines, la règle de rejet pré-enregistrée et le seuil à temps mural égal.

3 Travaux connexes et positionnement

Ce chapitre est organisé par problème plutôt que par article : les programmes jouant à Hive ; le coût de l'auto-jeu guidé par réseau ; les représentations des états et des ensembles d'actions variables ; les comparaisons directes grille-contre-graphe ; et les standards de preuve pour les études à peu d'exécutions. Seuls les travaux lus intégralement au cours de l'étude sont cités. Tableau 2 compare les plus proches d'entre eux avec la présente conception, et Chapitre 3.7 énonce ce qui est reproduit, modifié, maintenu identique et revendiqué.

3.1 Les programmes qui jouent à Hive

Les travaux savants sur Hive sont rares et presque entièrement classiques. Kampert et al. (2021) ont construit des agents heuristiques minimax et MCTS sur le moteur BeeKeeper et documenté la taille du jeu : le facteur de branchement moyen se stabilise autour de 60, environ le double de celui des échecs (≈30), et même un moteur hautement optimisé ne génère que ≈400k nœuds par seconde en fin de partie. Leur résultat le plus robuste est négatif : le décompte intuitif des tuiles autour de la reine adverse porte peu de signal d'évaluation une fois son poids réglé, alors qu'un terme de distance à la reine aide ; et le taux de victoire contre un joueur aléatoire sature et ne permet pas de séparer des agents convenables. Leurs agents, comme les tentatives de niveau mémoire qu'ils recensent, sont restés en deçà du jeu humain fort ; leur suggestion finale, insérer un réseau de neurones léger, est ce que le travail suivant a repris.

AZ-Hive (de Goede et al., 2022) est le travail antérieur le plus proche et la motivation directe de cette étude : une boucle de type AlphaZero sur Hive de base à travers le même moteur, le treillis hexagonal projeté sur un tableau 26×26 à axes obliques lu par un CNN à 4 couches avec augmentation par les 12 symétries, et cinq encodages de plateau croisés avec deux encodages d'action (coordonnées absolues contre relatif à la tuile), chaque configuration étant entraînée pendant 4 h avec 5 répétitions. Deux résultats importent ici. Le choix de l'encodage a un effet visible : avec les actions relatives à la tuile, l'encodage de plateau hybride l'emporte nettement tandis que les encodages original et simple ne montrent aucune amélioration du taux de victoire en 4 h, et avec les actions en coordonnées absolues l'ordre change. Et les moteurs appris sont restés faibles : après 24 h d'entraînement, le moteur d'auto-jeu était coté 1063 dans un tournoi toutes rondes BayesElo (400 matchs par moteur, avantage du trait neutralisé) contre 1181 pour un MCTS ordinaire et 1355 pour le minimax du moteur (non entraîné 704, aléatoire 778). Chaque encodage de leur espace de conception est un tableau dense sur un cadre fixe ; aucune représentation en graphe n'apparaît nulle part dans l'article (ce qui a été confirmé sur le texte intégral). Les auteurs estiment une exploration complète de cet espace à des dizaines d'années d'entraînement, le cadrage à calcul limité dont hérite cette étude.

Hors de la littérature, un projet amateur public (hiveGo ; Pfeifer, 2018–2026) attache un évaluateur à propagation avant à un chercheur alpha-bêta et un « petit GNN » (tiny GNN) à une boucle de type AlphaZero pour Hive dans une même base de code, avec un modèle à convolution hexagonale signalé comme défectueux. Il ne rapporte aucune évaluation contrôlée (ni adversaires fixes, ni nombres de parties, ni graines, ni intervalles) et ne

compare jamais ses familles de représentations dans des conditions appariées. Il est cité comme preuve d'existence qu'un encodeur en graphe pour Hive a été construit, et comme petit exemple de la lacune qui traverse tout le domaine.

3.2 Recherche guidée par réseau et coût de l'auto-jeu

AlphaZero (Silver et al., 2017/2018) a fixé le gabarit algorithmique dont hérite cette étude : un réseau politique-valeur guide une recherche PUCT en auto-jeu (800 simulations par coup), l'état est présenté sous forme de plans spatiaux empilés (8×8×119 pour les échecs) et la politique elle-même est exprimée en plans spatiaux, la paramétrisation grille canonique dont descend le bras grille. Son style de preuve est ce dont une étude à petit calcul doit s'écarter : une exécution d'entraînement par jeu, un budget libellé en matériel (5 000 TPU de première génération pour l'auto-jeu, 64 TPU de deuxième génération pour l'entraînement) et une évaluation contre un seul moteur de référence par jeu. AlphaZero ne fait jamais varier l'encodage à budget fixe.

KataGo (Wu, 2020) a montré à quel point ce coût est compressible : moins de 30 GPU V100 pendant 19 jours (≈1.4 GPU-années, 4.2M parties d'auto-jeu, ≈241M échantillons d'entraînement) ont atteint la force qu'ELF OpenGo avait atteinte avec ≈74 GPU-années, un gain revendiqué de ≈50×, et il a établi le standard de rapport budgétaire suivi ici : matériel, temps mural, parties et échantillons, force en fonction du coût cumulé, et une exécution d'ablation par technique. De ses économies, cette étude en adopte exactement une, la randomisation du plafond de simulations, parce qu'elle est agnostique à la représentation et s'applique à l'identique aux deux bras ; les caractéristiques d'entrée propres au go et les cibles auxiliaires, qui feraient entrer en contrebande de la connaissance du domaine dans un seul encodage, ne sont pas copiées. Le pooling global de KataGo donne à un corps convolutif un contexte global et compense en partie la localité d'un encodeur en grille, raison pour laquelle le bras grille conserve un biais de pooling global et le biais de pooling global propre au bras graphe fait l'objet d'une ablation plutôt que d'une supposition.

Polygames (Cazenave et al., 2020) est le plus fort représentant du paradigme grille bien exécuté : des corps entièrement convolutifs avec pooling global rendent le réseau indépendant de la taille du plateau, et le cadre logiciel a produit le premier programme à apprentissage sans connaissance préalable (zero-learning) à battre des joueurs humains forts à Hex 19×19. Cette invariance tient à l'intérieur du paradigme grille à topologie fixe : il met à l'échelle des plateaux dont l'adjacence est connue d'avance. Hive ne figure pas parmi ses jeux, et un jeu sans plateau, à empilement, n'offre aucun treillis fixe à mettre à l'échelle.

Jones (2021) est la caution méthodologique la plus proche pour des conclusions à petite échelle : des agents de type AlphaZero entraînés à Hex de 3×3 à 9×9 avec ≈500 GPU-heures au total ont montré des frontières calcul-performance lisses (≈500 Elo par ordre de grandeur de calcul d'entraînement au sein d'une taille de plateau), preuve que de petites expériences d'auto-jeu portent un signal extrapolable. Le même travail fournit un avertissement que le protocole encode : environ 10× plus de calcul d'entraînement remplaçait environ 15× plus de recherche à force constante, de sorte que les nombres de visites en évaluation doivent être épinglés, faute de quoi une comparaison de représentations devient une comparaison de budgets de recherche. Jones n'a utilisé que des réseaux entièrement connectés ; la représentation n'a jamais été un axe.

3.3 Représenter les états et les ensembles d'actions variables

GraphSAGE (Hamilton et al., 2017) fonde la famille inductive à passage de messages à laquelle appartient le bras graphe : des poids par couche partagés entre tous les nœuds plongent n'importe quel graphe, y compris des graphes jamais vus à l'entraînement, et chaque position de Hive est un graphe inédit. GraphSAGE vanille n'a pas de sémantique d'arêtes et garde une faible profondeur (deux couches apportaient l'essentiel de ses gains) ; le bras graphe ajoute des relations typées par direction (les six directions hexagonales, une matrice de poids chacune) et agrège sur les voisinages complets, puisqu'une cellule de Hive a au plus six voisins. La profondeur face au budget est un axe réel : Keller et al. (2023) ont eu besoin de quinze couches de passage de messages pour les dépendances à longue portée de Hex.

Les Pointer Networks (Vinyals et al., 2015) justifient la notation d'un ensemble variable de candidats : la distribution de sortie est une softmax sur les scores de compatibilité d'exactement les candidats présents dans l'entrée, de sorte qu'aucune capacité n'est dépensée sur un treillis de sortie fixe et pour l'essentiel illégal. Le décodeur d'actions partagé est la forme « jeu de plateau » de ce mécanisme : une distribution sur les paires légales (pièce, destination) de la position, que les deux bras émettent sur le même espace d'actions, construite, masquée, normalisée et entraînée de la même manière. Parce qu'on ne peut pointer vers un élément absent de l'entrée, le graphe contient les destinations candidates vides comme nœuds à part entière.

Les réseaux MDP-homomorphes (van der Pol et al., 2020) fournissent la théorie expliquant pourquoi la représentation pourrait compter à petit calcul : l'équivariance à la symétrie conjointe état-action rétrécit l'espace d'hypothèses et a accéléré l'apprentissage sur CartPole, un monde en grille et Pong à nombre égal de pas d'environnement. La construction requiert un groupe connu, petit et exact ; Hive de base n'a pas de bord de plateau, de sorte que son groupe de symétrie est grand (translations combinées au groupe hexagonal à douze éléments). Cette étude ne construit aucune couche équivariante. L'encodage en graphe est sans coordonnées par construction, mais le passage de messages avec des types d'arêtes directionnels n'est pas invariant par rotation, de sorte qu'aucune invariance n'a été supposée du réseau de graphe et qu'aucune n'est revendiquée.

3.4 Comparaisons directes grille-contre-graphe

Trois travaux comparent un encodeur en grille à un encodeur en graphe sur un jeu de plateau. Aucun ne porte sur Hive, et il n'en est pas deux qui partagent le jeu, l'algorithme d'apprentissage, le budget et les statistiques.

Keller et al. (2023) ont mené une comparaison CNN-contre-GNN à paramètres appariés sur Hex : un réseau SAGEConv à quinze couches (≈487K paramètres) contre un CNN résiduel et un U-Net (≈481K paramètres), chacun entraîné pendant ≈110 A100-heures en 11×11. Le GNN a commis 1 erreur sur une suite de dépendances à longue portée couvrant des plateaux de 8×8 à 25×25, là où les CNN en commettaient 31 et 36, a transféré sans réentraînement (zero-shot) vers des tailles de plateau inédites et a moins surappris, tandis que le CNN restait plus affûté sur les motifs locaux en jeu direct. Trois caractéristiques limitent sa portée ici. La comparaison contrôlée s'est déroulée sous RainbowDQN, et le bras CNN ne s'est jamais entraîné sous recherche arborescente : l'exécution de type AlphaZero (800 simulations par coup, trois A100 pendant ≈6 jours) n'existe que pour le

GNN. Le graphe est une réduction au jeu de Shannon propre à Hex, dans laquelle les cellules jouées sont contractées, de sorte que chaque nœud est une cellule vide et que les actions sont en bijection avec les nœuds ; les auteurs déclarent qu'aucune formulation d'efficacité comparable n'est connue pour d'autres jeux. Et le travail est une prépublication.

Rigaux et Kashima (2024) rapportent le signe opposé pour les échecs, dans un lieu de publication évalué par les pairs : un réseau d'attention sur graphe à caractéristiques d'arêtes (GATEAU) dont la politique est lue sur les arêtes de coup accroît la force de jeu un ordre de grandeur plus vite qu'un AlphaZero convolutif du même cadre logiciel (expérience de vitesse d'apprentissage : huit GPU RTX A5000 pendant ≈13 j 16 h), et un modèle entraîné aux échecs de Gardner 5×5 transfère vers le 8×8 (Elo 807±46 sans aucune exposition au 8×8, 1876±47 après 100 itérations d'affinage). Trois caractéristiques confondent une affirmation sur la représentation, chacune énoncée par les auteurs ou visible dans l'article : chaque modèle a été entraîné exactement une fois, de sorte que les intervalles rapportés (intervalles par la méthode delta sur une estimation Elo à a priori de Jeffreys) ne couvrent que l'incertitude d'échantillonnage des matchs ; les « paramètres comparables » vont de 1.0M pour le modèle graphe à 2.2M pour le CNN ; et la paramétrisation de l'action change en même temps que la représentation (une lecture sur les arêtes contre une tête grille fixe). Les échecs donnent en outre à chaque arête une riche sémantique conçue à la main (direction, promotion, capacités des pièces) qu'un graphe d'adjacence de Hive ne possède pas.

Ben-Assayag et El-Yaniv (2021) ont remplacé le CNN d'AlphaZero par un réseau d'isomorphisme de graphe à trois couches sur le treillis carré, plus un nœud factice global, pour s'entraîner sur de petits plateaux d'Othello, de Gomoku et de go et jouer sur de plus grands, sur un seul TITAN X : ≈3 jours d'entraînement sur petit plateau ont battu un AlphaZero entraîné jusqu'à 30 jours sur le plateau cible (54% à Othello 16×16, 84% en 20×20, 100% à Gomoku 17×17), chaque chiffre étant moyenné sur cinq exécutions indépendantes avec erreurs types. Il s'agit d'une affirmation de transfert sous budgets délibérément asymétriques, non d'une comparaison de représentations à budget égal, et ses actions vivent sur les nœuds. Il n'en présente pas moins la meilleure hygiène de réplication des trois, et son nœud factice rappelle qu'un passage de messages peu profond a besoin d'un chemin global explicite.

Lues ensemble, les trois comparaisons ne fixent pas de signe pour Hive. Elles rendent un avantage du graphe plausible (structure à longue portée, transfert en taille, moindre surapprentissage) tout en montrant qu'un CNN peut l'emporter sur les motifs locaux, que le résultat positif publié repose sur une exécution par modèle avec un décodeur confondu, et que ce que les arêtes encodent peut compter davantage que la structure de graphe elle-même. Le jeu de Hive est dominé par des tactiques d'adjacence locale autour de la reine, là où les données de Hex favorisent la grille.

3.5 Standards de preuve pour les comparaisons à peu d'exécutions

Agarwal et al. (2021) fournissent le cadre statistique de ce régime. En réanalysant des résultats publiés d'apprentissage par renforcement profond, ils montrent que les conclusions tirées d'estimations ponctuelles s'inversent fréquemment sous une analyse par intervalles, que les comparaisons à 3–5 exécutions ne sont souvent pas soutenues par

leurs propres données, et que l'incertitude est gravement sous-estimée en deçà d'une dizaine d'exécutions ; ils prescrivent des estimations par intervalles partout, une agrégation par exécution avant toute comparaison, un rééchantillonnage bootstrap stratifié des exécutions, et une statistique de probabilité d'amélioration à la place des moyennes nues. Avec un seul jeu, la stratification se réduit à un bootstrap sur les graines, de sorte que c'est le nombre d'exécutions d'entraînement indépendantes qui compte. Cette étude suit ces prescriptions avec la graine comme unité de rééchantillonnage : un score par graine et par adversaire, des intervalles sur la différence graphe–grille calculés sur les graines en préservant l'appariement, et une règle de décision écrite avant toute exécution. Par contraste, AZ-Hive a répété 5 fois chaque configuration de 4 h, Ben-Assayag et El-Yaniv ont moyenné cinq exécutions, Keller et al. et KataGo rapportent une exécution principale, et Rigaux et Kashima ont entraîné chaque modèle une fois.

3.6 Synthèse des travaux les plus proches

Tableau 2 place les huit travaux les plus proches à côté de la présente conception.

Tableau 2 : Travaux antérieurs les plus proches comparés à cette étude (environnement, représentation, recherche, budget, protocole de preuve et différence avec la présente étude).

Travail	**Environnement**	**Représentation**	**Recherche**	**Budget**	**Protocole de preuve**	**Différence avec cette étude**
AZ-Hive (de Goede et al., 2022)	Hive, jeu de base	5 encodages denses en tableau sur treillis hexagonal × 2 encodages d'action, tous CNN ; cadre 26×26	MCTS de type AlphaZero	4 h par configuration, 5 répétitions ; un moteur à 24 h	Courbes de perte ; taux de victoire contre l'aléatoire ; tournoi toutes rondes BayesElo	Pas de bras graphe ; conclusions sur la vitesse d'apprentissage initiale ; moteurs en deçà d'un MCTS ordinaire
Kampert et al. (2021)	Hive, jeu de base	Caractéristiques heuristiques artisanales	Minimax (alpha-bêta, table de transposition), variantes de MCTS ; 0.01–1 s par coup	Un nœud de calcul	Taux de victoire contre l'aléatoire ; tours jusqu'à la victoire ; tournoi toutes rondes Elo	Pas d'apprentissage, pas de question de représentation ; agents en deçà du niveau humain
Keller et al. (2023)	Hex 8×8–25×25 (graphe du jeu de Shannon)	GNN SAGEConv à 15 couches (≈487K) contre ResNet et U-Net (≈481K)	RainbowDQN pour la comparaison ; de type AlphaZero (800 simulations) pour le GNN seulement	≈110 A100-heures par modèle	Suite de tests à longue portée ; transfert en taille ; une exécution par modèle	Bras CNN jamais sous recherche en auto-jeu ; graphe propre à Hex ; actions par nœud ; prépublication
Rigaux & Kashima (2024)	Échecs (8×8) et Gardner 5×5	Attention sur graphe à caractéristiques d'arêtes (GATEAU) avec lecture de politique sur les arêtes, contre plans CNN	MCTS de type AlphaZero	8×A5000, ≈13 j 16 h	Elo relatif avec intervalles par la méthode delta ; une exécution par modèle	Exécution unique, pas de variance entre graines ; 1.0M contre 2.2M paramètres ; le décodeur change avec la

Travail	Environnement	Représentation	Recherche	Budget	Protocole de preuve	Différence avec cette étude
						représentation
Ben-Assayag & El-Yaniv (2021)	Othello, Gomoku, go ; petits plateaux → grands	GIN à 3 couches sur le treillis carré + nœud factice global ; politique sur les nœuds	MCTS de type AlphaZero, 100 simulations	1×TITAN X ; ≈3 j contre 30 j, asymétrique par conception	Taux de victoire contre l'aléatoire, le glouton et AlphaZero ; 5 exécutions avec erreurs types	Affirmation de transfert, non comparaison à budget égal ; actions sur les seuls nœuds ; code non publié
Polygames (Cazenave et al., 2020)	Hex, Havannah, Othello et autres ; pas Hive	Entièrement convolutif + pooling global (grille)	MCTS de type AlphaZero	Non comparable par exécution	Résultats de compétition ; victoires contre des humains forts à Hex 19×19	Paradigme grille seulement ; plateaux à topologie fixe
KataGo (Wu, 2020)	Go	Plans CNN + caractéristiques et pooling globaux	PUCT-MCTS avec randomisation du plafond de simulations	Moins de 30 V100 pendant 19 jours (≈1.4 GPU-années)	Elo bayésien contre une échelle externe ; ablations par technique ; une exécution principale	Étude de réduction de coût, non comparaison de représentations ; caractéristiques propres au go
Jones (2021)	Hex 3×3–9×9	Réseaux résiduels entièrement connectés	MCTS de type AlphaZero	≈500 GPU-heures au total, balayées	Elo ancré au jeu parfait ; frontières de calcul	Représentation jamais variée ; aucun ancrage au jeu parfait n'existe pour Hive
Cette étude	Hive, jeu de base	Grille : 77 plans sur un cadre 32×32, CNN résiduel (1.44M) ; graphe : graphe de cellules à six relations typées par direction, passage de messages (1.47M) ; décodeur d'actions partagé	Même PUCT-MCTS pour les deux bras ; 128/32 simulations avec randomisation du plafond de simulations ; 400 en évaluation	Une machine ; 10 générations × 500 parties par exécution ; lectures à exemples égaux et à temps mural égal	5 graines indépendantes par bras ; population gelée de trois adversaires ; intervalles bootstrap sur les graines ; règle de rejet pré-enregistrée	aucune

3.7 Positionnement

Ce qui est reproduit. La boucle de type AlphaZero telle que publiée (réseau politique-valeur, recherche PUCT avec bruit à la racine, cibles de politique en distribution de visites, perte conjointe politique-valeur ; Silver et al., 2018), avec la randomisation du plafond de simulations de KataGo en auto-jeu (Wu, 2020). Le bras grille reproduit dans

l'esprit la famille d'encodages d'AZ-Hive : le treillis hexagonal plongé dans un cadre fixe et lu par un CNN résiduel (de Goede et al., 2022), ici un cadre 32×32 déplié par parcours en largeur, centré sur la boîte englobante des cellules occupées et portant 77 plans. L'évaluation suit Kampert et al. (2021) et AZ-Hive en mesurant contre des adversaires fixes et non appris. Le moteur de règles et la recherche classique qui servent les deux bras sont antérieurs à l'étude (construits en juillet 2026 comme projet de moteur et validés contre les deux moteurs de référence listés dans la webographie) ; le pipeline propre à l'étude, les deux encodeurs, le réseau de graphe, les adversaires de base et le protocole ont été construits à partir de septembre 2026.

Ce qui est modifié. L'axe de représentation gagne le bras graphe qu'aucun des travaux sur Hive ne possède : un graphe de cellules sur l'ensemble candidat de la position (chaque cellule occupée plus son anneau de voisines vides, de sorte que chaque destination pouvant être notée est un nœud), avec 56 caractéristiques de nœud portant la composition complète de la pile, six relations typées par direction réalisées comme matrices de poids propres à chaque relation, un vecteur global et un biais de pooling global. Les deux bras sont à capacité appariée à +1.5% près (1.44M contre 1.47M paramètres), bien plus serré que la comparaison 1.0M contre 2.2M de Rigaux et Kashima (2024). Et le protocole de preuve remplace les courbes Elo à exécution unique par la discipline à peu d'exécutions d'Agarwal et al. (2021) : cinq graines indépendantes par bras, des intervalles bootstrap sur les graines pour la différence graphe–grille, et une règle de rejet inscrite dans un protocole gelé le 10 septembre 2026 contre une population d'adversaires gelée le 9 septembre 2026, l'un et l'autre avant toute exécution d'entraînement.

Ce qui reste identique entre les bras. Tout, hormis l'encodeur d'état et le corps du réseau : le moteur, la recherche arborescente et ses budgets (128 ou 32 simulations par décision avec randomisation du plafond de simulations en auto-jeu, 400 en évaluation), l'espace d'actions sur les paires (pièce, destination) avec son masque de légalité, les cibles et la perte, la boucle d'entraînement, le pipeline de données, le calendrier des générations, le harnais d'évaluation, les ouvertures appariées pré-tirées, les trois adversaires (aléatoire légal, heuristique, et recherche à 6 400 simulations sans réseau) et la machine. Le budget est lu deux fois, comme le même nombre d'exemples d'entraînement et comme le même temps mural sur le même matériel, parce qu'une représentation meilleure par exemple mais plus lente par exemple peut perdre la seconde lecture ; les deux sont rapportées côte à côte.

Ce qui est revendiqué. Aucun travail antérieur n'exécute de comparaison grille-contre-graphe contrôlée et à budget apparié pour Hive avec les deux bras sous un même pipeline d'auto-jeu. La contribution à portée délimitée est donc la première comparaison grille-contre-graphe contrôlée et à graines multiples pour Hive dont le budget est apparié sous la lecture à exemples égaux comme sous la lecture à temps mural égal, les deux bras étant entraînés sous un même pipeline d'auto-jeu de type AlphaZero, dans un jeu sans cadre et à empilement dont les actions sont des paires (pièce, destination) et dont le graphe doit porter des destinations candidates vides. Elle n'est délibérément pas « la première comparaison grille-contre-graphe dans un jeu de plateau », ce que Keller et al. (2023) et Rigaux et Kashima (2024) excluent. Le périmètre comprend une variante (Hive de base), un réseau grille et un réseau relationnel à passage de messages à une capacité appariée, une machine, adversaires, ouvertures et protocole gelés, deux lectures budgétaires, cinq graines par bras. À l'intérieur de ce périmètre, la question est tranchée dans les chapitres de résultats ; en dehors, rien n'est affirmé. Le contraste avec le résultat positif aux

échecs et l'asymétrie longue-portée-contre-locale à Hex est repris dans le chapitre de discussion.

Partie II. Construction du système

4 Comment et quand le système et l'étude ont été construits

Le logiciel sous-jacent à cette étude a trois couches de provenance, de poids probant différent. Le moteur de règles, la recherche classique et le premier pipeline neuronal ont été construits en juillet 2026 dans le cadre d'un projet de moteur visant la force de jeu, non une comparaison contrôlée. Une première boucle d'auto-jeu de type AlphaZero a tourné sur ce moteur de juillet à août 2026 ; c'est une démonstration et rien de plus. Tout ce qui porte une preuve ici (corpus, population d'adversaires, protocole, les deux encodeurs et les deux réseaux, campagnes et analyses) a été construit et exécuté à partir du 9 septembre 2026 (Tableau 3, Fig. 2).

4.1 Le projet de moteur (juillet 2026)

Le projet de moteur s'est donné pour objectif de construire un programme de Hive couvrant les huit types de partie du protocole universel Hive (Universal Hive Protocol, UHP), c'est-à-dire le jeu de base et chaque combinaison des extensions Mosquito, Ladybug et Pillbug (moustique, coccinelle et cloporte), assez fort pour disputer des matchs automatisés contre les deux moteurs de référence publics, MzingaEngine et nokamute. Sa conception était échelonnée : d'abord un moteur alpha-bêta classique doté d'une évaluation construite à la main, comme partenaire d'entraînement et oracle de test ; puis une combinaison de type AlphaZero d'une recherche arborescente Monte-Carlo et d'un réseau de neurones entraîné par auto-jeu. Le cœur a été écrit en Rust, l'entraînement en Python et PyTorch, et l'inférence dans le moteur utilisait ONNX avec le fournisseur d'exécution CoreML à formes fixes. Le plan était dimensionné pour un seul Apple M4 Pro (14 cœurs, 24 Go).

Les premiers commits du dépôt et la note d'état du projet portent tous deux la date du 12 juillet 2026. La note consigne ce que cette première journée de construction a atteint : noyau de règles, entrées-sorties du protocole, moteur alpha-bêta et arène de matchs achevés ; comptes perft conformes aux tables de référence publiées jusqu'à la profondeur 7 pour les huit types de partie ; fuzzing différentiel contre MzingaEngine et nokamute passé sur 27k et 66k positions ; conformité 21/21 ; pipeline neuronal (cadre, encodage de la politique, enregistrements, génération de données, réseau de 1.44M paramètres, export ONNX) achevé ; vérification go/no-go de l'inférence passée à 2 246 évaluations par seconde au lot 128 sur CoreML ; recherche guidée par réseau entamée. La note consigne aussi des chiffres de force (100% contre MzingaEngine, 79.2% contre nokamute à 1 s par coup, +232 Elo) ; ce sont les chiffres propres au projet de moteur, et l'étude n'en fait aucun usage.

4.2 La boucle d'auto-jeu antérieure (juillet–août 2026)

Un amorçage supervisé à partir de parties alpha-bêta a tourné pendant la nuit du 12 au 13 juillet 2026, et la première génération de la boucle d'auto-jeu a démarré le 13 juillet 2026. Chaque génération jouait 2000 parties à 600 simulations complètes et 150

simulations économiques par décision sur 2 threads, en environ 150 000–157 000 s ; un candidat était soumis à une barrière de promotion de 60 parties à 2 s par coup contre le tenant, et promu au-dessus de 50%. Dix-neuf générations ont été menées à terme. Les promotions furent intermittentes (générations 6, 7, 11, 14 et 19 parmi celles consignées, scores de barrière d'environ 45% à 63%), les métriques d'entraînement ont fluctué plutôt que progressé (top-1 de la politique autour de 43–47%, exactitude de la valeur autour de 58–63%), et la génération 19 a été promue à 56.7%. La génération 20 a avorté le 14 août 2026 lorsque le disque s'est rempli ; la boucle n'a pas tourné depuis.

La boucle est classée comme démonstration, non comme preuve, pour des raisons fixées par écrit le 9 septembre 2026, avant tout travail d'étude : graine unique, architecture unique, aucun budget de calcul contrôlé, aucune population d'adversaires fixée à l'avance. Elle montre qu'un pipeline d'auto-jeu de bout en bout a été construit et a tourné ; elle ne montre rien sur la qualité des représentations ni sur l'efficacité de l'apprentissage. L'étude hérite du moteur et de ses preuves de validation, non des résultats de la boucle. L'auteur a arrêté la boucle définitivement le 9 septembre 2026, a exclu son point de contrôle de la génération 19 de la population d'adversaires gelée le même jour, et a supprimé l'espace de travail d'origine le 10 septembre 2026, laissant la copie de recherche comme seule copie des travaux antérieurs.

4.3 Le programme de recherche (à partir du 9 septembre 2026)

9 septembre 2026 : inventaire, validation, adversaires de base. Le programme s'est ouvert par un inventaire séparant les démonstrations fonctionnelles des comparaisons reproductibles, et par deux choix de cadrage : le jeu de base uniquement, et une partie atteignant le plafond de coups n'est jamais comptée comme nulle. Les suites de validation du moteur ont été réexécutées sur la machine d'étude, un Apple M1 Pro (10 cœurs, 16 Go) : perft jusqu'à la profondeur 6 pour les huit types de partie en 8.32 s, conformité 21/21, fuzzing différentiel sur 27 829 positions contre chaque moteur de référence, contre-vérification des plans sur 240 positions. Un corpus de 30 cas de positions critiques, annoté à la main à partir des règles de l'éditeur et consigné par commit avant toute exécution du moteur, a donné 29/30 à la première exécution, l'unique désaccord ayant été résolu contre le corpus, et 30/30 à la seconde. Des vérifications d'invariants sur parties aléatoires à graine fixée ont appliqué puis annulé 10.9M transitions sans une seule violation, et un profil de débit a mesuré les coûts que le protocole porterait plus tard. Les trois adversaires de base ont été construits, caractérisés sur 100 parties appariées par confrontation, et gelés par l'auteur le jour même, l'adversaire de recherche étant fixé à 6 400 simulations et le point de contrôle antérieur exclu ; l'auteur a reporté le gel du protocole jusqu'à ce qu'un pilote ait remesuré les budgets proposés avec un réseau à l'échelle de l'étude. Le défaut d'inférence CoreML découvert pendant le profilage a été corrigé le soir même.

9–10 septembre 2026 : pipeline, pilote, gels, encodeurs, lancement. Le pipeline propre à l'étude a été construit le 9 septembre (un format d'enregistrement avec une issue de troncature distincte, un entraînement masqué sur l'ensemble légal, sept vérifications automatisées pré-entraînement) ; son auto-jeu pilote a commencé à 20 h 44 ce soir-là, et l'unique génération du pilote (300 parties, entraînement et évaluation) s'est achevée le 10 septembre. Sur les mesures du pilote, l'auteur a gelé le protocole en version 1.0 le 10 septembre 2026, son hachage et son commit étant consignés. Le même jour,

l'encodeur grille a été réutilisé avec une vérification de bornes durcie, l'encodeur et le réseau graphe ont été écrits et testés par propriétés, les deux encodeurs ont été épinglés par des tests inter-langages sur fichiers de référence (golden) (240 et 160/160 positions), 250 ouvertures de quatre plis ont été générées à l'aveugle et gelées par l'auteur, et l'auteur a approuvé la campagne à sa pleine taille, lancée à 12 h 42.

10–19 septembre 2026 : campagne principale et analyse à trois graines. Six exécutions (deux bras × trois graines) se sont enchaînées en séquence et se sont achevées le 17 septembre 2026 à 22 h 31. Le 16 septembre, cinq des six exécutions étant terminées, le seuil à temps mural égal a été calculé à partir des seuls temps muraux du bras grille, comme pré-enregistré (18.77 h), avant qu'aucun nombre inter-bras n'existe, et le volume d'évaluation a été maintenu à 100 parties par confrontation après une vérification de précision. Les évaluations des points de contrôle au seuil ont tourné le 18 septembre (environ 11 h) ; ce jour-là, la règle selon laquelle chaque position d'échec sélectionnée doit être reproduite a mis au jour un défaut de perte d'enregistrements dans le lanceur de matchs, corrigé après qu'un audit eut montré qu'aucun fichier de campagne n'était touché. L'analyse à trois graines du 19 septembre 2026 a rejeté l'hypothèse sous la règle pré-enregistrée.

19 septembre – 9 octobre 2026 : ablations, extension, analyse finale. Les configurations d'ablation ont été préparées le 19 septembre, avec une substitution décidée ce jour-là ; l'auteur a approuvé la campagne d'ablations le 20 septembre 2026, les deux ablations à pleine parité de trois graines avec la comparaison principale. La divergence de l'ablation du typage des arêtes (sorties non finies dès la génération 0 pour 3/3 graines) a été diagnostiquée le 23 septembre ; les exécutions de l'ablation du pooling global se sont achevées les 24, 26 et 27 septembre et ont été analysées le 2 octobre. Le 26 septembre 2026, l'auteur a approuvé, avec des pré-engagements énoncés avant toute nouvelle exécution, une extension symétrique à cinq graines par bras et le supplément à deux composants à l'ablation divergée. Les graines 4 et 5 ont tourné du 27 septembre au 2 octobre, avec les évaluations au seuil le 6 octobre ; le supplément a tourné du 2 au 6 octobre. L'analyse finale à cinq graines du 9 octobre 2026 a confirmé le rejet ; le rapport a été assemblé en anglais et en français le même jour, puis développé à la demande de l'auteur sans qu'aucun nombre, aucune affirmation ni aucun artefact gelé ne soit modifié.

9–27 octobre 2026 : publication et seconde étude. Le 9 octobre 2026, l'auteur a approuvé la publication : le dépôt a été rendu public sous son étiquette avec le rapport dans les deux langues, l'article court et l'archive des enregistrements. Le même jour, l'étude complémentaire a été mise en file d'attente et le support qu'elle exige du pilote de campagne a été écrit : deux options du pilote de campagne, démarrage à chaud et fenêtre de rejeu, désactivées par défaut, avec un test de non-régression montrant que les lignes de commande par défaut reproduisent les manifestes enregistrés de la première étude sur quatre exécutions et quarante générations. Un pilote approuvé le 12 octobre 2026 a exécuté deux générations de 200 parties par bras sous le nouveau régime ainsi que la batterie de vérifications sur ses segments fenêtrés, ce qui a conduit à généraliser une vérification ; le 13 octobre 2026, l'auteur a gelé le protocole en version 2.0 (option A d'hyperparamètres, fenêtre de trois générations, règle de seuil) et approuvé la campagne, lancée à 15 h 02. Dix exécutions se sont enchaînées en séquence sans incident et se sont achevées le 26 octobre 2026 à 23 h 00 ; le seuil à temps égal de la seconde étude a été calculé à partir des cinq horloges grille le 25 octobre, avant qu'aucun score n'ait été lu ; les évaluations au seuil ont tourné pendant la nuit et l'analyse du 27 octobre 2026 n'a trouvé

aucun avantage du graphe sous l'une ou l'autre lecture. La présente révision du rapport a suivi le même jour.

4.4 Chronologie

Tableau 3 : Chronologie du projet de moteur et de l'étude, d'après l'historique des commits du dépôt, la note d'état du 12 juillet 2026, les journaux de la boucle, les entrées de journal datées et les enregistrements de décision portant les approbations de l'auteur ; la dernière colonne nomme l'artefact qui documente chaque jalon.

Date	Jalon	Preuve produite
12 juillet 2026	Projet de moteur : premiers commits ; noyau, protocole, alpha-bêta, arène, interface neuronale, recherche arborescente	Note d'état : perft jusqu'à la profondeur 7 (8 types), fuzz sur 27k/66k positions, conformité 21/21, inférence à 2 246 évaluations/s au lot 128
12–13 juillet 2026	Amorçage supervisé à partir de parties alpha-bêta	Journaux d'amorçage (démonstration seulement)
13 juillet – 14 août 2026	Boucle d'auto-jeu antérieure : 19 générations ; promotions aux générations 6, 7, 11, 14, 19 ; génération 20 avortée sur disque plein	Journaux de la boucle ; point de contrôle de la génération 19 (conservé, exclu)
9 septembre 2026	Ouverture du programme : inventaire ; suites de validation réexécutées ; corpus 30/30 ; vérifications d'invariants sur 10.9M transitions ; profil de débit ; correctif CoreML ; adversaires de base caractérisés et gelés	Inventaire ; entrées de validation ; enregistrement du gel de la population, avec hachages
9–10 septembre 2026	Pipeline ; pilote (300 parties) ; protocole gelé v1.0 ; encodeurs testés sur fichiers de référence (240, 160/160) ; 250 ouvertures gelées ; campagne lancée à 12 h 42	Entrée du pilote ; hachage du protocole ; entrées des encodeurs ; hachage des ouvertures ; journal de campagne
10–17 septembre 2026	Campagne principale : 6 exécutions en séquence, achevées le 17 septembre à 22 h 31	Journaux de temps mural par exécution, points de contrôle, enregistrements d'évaluation
16 septembre 2026	Seuil à temps mural égal issu des temps muraux du bras grille (18.77 h), avant toute comparaison inter-bras ; 100 parties par confrontation maintenues	Entrée de suivi avec la dérivation
18–19 septembre 2026	Évaluations au seuil ; figures ; défaut de perte d'enregistrements corrigé ; analyse à trois graines : hypothèse rejetée	Tables de résultats, contrastes entre bras, figures ; entrée du journal méthodologique
20 septembre 2026	Campagne d'ablations approuvée et lancée (deux ablations × 3 graines)	Diffs de configuration à un composant
23 septembre 2026	Ablation du typage des arêtes diagnostiquée : valeurs non finies dès la génération 0 pour 3/3 graines ; gardes ajoutées	Entrée de divergence
26 septembre 2026	Extension à cinq graines et supplément à deux composants approuvés avec pré-engagements	Enregistrements de décision citant les pré-engagements
27 septembre – 6 octobre 2026	Graines 4–5 (jusqu'au 2 octobre), évaluations au seuil (6 octobre) ; supplément (2–6 octobre) ; ablation du pooling analysée (2 octobre)	Journaux d'exécution ; entrées d'ablation
9 octobre 2026	Analyse finale à cinq graines (le rejet tient) ; supplément analysé ; rapport assemblé en anglais et en français, puis développé	Tables de résultats finales ; rapport
9 octobre 2026	Dépôt rendu public à l'étiquette v2.0-report avec le rapport, l'article court et l'archive des enregistrements ; seconde étude mise en file d'attente ; options du pilote de campagne pour le démarrage	Enregistrement de publication ; entrée du test du pilote de campagne

Date	Jalon	Preuve produite
	à chaud et la fenêtre de rejeu écrites et épinglées à la première étude par un test de non-régression	
12–13 octobre 2026	Pilote de la seconde étude (deux générations × 200 parties par bras ; batterie de vérifications sur segments fenêtrés) ; protocole v2 gelé ; campagne lancée à 15 h 02	Entrée du pilote ; hachage du protocole v2 ; journal de campagne
13–26 octobre 2026	Seconde campagne : dix exécutions à démarrage à chaud en séquence, achevées le 26 octobre à 23 h 00 ; seuil à temps égal de la seconde étude (17.63 h) calculé à partir des cinq horloges grille le 25 octobre, avant qu'aucun score n'ait été lu	Journaux d'horloge par exécution, manifestes, enregistrements d'évaluation ; entrée du seuil
26–27 octobre 2026	Évaluations au seuil (21 ensembles) ; analyse de la seconde étude : aucun avantage du graphe détecté ; rapport révisé	Tables de résultats et figures de la seconde étude ; entrée d'analyse

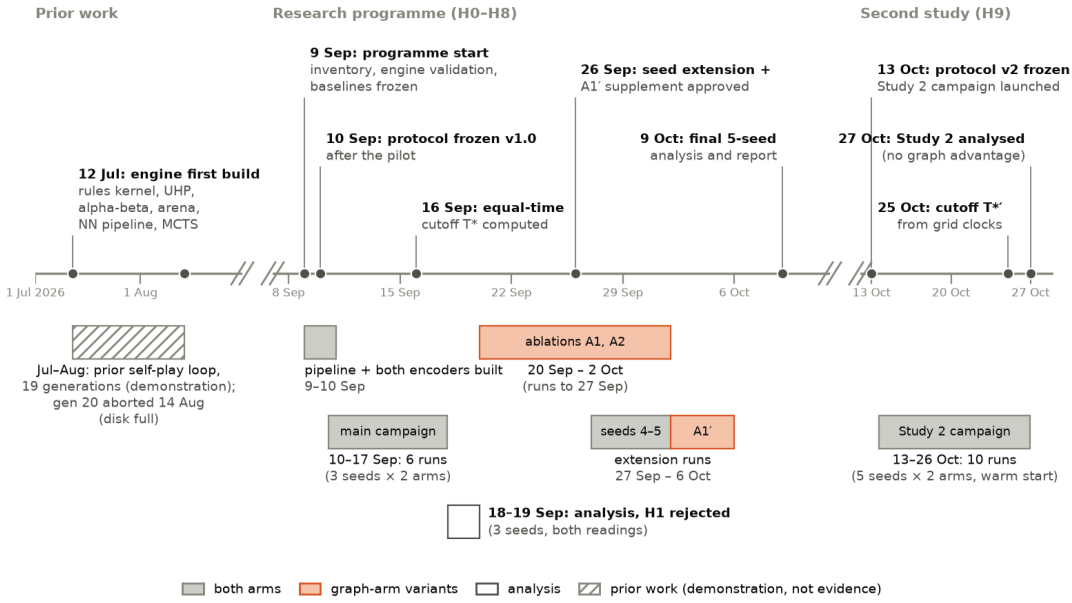


Fig. 2 : Chronologie du projet, juillet–octobre 2026 : projet de moteur (12 juillet), boucle d'auto-jeu antérieure classée comme démonstration (13 juillet – 14 août) et programme de recherche (9 septembre – 9 octobre) avec les gels (population le 9 septembre ; protocole et ouvertures le 10 septembre), la campagne principale (10–17 septembre), le seuil à temps mural égal (16 septembre), l'analyse à trois graines (19 septembre), les ablations (20 septembre – 6 octobre), l'extension des graines (27 septembre – 6 octobre) et l'analyse finale à cinq graines (9 octobre) ; seconde étude (13–27 octobre) : protocole v2 gelé et campagne lancée (13 octobre), dix exécutions à démarrage à chaud (13–26 octobre), seuil à temps mural égal issu des horloges grille (25 octobre), analyse (27 octobre).

5 Le moteur Hive : conception, implémentation et validation

Chaque bras, chaque adversaire et chaque évaluation de cette étude partagent le moteur. Une étude par auto-jeu s'entraîne sur des positions que le moteur génère et évalue lui-même, de sorte qu'un défaut de règles enseignerait aux deux bras le même jeu faux et invaliderait silencieusement la comparaison, au lieu de simplement ajouter du bruit. Le moteur a été construit en juillet 2026 comme projet indépendant (Chapitre 4) et validé selon quatre lignes de preuve indépendantes le 9 septembre 2026, avant tout travail sur les adversaires de base ou l'entraînement. Ce chapitre couvre ses objectifs, son architecture, sa représentation du plateau, sa génération de coups, sa validation, son débit, et les deux correctifs consignés avant l'entraînement.

5.1 Objectifs de conception

Le projet de moteur a fixé cinq objectifs. Le premier était la fidélité aux règles : les huit types de partie du protocole universel Hive (Universal Hive Protocol, UHP), avec pour critères d'acceptation l'accord exact avec les tables perft publiées du moteur de référence et l'accord, à chaque pli, des ensembles de coups légaux avec deux moteurs de référence indépendants. Le deuxième était l'interopérabilité du protocole sur l'entrée et la sortie standard, de sorte que MzingaEngine et nokamute puissent être pilotés comme sous-processus. Le troisième était une force échelonnée : d'abord un moteur alpha-bêta classique doté d'une évaluation construite à la main, comme partenaire d'entraînement et oracle de test, puis une recherche arborescente de type AlphaZero guidée par un réseau entraîné par auto-jeu (Silver et al., 2018). Le quatrième était la faisabilité sur une seule machine, c'est-à-dire un noyau Rust sans allocation, des exports ONNX à formes fixes pour le fournisseur d'exécution CoreML, et une vérification go/no-go de l'inférence avant tout auto-jeu. Le cinquième était une vérification intégrée dès le départ : tests unitaires par règle, perft rapide dans la suite standard et profond dans l'exécution nocturne, fuzzing différentiel inter-moteurs, tests par propriétés du hachage en make/unmake, et une suite de régression tactique.

Deux propriétés supplémentaires ont compté davantage pour l'étude que la force : le noyau de règles n'effectue aucune entrée-sortie et ne détient aucun état global, de sorte qu'un seul noyau arbitre les parties d'arène, génère l'auto-jeu et sert le protocole ; et chaque composant stochastique est à graine fixée, de sorte que toute partie enregistrée se rejoue de manière déterministe.

5.2 Architecture

Le moteur est un espace de travail Rust composé de petits crates aux dépendances à sens unique : le noyau de règles à la base, le protocole et l'évaluation au-dessus, les deux moteurs de recherche (backends) au-dessus de ceux-ci, et l'arène, les générateurs d'auto-jeu et le binaire livré au sommet (Tableau 4). Trois choix méritent un mot. La recherche classique ne pratique aucun élagage par coup nul (null move), parce que les états de

passe forcée de Hive rendent l'élagage fondé sur le tempo incorrect précisément dans les positions qui comptent, les courses aux reines ; sa quiescence ne couvre que les coups forçants de Hive, ceux qui atterrissent sur la reine ennemie ou à côté d'elle. La recherche guidée par réseau est générique sur un évaluateur à contrat de sortie unique, de sorte que l'évaluation construite à la main avec des a priori uniformes (l'adversaire sans réseau de l'étude), le réseau grille sur CoreML et le réseau graphe sur le fournisseur CPU passent tous par la même recherche arborescente. Le générateur d'auto-jeu implémente les économies de KataGo de Wu (2020) : randomisation du plafond de simulations, bruit de Dirichlet à la racine, température sur les premiers coups, et abandon avec une fraction d'audit. Le bruit à la racine reste désactivé dans tout autre backend.

Tableau 4 : Composants du moteur, leurs rôles et les propriétés sur lesquelles l'étude s'appuie. Un seul noyau de règles génère, arbitre et sert chaque partie de ce rapport ; les trois adversaires gelés et les deux bras de l'étude sont des backends du même binaire.

Composant	**Rôle**	**Propriétés sur lesquelles l'étude s'appuie**
Noyau de règles	État, génération de coups, make/unmake, hachage, notation, perft, canonisation	Aucune entrée-sortie, aucun état global ; les huit types de partie ; arbitre chaque partie
Serveur et client de protocole	UHP sur flux standard ; pilote de sous-processus pour les autres moteurs	Chaque bloc de réponse se termine par `ok` ; tout backend se branche derrière un seul trait
Évaluation construite à la main	Évaluation statique, convention negamax	Libertés de la reine dominantes, activité des pièces, tempo ; poids épinglés par hachage pour l'adversaire heuristique
Recherche classique	Recherche à variation principale, approfondissement itératif	Table de transposition partagée sans verrou (validée par XOR), SMP paresseux (lazy SMP), ordonnancement killer/history, réductions des coups tardifs, fenêtres d'aspiration ; aucun coup nul ; quiescence ciblant la reine
Interface neuronale	Cadre 32×32 de 77 plans ; index de politique (pièce, destination) à 28 673 entrées ; enregistrements d'entraînement (112 octets en one-hot, 176 octets avec distribution de visites, version de l'étude avec estampille du modèle, liste des indices légaux, issue de troncature)	Identique à l'octet près au décodeur Python (testé sur fichiers de référence)
Recherche guidée par réseau	Recherche arborescente PUCT par lots sur un évaluateur	Descente par rejeu, perte virtuelle ; trois évaluateurs, un contrat de sortie ; bruit désactivé sauf si l'auto-jeu l'active
Arène de matchs	Parties appariées, à couleurs échangées, à partir d'ouvertures aléatoires à graine fixée ou d'un fichier fixe (la paire i joue la ligne i)	Chaque réponse validée par le noyau ; plafond de 300 plis comme issue distincte, exclue du score et rapportée comme un taux
Générateurs d'auto-jeu	Données d'amorçage alpha-bêta ; auto-jeu par recherche arborescente	Randomisation du plafond de simulations, bruit à la racine, température, abandon avec audit ; à graine fixée
Binaire livré	Frontal de protocole sur tous les backends	Alpha-bêta par défaut ; recherche arborescente avec heuristique, réseau grille ou réseau graphe ; aléatoire légal ; options de graine et de simulations

5.3 Représentation du plateau et hachage

Hive n'a pas de plateau : les pièces définissent la surface de jeu, et une ruche dérive sur le plan à mesure qu'elle grandit. Le moteur utilise néanmoins une grille fixe de 64×64 cellules en coordonnées axiales, refermée en tore, sur un argument de comptage : une ruche contient au plus 28 pièces, donc elle s'étend sur au plus 28 cellules le long de tout axe, et sur un tore 64×64 toute requête locale (adjacence, glissements, sauts) est indiscernable de la même requête sur un plan infini. Chaque cellule stocke sa pièce du dessus, les piles vivent dans une table annexe, et la hauteur maximale d'une pile est 7 (une pièce au sol sous 4 scarabées et 2 moustiques).

Parce que la ruche ne quitte jamais le tore, les coordonnées n'ont jamais besoin d'être renormalisées au cours d'une partie, ce qui garde le hachage de position entièrement incrémental : make et unmake font entrer et sortir par XOR les clés des pièces déplacées, et aucun recentrage ne force jamais un recalcul ni n'invalide les entrées de la table de transposition. La symétrie par translation est donc sans objet à l'intérieur d'un arbre de recherche ; la canonisation sous translation et sous les 12 symétries hexagonales est isolée à la frontière avec le cadre du réseau et le livre d'ouvertures. Le hachage de Zobrist est calculé par une fonction de mélange plutôt que par des tables de consultation (une table complète pèserait environ 6 Mo ; mélanger une clé compactée (pièce, cellule, niveau) coûte environ 2 ns sans aucune mémoire) et intègre le camp au trait, la dernière pièce déplacée et une composante de phase de début de partie. La détection des répétitions abandonne la composante de phase, de sorte que le même arrangement atteint à des plis différents est comparé égal, mais conserve le camp au trait et la dernière pièce déplacée, c'est-à-dire les droits d'étourdissement, tout comme les clés de répétition aux échecs conservent les droits de prise en passant ; une troisième occurrence est une nulle, sauf si la partie est déjà décidée par encerclement.

5.4 Génération de coups et cas limites des règles

La génération de coups suit les conventions du moteur de référence, parce que ce sont elles que comptent ses tables perft publiées : la reine ne peut être placée au premier tour d'aucun des deux joueurs (la règle d'ouverture de tournoi), pour des insectes identiques en réserve seul l'ordinal le plus bas est plaçable, un joueur sans placement ni déplacement reçoit l'unique coup `pass`, et une partie terminée ne génère aucun coup.

La légalité des glissements et des portes est évaluée sur une vue *soulevée* du plateau, l'occupation avec la pièce mobile retirée de sa cellule d'origine. Sur cette vue, la **règle de liberté de mouvement** pour un pas de glissement au niveau du sol s'énonce ainsi : la destination doit être vide, et des deux cellules adjacentes à la fois à l'origine et à la destination, exactement une doit être occupée ; avec zéro, la pièce se détacherait en transit, et avec deux, la porte serait trop étroite pour passer. La **porte de hauteur** pour les coups d'escalade (pas du scarabée et de la coccinelle, les deux segments d'un lancer du cloporte (Pillbug)) bloque un pas si et seulement si les deux voisines communes sont strictement plus hautes à la fois que le niveau au-dessus duquel la pièce mobile part et que la pile de destination ; la porte du scarabée au-dessus du sol de la FAQ de tournoi découle de cette règle sans cas particulier. La **règle One-Hive** est un calcul de points d'articulation : une pièce au niveau du sol peut être soulevée si et seulement si sa cellule n'est pas un sommet de coupe (cut vertex) du graphe des cellules occupées, déterminé par un parcours en profondeur avec lowlink une fois par génération de coups ; les sommets des piles en sont

exemptés, puisque la cellule conserve son nœud, et un Pillbug cloué peut encore lancer, puisqu'il ne se déplace pas lui-même.

La **règle d'étourdissement (stun)** de l'extension Pillbug, selon laquelle une pièce déplacée au tour précédent ne peut ni se déplacer ni être déplacée, ne requiert qu'un seul champ : la pièce physiquement déplacée ou placée au pli précédent. Une pièce est étourdie si et seulement si elle est cette pièce et appartient au camp au trait ; un Pillbug ne peut pas lancer cette pièce ; l'étourdissement expire après un tour de l'adversaire ; le placement renseigne aussi le champ. Le **moustique** copie chaque type d'insecte adjacent et est un pur scarabée au sommet de la ruche ; la **coccinelle** se déplace d'exactement deux pas sur le dessus et d'un pas vers le bas. Parce que la copie multiple du moustique, les chemins multiples de la coccinelle et la marche-contre-lancer peuvent dériver le même coup plus d'une fois, la liste générée est triée et dédoublonnée ; si elle est alors vide, `pass` est généré.

5.4.1 Pourquoi une paire (pièce, destination) identifie un coup

Les espaces de politique des deux bras et la déduplication ci-dessus reposent sur l'affirmation que (pièce, cellule de destination) détermine l'état successeur. Un placement fait passer la pièce de la réserve à une cellule vide, donc la paire fixe le résultat. Un déplacement part de l'emplacement courant de la pièce, que l'état détermine de manière unique, et se termine au sommet de la pile de destination, dont l'état détermine aussi la hauteur ; la paire fixe donc le plateau. La seule façon pour deux actions distinctes de partager une paire est le lancer par un Pillbug d'une pièce vers une cellule qu'elle aurait aussi pu atteindre en marchant. Les deux produisent le même plateau, de sorte que les successeurs ne pourraient différer que par la trace de la pièce déplacée en dernier, et le moteur ne stocke que *quelle* pièce s'est déplacée, non comment. Cela suffit : au début du tour d'un joueur, si la dernière pièce déplacée est l'une des siennes, l'adversaire ne peut que l'avoir lancée, donc elle est immobilisée pour ce tour ; si c'est une pièce de l'adversaire, le Pillbug du joueur ne peut pas la lancer dans un cas comme dans l'autre, et rien d'autre ne dépend de la distinction. La collision marche–lancer produit donc des états identiques, et une tête de politique indexée par (pièce, destination) plus la passe ne perd rien.

5.5 Campagne de validation (septembre 2026)

Toutes les preuves de validation ont été rétablies le 9 septembre 2026 sur la machine d'étude, un Apple M1 Pro (10 cœurs, 16 Go), avec les moteurs de référence fraîchement récupérés (MzingaEngine v0.16.0, l'implémentation de référence du protocole, et nokamute 1.0.3 compilé depuis les sources), avant tout travail sur les adversaires de base ou l'entraînement. Quatre lignes de preuve bornent quatre modes de défaillance différents ; une cinquième vérification épingle le chemin de données au code d'entraînement.

5.5.1 Perft contre les tables publiées

Le perft compte les chemins de coups légaux jusqu'à une profondeur donnée sous les conventions du moteur de référence, de sorte que ses tables publiées servent de vérité terrain ; il est exhaustif à sa profondeur, et toute divergence de la génération de coups (un coup manquant, un coup en trop, une règle d'empilement erronée) décale un compte. La suite standard vérifie la profondeur ≤5 pour les huit types de partie à chaque build, une

exécution dédiée vérifie la profondeur 6, et l'exécution nocturne vérifie la profondeur 7. Le 9 septembre 2026, la suite standard a passé et l'exécution à la profondeur 6 s'est achevée en 8.32 s avec chaque compte conforme ; la profondeur 7 n'a pas été répétée ce jour-là, de sorte que l'étude revendique la profondeur ≤6, la profondeur 7 ayant été conforme pour la dernière fois en juillet 2026 (Tableau 5, Tableau 6).

Tableau 5 : Comptes de nœuds perft pour le jeu de base, profondeurs 1–6, tels que publiés pour le moteur de référence et reproduits exactement par le noyau de règles le 9 septembre 2026 (pas de reine au premier tour d'un joueur ; insectes identiques en réserve comptés une fois ; la passe forcée compte comme un coup ; une partie terminée n'en génère aucun).

Profondeur	Nœuds
1	4
2	96
3	1 440
4	21 600
5	516 240
6	12 219 480

Tableau 6 : Comptes de nœuds perft aux profondeurs 4–6 pour les huit types de partie (M = Mosquito, L = Ladybug, P = Pillbug), tels que publiés pour le moteur de référence et reproduits par le noyau de règles ; les variantes Mosquito et Ladybug partagent leurs comptes jusqu'à la profondeur 5 et ne se séparent qu'à la profondeur 6, d'où l'exécution à la profondeur 6.

Type de partie	Profondeur 4	Profondeur 5	Profondeur 6
Base	21 600	516 240	12 219 480
Base+M	45 414	1 252 800	34 233 432
Base+L	45 414	1 252 800	34 233 672
Base+P	45 414	1 255 932	34 395 984
Base+ML	86 400	2 725 920	85 201 200
Base+MP	86 400	2 730 888	85 492 248
Base+LP	86 400	2 730 240	85 457 136
Base+MLP	151 686	5 427 108	192 353 904

5.5.2 Accord avec les moteurs de référence

Le harnais de conformité au protocole livré avec nokamute a passé 21/21. De manière plus exigeante, le fuzzing différentiel joue des parties aléatoires à graine fixée en vérifiant, après chaque pli, l'*égalité ensembliste* des ensembles de coups légaux renvoyés par le moteur et par une référence. À la graine 20260909 avec 25 parties par type de partie, les deux références ont été d'accord sur chacune des 27 829 positions (le compte identique est attendu : même graine, mêmes parties) ; l'exécution nocturne répète cela à 200 parties par type contre nokamute et 100 contre MzingaEngine. L'accord avec deux bases de code indépendantes borne une mauvaise lecture partagée avec le moteur, non une mauvaise lecture partagée par toute la communauté des implémentations, ce que traite la ligne suivante.

5.5.3 Le corpus critique annoté à la main

Trente positions critiques ont été annotées à la main à partir des règles de l'éditeur : la feuille de règles du jeu de base citée par page, la feuille Pillbug, et la FAQ des règles des World Hive Tournaments pour les portes au-dessus du sol et les clarifications sur

l'étourdissement. La règle d'ouverture de tournoi a été déclarée comme convention de l'étude. Chaque attendu a été écrit à partir du texte des règles, jamais à partir du moteur ni contre lui, et le corpus a été consigné par commit avant la première exécution. Chaque cas est une séquence de coups au format du protocole depuis le plateau vide, donc atteignable et vérifiable indépendamment, avec un attendu de l'un de sept types : l'ensemble exact des coups d'une pièce focale (éventuellement vide), un coup unique légal ou illégal, chaque coup étant un placement d'une pièce nommée, aucun coup de déplacement, passe forcée, ou un état terminal. Le lanceur pilote le moteur via le protocole, résout les chaînes de coups en cellules avec un analyseur purement géométrique qui ne connaît aucune règle du jeu, et compare les ensembles (pièce, destination). Tableau 7 en liste la couverture ; le corpus complet, avec chaque justification, figure dans Annexe A.

Tableau 7 : Couverture du corpus critique de 30 cas annoté à la main, par domaine de règles, avec les identifiants de cas tels qu'utilisés dans l'annexe du corpus. Les attendus ont été écrits à partir des règles de l'éditeur et consignés par commit avant toute exécution du moteur ; trois cas Pillbug gardent la logique d'étourdissement du noyau partagé et se situent hors du périmètre de jeu de base de l'étude.

Domaine de règles	**Cas**
Règles d'ouverture et de placement	C001–C005
Glissement, liberté de mouvement, portes	C006–C012
Scarabée et empilement, porte du scarabée au-dessus du sol, couleur de la pile	C013–C018, C028
One-Hive, anneaux, sauterelle en point d'articulation	C019–C020, C027
États terminaux : victoire, nulle par encerclement simultané	C021–C022
Passe forcée	C023
Capacité du Pillbug, étourdissement, gardes du dernier-déplacé (gardes du noyau, Base+P)	C024–C026
Compte de pas de l'araignée ; énumérations des coups de la reine	C008, C029 ; C006, C011, C030

La première exécution a renvoyé 29 succès et une erreur de mise en place. Le cas litigieux était la nulle par encerclement simultané : sa mise en place déplaçait la reine noire d'un pas avec les deux cellules flanquantes vides, de sorte que la ruche aurait été « laissée disjointe pendant que la pièce est en transit », ce qui est précisément l'exemple de coup illégal que la feuille de règles donne pour la règle One-Hive ; le rejet par le moteur était le comportement prescrit par la règle. Le désaccord a été résolu *contre le corpus* : la mise en place a été réacheminée par une cellule intermédiaire de sorte que chaque pas conserve une cellule flanquante occupée, l'issue attendue (une nulle) a été laissée inchangée, et la correction a été documentée dans le fichier du cas. La seconde exécution a renvoyé 30/30. La trace de l'erreur du corpus est conservée délibérément : des attendus consignés avant l'exécution tranchent dans les deux sens. Deux limites sont déclarées : aucune relecture externe par un lecteur connaisseur de Hive pour l'instant, et 30 cas se situent au bas de la fourchette de 30–50 que le plan de recherche proposait ; les cas les plus forts sont les énumérations (13 destinations dans un cas de glissement, 6 dans un cas de scarabée).

5.5.4 La suite tactique

Un second ensemble annoté à la main vérifie la *recherche* plutôt que les règles. Il comprend cinq positions à l'issue favorable connue : une victoire en un coup par une pièce qui marche et par une pièce qui saute, chacune pour les deux couleurs, et une position où le joueur au trait doit éviter d'achever l'encerclement de sa propre reine. Puisque plusieurs

pièces peuvent souvent atteindre la cellule gagnante, les attendus sont fondés sur la destination : le coup du chercheur doit, ou ne doit pas, atterrir sur une cellule nommée. La recherche arborescente sans réseau a résolu 5/5 à 400, 1 600 et 6 400 simulations. Les signes de la valeur sous alternance des joueurs sont couverts par des tests automatisés (l'évaluation change de signe quand le camp au trait bascule ; les scores de mat en un coup et les valeurs à la racine sont positifs pour le joueur au trait, pour les deux couleurs). Une bévue de mise en place, une erreur d'ordre des pièces dans un cas, a été attrapée dès la première exécution parce que les cas avaient été consignés par commit au préalable ; le moteur avait raison, la mise en place a donc été corrigée et l'incident consigné. La même limite de relecture externe s'applique.

5.5.5 Vérifications d'invariants aléatoires

Des parties aléatoires à graine fixée, sur l'ensemble des huit types de partie, vérifient quatre invariants après chaque transition : chaque coup généré est accepté par le chemin d'application et l'annulation restaure exactement la chaîne de partie ; la ruche reste connexe ; la chaîne de partie du protocole fait l'aller-retour vers la même position, le même résultat et le même ensemble trié de coups légaux ; et la passe est jouée exactement quand la liste de coups est vide. Une violation arrêterait l'exécution avec le type de partie, la graine, la partie, le pli et la chaîne de partie, à archiver comme cas de régression ; aucune n'est survenue (Tableau 8). L'exécution par défaut figure dans la suite standard à chaque build, l'exécution approfondie dans le script nocturne.

Tableau 8 : Vérifications d'invariants sur parties aléatoires à graine fixée du 9 septembre 2026, sur les huit types de partie (exactitude application/annulation, connexité de la ruche, aller-retour de la chaîne de partie, passe exactement quand aucun coup n'existe). Déterministes étant donné les graines ; durées sur la machine d'étude.

Exécution	Parties	Plis parcourus	Coups générés appliqués et annulés	Violations	Durée
Par défaut (chaque build)	4 par type × 8 types, ≤150 plis	4 029	253 936	0	2.7 s
Approfondie (nocturne)	3 tours × 20 par type × 8 types, ≤400 plis	161 546	10 665 686	0	214 s

5.5.6 Vérification inter-langages du chemin de données et portée

L'encodeur de plans du bras grille en Rust et le décodeur du code d'entraînement Python doivent être identiques à l'octet près ; une contre-vérification sur fichiers de référence (golden files) portant sur 240 positions a confirmé l'accord exact le 9 septembre 2026 et s'exécute chaque nuit. L'encodeur graphe construit le 10 septembre a reçu le même traitement, 160 positions sur 160, sur l'ensemble des huit types de partie, concordant exactement dès la première exécution.

Ces lignes bornent des modes de défaillance différents (exhaustivité à profondeur donnée, accord avec les références, fidélité au texte des règles, correction de la recherche, stabilité des invariants), mais aucune ne prouve la perfection sur parties complètes, et la couverture de code n'est délibérément pas revendiquée comme preuve de correction. La règle d'arrêt selon laquelle un moteur incorrect suspend tout le travail côté entraînement

n'a pas été déclenchée ; le travail sur les adversaires de base et le pipeline s'est poursuivi le même jour.

5.6 Profil de débit

Avant que tout budget de simulations ou plafond de coups ne soit fixé, les coûts du moteur ont été mesurés sur la machine d'étude par de courtes exécutions des binaires existants, le point de contrôle de la génération 19 de la boucle antérieure ne servant que de charge d'inférence réaliste (Tableau 9).

Tableau 9 : Profil de débit mesuré le 9 septembre 2026 sur la machine d'étude (Apple M1 Pro, 10 cœurs, 16 Go), builds en mode release, graine de profilage 20260909, graines de longueur de partie 1001–1030 (les longueurs incluent 6 plis d'ouverture aléatoires) ; inférence réseau mesurée avec le point de contrôle de la génération 19 de la boucle antérieure (entrée 77×32×32) comme charge.

Mesure	**Valeur**
Perft(5), jeu de base	516 240 nœuds en 1.36 ms (≈380 MN/s)
Perft(6), jeu de base	12.2 M nœuds en 26.5 ms (≈461 MN/s)
Alpha-bêta, position d'ouverture, profondeur 10	7.20 Mnps
Alpha-bêta, position de milieu de partie, profondeur 8	4.67 Mnps
Décision alpha-bêta à la profondeur 4	≈40 ms, mono-thread
Parties complètes à la profondeur 4 (6 threads, 100 parties)	22 s de temps mural, 4.6 parties/s, 4 397 positions enregistrées
Longueur des parties à la profondeur 4 (30 parties à graine fixée)	moyenne 55.0, médiane 39, 90e centile 80, maximum 202 plis ; aucune n'a atteint le plafond de 300 plis
Inférence réseau, paquet Python ONNX, fournisseur CPU, lot 1	23.5 ms par évaluation (43 évaluations/s)
Inférence réseau, paquet Python ONNX, fournisseur CoreML, lot 1	2.62 ms par évaluation (382 évaluations/s)
Inférence réseau, liaisons Rust, fournisseur CoreML	plantage (corrigé ci-dessous)
Aller-retour du protocole depuis Python	21.7 µs par `validmoves` (46 051 requêtes/s) ; 21.6 µs par paire play–undo

Trois conclusions en ont découlé, chacune une proposition mesurée, confirmée ensuite par le pilote et gelée dans le protocole. La génération des coups légaux est très loin d'être un goulot d'étranglement ; l'évaluation par le réseau domine chaque décision, et le chemin CoreML décide de la faisabilité : à 2.62 ms par évaluation, une décision à 128 simulations coûte environ 0.34 s et une décision à 64 simulations environ 0.17 s sur un thread, tandis que le chemin CPU est environ 9× plus lent. Les longueurs de partie observées ont étayé un plafond de 300 plis, au-dessus du maximum observé de 202 mais borné, avec la troncature comme issue à part entière. Enfin, un aller-retour du protocole à 21.7 µs se situe trois à quatre ordres de grandeur sous tout coût par décision ; comme la génération d'auto-jeu vit entièrement en Rust et échange ses données avec Python par des fichiers shards binaires, aucune liaison en processus n'a été construite.

5.7 Deux correctifs côté moteur consignés avant l'entraînement

Le profilage a fait apparaître deux défauts, chacun consigné d'abord comme constat puis corrigé ensuite dans le code propre de l'étude, plutôt que rapiécé silencieusement.

Fournisseur d'exécution CoreML. Le chemin d'inférence Rust paniquait sur la machine d'étude avec le message « Unable to compute the prediction using a neural network model », alors que le paquet Python ONNX exécutait le même modèle sur CoreML à 2.62 ms par évaluation (55 nœuds sur 64 sur CoreML, 5 partitions). Le message nomme la cause : les liaisons Rust (version 2.0.0-rc.12) utilisent par défaut le format historique *NeuralNetwork* de CoreML, alors que le paquet Python utilise *MLProgram*. Demander MLProgram, un changement d'une seule option, a rétabli l'inférence CoreML, vérifiée le 9 septembre 2026 avec le générateur d'auto-jeu : à 600 simulations complètes et 150 économiques, 2 parties sur 2 threads ont pris 44 s (≈44 thread-secondes par partie, contre ≈320 sur le repli CPU et ≈150 sur le chemin fonctionnel de la boucle antérieure) ; au budget final de l'étude de 128/32, 8 parties sur 4 threads ont pris 24 s, soit environ 3 s de temps mural et 12 thread-secondes par partie, ce qui situait une génération de 2000 parties aux alentours de 1.7 h. La suite de tests complète, la suite tactique (5/5) et le corpus critique (30/30) étaient au vert après le changement. Ces sondages sont des ancrages de faisabilité ; le pilote a remesuré avec les réseaux propres de l'étude.

La troncature n'est pas une nulle. Les deux générateurs des travaux antérieurs plafonnaient les parties à 300 plis et convertissaient le plafond en nulle, et les déroulements internes de la recherche arborescente plafonnaient à 120 plis. Compter une partie tronquée comme une nulle laisserait le plafond fabriquer ou effacer un avantage sans que cela soit détecté ; la convention de l'étude, fixée le 9 septembre 2026 avant l'écriture de tout code de mesure, fait donc de la troncature une quatrième catégorie d'issue partout : exclue du score principal, rapportée comme un taux séparé, comptée 0.5 dans une vérification de sensibilité. Le code des travaux antérieurs a été laissé en l'état, comme constat consigné ; le format d'enregistrement de l'étude porte une valeur de troncature distincte dans l'octet d'issue, l'arène exclut les parties tronquées du score et rapporte le taux, et l'entraînement exclut les enregistrements tronqués de la perte de valeur. Le tout a été livré avec le format d'enregistrement de l'étude les 9–10 septembre 2026, avant le pilote.

Le moteur validé, profilé et corrigé sur ces deux points, le pipeline propre à l'étude, les adversaires et les deux encodeurs ont été construits par-dessus ; les chapitres suivants les décrivent.

6 Le pipeline d'apprentissage par auto-jeu

Les deux bras de la comparaison sont entraînés et évalués par un seul pipeline, construit en septembre 2026 au-dessus du moteur de Chapitre 5. Il est délibérément ordinaire, une boucle de style AlphaZero au sens de Silver et al. (2018) avec les dispositifs d'économie de calcul de Wu (2020), puisque sa seule exigence est d'être identique pour les deux bras. La recherche qui génère les parties, l'enregistrement qui les stocke, la perte, le chemin d'export, l'arène d'évaluation et les vérifications automatisées sont tous partagés ; seuls l'encodeur d'état et le corps du réseau (Chapitre 7) diffèrent entre le bras grille et le bras graphe. Fig. 3 en donne la vue d'ensemble ; les sections ci-dessous décrivent chaque étape, puis le pilote du 10 septembre 2026 qui a validé la boucle de bout en bout, et l'unique défaut de harnais qu'il a détecté.

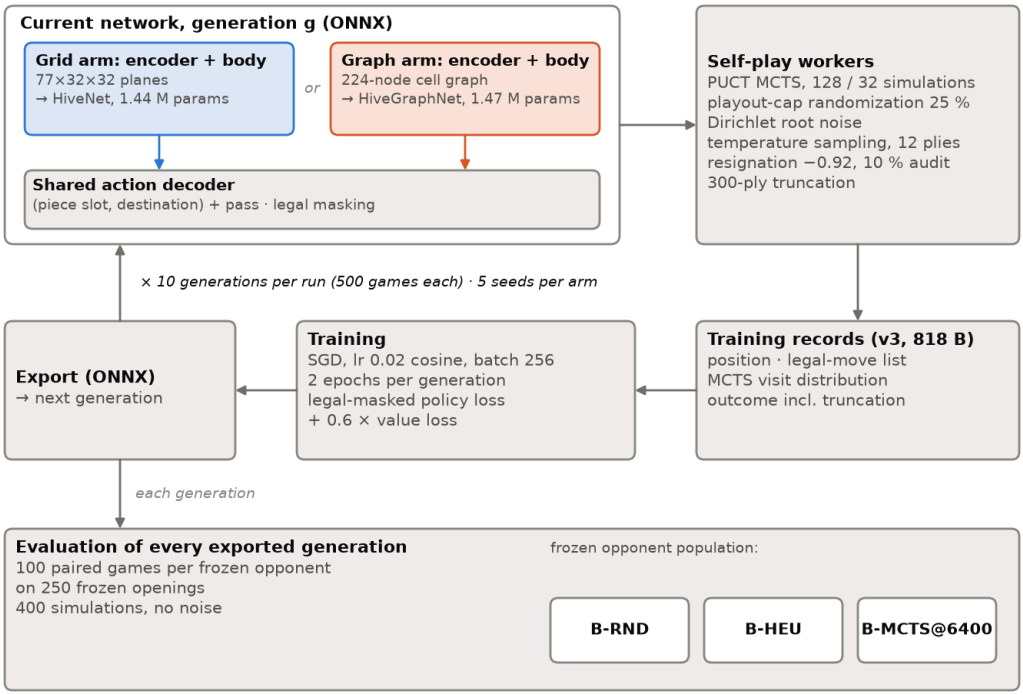


Fig. 3 : La boucle de générations partagée par les deux bras. L'auto-jeu avec recherche arborescente Monte-Carlo produit des enregistrements d'entraînement ; les enregistrements entraînent un réseau ; le réseau est exporté en ONNX et pilote l'auto-jeu de la génération suivante ; à des générations fixées, le réseau exporté est évalué contre la population d'adversaires gelée (B-RND, B-HEU, B-MCTS) sur des ouvertures gelées. Le bras grille et le bras graphe ne diffèrent que par l'encodeur d'état et le corps du réseau à l'intérieur des boîtes marquées « réseau ».

6.1 La boucle de générations

Une exécution d'entraînement est identifiée par un bras et une graine. Le réseau de la génération 0 est une initialisation aléatoire à graine fixée, exportée en ONNX avant qu'aucune partie ne soit jouée ; la graine de base de l'exécution s est $100,000 \times s$, et la génération g utilise la graine de base $+ g$ pour l'auto-jeu comme pour l'entraînement, de sorte que deux exécutions ne partagent jamais un flux aléatoire. Dans la configuration

sous laquelle toutes les exécutions de l'étude se sont déroulées, une exécution comprend 10 générations de 500 parties d'auto-jeu chacune, avec quatre threads de travail, sur une seule machine.

Chaque génération se déroule comme dans le pseudo-code ci-dessous. Le réseau courant joue contre lui-même les parties de la génération ; les positions recherchées au budget complet de simulations deviennent des enregistrements, estampillés du numéro de génération et d'un hachage du réseau générateur. Un réseau est entraîné sur ces enregistrements, exporté, et devient le réseau courant ; un point de contrôle est conservé à chaque génération afin que la lecture à temps mural égal (Chapitre 9) puisse ensuite sélectionner le dernier point de contrôle achevé avant le seuil à temps mural égal. Les générations sont numérotées de 1 à 10 dans tout ce rapport ; les journaux d'exécution et le pseudo-code ci-dessous les comptent à partir de 0. Après la cinquième et la huitième génération, et après la dixième et dernière, le réseau exporté est évalué contre la population d'adversaires gelée. Le lanceur ne transmet à l'entraîneur ni point de contrôle de démarrage à chaud ni point de contrôle de reprise : le réseau de la génération g est une initialisation fraîche à graine fixée, entraînée pendant deux époques sur les seuls enregistrements de la génération g, lesquels ont été produits par le réseau de la génération $g-1$ (par l'initialisation aléatoire lorsque $g=0$). Il n'existe aucune fenêtre de rejeu entre générations.

```
exécuter(bras, graine s):
    θ ← initialisation aléatoire à graine fixée (graine de base 100 000 × s),
exportée en ONNX
    pour g de 0 … 9 :
        D_g ← AUTO-JEU(θ, 500 parties ; graine base + g)
                  128 / 32 simulations, 25 % des décisions à 128 (enregistrées)
                  bruit de Dirichlet à la racine ε = 0.25 ; échantillonnage
proportionnel aux visites pendant 12 plis
                  abandon sous −0.92 (10 % des parties n'abandonnent jamais) ;
troncature à 300 plis
        θ ← ENTRAÎNER(initialisation fraîche à graine fixée, D_g ; graine base +
g)
                  2 époques, lot 256, SGD lr 0.02 (cosinus), momentum 0.9, wd 1e-4
                  perte = entropie croisée de politique masquée + 0.6 × entropie
croisée de valeur
                  (enregistrements tronqués exclus du terme de valeur)
        EXPORTER(θ) → ONNX, formes statiques ; point de contrôle conservé ; temps
mural de g journalisé
        si g ∈ {4, 7} ou g = 9 :
            ÉVALUER(θ) contre B-RND, B-HEU, B-MCTS sur les ouvertures gelées
                  400 simulations, sans bruit, argmax ; 20 parties/adversaire (g ∈
{4, 7}), 100 (g = 9)
```

La batterie de sept vérifications de Chapitre 6.7 est une commande distincte exécutée contre des segments (shards) réels. Le lanceur du pilote l'exécutait après l'auto-jeu de chaque génération ; le lanceur de campagne qui a exécuté la matrice gelée ne l'a pas relancée à chaque génération, s'appuyant sur le fait que la batterie avait réussi sur des segments réels et sur les tests côté moteur qui s'exécutent à chaque build.

6.2 Travailleurs d'auto-jeu et réglages de recherche

L'auto-jeu s'exécute entièrement dans le moteur Rust ; le côté Python n'intervient jamais dans une boucle par coup et reçoit ses données sous forme de fichiers de segments binaires. La recherche est une recherche arborescente Monte-Carlo PUCT avec une constante d'exploration $c = 1.4$, une évaluation des feuilles par lots et une remontée exacte des valeurs terminales. Quatre dispositifs, appliqués identiquement aux deux bras, façonnent les parties en données d'entraînement.

Randomisation du plafond de simulations (playout-cap randomization) (Wu, 2020). Chaque décision est recherchée au budget complet de 128 simulations avec une probabilité 0.25 et au budget économique de 32 simulations sinon ; seules les décisions au budget complet sont enregistrées. Le dispositif échange la qualité de la cible de politique sur un quart des coups contre beaucoup plus de parties par heure, ce dont la tête de valeur a besoin. La paire 128/32 n'a pas été reprise du plan de recherche : elle a été mesurée sur la machine d'étude après la restauration du fournisseur d'exécution CoreML le 9 septembre 2026 et confirmée avec un réseau à l'échelle de l'étude lors du pilote.

Exploration. Un bruit de Dirichlet avec $\varepsilon = 0.25$ est mélangé à l'a priori de la racine, et pendant les 12 premiers plis le coup est échantillonné proportionnellement aux comptes de visites de la racine ; à partir du pli 12, le coup le plus visité est joué. Le chemin d'évaluation porte $\varepsilon = 0$ comme valeur par défaut de son code et aucun échantillonnage, une séparation imposée par test (vérification 6 ci-dessous).

Abandon avec audit. Un camp abandonne lorsque la valeur de la racine, de son point de vue, tombe sous -0.92 ; dans 10 % des parties, tirées au sort au début de la partie, l'abandon est désactivé afin que le seuil puisse être audité contre des issues jouées jusqu'au bout.

Troncature. Une partie qui atteint 300 demi-coups sans résultat est tronquée. La troncature est une issue distincte (la quatrième valeur de l'octet d'issue) et n'est jamais enregistrée comme une nulle ; le plafond se situe au-delà de la plus longue partie guidée par la recherche observée lors du profilage du moteur (202 demi-coups). Le jeu de la génération 0 à partir d'une initialisation aléatoire tronque souvent (56.7 % des parties dans le pilote), ce que la catégorie séparée absorbe par conception et que la procédure de sensibilité au plafond du protocole existe pour sonder.

6.3 L'enregistrement d'entraînement

Chaque position enregistrée est un enregistrement de longueur fixe de 818 octets (version 3 du format), écrit par le générateur et lu identiquement par les chargeurs de données des deux bras. Il stocke la position en coordonnées absolues des pièces : 28 pièces × (x, y, niveau), avec une sentinelle pour les pièces en main. À côté de la position, il stocke le camp au trait, la dernière pièce déplacée (pour la règle d'étourdissement), le pli, les bits de type de partie, les libertés des reines et les comptes de réserves des deux camps, et le masque de bits des clouages One-Hive. Tout consommateur reconstruit exactement l'état, son cadre et son ensemble candidat à partir de ces octets, de sorte que l'encodeur grille et l'encodeur graphe dérivent de la même source.

Trois champs ont été ajoutés pour cette étude. Une *estampille du modèle* (numéro de génération et hachage du réseau) aux octets 100–107 répond, pour chaque segment, à la question de savoir quel réseau l'a généré ; la batterie de vérifications affirme que

l'estampille de chaque enregistrement correspond au manifeste de l'exécution. La *liste d'indices des coups légaux* (octets 178–817, plafonnée à 320 entrées ; le plus grand facteur de branchement mesuré en jeu réel est 213) permet l'entraînement masqué sur l'ensemble légal dans les deux bras sans relancer la génération de coups. L'*octet d'issue* prend quatre valeurs du point de vue du camp au trait : 0 défaite, 1 nulle, 2 victoire, 3 tronquée. La cible de politique est la distribution de visites à la racine de la recherche sur l'espace d'actions partagé, stockée sous forme de ses entrées top-15 avec le compte total de visites.

Les enregistrements de la boucle de démonstration antérieure (versions 1 et 2) ne portent ni estampille ni liste légale, et leurs parties plafonnées étaient étiquetées comme nulles ; ils ne sont pas des entrées d'entraînement pour l'étude. Chaque exécution conserve trois arborescences séparées, pour les enregistrements d'auto-jeu (les entrées d'entraînement), les points de contrôle et les sorties d'évaluation ; un audit affirme qu'aucun fichier d'évaluation n'est jamais référencé par une configuration d'entraînement (vérification 7).

6.4 La procédure d'entraînement

L'entraînement est identique pour les deux bras jusqu'à l'état de l'optimiseur : descente de gradient stochastique avec un taux d'apprentissage 0.02 recuit en cosinus jusqu'à un centième de sa valeur initiale sur la génération, momentum 0.9, décroissance des poids 10^{-4}, taille de lot 256, deux époques par génération, et un poids de la perte de valeur de 0.6. Un enregistrement sur cinquante est retenu comme partition de validation, sur laquelle l'argmax de politique (top-1) (masqué comme à l'entraînement) et l'exactitude de la valeur (sur les enregistrements non tronqués) sont rapportés après chaque époque. Aucune recherche d'hyperparamètres n'a été effectuée pour l'un ou l'autre bras.

La perte est celle écrite en Équation 2 de Chapitre 2 : l'entropie croisée entre la distribution de visites enregistrée et la politique du réseau sur l'ensemble légal, plus 0.6 fois l'entropie croisée d'issue à trois classes, le terme de valeur étant moyenné sur les seuls enregistrements non tronqués du lot. La distribution de politique $p_\theta(\cdot \mid s)$ est normalisée sur exactement l'ensemble légal : le bras grille remplit les entrées illégales de son tenseur plat de logits avec -10^9 avant le log-softmax, tandis que le bras graphe ne calcule des logits que pour les lignes (emplacement, destination) légales, de sorte que les deux bras produisent la même famille de distributions sur les mêmes ensembles légaux. La tête de valeur est un classifieur à trois classes victoire/nulle/défaite ; un enregistrement tronqué contribue au terme de politique mais est exclu du terme de valeur, jamais entraîné comme une nulle.

Les points de contrôle portent l'état du modèle, de l'optimiseur et du planificateur, les compteurs de pas et d'époques, l'état du générateur de nombres aléatoires et la configuration de l'exécution ; le mélange par époque est re-semé à partir de la graine de l'exécution et de l'indice d'époque, de sorte qu'une exécution reprise à une frontière d'époque rejoue l'ordre de lots identique (vérification 5). Une seule modification a été apportée à la boucle après la campagne principale : le 23 septembre 2026, après qu'une exécution d'ablation eut divergé vers une perte non finie et poursuivi silencieusement l'auto-jeu sur celle-ci pendant dix générations, un garde-fou a été ajouté qui arrête l'entraînement sur une perte non finie ; il laisse inchangé tout calcul fini. Le débit d'entraînement a été mesuré le 10 septembre 2026 à 274 positions/s pour le réseau grille et 138 positions/s pour le réseau graphe (lot 128, passe avant et arrière, sur le

backend GPU de la machine) ; les exécutions de la campagne ont chargé leurs données dans le processus plutôt que par des processus de travail, de sorte que leurs phases d'entraînement ont tourné à ces chiffres ou en dessous. La génération des parties plutôt que l'entraînement domine le temps mural d'une génération dans l'un et l'autre bras.

6.5 Export et inférence

Après l'entraînement, le point de contrôle est exporté en ONNX avec des formes d'entrée statiques, un graphe par taille de lot (la campagne a exporté le graphe de lot 1 qu'utilisent le jeu en match et l'auto-jeu). Les formes statiques sont exigées par le fournisseur d'exécution CoreML ; le bras graphe les obtient en complétant chaque position jusqu'à une capacité de nœuds fixe et une capacité de coups fixe, avec des masques. Le graphe exporté s'exécute ensuite dans la recherche Rust par le même chemin d'inférence pour les deux bras.

Le fournisseur d'exécution est le point où les deux bras diffèrent réellement en coût, et la différence est rapportée plutôt qu'égalisée (Tableau 10, mesure du 10 septembre 2026). Le réseau convolutif du bras grille s'exécute le plus vite sur CoreML ; le réseau du bras graphe, riche en opérations de collecte (gather), est scindé par CoreML en 15 sous-graphes avec 147 nœuds pris en charge sur 287, et s'exécute le plus vite sur le fournisseur CPU. Chaque bras a donc utilisé son meilleur fournisseur disponible : la grille sur CoreML à 2.62 ms par évaluation, le graphe sur CPU à 3.67 ms, soit un rapport de ≈1.4× au détriment du bras graphe. La lecture à temps mural égal du protocole impute à chaque bras ce coût réel sur cette machine ; la lecture à exemples égaux n'en est pas affectée.

Tableau 10 : Latence d'inférence des deux réseaux à capacité appariée (grille 1.44 M paramètres, graphe 1.47 M) à une taille de lot de 1, en millisecondes par évaluation du réseau, mesurée sur la machine d'étude (Apple M1 Pro, 10 cœurs, 16 Go) le 10 septembre 2026 sur trois chemins d'exécution ; une série de mesures par cellule, aucun intervalle rapporté. Le gras marque le fournisseur que chaque bras a utilisé en auto-jeu et en évaluation.

Chemin d'exécution	Bras grille	Bras graphe
PyTorch, CPU	11.03	10.51
ONNX Runtime, CPU	23.5	**3.67**
ONNX Runtime, CoreML	**2.62**	9.84
Fournisseur utilisé dans l'étude	2.62 (CoreML)	3.67 (CPU)

Le chemin CoreML lui-même a d'abord dû être réparé : le 9 septembre 2026, la couche d'inférence Rust plantait sous CoreML parce qu'elle demandait le format de modèle hérité du fournisseur ; demander le format moderne l'a rétablie, et une sonde à 128/32 simulations avec le réseau de la boucle antérieure comme charge de travail a tourné à ≈3 s par partie de temps mural sur quatre threads. Cette sonde est l'origine de l'estimation « ≈3 s/partie avec un réseau entraîné » citée dans le protocole.

6.6 L'arène d'évaluation

L'évaluation est indépendante de l'entraînement par ses données, ses réglages et ses graines. L'arène joue des parties *appariées* : la paire i de chaque affrontement joue la ligne d'ouverture i d'un fichier gelé de 250 ouvertures de jeu de base uniques et légales de quatre demi-coups, une fois avec chaque couleur, de sorte que chaque bras, chaque graine et chaque adversaire fait face à un calendrier d'ouvertures et de couleurs identique

(vérifié par un test qui hache les calendriers). Les ouvertures ont été générées à l'aveugle par une marche aléatoire à graine fixée sur les coups légaux du moteur et gelées le 10 septembre 2026, avant toute exécution de comparaison, leur hachage de contenu étant enregistré. Les évaluations finales utilisent 100 parties par adversaire (ouvertures 0–49), les évaluations intermédiaires 20.

Le réseau en cours d'évaluation recherche 400 simulations par décision, sans bruit de Dirichlet et sans température : le coup le plus visité est joué, de façon déterministe. Ces réglages ont été épinglés le 9 septembre 2026 dans une configuration versionnée et par un test unitaire affirmant que les valeurs par défaut de la recherche portent $\varepsilon = 0$; l'interface de jeu en match n'expose aucun moyen d'activer le bruit, de sorte que le chemin d'évaluation ne peut pas explorer par accident. Le compte de visites est fixé pour une raison : Jones (2021) mesure un compromis de calcul entre temps d'entraînement et temps de test sous lequel un budget d'évaluation flottant laisserait dériver la force mesurée. Les adversaires sont invoqués exactement tels que gelés (Chapitre 8), avec leurs propres graines fixes ; le côté réseau utilise la graine 9000 plus l'indice de génération (9500 pour les évaluations au seuil à temps mural égal). Les parties sont plafonnées à 300 demi-coups ; les troncatures sont exclues du score, rapportées comme un taux séparé, et comptées 0.5 dans une ligne de sensibilité. Une partie d'évaluation coûte ≈23–32 s à ce budget.

Les scores sont victoire = 1, nulle = 0.5, défaite = 0 sur les parties non tronquées, agrégés en un score par cellule (graine, adversaire) avant tout calcul statistique ; la graine plutôt que la partie est l'unité de rééchantillonnage (Chapitre 9).

6.7 Sept vérifications automatisées pré-entraînement

Avant de faire confiance à toute sortie d'entraînement, sept propriétés du chemin de données et de la boucle d'entraînement ont été transformées en assertions automatisées et exécutées en une seule commande contre des segments d'auto-jeu réels (Tableau 11). Elles vont au-delà de tests unitaires de fonctions isolées : les vérifications 1–3 s'exécutent sur des positions enregistrées à travers une vraie passe avant, la vérification 4 est une courte expérience avec un critère énoncé, et les vérifications 5–7 exercent de bout en bout les contrats de reprise, d'évaluation et d'agencement.

Tableau 11 : Les sept vérifications automatisées pré-entraînement, exécutées en une seule commande contre des segments d'auto-jeu réels avant de faire confiance à toute sortie d'entraînement. Chaque ligne énonce la propriété affirmée et le mécanisme qui l'affirme ; la vérification 4 est une expérience avec un critère numérique, les autres sont des assertions qui réussissent ou échouent.

Vérification	Ce qu'elle affirme	Comment elle est testée
1 Normalisation sur l'ensemble légal	La politique masquée place une probabilité exactement nulle sur les actions illégales et somme à un sur l'ensemble légal	Vraie passe avant sur des positions enregistrées ; masse illégale affirmée égale à 0
2 Identité des indices de coups	Encoder → indexer → décoder est l'identité sur tous les coups légaux à travers les types de partie ; chaque cible stockée et chaque indice joué sont légaux	Test aller-retour en Rust ; balayage côté données des entrées cibles de chaque enregistrement contre sa liste légale
3 Perspective de l'issue	L'octet d'issue est l'une de quatre valeurs, correcte pour le camp au trait dans les deux couleurs ; les enregis-	Batteries de signes en Rust (évaluation, alpha-beta, racine de recherche) ; comparaison côté don-

Vérification	Ce qu'elle affirme	Comment elle est testée
	trements tronqués sont exclus de la perte de valeur	nées de la perte de valeur avec sa référence filtrée des troncatures
4 Sur-apprentissage d'un petit lot	Un petit lot fixe atteint un ajustement quasi parfait	Argmax de politique 15/15, valeur 15/15, KL résiduelle 0.09 contre les cibles douces
5 Sauvegarde et reprise	Une exécution interrompue puis reprise depuis un point de contrôle reproduit un état du modèle et de l'optimiseur identique au bit près et conserve le calendrier de taux d'apprentissage prévu	Deux tronçons d'entraînement sur CPU ; chaque tenseur du modèle et chaque tenseur de l'optimiseur comparés pour égalité exacte
6 Pas d'exploration à l'évaluation	Le chemin d'évaluation porte $\varepsilon = 0$ et aucune température	Test unitaire sur les valeurs par défaut de la recherche ; réglages épinglés dans une configuration versionnée
7 L'évaluation n'alimente jamais l'entraînement	Aucune partie d'évaluation n'est une entrée d'entraînement	Audit de la liste de segments de chaque configuration d'entraînement contre l'arborescence d'évaluation

La vérification 4 mérite une note sur son critère. Les distributions de visites sont des cibles douces à entropie irréductible, de sorte que « la perte tend vers zéro » est le mauvais test ; le critère est la divergence de Kullback–Leibler rapportée au plancher d'entropie de la cible. La première exécution de la vérification a été rapportée comme un échec pour deux raisons sans rapport avec le réseau : la perte avait été comparée à zéro plutôt qu'au plancher, et le jeu de validation retenu avait été inclus. Le critère a été corrigé et la correction enregistrée. Sur le segment du pilote, la vérification corrigée a donné KL 0.093, argmax de politique 15/15, valeur 15/15.

6.8 Le pilote du 10 septembre 2026

La boucle a d'abord été exécutée de bout en bout à un budget délibérément petit : une génération à partir d'une initialisation aléatoire à graine fixée du réseau grille (1.44 M paramètres), 300 parties d'auto-jeu à 128/32 simulations avec randomisation du plafond de simulations (playout-cap randomization), trois époques d'entraînement (la campagne en a ensuite utilisé deux), lot 256, la perte de politique masquée, et une évaluation indépendante contre la population gelée sous les réglages épinglés avec 30 parties par adversaire. L'auto-jeu a commencé le soir du 9 septembre 2026 ; l'évaluation s'est achevée le 10 septembre.

Génération. Les 300 parties ont produit 17 237 positions enregistrées ; 170 parties sur 300 (56.7 %) ont été tronquées au plafond de 300 demi-coups et aucune ne s'est terminée par abandon, puisqu'une tête de valeur issue d'une initialisation aléatoire ne franchit jamais le seuil. La génération a pris ≈60 min sur quatre threads, ≈12 s par partie : le jeu non entraîné dure longtemps, alors que la sonde avec réseau entraîné de la veille mesurait ≈3 s par partie au même budget, de sorte que le coût par génération diminue à mesure que le jeu s'affine.

Vérifications. Les sept ont réussi sur le segment réel : masse de politique illégale exactement 0 et masse légale 1 ; chaque indice stocké et joué légal ; un domaine d'issue valide avec 12 806 enregistrements tronqués exclus de la perte de valeur ; le résultat de sur-apprentissage ci-dessus ; sauvegarde et reprise identiques au bit près au pas 2110 ; bruit

d'évaluation structurellement désactivé ; arborescences d'évaluation et d'entraînement disjointes.

Entraînement. Sur 16 893 enregistrements d'entraînement et 344 de validation, les pertes ont diminué (politique ≈3.4 nats à la fin contre ≈4.1 pour une distribution uniforme sur ≈60 coups légaux) ; l'argmax de politique (top-1) en validation a atteint 4–5 % contre un niveau de hasard uniforme sur l'ensemble légal de ≈1–2 % ; l'exactitude de la valeur en validation était de 42–46 % sur les 94 enregistrements de validation non tronqués ; débit ≈772 positions/s. Le signal de valeur est mince à la génération 0 parce que 74 % des enregistrements sont masqués par troncature.

Évaluation. Tableau 12 en donne le résultat. Le réseau de la génération 0 a battu l'aléatoire légal dans chaque partie décidée (28 victoires, 2 troncatures) et a obtenu 3.3 % contre l'heuristique comme contre la recherche à 6 400 simulations.

Tableau 12 : Évaluation indépendante du réseau grille de la génération 0 du pilote (une exécution d'entraînement, 300 parties d'auto-jeu à 128/32 simulations, trois époques) contre les trois adversaires gelés, à 400 simulations par décision sans bruit, 30 parties appariées à couleurs échangées par adversaire. Score = victoires + ½ nulles sur les parties non tronquées, en % ; les troncatures au plafond de 300 demi-coups sont comptées séparément. Volume de pilote : aucun intervalle n'est rapporté et ceci n'est pas un résultat de l'étude.

Adversaire	**V/N/D**	**Score**	**Tronquées**
B-RND (aléatoire légal)	28/0/0	100.0 %	2/30
B-HEU (heuristique)	0/2/28	3.3 %	0/30
B-MCTS (recherche, 6 400 simulations)	0/2/28	3.3 %	0/30

Ce que le pilote a diagnostiqué. Le profil (domination sur l'aléatoire légal, défaite quasi totale contre les deux adversaires de la bande intermédiaire) a été interprété dans l'ordre prescrit : règles, signes de la valeur, recherche, données. Les trois premiers avaient été validés indépendamment (Chapitre 5, Chapitre 8), de sorte que l'écart après une génération relève des données et des itérations plutôt que d'un défaut ; le levier indiqué est davantage de générations plutôt que davantage de capacité, et aucun changement de capacité n'a été fait. Le pilote a aussi fourni les deux confirmations budgétaires qu'attendait le gel du protocole : 128/32 simulations sont faisables à l'échelle de l'étude (pire cas ≈12 s par partie à la génération 0, s'améliorant vers ≈3 s), et le plafond de 300 demi-coups est praticable précisément parce que la troncature est une issue rapportée séparément. Le protocole a été gelé avec ces valeurs le jour même. Les limites du pilote sont celles de sa taille : une graine, 30 parties par adversaire, la génération 0 seulement, le bras grille seulement. L'itération de la boucle a été exercée dans sa structure mais n'a pas été réellement exécutée.

6.9 Un défaut de harnais détecté pendant le pilote

La première passe d'évaluation du pilote n'a pas produit Tableau 12. Elle a renvoyé 0/2/28 contre les trois adversaires, y compris l'aléatoire légal, contre lequel un réseau ayant appris quoi que ce soit ne devrait pas perdre. En suivant le même ordre d'investigation, les enregistrements de parties ont été inspectés : les réponses des adversaires étaient identiques dans les trois appariements. La boucle de shell interactive utilisée pour cette première passe avait transmis les options de ligne de commande de chaque adversaire comme un seul argument, de sorte que chaque adversaire s'est rabattu sur le backend de recherche par défaut du moteur et que les trois affrontements avaient en fait été

joués contre l'heuristique. Le lanceur du pilote proprement dit, qui construit les listes d'arguments explicitement, n'a pas ce défaut ; relancée avec des arguments explicites, l'évaluation a produit la table cohérente ci-dessus.

L'incident a été conservé comme exemple commenté plutôt qu'écarté, parce que des résultats identiques sous des conditions supposément différentes doivent être traités comme une alarme de harnais avant d'être lus comme un résultat. La règle a montré son utilité deux semaines plus tard, lorsque trois graines d'ablation « indépendantes » ont renvoyé des évaluations identiques jusqu'au nombre de parties et que l'alarme a conduit directement à la divergence silencieuse décrite dans Chapitre 13. Un second accroc mineur du même pilote, où la batterie de vérifications invoquait initialement un interpréteur Python système doté d'une installation d'apprentissage profond défectueuse pour la vérification 5, a été corrigé en épinglant l'environnement propre du projet.

7 Les deux représentations d'état et leurs réseaux

La variable indépendante de l'étude est la représentation d'état accompagnée du corps de réseau qui la lit, et rien d'autre. Ce chapitre spécifie les deux bras tels qu'ils ont été exécutés : encodages, réseaux, décodeur d'actions partagé, appariement de capacité, asymétries de coût mesurées plutôt que supprimées, tests qui épinglent chaque encodeur, et variantes d'ablation. Fig. 1 montre une position sous les deux encodages, Fig. 4 les deux réseaux, et Annexe B les tables de couches, l'arithmétique du décodeur et l'agencement de l'enregistrement.

Les deux côtés n'ont pas la même histoire. Le cadre grille, ses plans et le réseau convolutif ont été construits en juillet 2026 au sein du projet de moteur et ont porté la démonstration d'auto-jeu antérieure de 19 générations ; le 10 septembre 2026, ils ont été réutilisés sans modification comme représentation de référence, avec un seul durcissement de la vérification de débordement du cadre. Tout ce qui est du côté graphe (encodage, réseau, constructeurs de tenseurs dans les deux langages d'implémentation, et leurs tests) a été construit le 10 septembre 2026 pour cette étude, contre un contrat de décodeur fixé la veille.

7.1 Ce qui varie entre les bras, et ce qui, de manière démontrable, ne varie pas

Les deux bras partagent le moteur de règles, la recherche, la population d'adversaires gelée et les réglages d'évaluation épinglés, le format d'enregistrement, les cibles d'entraînement et les conventions d'issue (la troncature comme quatrième issue, exclue de la perte de valeur), l'espace d'actions avec sa normalisation sur l'ensemble légal, et la boucle d'entraînement (deux époques par génération, lot 256, taux d'apprentissage 0.02, poids de la perte de valeur 0.6, descente de gradient stochastique avec momentum 0.9 sous un calendrier cosinus). Aucun des deux bras n'a été réglé au-delà de l'appariement de capacité. Chaque élément partagé est imposé plutôt qu'affirmé : un seul contrat de décodeur avec masquage identique, un seul format d'enregistrement, un test doré inter-langages par encodeur, et une seule recherche arborescente Monte-Carlo en Rust évaluant les deux bras à travers des exports ONNX à formes statiques. Ce qui diffère est exactement énumérable : la représentation, le corps du réseau, et leurs interactions matérielles mesurées.

7.1.1 Le décodeur d'actions partagé

Un coup de Hive est identifié de manière unique par (pièce, cellule de destination) : une marche et un lancer amenant la même pièce sur la même cellule produisent des états successeurs identiques. Le décodeur adresse la pièce par un emplacement relatif au camp au trait : les 14 pièces du joueur au trait dans l'ordre de l'effectif, puis les 14 de l'adversaire aux emplacements 14–27 (ils existent parce que l'extension Pillbug déplace les pièces ennemies ; dans le jeu de base, ils ne sont jamais légaux). Il adresse la destination par une cellule de l'ensemble candidat, chaque cellule occupée plus l'anneau

des cellules vides adjacentes à la ruche, qui contient chaque destination légale par construction. Une action supplémentaire, la passe, est légale exactement lorsqu'aucun coup n'existe. Le bras grille matérialise cet espace comme un vecteur plat de 28 673 logits indexé par emplacement × 1024 + y × 32 + x sur son cadre 32 × 32, le dernier indice étant la passe. Le bras graphe ne matérialise aucun vecteur de ce type : pour chaque paire (emplacement, destination) légale, il calcule un logit à partir des plongements du nœud de destination, de la source de la pièce et de l'emplacement, et note une ligne de passe de la même façon. L'action à laquelle un logit se réfère est la paire identique dans les deux bras ; c'est le mécanisme du réseau pointeur (Vinyals et al., 2015) qui note chaque élément d'un ensemble candidat variable et normalise sur exactement cet ensemble.

Quatre propriétés sont partagées mot pour mot. Masquage : les logits n'existent que pour l'ensemble légal que le moteur génère, la softmax s'exécute sur exactement cet ensemble, et la masse de probabilité sur les actions illégales est identiquement nulle ; la première des sept vérifications automatisées pré-entraînement l'affirme par une vraie passe avant pour le bras testé, quel qu'il soit. Indépendance à l'ordre : les scores s'attachent aux paires (emplacement, destination), jamais aux positions dans la liste légale. Départage : les égalités d'argmax se départagent vers l'indice plat le plus bas, l'indexation de la grille définissant le départage pour les deux bras. Cibles : la distribution de visites à la racine de la recherche sur le même espace, stockée sous forme de ses 15 premières entrées (indice, poids de visites) avec le compte total de visites ; la cible de valeur est un scalaire dans [−1, 1] du point de vue du camp au trait (victoire +1, défaite −1, nulle 0), et la troncature ne contribue à aucune cible de valeur dans l'un ou l'autre bras. Le contrat a été fixé le 9 septembre 2026, avant que l'un ou l'autre encodeur ne soit construit pour l'étude. La justification est que le facteur de confusion est contrôlé par l'identité de l'espace d'actions, du masque et des cibles plutôt que par la production du même tenseur par les deux bras ; imposer le tenseur plat au réseau graphe aurait transporté le cadre, un artefact de la grille, dans le bras graphe.

7.2 L'encodage grille

7.2.1 Cadre, ancrage et débordement

Le plateau du moteur est une grille d'octets 64 × 64 à enroulement (un tore) en coordonnées axiales absolues. Pour l'encodage, une position est dépliée depuis le tore par un parcours en largeur à partir d'une cellule occupée arbitraire (l'enroulement ne peut pas scinder la ruche, qui est connexe par règle) et translatée de sorte que le centre de la boîte englobante occupée tombe en (16, 16) d'un cadre fixe 32 × 32. L'adjacence hexagonale en coordonnées axiales est un sous-ensemble de 7 cellules du voisinage 3 × 3, de sorte que des convolutions 3 × 3 ordinaires la couvrent, les deux coins non voisins de chaque noyau devenant des poids morts apprenables. L'ancrage se fait par le centre de la boîte englobante seulement : aucune canonicalisation par rotation ou réflexion, et aucune augmentation par symétries (voir la fin de ce chapitre).

Une ruche de 28 pièces s'étend sur au plus 28 cellules par axe après dépliage, de sorte que la boîte occupée plus l'anneau complet des destinations candidates tient dans le cadre avec marge, par construction. L'argument est aussi imposé à l'exécution : le constructeur du cadre porte une assertion toujours active, présente dans les builds release, de sorte que toute cellule projetée hors du cadre arrête bruyamment le programme et qu'aucune

pièce ni destination ne peut disparaître ou se replier silencieusement. Jusqu'au 10 septembre 2026, c'était une assertion de débogage seulement, retirée à la compilation des builds release, un risque de corruption silencieuse que la revue de débordement a clos. Des tests extrémaux placent les 28 pièces en ligne droite le long de chaque axe et prouvent que chaque cellule occupée et chaque cellule de l'anneau se projette sans repliement ; un long test de dérive sur parties aléatoires couvre le jeu ordinaire.

7.2.2 Les 77 plans de caractéristiques

Tableau 13 : Les 77 plans d'entrée de l'encodage grille. Chaque plan est une carte 32 × 32 en float32 à valeurs dans [0, 1] ; un plan « constant » diffuse un scalaire sur tout le cadre. Le propriétaire est relatif au camp au trait, en accord avec les emplacements du décodeur et la perspective de la tête de valeur.

Plans	**Contenu**
0–63	Plans de pièces : propriétaire (joueur au trait = 0, adversaire = décalage 32) + type d'insecte (8 types : Q, S, B, G, A, M, L, P) × 4 + min(niveau de pile, 3) ; propriétaire, type et hauteur sont encodés conjointement, un plan par combinaison
64	Pièces de sommet clouées par la règle One-Hive (cellules d'articulation de la ruche)
65	Cellule de la dernière pièce déplacée (état pertinent pour l'étourdissement)
66	Cellules de placement légal pour le camp au trait
67	Cellules de placement légal pour l'adversaire
68	Le camp au trait est blanc (plan constant)
69, 70	Libertés des reines (joueur au trait, adversaire) / 6 (plans constants ; 0 si la reine n'est pas placée)
71	Pli / 100 (plan constant)
72–74	Bits de type de partie M, L, P (plans constants)
75, 76	Comptes de réserves (joueur au trait, adversaire) / 14 (plans constants)

Les plans de pièces conjoints enregistrent, par cellule, qui possède la pièce à chaque niveau de pile, ce qu'elle est et à quelle hauteur elle se trouve, les niveaux 3 et supérieurs étant fusionnés. Les plans de placement suivent la règle d'adjacence standard (une cellule vide adjacente à au moins une pièce de sommet du joueur et à aucune de l'adversaire) sans les exceptions des tours d'ouverture ; ce sont seulement des indices, puisque la légalité elle-même est fournie aux deux bras par le masque du moteur. L'encodeur graphe utilise la même règle simplifiée.

7.2.3 Le réseau grille

HiveNet est un réseau convolutif résiduel dans le style de KataGo (Wu, 2020), dimensionné à 96 canaux et 8 blocs résiduels. Un tronc convolutif 3 × 3 fait passer les 77 plans à 96 canaux (normalisation par lots, ReLU) ; chaque bloc résiduel applique deux convolutions 3 × 3 avec normalisation par lots, une addition résiduelle et une ReLU ; les blocs 2 et 5 (en comptant à partir de 0) ajoutent un biais de pooling global avant l'addition résiduelle : la moyenne et le maximum par canal sur le cadre sont concaténés, passés par une couche linéaire et réinjectés par canal. Cela injecte deux fois un signal global, la sécurité de la reine étant une propriété globale. La tête de politique est une convolution 1 × 1 vers 28 plans d'emplacements de pièces, aplatie en 28 672 logits spatiaux, plus un logit de passe issu des caractéristiques agrégées par moyenne : 28 673 sorties. La tête de valeur fait passer les caractéristiques agrégées par moyenne par une couche cachée de 64 unités vers trois logits (victoire, nulle, défaite du point de vue du camp au trait). Le réseau compte 1.44 M paramètres, comptés par le code d'entraînement.

7.3 L'encodage graphe

7.3.1 Nœuds, pièces et destinations

L'encodage graphe est sans coordonnées : aucune coordonnée absolue n'y apparaît. Ses nœuds sont les cellules de l'ensemble candidat (chaque cellule occupée et chaque cellule vide adjacente à la ruche), qui est exactement l'univers de destinations du décodeur, de sorte que chaque destination pouvant être notée est un nœud à part entière. Le choix découle des règles : les destinations et les contraintes de glissement de Hive sont des propriétés de l'espace vide, et un graphe sur les seules cellules occupées n'aurait rien à noter pour la plupart des coups. Keller et al. (2023) sont parvenus à la conclusion analogue pour Hex, dont la formulation ne conserve que les cellules vides comme nœuds, et un décodeur de type pointeur ne peut pointer que vers des éléments qui existent (Vinyals et al., 2015). Une caractéristique d'occupation explicite distingue les candidats vides des nœuds occupés. Les pièces ne sont pas des nœuds séparés : puisque le décodeur adresse un coup comme (emplacement, cellule de destination), l'identité d'une pièce entre dans la politique par son plongement d'emplacement et sa localisation par le nœud sur lequel elle se tient ; les nœuds de cellule portent la composition complète de la pile niveau par niveau, de sorte qu'aucune information de pièce n'est perdue et que le graphe reste deux fois plus petit qu'il ne le serait avec des nœuds de pièces. C'est un choix de conception documenté.

7.3.2 Caractéristiques de nœud, relations typées et caractéristiques globales

Tableau 14 : Les 56 caractéristiques de nœud de l'encodage graphe, un vecteur par cellule de l'ensemble candidat. L'empilement est représenté niveau par niveau jusqu'à la hauteur 5 (les hauteurs supérieures à 5 ne peuvent pas survenir dans le jeu de base ; le scalaire de hauteur les enregistre néanmoins). Le propriétaire est relatif au camp au trait.

Caractéristiques	Contenu
0–49	Cinq niveaux de pile (0–4), dix caractéristiques chacun : bit de présence, bit propriétaire-au-trait, one-hot du type d'insecte sur les 8 types
50	Hauteur de pile / 5
51	Bit candidat-vide (1 pour une cellule vide de l'anneau)
52	Bit cloué One-Hive (la pièce de sommet est un point d'articulation de la ruche)
53	Bit dernier-coup (pertinent pour l'étourdissement)
54	Bit de placement légal pour le camp au trait
55	Bit de placement légal pour l'adversaire

Les arêtes sont les adjacences dirigées entre cellules de l'ensemble candidat, typées par les six directions hexagonales (est, nord-est, nord-ouest, ouest, sud-ouest, sud-est), stockées sous forme d'un tenseur d'indices de voisins (par nœud, l'indice de son voisin dans chaque direction, avec une sentinelle en l'absence de voisin) et réalisées dans le réseau comme six matrices de poids propres à chaque relation. L'inverse de la direction d est (d + 3) mod 6, une symétrie que les tests de propriétés vérifient. L'adjacence aux cellules hors de l'ensemble candidat est exclue : ces cellules sont vides et non adjacentes à la ruche, de sorte qu'elles ne peuvent influencer ni la légalité ni la valeur.

Tableau 15 : Les 23 caractéristiques globales de l'encodage graphe. Le vecteur est concaténé à l'entrée de chaque nœud et de nouveau à la représentation agrégée dans la tête de valeur.

Caractéristiques	Contenu
0	Le camp au trait est blanc
1	Pli / 100
2, 3	Libertés des reines (joueur au trait, adversaire) / 6 ; 0 si la reine n'est pas placée
4–11	Compte de réserves du joueur au trait par type d'insecte / 3 (8 types)
12–19	Compte de réserves de l'adversaire par type d'insecte / 3 (8 types)
20–22	Bits de type de partie M, L, P

Les réserves entrent par type d'insecte parce que la légalité des placements et la planification du matériel dépendent des insectes qui restent en main ; le bras grille porte les deux totaux comme plans constants et peut retrouver les comptes par type à partir de ses plans de pièces, de sorte qu'aucun bras ne reçoit une information que l'autre ne peut reconstruire.

7.3.3 Capacités fixes et débordement bruyant

Les tenseurs ont des formes fixes, 224 nœuds et 321 lignes de coups (320 coups légaux, le plafond de l'enregistrement, plus une ligne de passe), et, comme le cadre, une vérification de débordement toujours active : une position dépassant l'une ou l'autre capacité échoue bruyamment plutôt que silencieusement. Les maxima mesurés sur 300 enregistrements d'auto-jeu réels étaient de 67 nœuds et 124 coups légaux, bien en deçà de la capacité. Les formes fixes gardent l'export ONNX statique, de sorte que le bras graphe s'exécute sous la même recherche Rust et le même chemin d'inférence que le bras grille ; des formes dynamiques auraient imposé une voie d'inférence différente et une asymétrie par bras là où la conception doit être identique.

7.3.4 Couverture de l'état du moteur, et ce qui n'est pas donné au réseau

Tableau 16 : Carte de couverture des composants de l'état de jeu du moteur vers les éléments de l'encodage graphe. Les deux dernières lignes énoncent ce qui est délibérément absent.

Composant de l'état (influence la légalité ou l'issue)	Élément du graphe
Positions, propriétaires et types des pièces	caractéristiques de nœud par niveau de pile
Empilement (montées de scarabée, pièces enfouies)	caractéristiques par niveau et hauteur
Destinations candidates vides	nœuds candidats vides
Géométrie d'adjacence et directions	arêtes typées (6 directions)
Clouages One-Hive	bit de clouage (articulation calculée par le moteur)
État d'étourdissement (dernière pièce déplacée)	bit dernier-coup
Camp au trait	caractéristiques relatives au joueur au trait et bit global
Échéance de placement de la reine	pli global et caractéristiques de réserves
Réserves	comptes globaux par type
Type de partie	bits globaux
Régions de légalité de placement	bits de placement (règle dérivée du moteur)
La légalité des coups elle-même	exclue : fournie par position par le moteur à travers le masque légal du décodeur, identiquement au bras grille ; le réseau ne calcule jamais la légalité
Coordonnées absolues du plateau	exclues : sans coordonnées par conception, de sorte que les questions d'ancrage ne se posent pas

Aucune invariance et aucune règle ne sont accordées gratuitement. L'encodage ne contient aucune coordonnée absolue, mais la fonction apprise n'en est pas pour autant invariante par translation ou par rotation : le passage de messages avec relations typées par direction n'est pas invariant par rotation, les champs réceptifs sont limités par la profondeur (un saut par couche), et aucune règle de Hive n'est connue du réseau ; la légalité arrive du moteur par le masque, pour les deux bras de la même façon. Le protocole traite toute invariance comme une question à mesurer plutôt qu'à présumer, et ce chapitre n'en affirme aucune.

7.3.5 Le réseau graphe

HiveGraphNet commence par une couche linéaire qui fait passer les 56 caractéristiques de chaque nœud, concaténées aux 23 caractéristiques globales, à 152 canaux, masquée aux nœuds réels ; une ligne nulle à l'indice 224 tient lieu de voisin absent. Huit couches relationnelles à passage de messages suivent, chacune calculant

$$h_i' = \mathrm{ReLU}\left(h_i + W_{\mathrm{self}}\, h_i + b + \sum_{d=1}^{6} W_d\, h_{n_i(d)} \right) \tag{4}$$

où $n_i(d)$ est le voisin du nœud i dans la direction d (la ligne nulle en son absence), W_{self} porte le biais b, et les six W_d sont les matrices typées par direction, sans biais ; la sortie est masquée aux nœuds réels. Les couches 2 et 5 (en comptant à partir de 0), soit une couche sur trois, ajoutent un biais de pooling global à l'intérieur de la non-linéarité : la moyenne masquée et le maximum masqué sur les nœuds réels sont concaténés, passés par une couche linéaire et ajoutés à chaque nœud. Le passage de messages seul est limité par la profondeur, alors que la sécurité de la reine est une propriété globale.

La tête de valeur concatène la moyenne masquée, le maximum masqué et le vecteur global et les fait passer par une couche cachée de 64 unités vers les trois mêmes logits victoire/nulle/défaite que le bras grille. La tête de politique est la fonction de notation par candidat du décodeur : pour chaque ligne (emplacement, destination) légale, elle concatène le plongement du nœud de destination, un plongement de source et un plongement d'emplacement de dimension 32 et les fait passer par une couche cachée de 128 unités vers un logit ; le plongement de source est le nœud sur lequel la pièce se tient pour un mouvement et un vecteur de réserve appris pour un placement. La ligne de passe est notée par la même fonction de notation à partir du plongement de l'emplacement de passe, du vecteur de réserve et de la ligne nulle ; c'est une constante apprise et sans incidence, parce que la passe n'est légale que lorsqu'aucun coup n'existe et remporte alors seule la softmax. Les lignes illégales sont masquées avant la softmax, qui s'exécute sur les lignes légales dans la perte et dans l'évaluateur d'inférence exactement comme la softmax masquée du bras grille. Le réseau compte 1.47 M paramètres, +1.5 % par rapport au réseau grille. C'est un encodeur inductif à passage de messages au sens de Hamilton et al. (2017), étendu avec des poids propres à chaque relation par direction sur des voisinages exacts de degré au plus six, ce qui est exactement ce que le protocole prescrit.

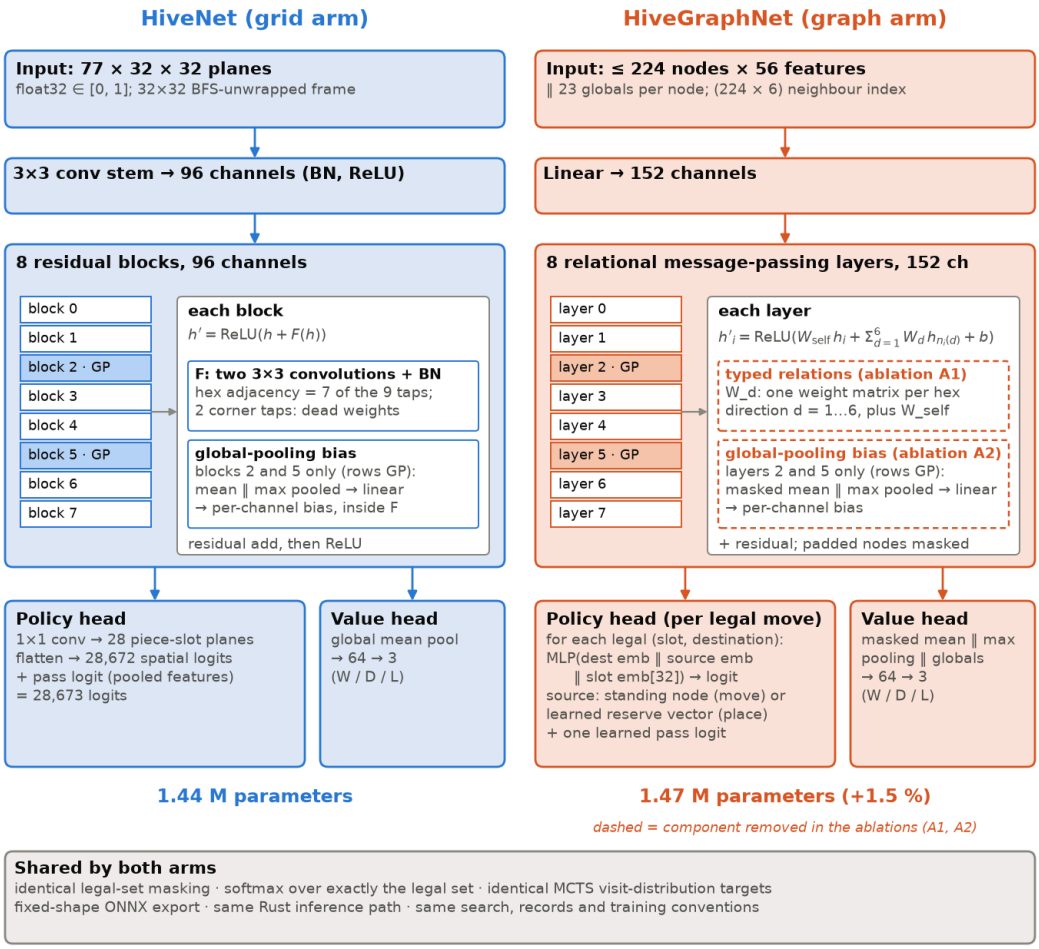


Fig. 4 : Schémas par blocs des deux réseaux. À gauche, HiveNet (bras grille) : tronc convolutif 3 × 3, 8 blocs résiduels de 96 canaux avec un biais de pooling global dans les blocs 2 et 5, une tête de politique plate à 28 673 sorties et une tête de valeur à 3 classes, 1.44 M paramètres au total. À droite, HiveGraphNet (bras graphe) : couche linéaire d'entrée vers 152 canaux, 8 couches relationnelles à passage de messages avec six matrices typées par direction et un biais de pooling global une couche sur trois, une fonction de notation de politique par candidat sur les plongements de destination, de source et d'emplacement, et une tête de valeur moyenne‖max masquée, 1.47 M paramètres au total. Les deux réseaux alimentent le décodeur d'actions partagé ; seuls l'encodeur et le corps diffèrent.

7.4 Appariement de capacité et asymétries de coût mesurées

La capacité a été appariée en dimensionnant la largeur et la profondeur du réseau graphe (152 canaux, 8 couches) sur le compte de paramètres du réseau grille (96 canaux, 8 blocs) : 1.47 M contre 1.44 M, +1.5 %, rapporté. Ce qui n'a pas pu être apparié est le coût d'exécution de chaque réseau sur la machine d'étude, où les deux représentations interagissent avec le matériel de façons opposées.

Tableau 17 : Coût d'inférence par évaluation de position à une taille de lot de 1 pour les deux réseaux, mesuré le 10 septembre 2026 sur la machine d'étude (Apple M1 Pro, 10 cœurs, 16 Go, macOS 15.3.1). Millisecondes par évaluation ; une seule configuration de mesure, aucun intervalle.

Chemin d'inférence	Grille (HiveNet)	Graphe (HiveGraphNet)
PyTorch, CPU	11.03 ms	10.51 ms
Inférence ONNX, fournisseur CPU	23.5 ms	3.67 ms
Inférence ONNX, fournisseur CoreML	2.62 ms	9.84 ms
Meilleur fournisseur disponible	2.62 ms (CoreML)	3.67 ms (CPU)

Le réseau convolutif s'exécute le plus vite sur l'accélérateur CoreML ; le réseau graphe s'exécute le plus vite sur le CPU, parce que sous CoreML seuls 147 de ses 287 opérateurs sont pris en charge, les opérations riches en collecte (gather) se rabattent sur 15 partitions, et le chemin de l'accélérateur finit par être plus lent que le chemin CPU. Au meilleur fournisseur de chaque bras, le rapport de coût par évaluation est de ≈1.4× au détriment du bras graphe. L'auto-jeu et l'évaluation ont donc exécuté chaque bras sur son meilleur fournisseur à travers la même recherche.

Tableau 18 : Débit d'entraînement à une taille de lot de 128 sous conditions appariées pour les deux réseaux, en positions par seconde, mêmes machine et date que la table d'inférence. L'étude entraîne sur le chemin MPS.

Chemin d'entraînement	Grille (HiveNet)	Graphe (HiveGraphNet)
PyTorch, CPU, passe avant seulement	138 pos/s	478 pos/s
PyTorch, MPS, passe avant + arrière	274 pos/s	138 pos/s

L'entraînement montre le motif inverse : sur le CPU, le réseau graphe est ~3.5× plus rapide par position, sur le chemin MPS utilisé pour l'entraînement ~2× plus lent, les opérations de collecte (gather) et de dispersion (scatter) y dominant. L'entraînement est de toute façon une part mineure du temps mural d'une génération (dans le pilote, ≈3.5 min d'entraînement contre ≈60 min d'auto-jeu pour la génération 0), de sorte que l'asymétrie d'inférence pilote la différence de coût à l'échelle de la campagne. Les chiffres proviennent d'une seule machine et d'une seule configuration, excluent le coût de construction en Python du chargeur de données graphe, et les nombres MPS sont une simple passe avant et arrière sans le pas de l'optimiseur. Ces asymétries sont rapportées plutôt qu'égalisées. Ce sont de véritables interactions entre une représentation et le matériel, et les égaliser, que ce soit en bridant le bras grille ou en forçant le bras graphe sur un fournisseur plus lent, fabriquerait une parité qu'aucun utilisateur de l'une ou l'autre représentation ne rencontrerait. Le protocole lit plutôt la comparaison deux fois : sous la lecture à exemples égaux, les asymétries sont sans objet, les deux bras jouant le même nombre de parties avec les mêmes réglages de générateur ; sous la lecture à temps mural égal, chaque bras se voit imputer son coût réel sur cette machine. La conséquence à l'échelle de la campagne est rapportée avec les résultats de coût.

7.5 Épinglage des encodeurs : tests dorés et tests de propriétés

Chaque encodeur existe deux fois, en Rust dans le moteur et la recherche et en Python dans la boucle d'entraînement, et l'accord entre les deux est prouvé plutôt que supposé. Pour le bras grille, l'encodeur Rust et le décodeur Python sont identiques à l'octet près par contrat, ce qu'impose une vérification croisée dorée nocturne sur 240 positions ; après le durcissement de l'assertion, la vérification croisée a réussi et la suite du crate d'interface s'établissait à 7/7, y compris les nouveaux tests extrémaux ; tout changement ultérieur doit la maintenir au vert ou être enregistré comme un résultat.

Pour le bras graphe, l'entraînement construit les tenseurs en Python à partir des enregistrements générés par le moteur (et eux-mêmes contre-vérifiés). Une batterie de propriétés sur 300 enregistrements réels a réussi intégralement : aucune perte d'information (chaque pièce, niveau de pile, compte de réserves, état d'étourdissement et donnée de trait de l'enregistrement apparaît dans les tenseurs) ; capacités respectées, avec une sonde de débordement levant une erreur bruyamment ; symétrie des arêtes ; validité du tenseur de coups légaux, chaque cible stockée étant dans la liste légale ; et masse de probabilité illégale nulle par une vraie passe avant. Une seule correction du constructeur a été nécessaire : le plateau vide au pli 0 a un ensemble candidat vide, et le constructeur reflète désormais la première cellule canonique du cadre, (16, 16). Le constructeur Rust utilisé à l'inférence reflète le constructeur Python (même ordre des cellules, même agencement des caractéristiques, même ordre des directions, même règle de placement simplifiée et même débordement bruyant), et une vérification croisée dorée sur 160 positions pseudo-aléatoires couvrant les 8 types de partie a trouvé 160/160 exactement égales dès sa première exécution ; elle s'exécute chaque nuit à côté de la vérification croisée des plans. L'évaluateur graphe à l'intérieur de la recherche construit les tenseurs et les lignes de coups dans l'ordre des coups de la recherche, exécute l'export à formes statiques sur le fournisseur CPU, applique la softmax sur les lignes légales et convertit la sortie à trois classes en P(win) − P(loss), ce qui est le contrat de sortie de l'évaluateur grille.

Une validation du câblage (deux époques sur les données du pilote de la génération 0, une graine) a tourné à 194 positions par seconde sur MPS, chargeur de données inclus, avec un argmax de politique (top-1) en validation de 4.7 % et une exactitude de la valeur de 40.4 %, le même profil que le bras grille sur les données identiques (4–5 %, ~43 %) ; un match de fumée de 6 parties à 200 simulations contre l'adversaire aléatoire légal a donné 2 victoires, 0 défaite et 4 troncatures sans aucune réponse illégale. Ces chiffres établissent seulement que le câblage est sain ; ils ne constituent pas un résultat de comparaison.

7.6 Les deux variantes d'ablation

Tableau 19 : Les variantes d'ablation du bras graphe. Chacune diffère du bras graphe complet par exactement un composant du réseau ; l'encodeur, le décodeur, les réglages d'entraînement, les budgets, les réglages d'évaluation, les adversaires et les ouvertures sont identiques. Paramètres en millions, mesurés par le compteur de paramètres du code d'entraînement.

Variante	Composant unique modifié	Paramètres
Bras graphe complet (référence)	aucun	1.47 M
Arêtes non typées (« adjacence naïve »)	les six matrices typées par direction remplacées par une seule matrice partagée	0.54 M
Sans pooling global	le biais de pooling global retiré de chaque couche	1.37 M

Les différences de paramètres sont inhérentes aux composants retirés et sont rapportées plutôt qu'égalisées ; élargir le réseau non typé pour compenser modifierait un second composant. La variante à adjacence naïve est l'ablation que le plan de recherche demandait ; elle teste la prémisse, inscrite dans le protocole avant toute exécution, selon laquelle un graphe à adjacence naïve pourrait ne pas suffire. La variante sans pooling global a été substituée le 19 septembre 2026 à l'ablation prévue de retrait de

l'augmentation, qui n'avait plus rien à retirer une fois l'augmentation exclue de la méthode complète ; elle a été choisie parce que le signal de mécanisme de la comparaison principale (la force relative du bras graphe résidait dans sa tête de valeur tandis que sa politique restait localement faible) faisait du signal agrégé la question à composant unique la plus nette qui restait. Une variante supplémentaire à deux composants, arêtes non typées plus un écrêtage global de la norme du gradient à 1.0, a été approuvée le 26 septembre 2026 après que la variante non typée se fut révélée non entraînable aux réglages de parité, divergeant vers des valeurs non finies à la génération 0 pour 3 graines sur 3 ; ses nombres sont étiquetés comme une différence à deux composants partout où ils apparaissent et rien de ce qu'elle montre n'est attribué au seul typage des arêtes. Les issues sont rapportées avec les résultats d'ablation.

7.7 Augmentation par symétries : exclue par décision

Aucun des deux bras ne s'entraîne avec augmentation par symétries, par une décision du 10 septembre 2026 prise avant toute exécution de comparaison. L'exclusion rend trivialement vraie la règle des données identiques : implémenter les symétries hexagonales de manière cohérente à travers deux représentations (rotations et réflexions du cadre d'un côté, permutations des types de direction de l'autre) est subtil, et une asymétrie à cet endroit contaminerait la comparaison principale. Elle garde aussi la question secondaire des symétries séparable comme ablation additive, et elle correspond à la référence caractérisée : le pipeline antérieur avait documenté une augmentation par 12 symétries comme intention mais ne l'avait jamais implémentée. Le protocole gelé n'a jamais spécifié d'augmentation ; les deux bras voient donc chaque position dans l'orientation que la partie a produite.

7.8 Limites de ce que ce chapitre établit

Ce chapitre établit l'identité de tout sauf la représentation ; il ne dit rien du mérite de l'une ou l'autre représentation. Les chiffres de coût proviennent d'une seule machine dans une seule configuration ; un autre accélérateur pourrait inverser l'asymétrie d'inférence. L'appariement de capacité, à +1.5 %, est un appariement des comptes de paramètres plutôt que du calcul. Le bras graphe est un point dans un vaste espace de conception (nœuds de cellule, six types de direction, un biais agrégé une couche sur trois, une fonction de notation par candidat), et l'étude compare un réseau grille à un réseau graphe à une seule capacité et un seul budget ; rien ici ne dit qu'une autre conception de graphe se comporterait de même. Enfin, les symétries hexagonales ne sont ni canonicalisées ni augmentées dans l'un ou l'autre bras, de sorte que toute différence liée aux symétries entre les bras est une propriété des fonctions apprises plutôt que des encodages.

8 La population d'adversaires gelée

8.1 Pourquoi une population fixe plutôt qu'un classement contre des cibles mouvantes

La métrique primaire de l'étude est un score moyen contre une population fixe d'adversaires (victoire = 1, nulle = 0.5, défaite = 0 sur les parties non tronquées, les troncatures étant rapportées comme un taux à part) plutôt qu'un classement Elo ; la raison en est la comparabilité. Un classement estimé à partir de parties entre les agents en cours d'entraînement se déplace chaque fois que les agents se déplacent : un point de contrôle ultérieur modifie l'échelle sur laquelle un point de contrôle antérieur a été mesuré, et deux bras entraînés séparément ne partagent aucune échelle. Le go dispose d'une échelle externe de réseaux publics et Hex d'une ancre de jeu parfait, que Wu (2020) et Jones (2021) utilisent respectivement pour fixer leurs classements ; Hive n'a ni l'une ni l'autre. Une population fixée à l'avance et partagée par les deux bras, toutes les graines et les deux lectures de budget en est le substitut : elle rend les scores comparables à travers tout ce que l'étude fait varier, et elle ferme le canal par lequel un adversaire pourrait être ajusté après l'observation des résultats. La population a donc été gelée le 9 septembre 2026, avant toute exécution d'entraînement, et aucun membre n'y a été ajouté, retiré, réajusté ou re-versionné depuis. Là où l'outillage imprime un nombre de type Elo dans ce rapport, il est descriptif et relatif à cette population uniquement.

8.2 Les trois adversaires

Les trois adversaires s'exécutent sur le moteur validé, le même noyau de règles qui engendre les parties d'entraînement des deux bras, via l'Universal Hive Protocol ; ils sont entièrement contrôlés par graine et reproductibles.

B-RND, aléatoire légal. Un tirage uniforme parmi les coups légaux de la position. Le générateur avance à chaque requête et est mélangé au hachage de la position, de sorte que l'agent est déterministe étant donné sa graine et l'historique de la partie. Il constitue le plancher de la population et ancre l'échelle des scores ; tout agent appris ou heuristique doit le dominer.

B-HEU, heuristique documentée. Un argmax glouton à un demi-coup de l'évaluation construite à la main du moteur, en mono-thread, étendu par la quiescence du moteur de recherche sur les coups ciblant la reine adverse. L'évaluation est la somme pondérée des caractéristiques de Tableau 20, en unités de type centipion, les valeurs positives favorisant le joueur au trait ; sa négation sous alternance des joueurs est épinglée par un test automatisé. Les poids sont les valeurs par défaut du moteur tel que construit en juillet 2026, hérités inchangés et fixés pour l'étude par un hachage cryptographique du fichier de poids et par un test qui échoue si une valeur quelconque change : aucun réglage n'a eu lieu pendant la caractérisation de la population et aucun ne peut avoir lieu ensuite sans casser la suite de tests. Un réglage après observation des résultats est donc exclu par construction.

Tableau 20 : Caractéristiques et poids de l'adversaire heuristique. L'évaluation est la somme pondérée de ces termes pour le joueur au trait moins la même somme pour l'adversaire, en unités de type centipion (positif = favorable au joueur au trait). Les poids sont les valeurs par défaut du moteur de juillet 2026, épinglées par hachage et imposées par test ; le terme du cloporte appartient au noyau partagé et ne se déclenche jamais dans le jeu de base utilisé par l'étude.

Caractéristique	**Poids**	**Justification**
Libertés de la reine (cellules vides autour de sa propre reine, 0 à 6)	−2000, −700, −350, −150, −50, 0, +20	Terme dominant : la distance à l'encerclement est l'objectif du jeu ; fortement convexe à mesure que les libertés disparaissent
Pièce adverse adjacente à sa propre reine	−90 chacune	Les voisins adverses sont du matériel d'encerclement permanent
Pièce amie adjacente à sa propre reine	−20 chacune	Les pièces propres encombrent les cellules de fuite et peuvent y être clouées
Pièce adverse au-dessus de sa propre reine	−180	Une reine couverte ne peut pas fuir et la cellule compte contre elle
Pièce libre de se déplacer (par insecte : Q, S, B, G, A, M, L, P)	15, 35, 55, 40, 80, 55, 50, 40	La mobilité approxime le matériel à Hive ; la fourmi est la pièce mobile la plus précieuse
Pièce clouée ou immobile (même ordre)	0, 4, 8, 6, 10, 6, 6, 4	Une pièce clouée est du matériel presque mort ; faible résidu pour la valeur latente
Pièce en main	+6 chacune	Souplesse de placement et tempo
Propre cloporte adjacent à sa propre reine	+40	Disponibilité du lancer de secours ; inerte dans le jeu de base

B-MCTS, recherche sans réseau. Recherche arborescente Monte-Carlo PUCT sur le même moteur, avec des a priori uniformes et la même évaluation construite à la main, écrasée par une tangente hyperbolique, comme valeur aux feuilles ; aucun réseau de neurones et aucun bruit d'exploration en match. Son budget est de 6 400 simulations par décision, mesuré à ≈27 ms par décision sur un thread de la machine d'étude ; le budget a été fixé à partir de cette mesure plutôt qu'à partir des budgets provisoires du plan de recherche.

8.3 Vérification tactique avant usage

L'adversaire de recherche a été vérifié sur cinq positions tactiques annotées à la main à partir des règles de l'éditeur et consignées par commit avant toute exécution, selon la même discipline que le corpus de validation du moteur de Chapitre 5. Les positions sont un mat en un coup par marche sur le périmètre et par saut de sauterelle, chacun pour les deux couleurs, et une position dans laquelle le seul coup sûr évite d'encercler sa propre reine. B-MCTS a résolu 5 cas sur 5 à 400, 1 600 et 6 400 simulations. Les signes de la valeur sous alternance des joueurs sont épinglés par des tests automatisés à trois niveaux : l'évaluation se nie exactement quand le trait change de camp ; l'alpha-bêta note un mat en un coup au-dessus du seuil de mat pour le joueur au trait, quelle que soit la couleur qui joue ; et la valeur à la racine de la recherche est fortement positive pour le joueur au trait gagnant, quelle que soit la couleur qui joue. Une erreur d'ordre des pièces dans la mise en place des cas tactiques a elle-même été attrapée par cette procédure (le moteur avait raison et la mise en place tort) et a été consignée plutôt que corrigée en silence.

8.4 Caractérisation

Les trois agents ont été opposés en tournoi toutes rondes le 9 septembre 2026 : 100 parties appariées, à couleurs échangées, par confrontation, à partir d'ouvertures aléatoires communes de quatre demi-coups à graine fixée, avec des coups validés par l'arbitre et un plafond de 300 demi-coups, les troncatures étant exclues du score et rapportées séparément. L'ensemble de la caractérisation a pris environ dix minutes sur la machine d'étude ; Tableau 21 en donne les résultats.

Tableau 21 : Caractérisation en tournoi toutes rondes des trois adversaires le 9 septembre 2026, 100 parties appariées à couleurs échangées par confrontation à partir d'ouvertures communes de quatre demi-coups à graine fixée, plafond de 300 demi-coups ; B-HEU à profondeur 1, B-MCTS à 6 400 simulations par décision. Score = victoires + ½ nulles sur les parties non tronquées, en % ; les troncatures sont comptées séparément (aucune n'est survenue). La colonne Elo est descriptive seulement : une transformation logistique du score avec un intervalle à 95 % en approximation normale qui traite les parties comme indépendantes, ce que le plan apparié rend légèrement conservateur ; un score parfait n'a pas d'Elo fini, de sorte que le résultat 100–0 est montré comme la valeur approximative qu'imprime l'outillage.

Confrontation (A vs B)	V/N/D pour A	Score de A	Tronquées	Différence Elo (descriptive)
B-HEU vs B-RND	100/0/0	100.0 %	0/100	≈+2400
B-MCTS vs B-RND	99/1/0	99.5 %	0/100	+920 [+730, +1200]
B-MCTS vs B-HEU	23/29/48	37.5 %	0/100	−89 [−150, −32]

L'heuristique a gagné les 100 parties contre l'aléatoire légal ; la recherche en a gagné 99 et annulé 1 ; contre l'heuristique, la recherche a obtenu 23 victoires, 29 nulles et 48 défaites. La dernière confrontation correspond à un score de 37.5 %, soit −89 Elo avec un intervalle [−150, −32], qui exclut la parité ; aucune des 300 parties n'a atteint le plafond de 300 demi-coups, ce qui est cohérent avec le profil de longueur des parties mesuré pendant la validation du moteur. Au sein de cette population, l'aléatoire légal est un plancher net, et l'heuristique et la recherche forment une bande intermédiaire séparée d'environ 90 Elo, aux styles différents, tactique gloutonne tranchante d'un côté et recherche échantillonnée de l'autre. Quant aux limites, 100 parties par confrontation relèvent du volume d'un pilote, donnant des intervalles d'environ ±5 points de pourcentage pour des scores proches de 40–60 % ; l'intervalle de l'arène est une approximation non appariée ; tout a été mesuré sur une seule machine, et les chiffres de coût de la recherche sont spécifiques à la machine par conception.

Un ordre n'était pas attendu et est rapporté tel que constaté : la recherche à 6 400 simulations se situe *en dessous* de l'heuristique à un demi-coup. Le diagnostic consigné à l'époque est que des a priori uniformes répartissent 6 400 simulations trop finement sur le facteur de branchement de Hive, d'environ 60 coups, produisant une recherche effectivement peu profonde, tandis que l'argmax à profondeur 1 de l'heuristique, avec sa quiescence ciblant la reine, est tactiquement tranchant ; la valeur aux feuilles écrasée par la tanh sature aussi sur les positions de danger pour la reine, aplatissant le signal que reçoit la recherche. Un sondage diagnostique étaye cette lecture : à 25 600 simulations (quatre fois le budget, ≈110 ms par décision), la recherche a obtenu 9/9/6 = 56.2 % contre l'heuristique sur 24 parties, Elo descriptif [−66, +163], de sorte que la recherche dépasse bien l'heuristique quand le budget augmente. Ce constat est cohérent avec la littérature sur Hive, où les moteurs appris perdent contre la recherche simple (de Goede et al., 2022) et où un minimax heuristique fort est difficile à battre (Kampert et al., 2021).

Aucun adversaire n'a été réajusté après cette observation : la population est allée au gel exactement telle que caractérisée, le choix entre 6 400 simulations (caractérisées, 27 ms par décision) et 25 600 (près de la parité, 110 ms par décision, exigeant une re-caractérisation) étant laissé à l'humain.

8.5 Le gel du 9 septembre 2026

La population a été présentée pour gel avec ses configurations, le hachage du fichier de poids, la caractérisation ci-dessus, le sous-choix budgétaire pour l'adversaire de recherche, et une question ouverte : fallait-il inclure le point de contrôle promu par la boucle d'auto-jeu antérieure d'août 2026 comme quatrième membre, dit « réseau gelé » ? L'auteur a approuvé le gel le 9 septembre 2026, l'adversaire de recherche étant fixé à 6 400 simulations et le point de contrôle antérieur exclu ; le journal des décisions consigne l'approbation et les hachages.

La population gelée est exactement constituée des trois agents ci-dessus, B-MCTS à 6 400 simulations, identifiés par le hachage de chaque fichier de configuration et du fichier de poids, ainsi que par le commit du code du moteur au moment du gel. À partir de cet instant, toute retouche de tout adversaire constitue une nouvelle étude. Des commits ultérieurs du moteur ont touché le format des enregistrements, le bras graphe et les entrées/sorties de l'arène, mais aucun moteur de recherche, aucun poids d'évaluation ni aucun réglage par défaut de la recherche ; le test d'épinglage des poids était vert à chaque commit. Les moteurs de référence tiers utilisés pour les tests différentiels des règles (Chapitre 5) ne sont pas membres de la population, et aucun résultat de l'étude n'en dérive.

L'exclusion du point de contrôle antérieur a été délibérée, proposée avant que l'humain n'en décide. Ce point de contrôle est la dix-neuvième génération d'une boucle exécutée en août 2026 avec 2000 parties par génération à 600 simulations complètes et 150 simulations économiques, filtrée par 60 parties contre le tenant et promue à 56.7 %. Il repose sur une seule graine et une seule architecture, sans budget de calcul contrôlé ni population d'adversaires fixée à l'avance. Trois raisons ont été consignées. Sa provenance n'ajoute aucune information contrôlée et invite à la méprise : tout score contre lui ressemblerait à une comparaison avec « l'ancienne IA ». L'exécuter à l'époque dépendait d'un chemin d'inférence en cours de réparation ce jour-là même, ce qui aurait couplé un artefact gelé à un correctif non résolu. Et les trois agents couvrent déjà le plancher et une bande intermédiaire de deux styles distincts. Le point de contrôle est conservé et peut apparaître comme contexte descriptif ; il n'est jamais membre de la population et aucun nombre de ce rapport n'est mesuré contre lui.

Deux limites subsistent. Le volume de caractérisation est de 100 parties par confrontation avec un intervalle en approximation non appariée, ce qui suffit pour situer les adversaires les uns par rapport aux autres mais non pour étayer les conclusions de l'étude ; celles-ci reposent sur l'analyse au niveau des graines de Chapitre 9. Et les cinq cas tactiques, comme le corpus de validation du moteur, n'ont pas été relus par un lecteur externe connaisseur de Hive.

Partie III. Méthodologie

9 Méthodologie expérimentale : un protocole pré-enregistré et gelé

Ce chapitre énonce ce que la comparaison a fixé à l'avance, comment, quand et surtout pourquoi. Le protocole directeur a été gelé en version 1.0 le 10 septembre 2026, avant toute exécution de comparaison et après qu'un pilote eut confirmé chacune des valeurs mesurées qu'il contient ; il déclare que tout changement ultérieur constitue une nouvelle étude, et aucun n'a été fait. Les constantes, les hyperparamètres et les graines sont tabulés dans Annexe C ; les tables brutes par graine et la carte des points de contrôle au seuil dans Annexe D. Un second protocole, version 2.0, a été gelé le 13 octobre 2026 pour l'étude complémentaire rapportée dans Chapitre 15 ; il change le régime d'entraînement et rien d'autre, et y est décrit.

9.1 Pourquoi pré-enregistrer, et pourquoi geler, à petit budget de calcul

Une comparaison sur une seule machine avec une poignée de graines laisse à l'expérimentateur une grande liberté après coup : quel point de contrôle compte comme « le » résultat ; quels adversaires, à quelle force ; quelles graines sont rapportées ; si une partie qui atteint le plafond de coups est une nulle, une défaite ou rien ; quelle métrique devient le titre. Chaque choix, fait une fois les nombres visibles, peut fabriquer ou effacer un effet exactement de la taille qu'une telle étude peut détecter ; le jardin des chemins qui bifurquent n'a besoin d'aucune malhonnêteté consciente. Deux risques supplémentaires sont le réajustement silencieux d'une ligne de base au vu du résultat du test, et une attention asymétrique portée au bras dont on attend la victoire.

Le pré-enregistrement supprime ces degrés de liberté par une seule règle : chaque engagement susceptible de biaiser une observation est pris avant que l'observation n'existe. Cela a signifié ici des artefacts gelés sous un hachage de contenu, chacun daté et approuvé par l'auteur, et un protocole qui les nomme et fixe la métrique, les lectures de budget, l'unité de rééchantillonnage et l'issue qui compterait contre l'hypothèse. Le protocole énonce que l'hypothèse « n'est pas présumée vraie » et qu'« un résultat négatif ou nul est une issue publiable de cette étude ; le travail n'a pas besoin de confirmer H1 pour compter ». La réponse négative finalement obtenue était donc un livrable planifié plutôt qu'un échec à justifier.

Le gel a suivi un ordre fixe (Tableau 23) : la population d'adversaires (Chapitre 8) et les réglages de recherche de l'évaluation le 9 septembre 2026 ; le protocole le 10 septembre 2026, une fois que le pilote eut remesuré les budgets et le plafond de coups avec les réseaux propres à l'étude, de sorte que le texte gelé ne porte aucune valeur de substitution ; les 250 ouvertures partagées et la matrice de comparaison avec sa règle de seuil le même jour. La première exécution de comparaison a démarré le 10 septembre 2026 à 12:42. Geler après le pilote a été le choix de l'auteur : cela a permis de confirmer les budgets sur les bras réels et d'éviter un second gel, qui aurait affaibli ce qu'un gel signifie.

Les règles d'arrêt du plan de recherche ont aussi borné la conception : un moteur incorrect suspend l'entraînement, de sorte que la validation du moteur (Chapitre 5)

précède tout entraînement ; un dépassement de budget réduit les configurations, jamais l'honnêteté ; deux semaines sans résultat interprétable réduisent la portée ; une fois l'ensemble d'évaluation final consulté, la méthode n'est plus ajustée contre lui. Les ablations ont donc été déclarées secondaires et supprimables dès le départ, et aucun bras n'a reçu de seconde architecture lorsque le bras graphe a sous-performé.

9.2 Questions de recherche et hypothèse

Le protocole gelé pose une question primaire et une hypothèse, reproduites telles que gelées.

RQ-H1. À budget d'entraînement comparable, une architecture en graphe apprend-elle une meilleure politique qu'une architecture en grille pour Hive en jeu de base, au sein d'un pipeline d'auto-jeu de type AlphaZero ?

Hypothèse H1. Un réseau de graphe simple à passage de messages, recevant la ruche sous forme de graphe, atteint un score moyen plus élevé contre une population d'adversaires fixe qu'un CNN en grille recevant le cadre 32×32 déplié par parcours en largeur, à budget d'entraînement égal.

Le protocole donne lui-même des raisons de ne pas présumer H1. L'empilement et surtout les destinations candidates *vides* doivent être représentés correctement, et un graphe d'adjacence naïve sur les cellules occupées peut ne pas suffire, parce que les destinations et les contraintes de glissement sont des propriétés de l'espace vide. Les indications antérieures étaient aussi partagées, les réseaux de graphes favorisant la structure à longue portée et les convolutions les motifs locaux dans Hex (Keller et al., 2023).

La règle de rejet, écrite avant toute exécution de comparaison, est citée verbatim ; ses références de section renvoient aux sections propres du protocole sur les lectures de budget et sur les graines et l'incertitude :

> *Ce qui rejetterait H1 [...] : contre la population d'adversaires gelée, sous les* ***deux*** *lectures de budget du §5, le bras graphe ne montre aucun avantage cohérent entre graines (aucun ordre par graine cohérent en sa faveur),* ***et*** *l'intervalle sur la différence de score — calculé avec l'unité de rééchantillonnage énoncée au §6 — exclut un avantage graphe substantiel. Cette issue est rapportée telle quelle : « à ce budget, sur cette variante, la représentation en graphe n'a pas aidé. »*

La règle est conjonctive : l'ordre par graine *et* l'intervalle doivent tous deux ne pas favoriser le bras graphe, de sorte qu'une graine chanceuse ne peut pas sauver l'hypothèse et qu'une graine malchanceuse ne peut pas la rejeter. Elle s'applique sous les deux lectures de budget, et elle fixe l'unité de rééchantillonnage à l'avance. Une limite mérite d'être énoncée clairement : le texte gelé dit « substantiel » sans seuil numérique. Le verdict rapporte donc la plus grande borne supérieure de tous les intervalles graphe–grille, afin qu'un lecteur puisse appliquer tout seuil qu'il juge substantiel ; l'étude n'en revendique aucun qu'elle n'ait pré-enregistré.

Deux questions secondaires suivent. RQ-H2 demande si la comparaison change lorsque le budget est lu par exemple plutôt que par heure ; elle est pré-enregistrée dans la matrice sous la forme des deux lectures de budget. RQ-H3 demande ce que contribuent les composants distinctifs du bras graphe ; ses spécifications d'ablation ont été écrites le

19 septembre 2026, après que le résultat principal fut connu (Chapitre 9.8). La question du protocole conditionnée au temps disponible, la robustesse à la translation et à la symétrie, a été déclarée la première chose à abandonner sous la pression du calendrier, et elle a été abandonnée.

9.3 Conception : deux bras, cinq graines, dix générations

Seuls l'encodage d'état et le corps du réseau diffèrent entre les bras (Chapitre 7) : un CNN résiduel sur le cadre 32×32 déplié par parcours en largeur et centré sur la boîte englobante (1.44 M paramètres) contre un réseau relationnel à passage de messages sur le graphe des cellules (1.47 M, +1.5%). Les deux partagent le moteur de règles, la recherche, le décodeur d'actions partagé, la boucle d'entraînement, les budgets et le harnais d'évaluation. Aucun n'a reçu de recherche d'hyperparamètres.

Chaque exécution comprend 10 générations de 500 parties d'auto-jeu. L'auto-jeu recherche 128 simulations par décision sur un quart aléatoire des décisions et 32 sur le reste, suivant la randomisation du plafond de simulations de Wu (2020), et seules les décisions à plein budget produisent des cibles. Il échantillonne la distribution de visites pendant 12 plis, abandonne sous −0.92 avec un audit sans abandon de 10%, et tronque à 300 demi-coups. Chaque génération entraîne 2 époques au lot 256 et au taux d'apprentissage 0.02 avec la perte de politique masquée sur l'ensemble légal. Un point de contrôle est conservé et le temps mural journalisé à chaque génération, ce qui rend possible la seconde lecture de budget.

Ces valeurs ont été mesurées plutôt que reprises des points de départ du plan de 64 et 128 simulations. Le profilage du 9 septembre 2026 a situé une évaluation du réseau à 2.62 ms sur le fournisseur accéléré, et le pilote du 10 septembre 2026 a confirmé ≈12 s par partie à la génération 0, descendant vers ≈3 s à mesure que le jeu s'affine. Le plafond de 300 demi-coups se situe au-delà de la plus longue partie guidée par la recherche observée (202 demi-coups). Le pilote a montré que l'auto-jeu à initialisation aléatoire tronquait 56.7% des parties à ce plafond, ce qui n'est tolérable que parce que la troncature est une issue rapportée en propre.

Les graines sont venues en deux étapes, toutes deux divulguées. La matrice pré-enregistrait trois graines par bras (1–3 ; graine de base 100 000 × graine) et notait une option à cinq graines « si le budget le permet ». La campagne à trois graines, approuvée le 10 septembre 2026, s'est exécutée séquentiellement sur une seule machine, quatre threads de travail par exécution, sous une estimation de ≈4.6 jours et une garde de 20 Go d'espace disque libre. Après l'analyse des deux lectures de la matrice à trois graines (19 septembre 2026), l'auteur a approuvé le 26 septembre 2026 les graines 4 et 5 pour les deux bras sous trois pré-engagements écrits avant toute nouvelle exécution : (a) les cinq graines par bras entrent dans l'analyse finale quelle que soit la direction des nouvelles graines ; (b) le seuil à temps mural égal reste à sa valeur déjà calculée ; (c) la collecte en deux étapes est divulguée. Des graines ajoutées symétriquement sous une règle inchangée sont une décision de puissance statistique plutôt qu'un canal de réglage ; le verdict à trois graines reste consigné.

9.4 Les deux lectures de budget et le seuil à temps mural égal

Un réseau qui apprend davantage par exemple mais coûte davantage par exemple peut perdre à heures égales, de sorte que le protocole impose deux lectures d'une même campagne. L'asymétrie est réelle : le meilleur fournisseur disponible donne 2.62 ms par évaluation pour le bras grille (accéléré) mais 3.67 ms pour le bras graphe (sur le CPU, puisque ses opérations riches en collecte se rabattent hors de l'accélérateur), et les exécutions graphe ont pris 2.0× le temps mural d'entraînement de la grille (36.7 contre 18.0 h sur cinq graines ; 35.7 contre 18.2 h sur les trois paires originales). L'interaction est rapportée et imputée plutôt que neutralisée par égalisation.

Lecture à exemples égaux. Les deux bras s'entraînent pendant 10 générations × 500 parties avec des réglages de générateur identiques, donc sur le même nombre de parties au même budget de simulation ; chaque exécution est lue à son point de contrôle final (indice de génération 9 dans les journaux d'exécution, qui comptent à partir de 0).

Lecture à temps mural égal. Le seuil (T* dans les tables et les figures) a été défini par une règle de la matrice pré-enregistrée : la médiane du temps mural d'exécution complète des trois exécutions grille, chaque exécution des deux bras étant ensuite lue à son dernier point de contrôle achevé à ce seuil ou avant. La règle donne au bras grille son plein budget par construction, impute au bras graphe son coût réel, et est indépendante des scores. Elle a été exécutée le 16 septembre 2026, cinq des six exécutions étant achevées et la dernière exécution graphe encore en entraînement, à partir des seuls chronomètres de la grille et avant qu'aucun nombre inter-bras n'existe : médiane(18.77, 16.73, 19.07) = 18.77 h. Les chronomètres par génération comptent l'auto-jeu, l'entraînement et l'export ; le temps d'évaluation est exclu pour toutes les exécutions de la même manière. La médiane exacte, qui est le total propre de la première exécution grille, est la valeur opérante. Lors de la régénération des tables pour cinq graines, la constante arrondie a brièvement exclu le dernier point de contrôle de cette exécution à sa borne inclusive ; le rétablissement de la valeur exacte a reproduit exactement l'enregistrement du 16 septembre.

La sélection est mécanique et son issue fait partie du résultat (Tableau 67) : indices de génération 9, 9, 8, 9, 9 pour les exécutions grille et 3, 4, 4, 2, 5 pour les exécutions graphe. À temps mural égal, le bras graphe avait achevé 3–6 de ses dix générations, le bras grille neuf ou dix. Les points de contrôle au seuil ont été évalués après les campagnes (18 septembre 2026 ; 6 octobre 2026 pour l'extension) au même volume de 100 parties ; lorsque le point de contrôle au seuil est le point final, l'évaluation finale est réutilisée.

9.5 Évaluation : population gelée, ouvertures gelées, recherche épinglée

Population d'adversaires. La population comprend trois adversaires, gelés par l'auteur le 9 septembre 2026 avant toute exécution d'entraînement : B-RND, uniforme sur les coups légaux du moteur, à graine fixée ; B-HEU, l'évaluation documentée écrite à la main jouée de manière gloutonne à profondeur 1, poids épinglés par hachage et par un test ; B-MCTS, recherche sans réseau à 6 400 simulations par décision. Le point de contrôle de la boucle de démonstration antérieure a été exclu : sa provenance à graine unique et non contrôlée aurait invité à une lecture comme « l'ancienne IA ». Les 37.5% de l'adversaire de recherche contre l'heuristique ont été diagnostiqués et délibérément non réajustés,

puisque le gel interdit d'ajuster un adversaire au vu des résultats ; aucun adversaire n'a été touché ensuite.

Ouvertures et appariement. Les parties partent de 250 ouvertures uniques et légales de quatre demi-coups, générées à l'aveugle par une marche aléatoire à graine fixée sur les coups légaux du moteur (graine du générateur 20260910), gelées le 10 septembre 2026 sous leur hachage de contenu. La paire *i* de chaque affrontement joue la ligne *i* une fois avec chaque couleur, de sorte que chaque bras, graine, point de contrôle et adversaire fait face à un calendrier d'ouvertures et de couleurs identique (un test du harnais a montré des calendriers identiques à l'octet près pour des agents et des graines d'affrontement différents). Les évaluations finales et au seuil jouent 100 parties appariées par adversaire (ouvertures 0–49, les deux couleurs) ; les évaluations intermédiaires aux générations 4 et 7 jouent 20 parties par adversaire et ne servent qu'aux figures de trajectoires.

Pourquoi 100 parties. Le volume final a été recalibré après les premières évaluations finales : le 16 septembre 2026, la dispersion de graine à graine contre l'heuristique (0.080–0.190, écart-type ≈ 0.056) valait environ deux fois le bruit de parties par appariement à 100 parties (erreur-type ≈ 0.03), de sorte que davantage de parties ne pouvaient pas resserrer substantiellement l'intervalle au niveau des graines. Le volume est resté à 100 ; le seul levier est davantage de graines, ce que l'extension a ensuite fourni.

Recherche épinglée. Chaque évaluation d'un réseau entraîné exécute 400 simulations par décision, sans bruit de Dirichlet à la racine (la valeur par défaut du code, vérifiée par un test) et avec un meilleur coup déterministe ; ces réglages ont été épinglés le 9 septembre 2026 par une configuration versionnée et jamais changés. Des nombres de visites fixes importent parce que calcul d'entraînement et calcul de test se compensent (Jones, 2021) ; un budget d'évaluation flottant laisserait la force mesurée dériver avec la recherche plutôt qu'avec le réseau. Le réseau évalué reçoit la graine 9000 + génération pour les évaluations en cours d'exécution et 9500 pour les ensembles au seuil, B-RND 9101 et B-MCTS 9201 (B-HEU est déterministe). Les parties d'évaluation ne réalimentent jamais l'entraînement ; une vérification d'audit contrôle la séparation.

9.6 Statistiques et politique de troncature

Métrique et unité. La métrique primaire est le score moyen contre la population (victoire 1, nulle 0.5, défaite 0) sur les parties non tronquées d'une cellule (graine, adversaire) ; les données au niveau des parties sont d'abord agrégées en un score par cellule, et la graine est l'unité d'analyse. Les parties jouées par un réseau entraîné ne sont pas des échantillons indépendants de la *méthode* : elles partagent les poids de ce réseau, et la qualité avec laquelle une représentation apprend varie entre exécutions indépendantes bien plus qu'entre parties d'une même exécution. Agréger des milliers de parties d'un seul modèle produit des intervalles précis sur la mauvaise chose (Agarwal et al., 2021) ; aucune table de ce rapport ne le fait.

Intervalles. Les moyennes au niveau des graines et leurs intervalles à 95% sont des bootstrap percentiles à 10 000 rééchantillonnages sur les graines. Pour le contraste graphe–grille, les deux ensembles de graines sont rééchantillonnés indépendamment : les graines d'entraînement ne sont pas appariées entre bras, tandis que l'appariement qui existe bel et bien (ouvertures et couleurs identiques) est maintenu constant entre bras et absorbé dans chaque cellule. Chaque table montre chaque graine ; aucune ne

rapporte une meilleure graine. L'Elo, là où il est imprimé, est descriptif et relatif à cette seule population.

Troncature. Une partie qui atteint le plafond de 300 demi-coups n'est jamais une nulle : les troncatures sont partout une quatrième catégorie d'issue, et chaque table porte le taux de troncature à côté du score. Le score primaire exclut les parties tronquées ; une colonne de sensibilité les compte 0.5 ; deux traitements de bornage comptent chaque partie tronquée du bras testé comme une défaite et comme une victoire ; et la direction du contraste est rapportée sous les quatre traitements. Le traitement « victoire » borne ce que tout plafond plus grand pourrait apporter, ce pour quoi des bornes remplacent une réexécution à plafond plus grand. La politique s'est révélée nécessaire : contre l'adversaire aléatoire, le bras graphe a tronqué 20–57% de ses parties là où le bras grille en a tronqué 0–1%, et, comptée en nulles, cette pathologie aurait disparu dans des demi-points.

9.7 Critères d'exclusion et faits consignés

Une exécution ne peut être exclue que pour une défaillance de processus (un plantage, des données corrompues, ou une erreur du moteur telle qu'un défaut de règles exploité par un agent) ; un mauvais score n'est jamais un critère d'exclusion. Sur la campagne principale, l'extension et les ablations, aucune exécution n'a échoué, n'a été relancée ni n'a été exclue. Deux incidents ont touché la machinerie de mesure. Le 18 septembre 2026, on a constaté que le lanceur d'affrontements perdait ses enregistrements lorsque chaque partie d'un affrontement était tronquée ; un audit n'a montré aucun fichier de campagne affecté, et le défaut a été corrigé avant l'exécution des évaluations au seuil. Le 23 septembre 2026, on a constaté que les trois exécutions de la première ablation, achevées sans défaillance de processus, avaient divergé vers des sorties non finies à la génération 0 ; la divergence a été détectée parce que trois exécutions à graines indépendantes retournaient des évaluations identiques au nombre de parties près. La divergence est rapportée comme le résultat ; les exécutions ne sont pas exclues, mais leurs tables d'évaluation ne sont pas des mesures de force. Un arrêt bruyant sur perte non finie a été ajouté aux boucles d'entraînement le même jour ; les exécutions saines ne sont pas affectées.

9.8 Conception des ablations

Les règles d'ablation ont été fixées avant les exécutions d'ablation : au plus deux ablations, chacune retirant exactement un composant du bras graphe complet pour répondre à une question, à pleine parité avec la référence (graines 1–3, les mêmes budgets, adversaires et ouvertures gelés et évaluation épinglée) ; un changement de capacité inhérent au composant retiré est rapporté, jamais égalisé en changeant un second composant ; et là où plus d'un composant diffère, aucun effet n'est attribué au graphe. A1 remplace les six matrices d'arêtes typées par direction par une seule matrice partagée (adjacence naïve ; 0.54 M paramètres, les matrices typées étant le composant), ce qui teste la prémisse propre du protocole. A2 retire le biais de pooling global de chaque couche (1.37 M). A2 est une substitution : la première ablation du plan retire l'augmentation par symétrie, mais la méthode complète a été fixée sans augmentation le 10 septembre 2026, parce que des transformations de symétrie hexagonale cohérentes

entre deux représentations risquaient de contaminer la comparaison principale, de sorte qu'il n'y avait rien à retirer. La question de substitution a été choisie le 19 septembre 2026 au vu du signal de mécanisme du résultat principal (la force relative du bras graphe résidait dans sa tête de valeur). Il s'agit d'un choix de *question* postérieur au résultat, divulgué ici, sous une analyse inchangée. L'auteur a approuvé les deux ablations à trois graines le 20 septembre 2026.

A1 s'est révélée inentraînable à parité, de sorte qu'un supplément A1′ a été ajouté et explicitement étiqueté à deux composants : arêtes non typées plus un écrêtage global de la norme du gradient à 1.0, le seul changement d'optimiseur de l'étude. Approuvé le 26 septembre 2026, il ne répond qu'à la question « quel score obtient un réseau à adjacence naïve *entraîné* ? » ; aucun nombre d'A1′ n'est attribué au seul typage des arêtes.

9.9 Ressources réellement consommées

Toutes les exécutions ont tourné sur un seul Apple M1 Pro (10 cœurs, 16 Go, macOS 15.3.1), séquentiellement, quatre threads de travail par exécution ; aucun calcul ni aucune donnée payants n'ont été utilisés. Tableau 22 liste les quatre campagnes ; le temps mural d'entraînement par exécution somme les chronomètres par génération (auto-jeu, entraînement, export), parties d'évaluation exclues. Sur les dix exécutions de la comparaison principale, ces totaux allaient de 16.7 à 19.1 h (grille) et de 26.6 à 49.9 h (graphe), 273.5 h en tout ; la campagne à trois graines a occupé la machine ≈7.4 jours, ≈163 h en comptant ses évaluations au seuil ; une partie d'évaluation à 400 simulations prenait ≈23–32 s. Les exécutions de la première ablation sont courtes (7.61–8.01 h) parce qu'une politique non finie joue des parties dégénérées courtes.

Tableau 22 : Calcul consommé par les quatre campagnes sur l'unique machine d'étude (quatre threads de travail par exécution, exécutions séquentielles, aucun calcul payant). Le temps mural par exécution est le temps mural d'entraînement sous une même comptabilité pour chaque campagne (secondes d'auto-jeu, d'entraînement et d'export par génération sommées, parties d'évaluation exclues) ; les exécutions de la première ablation sont courtes parce que leur entraînement avait divergé.

Campagne	Exécutions	Dates (2026)	Temps mural par exécution
Comparaison principale, graines 1–3	6	10 sept. 12:42 → 17 sept. 22:31 ; évaluations au seuil 18 sept. (~11 h)	grille 18.77 / 16.73 / 19.07 h ; graphe 43.07 / 32.71 / 31.28 h
Ablations A1, A2	6	20 sept. 14:08 → 27 sept. 17:13	A1 7.75 / 8.01 / 7.61 h (divergée) ; A2 39.90 / 47.44 / 32.05 h
Extension, graines 4–5	4	27 sept. → 2 oct. ; évaluations au seuil 6 oct. (≈6 h)	grille 18.23 / 17.19 h ; graphe 49.86 / 26.57 h
Supplément A1′	3	2 oct. → 6 oct.	23.89 / 30.35 / 27.68 h

9.10 Ce qui a été fixé avant la première exécution de comparaison, et ce qui est venu après

Tableau 23 situe chaque engagement par rapport à la première exécution de comparaison ; les hachages et les valeurs sont dans Annexe C, l'index de provenance dans Annexe E.

Tableau 23 : Chronologie des engagements de l'étude par rapport à la première exécution de comparaison (10 septembre 2026, 12:42). Les lignes un à cinq ont précédé tout nombre de comparaison ; les lignes suivantes ont été ajoutées ensuite sous le garde-fou énoncé.

Élément	**Fixé le (2026)**	**Moment**	**Garde-fou**
Population d'adversaires ; point de contrôle antérieur exclu	9 sept.	avant l'entraînement	hachages ; jamais touchée
Réglages de recherche de l'évaluation	9 sept.	avant l'entraînement	configuration et test
Protocole v1.0, règle de rejet comprise	10 sept.	après le pilote, avant la première comparaison	hachage de contenu ; changement = nouvelle étude
250 ouvertures partagées	10 sept.	avant la première comparaison	générées à l'aveugle ; hachage de contenu
Matrice de comparaison, règle de seuil comprise	10 sept.	avant la première comparaison	règle indépendante des scores
Valeur du seuil à temps mural égal, 18.77 h	16 sept.	en milieu de campagne, avant tout nombre inter-bras	chronomètres de la grille seulement ; médiane exacte
Volume d'évaluation final, 100 parties par adversaire	16 sept.	après les premières évaluations finales	valeur préconfigurée conservée
Spécifications d'ablation A1, A2	19 sept.	après le résultat principal	règle à un composant ; choix divulgué
Graines 4–5	26 sept.	après l'analyse à trois graines	trois pré-engagements écrits
Supplément A1′	26 sept.	après la divergence d'A1	étiqueté à deux composants

9.11 Matrice question → expérience → résultat

Tableau 24 : Les trois questions de l'étude, l'expérience qui répond à chacune, et la forme et l'emplacement de son résultat. RQ-H1 est la question primaire pré-enregistrée ; RQ-H2 et RQ-H3 sont secondaires.

Question	**Expérience**	**Résultat et emplacement**
RQ-H1 (représentation) : le bras graphe apprend-il une meilleure politique que le bras grille à budget comparable ?	Grille contre graphe, 5 graines par bras, population et ouvertures gelées, lues aux mêmes exemples et au seuil à temps mural égal	Score et incertitude par graine et par adversaire ; contraste graphe–grille avec intervalles bootstrap sur les graines ; verdict sous la règle gelée (Chapitre 11 ; tables brutes dans Annexe D)
RQ-H2 (efficacité) : la réponse change-t-elle par exemple plutôt que par heure ?	La même campagne lue aux points de contrôle finaux, puis aux points de contrôle au seuil	Courbes du score en fonction du temps d'entraînement et table score/coût (Chapitre 12)
RQ-H3 (composants) : que contribuent les composants distinctifs du bras graphe ?	Un composant retiré à la fois à pleine parité, 3 graines chacun (A1 typage des arêtes ; A2 pooling global), plus le supplément A1′ étiqueté à deux composants	Table d'ablation : scores par graine, taux de troncature, intervalles sur les graines contre le bras graphe complet (Chapitre 13)

10 Méthodologie de travail : une recherche humain–IA à portes

Ce chapitre décrit comment l'étude a été menée, comme une collaboration entre un chercheur humain unique et un assistant IA, sous des règles de fonctionnement écrites qui réservent à l'humain toute décision irréversible ou scientifique, et pourquoi cet arrangement a été choisi. Il est la forme développée de la déclaration d'assistance par IA des pages liminaires ; les identifiants qui sous-tendent chaque énoncé fait ici sont listés dans Annexe E.

10.1 Pourquoi une méthode à portes

L'étude a été réalisée par un chercheur unique disposant d'un temps limité et d'un seul ordinateur portable (Apple M1 Pro, 10 cœurs, 16 Go) comme unique ressource de calcul. Un assistant de programmation par IA multiplie le débit sous de telles contraintes, mais introduit un risque spécifique : le texte et le code générés échouent de manière *plausible*, en ce qu'ils paraissent corrects exactement aux endroits que personne ne vérifie, et un assistant n'a aucune mémoire persistante digne de confiance entre les sessions de travail. Une étude produite ainsi peut accumuler des affirmations fluides mais intraçables. La réponse a été de faire de la qualité de la recherche une propriété d'**artefacts de processus** plutôt que de la mémoire ou de la confiance : une entrée de journal pour chaque mesure, des documents gelés identifiés par hachage cryptographique, un registre appariant chaque affirmation à sa preuve et à sa limite, et un journal des décisions en ajout seul dans lequel une décision annulée n'est jamais modifiée mais remplacée par une entrée qui renvoie à elle. Les sessions de travail sont sans état ; chacune reconstruit sa compréhension à partir de ces fichiers, de sorte qu'une session interrompue ne perd rien.

Trois autres choix en découlent. L'assistant est routé par *objectifs* : le plan définit des états finaux, chaque phase est un document exécutable avec des tâches ordonnées, des vérifications d'acceptation et des critères de sortie, et l'assistant optimise pour « la preuve de la phase existe » plutôt que pour « la modification demandée a été faite ». Les catégories d'actions irréversibles sont énumérées comme **portes** et réservées à l'humain, ce qui maintient l'autonomie de l'assistant là où les erreurs sont peu coûteuses et réversibles. Et les engagements précèdent les observations qui pourraient les biaiser (annotations avant la sortie du moteur, épinglage des comportements avant les changements de code, mesures avant les budgets), tandis que les productions substantielles de l'assistant, documents de fonctionnement compris, passent par des passes de vérification dont la consigne est de les *réfuter*. Les sections du rapport sont écrites pendant que le travail se fait, parce qu'un rapport assemblé après coup transforme la mémoire en récit.

10.2 Division du travail

L'humain est le chercheur principal. Il détient les questions de recherche, chaque engagement scientifique (gels du protocole, de la population et des ouvertures), chaque dépense de calcul, tout ce qui quitte la machine, et le dernier mot sur chaque affirma-

tion. La responsabilité scientifique de ce rapport est la sienne seule et ne peut pas être déléguée.

L'assistant IA exécute le plan. L'assistant est Claude (Anthropic), opérant en sessions Claude Code. À partir du plan et des fichiers d'état, il planifie, implémente, teste, mesure, journalise et rédige vers l'état final du plan ; les sessions partent d'une intention (« continue », ou un nom de phase) plutôt que d'une liste de tâches. Il a implémenté les encodeurs, le réseau graphe, le pipeline d'entraînement, les lignes de base et le harnais d'évaluation à l'intérieur du périmètre que fixe chaque document de phase ; épinglé le comportement existant avant de le modifier ; profilé avant de proposer des budgets et piloté avant les campagnes ; journalisé chaque expérience avec hypothèse, commit, configuration, graines, version des données, matériel, durée, coût, métriques, défaillances et interprétation ; et rédigé le rapport au fil du travail. Ses propres décisions sont consignées avec la même discipline, et plusieurs sont des *propositions* explicites qui ne sont devenues contraignantes que par l'approbation de l'auteur à une porte ; la proposition d'exclure le point de contrôle de démonstration antérieur de la population d'adversaires en est un exemple.

Ce que l'assistant ne décide jamais est listé dans les règles de fonctionnement : il ne franchit jamais une porte (aucun gel, aucune dépense, aucune publication, aucun contact ni aucune destruction sans décision humaine consignée) ; il n'écrit jamais un oracle après avoir vu la sortie d'un modèle, parce que cette contamination est irréversible ; il ne produit jamais un nombre intraçable ; il ne touche jamais un artefact gelé, puisque tout changement après gel est par définition une nouvelle étude ; et il ne formule jamais une affirmation sans une ligne dans le registre des affirmations.

10.3 Les six portes

La frontière est tracée mécaniquement plutôt que laissée au jugement. **G-FREEZE** : geler un protocole, une partition ou un ensemble de test, et toute retouche ultérieure d'un artefact gelé. **G-SPEND** : appels payants, achats, plafonds budgétaires, lancement ou relance de calculs longs. **G-PUBLIC** : tout ce qui quitte la machine (envois vers les dépôts distants, choix de licence, publication). **G-RIGHTS** : réutilisation de matériel dont la propriété n'est pas établie. **G-ADMIN** : contact institutionnel. **G-DESTRUCTIVE** : suppression de données, de modèles ou de résultats, écrasement de résultats bruts, arrêt de tâches en cours. À une porte, l'assistant consigne la demande en attente, met cette phase en attente et se route vers un autre travail ; un document de phase disant « faire X » n'est jamais une autorisation de franchir. Tableau 25 liste chaque décision de porte prise par l'auteur et résume le fondement de chacune ; le journal des décisions conserve les approbations de l'auteur dans leur formulation originale.

Tableau 25 : Décisions de porte prises par l'auteur pendant l'étude, dans l'ordre consigné dans le journal des décisions. G-ADMIN n'a pas été franchie ; G-RIGHTS a été résolue sans ambiguïté une fois établies les licences des moteurs de référence ; G-PUBLIC a été franchie une fois, le 9 octobre 2026, pour la publication de la première étude.

Date	Porte	Décision	Fondement et conditions consignés
2026-09-09	G-PUBLIC	Licence choisie (MIT, copyright 2026) ; dépôts distants créés, maintenus **privés** jusqu'à la fin de la recherche	Décidée sur instruction écrite de l'auteur ; les dépôts ne deviendront publics que par une nouvelle

Date	Porte	Décision	Fondement et conditions consignés
			décision à cette porte, une fois la recherche terminée
2026-09-09	G-SPEND / G-DESTRUCTIVE	Boucle d'auto-jeu antérieure définitivement arrêtée ; jamais relancée	Décidée sur instruction écrite de l'auteur ; le point de contrôle de la boucle conservé dans la copie de recherche ; aucune relance permise sans nouvelle décision
2026-09-09	G-FREEZE (revue)	Gel du protocole différé jusqu'à ce que les pilotes aient remesuré les budgets proposés avec les réseaux propres à l'étude	Décidée sur un projet portant des valeurs mesurées étiquetées comme propositions ; le gel programmé après les pilotes afin qu'il consomme des valeurs confirmées plutôt que des propositions
2026-09-09	G-FREEZE	Population d'adversaires gelée : aléatoire légal, heuristique, recherche à 6 400 simulations ; point de contrôle antérieur exclu ; hachages et commit du moteur consignés	Approuvée après le tournoi toutes rondes de caractérisation ; l'adversaire de recherche fixé à 6 400 simulations ; le point de contrôle antérieur exclu ; hachages des configurations et des poids et commit du moteur consignés
2026-09-10	G-FREEZE	Protocole gelé en version 1.0, hachage et commit consignés, sans valeur de substitution	Approuvée sur les valeurs mesurées du pilote ; hachage du protocole et commit consignés ; aucune valeur de substitution laissée
2026-09-10	G-DESTRUCTIVE	Archive de l'espace de travail antérieur à l'étude supprimée directement par l'auteur ; la copie de recherche est devenue l'unique copie du travail antérieur	Exécutée par l'auteur lui-même ; la copie de recherche déclarée unique copie du travail antérieur, ses fichiers antérieurs à l'étude placés sous la porte des actions destructrices
2026-09-10	G-FREEZE	250 ouvertures partagées de 4 demi-coups gelées, générées à l'aveugle à partir d'une graine documentée	Approuvée sur un fichier généré à l'aveugle, sans rien de réglable ; hachages du contenu et du fichier et graine du générateur consignés
2026-09-10	G-SPEND	Campagne principale : 2 bras × 3 graines, 10 générations × 500 parties, ≈4.6 jours estimés	Approuvée au dimensionnement complet parmi les options présentées à partir des coûts par partie mesurés ; exécutions séquentielles et reprenables ; résultats bruts jamais écrasés
2026-09-20	G-SPEND	Campagne d'ablations : A1 et A2, 3 graines chacune à pleine parité, ≈9 jours estimés	Approuvée à pleine parité de trois graines avec la comparaison principale, parmi les dimensionnements présentés à partir des coûts d'exécution mesurés du bras graphe
2026-09-26	G-SPEND	Exécution supplémentaire A1′ à deux composants, à un dimensionnement corrigé de ≈4 jours	Approuvée à un dimensionnement corrigé d'environ quatre jours, après révision de l'estimation initiale ; placée en file d'attente après l'extension ; ses résultats confinés à un supplément à deux composants étiqueté comme tel
2026-09-26	G-SPEND	Comparaison principale étendue à 5 graines par bras, ≈4.8 jours, sous trois pré-engagements énoncés avant toute nouvelle exécution	Approuvée avec trois pré-engagements écrits (analyse sur toutes les graines, seuil inchangé, collecte en deux temps divulguée) ; graines ajoutées symétriquement

Date	Porte	Décision	Fondement et conditions consignés
			aux deux bras sous des réglages identiques
2026-10-09	G-PUBLIC	Dépôt rendu public à l'étiquette v2.0-report avec le rapport dans les deux langues, l'article court et l'archive des enregistrements ; soumission d'une prépublication et billet public de l'auteur approuvés	Décidée sur instruction écrite de l'auteur après le relevé de contrôle final et un nettoyage préalable à la publication ; les autres dépôts du programme restent privés ; les moteurs de référence tiers s'étaient révélés sous licence MIT, ce qui a clos la question des droits sans ambiguïté
2026-10-12	G-SPEND	Pilote de la seconde étude : deux générations × 200 parties par bras sous démarrage à chaud et fenêtre de trois générations, une graine hors des ensembles de graines des deux études	Approuvée pour mesurer le coût du régime et exercer le chemin complet avant le gel ; ses scores n'entrent dans aucune analyse
2026-10-13	G-FREEZE	Protocole v2 gelé : démarrage à chaud, fenêtre de rejeu de trois générations, option A d'hyperparamètres (sans ajustement), graines 11–15, règle du seuil à temps égal ; hachage et commit consignés	Approuvée sur le budget mesuré du pilote ; option B (un petit budget de réglage identique par bras) examinée et écartée ; tout changement ultérieur constitue une nouvelle étude
2026-10-13	G-SPEND	Seconde campagne : 2 bras × 5 graines × 10 générations × 500 parties, ≈295 h estimées, évaluations au seuil comprises	Approuvée après le gel et lancée le même jour sous les règles d'exécution de la première étude (séquentielle, reprenable, garde disque, aucune autre charge sur la machine)

Deux entrées méritent un commentaire plus long que celui de la table. Le report du 9 septembre montre une porte agissant contre la hâte : devant un protocole aux valeurs mesurées, l'auteur a attendu la confirmation des pilotes afin que le gel consomme des nombres confirmés plutôt que des propositions. L'extension à 5 graines montre une décision de puissance a posteriori empêchée de devenir un canal de réglage : avant toute nouvelle exécution, il a été consigné que (a) l'analyse finale utiliserait les cinq graines par bras quelle que soit la direction des nouvelles graines, (b) le seuil à temps mural égal resterait à la valeur calculée le 16 septembre, jamais recalculée après l'observation des résultats, et (c) le rapport divulguerait que les graines 4–5 ont été collectées après l'analyse à 3 graines. Les chapitres de résultats honorent ces trois engagements. Le périmètre de ce rapport a lui-même été une décision de l'auteur, consignée le 9 octobre 2026 avec la contrainte qu'aucun artefact gelé, aucun nombre ni aucune affirmation ne pouvait changer.

10.4 Principes scientifiques appliqués à chaque phase

Au-delà des portes, les règles de fonctionnement lient chaque phase, quel que soit celui qui l'exécute.

1. **Les oracles avant la sortie du modèle.** Les positions de test et les cas de référence sont annotés avant que tout moteur ou modèle ne les voie ; ensuite, une attente non biaisée ne peut plus être écrite.

2. **Caractériser avant de changer.** Le comportement existant est épinglé par des tests avant modification ; un épinglage qui échoue est consigné comme un constat plutôt que corrigé en silence.
3. **Tout est journalisé** : identifiant, date, hypothèse, commit, configuration, graine, version des données, matériel, durée, coût, métriques, chemins des artefacts, défaillances, interprétation.
4. **Aucun nombre intraçable.** Chaque chiffre renvoie à une entrée de journal ou à un résultat brut ; les nombres hérités du plan sont étiquetés propositions de planification jusqu'à leur mesure.
5. **Les résultats négatifs sont conservés** et rapportés aux côtés des meilleures exécutions.
6. **Des lignes de base équitables.** Les méthodes comparées reçoivent le même modèle, le même budget et le même nombre de tentatives ; une ligne de base fantoche invalide une étude.
7. **La troncature n'est pas une nulle.** Une partie arrêtée au plafond de coups est une issue distincte, rapportée séparément et soumise à une analyse de sensibilité.
8. **Honnêteté multi-graines.** Aucune conclusion ne repose sur une seule graine ; les résultats par graine et l'incertitude sont rapportés, la graine étant l'unité de rééchantillonnage.
9. **Le travail antérieur est en lecture seule.** Le matériel antérieur à l'étude n'est jamais modifié sur place ; les importations portent une note de provenance ; les droits incertains vont à G-RIGHTS.
10. **Un registre des affirmations.** Chaque affirmation a une ligne (affirmation, preuve, section, limite) ou n'apparaît pas.
11. **Écrire au fil du travail.** Une phase n'est pas close tant que sa section de rapport n'existe pas : le protocole avant les expériences, la méthode pendant l'implémentation, les résultats uniquement à partir de tables brutes gelées.

10.5 Ce que la discipline a attrapé

Les règles de fonctionnement exigent un journal en ajout seul de chaque incident où la méthode a attrapé quelque chose, manqué quelque chose, changé une issue par une porte, ou coûté un surcoût réel ; aucune entrée n'est forcée. Tableau 26 rassemble ces entrées avec les incidents consignés dans les journaux d'expériences.

Tableau 26 : Incidents consignés par le journal méthodologique et les journaux d'expériences pendant l'étude : ce qui s'est passé, quelle règle ou vérification l'a attrapé, et ce qui a suivi.

Date	Incident	Comment il a été attrapé	Conséquence
2026-09-09	La première version des documents de phase contenait 33 défauts, dont un interblocage validation↔gel et deux tâches inexécutables	passe de vérification à consigne de réfutation	28 corrections, re-vérifiées propres avant toute exécution de recherche
2026-09-09	Une position critique annotée à la main était en désaccord avec le moteur	attentes consignées par commit avant l'exécution	résolu *contre* le corpus (cas de transit One-Hive) ; trace conservée
2026-09-09	Les générateurs d'auto-jeu antérieurs assimilaient le plafond de 300 demi-coups à une nulle	principe de troncature appliqué rétroactivement à la revue	corrigé avant la phase de pipeline ; la troncature est une issue distincte partout

Date	Incident	Comment il a été attrapé	Conséquence
2026-09-09	La frontière Python–Rust était sur le point de devenir un débat de préférences	la mesure d'abord : 21.7 µs par aller-retour contre ≥40 ms par décision	aucune liaison en processus construite
2026-09-09	Erreur de mise en place (ordre des pièces) dans l'ensemble de test tactique	cinq cas annotés à la main et consignés par commit avant d'être exécutés	moteur juste, mise en place fausse ; journalisé
2026-09-09	L'adversaire de recherche n'a obtenu que 37.5% contre l'heuristique ; un sondage à budget 4× a atteint 56.2%	diagnostiqué à la caractérisation	délibérément **non** réajusté : le gel de la population interdit le réglage a posteriori
2026-09-09	Deux sessions de travail concurrentes ont attribué le même identifiant de décision (**manqué** sur le moment)	recoupement ultérieur	entrée renumérotée ; règle de l'écrivain unique pour les fichiers d'état partagés
2026-09-09	Surcoût : les documents de fonctionnement ont consommé une session de travail entière (≈1M jetons d'agent, 17 sous-agents) avant tout travail de recherche	aucune	coût fixe concentré en amont, accepté
2026-09-10	La première évaluation pilote a retourné un identique 0/2/28 contre les trois adversaires, aléatoire légal compris	alarme des résultats identiques ; ordre de diagnostic règles → signes → recherche → données	une boucle shell avait passé les options de chaque adversaire comme un seul argument : chaque match était contre l'heuristique ; réexécution
2026-09-18	La reproduction d'une paire entièrement tronquée pour une figure d'échec n'a retourné aucun enregistrement	chaque position sélectionnée doit être reproduite et vérifiée contre son enregistrement	chemin latent de perte d'enregistrements corrigé ; audit : aucune évaluation de campagne affectée
2026-09-23	Les trois graines de l'ablation A1 ont retourné des évaluations identiques au nombre de parties près	la même alarme des résultats identiques	l'entraînement avait divergé vers NaN à la génération 0 et s'était auto-joué sur NaN pendant neuf générations de plus ; gardes contre les valeurs non finies ajoutées ; divergence conservée comme constat
2026-10-09	La carte régénérée des points de contrôle à temps égal pour les graines 1–3 différait de l'original journalisé	sorties régénérées vérifiées contre le journal avant usage	une constante de seuil arrondie avait rompu une borne inclusive ; corrigé, original reproduit exactement
2026-10-09	La légende générée du contraste entre bras indiquait encore « 3 par bras » sous des nombres à 5 graines	passe de rendu page par page	le générateur dérive le nombre de graines et réécrit le fichier en entier ; chaque nombre identique à l'octet près
2026-10-09	Nombres de graines périmés dans deux chapitres ; limites « en attente » périmées dans le registre des affirmations	balayage des valeurs de substitution avant l'assemblage	corrigé dans les deux langues ensemble
2026-10-09	Le générateur de la table des coûts divisait le temps mural de cinq exécutions par 3 (imprimant 30.0 h / 61.2 h pour 18.0 h / 36.7 h) et portait une ligne de débit d'entraînement	chaque nombre re-dérivé de sa source brute avec une définition explicite lors de la rédaction de ce rapport	générateur corrigé (référence journalisée 274 / 138 pos/s au lot 128, conditions énoncées) ; aucun score, intervalle ni affirmation affecté ; des citations de source par para-

Date	Incident	Comment il a été attrapé	Conséquence
	(≈770 / ≈195 pos/s) mesurée en conditions pilotes mais présentée comme débit de campagne		graphe et un vérificateur de traçabilité au niveau des jetons encadrent désormais le rapport
2026-10-09	La différence de capacité « +2.1% » citée dans chaque document était le rapport des comptes de paramètres arrondis (1.47/1.44 M) ; les comptes exacts donnent +1.5%	la tabulation couche par couche des architectures a recompté les paramètres à partir du code	corrigé partout sauf dans les journaux en ajout seul ; les comptes exacts sont désormais un artefact de résultats généré ; aucun verdict n'en dépend
2026-10-12	La vérification d'estampille de la batterie pré-entraînement a échoué sur l'ensemble de segments fenêtré de la seconde étude : des enregistrements de deux générations face à un seul manifeste	l'épinglage existant, exécuté sur le nouveau régime avant le gel du protocole	la vérification généralisée aux manifestes de chaque génération de la fenêtre plutôt qu'assouplie ; elle passe toujours sur les segments de la première étude et échoue toujours avec un mauvais manifeste
2026-10-27	Le seuil « substantiel » de la règle de rejet, laissé non chiffré dans les deux protocoles, est devenu opérant : la plus grande borne supérieure de la seconde étude (+0.090) n'est plus assez petite pour que le verdict soit indépendant d'un seuil	application de la règle telle que gelée	verdict énoncé avec ses bornes et les seuils sous lesquels il tient ; un seuil chiffré à pré-enregistrer dans toute étude successeur

Aucune de ces prises n'a demandé d'intuition ; chacune est venue d'une vérification mécanique. Une alarme (des conditions indépendantes ne peuvent pas s'accorder au nombre de parties près) a trouvé un défaut de harnais dans le pilote et une divergence numérique silencieuse dans une ablation ; une règle (consigner l'attente, puis exécuter) a trouvé une erreur d'annotation et une erreur de mise en place le même jour ; l'étape de reproduction-et-vérification derrière une figure a trouvé un chemin de perte d'enregistrements avant que les évaluations à temps égal (où les premiers points de contrôle du graphe peuvent réellement tronquer chaque partie) ne l'aient déclenché. Deux entrées sont des *effets de porte* : le gel de la population a visiblement empêché un réajustement de l'adversaire de recherche qui aurait flatté les résultats, et la règle mesurer-avant-de-s'engager a dissous un débat d'architecture avec un seul nombre. À cette échelle, l'erreur de harnais a été une menace plus grande que le bruit statistique, et seule la vérification mécanique l'a trouvée ; le registre des affirmations porte ceci comme une observation de processus issue d'une seule étude, dont la limite énoncée est qu'il n'existe aucun contrefactuel de ce qu'un flux de travail sans portes aurait attrapé.

Le volet coût est consigné avec la même honnêteté : le coût fixe de la méthode a été concentré en amont, et l'entrée « manqué » montre le premier mode de défaillance du fonctionnement multi-sessions : les fichiers d'état partagés exigent une discipline d'écriture autant qu'une discipline de lecture. La méthode n'a pas empêché les erreurs dans les brouillons ; elle les a fait remonter avant cette version par des passes de traduction, des vérifications d'identité numérique entre les deux versions linguistiques, des rendus de pages et des relectures adversariales. Une vérification comparant deux langues ne peut toutefois pas voir une obsolescence partagée par les deux. Le dernier incident a durci le standard : un nombre présent dans un fichier n'est pas pour autant

pourvu de provenance ; le fichier doit lui-même le dériver des données brutes selon une définition énoncée, et c'est pourquoi chaque paragraphe de ce rapport porte une citation de source vérifiée mécaniquement.

10.6 Ce que l'assistant n'a pas fait, et la responsabilité de l'auteur

L'assistant n'a choisi aucune hypothèse, n'a rien gelé, n'a rien dépensé, n'a rien publié, n'a rien supprimé et n'a décidé d'aucune affirmation ; il n'a écrit aucune attente de test après avoir vu la sortie du moteur et n'a pas recalculé le seuil à temps mural égal après que les résultats ont été connus. Les limites de l'arrangement sont consignées elles aussi : l'assistant peut mal lire le plan, et les documents de phase sont son interprétation (le journal des décisions consigne les endroits où la réalité les a corrigés) ; l'auto-vérification adversariale reste une auto-vérification au niveau du programme, de sorte que la relecture humaine externe d'un sous-ensemble d'annotations et du rapport, prévue dans le plan, n'est pas remplacée par elle ; et les données d'entraînement de l'assistant peuvent recouper la littérature publique sur Hive, de sorte qu'aucune affirmation de la forme « non vu par le modèle » n'est faite nulle part dans ce rapport.

Déclaration d'assistance par IA. L'implémentation, l'exécution des expériences, la journalisation et la rédaction de cette étude ont été réalisées avec l'assistance substantielle d'un agent de programmation par IA (Claude Code, Anthropic), opérant sous la méthodologie à portes décrite dans ce chapitre. Chaque décision scientifique, chaque engagement gelé et chaque dépense ont été pris par l'auteur à une porte, comme consigné dans Tableau 25 ; chaque affirmation a été formulée ou vérifiée par l'auteur, qui a personnellement vérifié les conclusions qu'il signe et assume l'entière responsabilité des résultats. La formulation exacte de cette déclaration suivra les règles du lieu de diffusion au moment de la diffusion.

Partie IV. Résultats

11 Résultats : la comparaison principale (RQ-H1)

Ce chapitre rapporte la comparaison pré-enregistrée exactement comme le protocole gelé l'a définie : deux bras, cinq graines indépendantes chacun, évalués contre la population gelée de trois adversaires sur les 250 ouvertures gelées, sous deux lectures de budget. Chaque table montre chaque graine. Les scores excluent les parties tronquées, qui sont rapportées dans leur propre colonne ; les intervalles sont des intervalles bootstrap percentiles à 95% sur les graines (10 000 rééchantillonnages). Les procédures statistiques et les tables brutes avec toutes les colonnes auxiliaires se trouvent dans Annexe D ; la provenance de chaque nombre est dans Annexe E.

11.1 Le verdict en un paragraphe

Sous les deux lectures de budget, le bras graphe ne montre aucun avantage cohérent entre graines contre aucun adversaire, et chaque intervalle sur la différence de score graphe−grille exclut un avantage graphe substantiel : la plus grande borne supérieure sur les six contrastes est +0.028. La règle de rejet pré-enregistrée se déclenche donc, et **H1 est rejetée**. À ce budget, sur Hive en jeu de base, la représentation en graphe n'a pas aidé : elle a obtenu un score plus faible contre deux des trois adversaires gelés sous les deux lectures, et pas meilleur contre le troisième. Le reste du chapitre présente la preuve derrière ce paragraphe, graine par graine.

11.2 Lecture 1 : nombre égal d'exemples d'entraînement

Dans la première lecture, les deux bras sont comparés à leurs points de contrôle de la génération 10 : chaque exécution a alors engendré dix générations de 500 parties d'auto-jeu sous des budgets de simulation identiques et s'est entraînée dessus, de sorte que les deux bras ont consommé le même nombre d'exemples d'entraînement.

Tableau 27 : Lecture à exemples égaux. Score au point de contrôle final (génération 10) de chaque exécution contre chaque adversaire gelé ; 100 parties appariées à couleurs échangées par adversaire et par exécution sur les 250 ouvertures gelées, à 400 simulations par coup sans bruit d'exploration. Score = moyenne sur les parties décidées (victoire 1, nulle 0.5, défaite 0). Troncature = part des 100 parties arrêtées au plafond de 300 demi-coups ; aucune partie contre B-HEU ou B-MCTS n'a été tronquée par l'un ou l'autre bras.

Bras	Graine	Score B-RND	Troncature B-RND	Score B-HEU	Score B-MCTS
grille	1	0.995	0%	0.150	0.125
grille	2	0.975	1%	0.080	0.125
grille	3	0.990	1%	0.190	0.075
grille	4	0.949	11%	0.105	0.085
grille	5	0.995	0%	0.120	0.210
graphe	1	0.733	57%	0.025	0.115
graphe	2	0.981	20%	0.050	0.125
graphe	3	0.722	55%	0.090	0.100

Bras	Graine	Score B-RND	Troncature B-RND	Score B-HEU	Score B-MCTS
graphe	4	0.688	44%	0.125	0.120
graphe	5	0.935	23%	0.035	0.115

Contre l'adversaire aléatoire légal, chaque graine grille obtient au moins 0.949 tandis que les graines graphe s'étendent de 0.688 à 0.981 ; contre l'heuristique, les graines grille s'étendent de 0.080 à 0.190 et les graines graphe de 0.025 à 0.125 ; contre l'adversaire de recherche, les deux bras se recouvrent (0.075–0.210 contre 0.100–0.125). Le bras graphe tronque 20–57% de ses parties contre l'adversaire aléatoire ; le bras grille 0–11%.

Tableau 28 : Lecture à exemples égaux. Score moyen au niveau des graines par bras et par adversaire, avec l'intervalle bootstrap à 95% sur les cinq graines.

Bras	vs B-RND	vs B-HEU	vs B-MCTS
grille	0.981 [0.964, 0.994]	0.129 [0.098, 0.165]	0.124 [0.087, 0.168]
graphe	0.812 [0.710, 0.920]	0.065 [0.034, 0.100]	0.115 [0.107, 0.121]

11.3 Lecture 2 : temps mural égal sur la même machine

Dans la seconde lecture, chaque exécution est prise au dernier point de contrôle qu'elle avait achevé dans le seuil à temps mural égal de 18.77 heures, la médiane du temps mural total des trois exécutions grille originales, calculée le 16 septembre 2026 avant toute comparaison inter-bras. Parce que les générations du bras graphe sont environ deux fois plus coûteuses sur cette machine, le seuil sélectionne la génération 9 ou 10 pour les exécutions grille (le point de contrôle de la génération 10 est atteint dans le seuil par quatre des cinq) et les générations 3 à 6 pour les exécutions graphe : les graines grille 1–5 sont lues aux générations 10, 10, 9, 10 et 10 ; les graines graphe 1–5 aux générations 4, 5, 5, 3 et 6 (en comptant le réseau initial comme génération 1 ; dans les journaux d'exécution, ces points de contrôle sont étiquetés gen009/gen009/gen008/gen009/gen009 et gen003/gen004/gen004/gen002/gen005).

Tableau 29 : Lecture à temps mural égal. Score de chaque exécution à son dernier point de contrôle achevé dans le seuil de 18.77 heures, contre chaque adversaire gelé ; même évaluation que Tableau 27 (100 parties appariées par adversaire, 250 ouvertures gelées, 400 simulations, sans bruit). Troncature = part des parties au plafond de 300 demi-coups ; aucune contre B-HEU ou B-MCTS.

Bras	Graine	Score B-RND	Troncature B-RND	Score B-HEU	Score B-MCTS
grille	1	0.995	0%	0.150	0.125
grille	2	0.975	1%	0.080	0.125
grille	3	0.939	1%	0.145	0.080
grille	4	0.949	11%	0.105	0.085
grille	5	0.995	0%	0.120	0.210
graphe	1	0.798	58%	0.045	0.075
graphe	2	0.926	39%	0.055	0.110
graphe	3	0.713	53%	0.060	0.105
graphe	4	0.631	39%	0.100	0.150
graphe	5	0.980	24%	0.045	0.095

Tableau 30 : Lecture à temps mural égal. Score moyen au niveau des graines par bras et par adversaire, avec l'intervalle bootstrap à 95% sur les cinq graines.

Bras	**vs B-RND**	**vs B-HEU**	**vs B-MCTS**
grille	0.971 [0.950, 0.991]	0.120 [0.098, 0.142]	0.125 [0.090, 0.168]
graphe	0.810 [0.697, 0.922]	0.061 [0.047, 0.081]	0.107 [0.087, 0.130]

Le tableau de la première lecture persiste à temps égal : les moyennes du bras grille sont plus élevées contre les adversaires aléatoire et heuristique et se recouvrent contre l'adversaire de recherche ; la troncature du bras graphe contre l'adversaire aléatoire est de 24–58%.

11.4 Le contraste entre bras

Tableau 31 : Le contraste pré-enregistré, soit la différence des scores moyens au niveau des graines (bras graphe moins bras grille), avec l'intervalle bootstrap à 95% sur les graines (cinq graines indépendantes par bras, rééchantillonnées indépendamment, 10 000 rééchantillonnages). Une valeur négative favorise le bras grille.

Adversaire	**Exemples égaux : graphe – grille**	**Temps mural égal : graphe – grille**
B-RND (aléatoire légal)	−0.169 [−0.272, −0.062]	−0.161 [−0.278, −0.048]
B-HEU (heuristique)	−0.064 [−0.111, −0.017]	−0.059 [−0.087, −0.029]
B-MCTS (recherche, 6 400 simulations)	−0.009 [−0.055, +0.028]	−0.018 [−0.066, +0.025]

Quatre des six contrastes excluent zéro, tous en faveur du bras grille ; les deux contrastes contre l'adversaire de recherche chevauchent zéro.

Contre l'adversaire aléatoire légal, la différence graphe−grille est de −0.169 [−0.272, −0.062] à exemples égaux et de −0.161 [−0.278, −0.048] à temps égal ; contre l'heuristique, −0.064 [−0.111, −0.017] et −0.059 [−0.087, −0.029] ; contre l'adversaire de recherche, −0.009 [−0.055, +0.028] et −0.018 [−0.066, +0.025]. La plus grande borne supérieure de tout intervalle est +0.028, soit le plus grand avantage graphe que les données laissent possible sous la lecture et l'adversaire les plus favorables.

La règle de rejet gelée exigeait, sous les deux lectures, l'absence d'un avantage graphe cohérent entre graines ainsi que des intervalles excluant un avantage graphe substantiel. Les deux conditions sont remplies : aucun adversaire ne montre un ordre par graine cohérent en faveur du bras graphe (Tableau 27, Tableau 29), et aucun intervalle n'admet un avantage supérieur à +0.028. H1 est rejetée. Le résultat est cohérent entre graines et tient sous les deux égalisations de budget ; il ne repose pas sur un seul intervalle.

Cinq graines par bras bornent la statistique : contre l'adversaire de recherche, où les deux bras obtiennent un score autour de 0.1, les données ne peuvent pas distinguer les bras. Le rejet concerne ce réseau relationnel simple à passage de messages, à cette capacité et à ce budget ; il ne dit rien des représentations en graphe à plus grands budgets ou avec d'autres architectures.

11.5 Comment l'analyse à cinq graines se rapporte à l'analyse à trois graines

Le protocole pré-enregistrait trois graines par bras avec une option de cinq. L'analyse à trois graines du 19 septembre 2026 rejetait déjà H1 sous les deux lectures (contrastes contre l'adversaire aléatoire −0.175 [−0.268, −0.007] et −0.158 [−0.254, −0.050] ; contre l'heuristique −0.085 [−0.143, −0.025] et −0.072 [−0.100, −0.028] ; contre l'adversaire de recherche +0.005 [−0.020, +0.035] et −0.013 [−0.040, +0.015]). Les graines 4 et 5 ont ensuite été ajoutées aux deux bras sous des pré-engagements écrits pris le 26 septembre 2026 : l'analyse finale utiliserait les cinq graines selon la même règle, le seuil resterait inchangé, et la collecte en deux étapes serait divulguée. L'extension a resserré quatre des six intervalles, absorbé l'unique paire de graines où le graphe faisait mieux observée contre l'heuristique (graine graphe 4 à 0.125 contre graine grille 4 à 0.105) et le meilleur score contre l'adversaire de recherche de toute exécution (graine grille 5 à 0.210), et n'a pas changé le verdict. Les tables à cinq graines ci-dessus sont les nombres finaux de l'étude.

11.6 Le plafond de coups peut-il sauver l'hypothèse ?

La troncature a été définie comme une issue distincte avant la première exécution, et le protocole exigeait que le résultat primaire soit recalculé sous des traitements alternatifs des parties tronquées. Le traitement le plus favorable au bras graphe compte chaque partie tronquée comme une victoire pour le bras testé, ce qui constitue une borne supérieure de ce que tout plafond plus grand pourrait apporter. Sous cette borne, calculée à partir des enregistrements par partie des trois graines originales, la différence graphe−grille contre l'adversaire aléatoire est encore de −0.072 à exemples égaux et de −0.058 à temps égal ; la direction du contraste est inchangée sous chaque traitement (exclues, comptées 0.5, comptées comme défaites, comptées comme victoires). La valeur du plafond ne peut donc pas renverser le verdict ; elle ne peut qu'en changer l'ampleur.

11.7 Où se loge le déficit

Mettre les trois adversaires côte à côte montre où se loge le déficit : il est le plus grand contre l'adversaire le plus faible et absent contre le plus fort. Contre l'aléatoire légal, où la force consiste surtout à achever un encerclement auquel rien ne résiste, le bras graphe gagne du matériel puis échoue à convertir ; la colonne de troncature est la trace de cet échec. Contre l'heuristique, qui attaque directement la reine, la défense du bras graphe est plus faible que celle du bras grille. Contre l'adversaire de recherche, les deux bras sont dans un régime de scores bas où dix générations d'auto-jeu à petit budget n'ont pas produit une attaque que l'un ou l'autre bras puisse soutenir. Chapitre 14 examine ces mécanismes sur des parties individuelles ; Chapitre 16 pèse les explications.

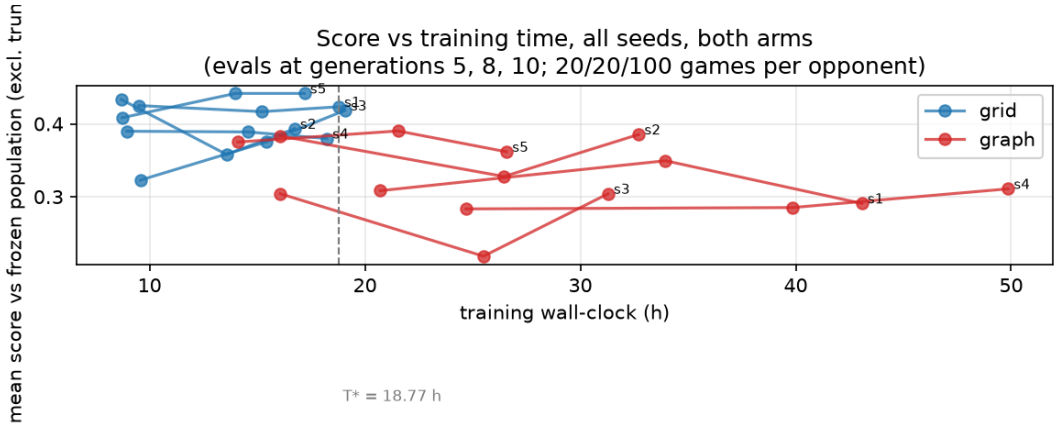


Fig. 5 : Score moyen contre la population gelée (moyenne des trois adversaires, troncatures exclues) en fonction du temps mural d'entraînement cumulé, pour les dix exécutions principales ; évaluations aux générations 5, 8 et 10 (20, 20 et 100 parties par adversaire). Bleu : bras grille ; rouge : bras graphe ; une ligne par graine. La ligne verticale en pointillés marque le seuil à temps mural égal de 18.77 heures.

Fig. 5 montre les deux lectures à la fois : à tout temps mural donné, le bras grille se situe au-dessus du bras graphe, et au seuil les exécutions graphe ont achevé moins de la moitié de leurs générations. Chapitre 12 quantifie le volet coût.

12 Résultats : coût et efficacité (RQ-H2)

La deuxième question de recherche demande comment les deux représentations se comparent lorsque le budget est lu comme un nombre d'exemples d'entraînement et lorsqu'il est lu comme un temps sur le même matériel. Le protocole exigeait les deux lectures et exigeait de chaque exécution qu'elle rapporte son temps mural, son matériel, le nombre d'états d'entraînement consommés et les simulations par décision. Ce chapitre rapporte ces coûts et met les deux lectures côte à côte.

12.1 Ce que coûte chaque bras sur la machine d'étude

Tableau 32 : Résumé des coûts et des scores par bras (H-T2). Le temps mural d'entraînement couvre la génération en auto-jeu et l'entraînement pour les dix générations d'une exécution, parties d'évaluation exclues, moyenné sur les cinq graines ; le coût d'auto-jeu équivalent le divise par les 5 000 parties d'une exécution. Les chiffres d'inférence et de débit d'entraînement sont des mesures de référence du 10 septembre 2026 sur la machine d'étude (Apple M1 Pro, 10 cœurs, 16 Go). Score contre la population = moyenne, sur les trois adversaires gelés, du score moyen au niveau des graines (troncatures exclues).

Métrique	Bras grille	Bras graphe
Paramètres	1.44 M	1.47 M (+1.5%)
Inférence au meilleur fournisseur, lot 1	2.62 ms (accélérateur neuronal)	3.67 ms (CPU)
Temps mural d'entraînement par exécution, moyenne de 5 graines (10 générations × 500 parties)	18.0 h	36.7 h (2.0×)
Coût d'auto-jeu équivalent	13.0 s/partie	26.4 s/partie
Référence de débit d'entraînement (lot 128, avant + arrière)	274 pos/s	138 pos/s
Score contre la population, lecture à exemples égaux	0.411	0.331
Score contre la population, lecture à temps mural égal (seuil 18.77 h)	0.405	0.326

Les deux réseaux sont appariés en taille. L'évaluation unitaire du bras graphe est 1.4× plus lente à son meilleur fournisseur, son débit d'entraînement est la moitié de celui du bras grille, et une exécution complète lui coûte le double de temps mural. Par exécution, les cinq graines grille ont pris 18.77, 16.73, 19.07, 18.23 et 17.19 heures de temps mural d'entraînement ; les cinq graines graphe ont pris 43.07, 32.71, 31.28, 49.86 et 26.57 heures.

Le rapport de temps mural mesuré est de 2.0× (36.7 h contre 18.0 h en moyenne). Au stade du dimensionnement, le rapport avait été estimé à 1.7× ; le rapport mesuré sur les cinq premières exécutions achevées était de 2.1×, et l'estimation a été remplacée par le chiffre mesuré dans toute la comptabilité budgétaire avant le calcul de toute comparaison.

La génération en auto-jeu domine le temps mural des deux bras ; dans le pilote, la génération 0 a pris environ 60 minutes d'auto-jeu contre environ 3.5 minutes d'entraînement. Le surcoût du bras graphe se paie donc surtout dans la recherche, par son inférence par position plus lente et son exécution sur CPU. L'asymétrie est une propriété du matériel de déploiement : le réseau convolutif se projette proprement sur l'accélérateur neuronal de la machine (2.62 ms par évaluation), tandis que le réseau graphe, riche en opérations

de collecte (gather), se replie sur le CPU pour une large part de ses opérations et s'y exécute le plus vite (3.67 ms sur CPU contre 9.84 ms sur l'accélérateur, avec 147 nœuds d'opérateurs pris en charge sur 287). Le protocole a choisi de rapporter et d'imputer cette asymétrie plutôt que de la neutraliser par égalisation. La lecture à temps mural égal reflète ainsi ce que chaque représentation coûte sur cette machine, tandis que la lecture à exemples égaux n'en est pas affectée.

Sur un autre accélérateur, le rapport pourrait changer dans l'une ou l'autre direction ; sur CPU seul, le réseau graphe est le plus rapide des deux à l'inférence (3.67 ms contre 23.5 ms). Les chiffres de coût de ce chapitre sont donc spécifiques à ce matériel par construction ; les comptes de paramètres et les chiffres de débit en sont la partie portable.

12.2 Les deux lectures côte à côte

Sous la lecture à exemples égaux, les deux bras sont comparés après dix générations ; le score du bras grille contre la population est de 0.411 et celui du bras graphe de 0.331. Sous la lecture à temps mural égal, chaque exécution est prise à son dernier point de contrôle dans les 18.77 heures (génération 9 ou 10 pour les exécutions grille, génération 3 à 6 pour les exécutions graphe), et les scores contre la population sont de 0.405 et 0.326. Les deux lectures donnent donc le même ordre et presque le même écart : 0.081 à exemples égaux et 0.079 à temps égal, en faveur du bras grille.

La raison pré-enregistrée de lire le budget deux fois était la possibilité d'une représentation qui apprend davantage par exemple mais moins par heure, auquel cas les deux lectures seraient en désaccord et toutes deux devraient être rapportées. Ici elles concordent : le bras graphe apprend moins par exemple et aussi moins par heure. La lecture à temps égal ne crée pas le déficit ; elle accentue un déficit que la lecture par exemple montre déjà, puisqu'à l'intérieur du seuil le bras graphe achève moins de la moitié de ses générations.

Le score contre la population moyenne trois adversaires de difficulté très différente ; il n'est donc qu'un résumé pour cette comparaison et masque la structure par adversaire du résultat ; les tables par adversaire de Chapitre 11 constituent la preuve primaire. Les scores aux générations intermédiaires (5 et 8) proviennent d'évaluations à 20 parties dimensionnées pour le suivi et n'entrent dans aucun intervalle.

12.3 Le score en fonction du temps

Fig. 5 dans Chapitre 11 trace le score contre la population de chaque exécution en fonction de son temps mural d'entraînement cumulé. Deux traits des courbes répondent directement à RQ-H2. Premièrement, à chaque temps mural auquel les deux bras disposent d'une évaluation, chaque exécution grille se situe au-dessus de chaque exécution graphe ; les bandes des bras ne se croisent pas. Deuxièmement, les courbes des exécutions grille s'aplatissent après leurs premières évaluations (contre l'adversaire aléatoire, elles sont près du plafond dès la génération 5), tandis que les courbes des exécutions graphe continuent de bouger de façon non monotone tout au long de leurs exécutions plus longues ; au seuil, les exécutions graphe ont eu 3 à 6 générations et sont encore dans le régime où la variation d'une graine à l'autre domine. Dans ce budget, le temps supplémentaire a donné au bras graphe davantage de générations sans produire de croisement.

12.4 Ressources consommées par l'étude

La comparaison principale a consommé dix exécutions d'entraînement totalisant environ 274 heures de temps mural d'entraînement (cinq exécutions grille, 90.0 h ; cinq exécutions graphe, 183.5 h), plus les parties d'évaluation (100 parties par adversaire et par point de contrôle à environ 23–32 secondes par partie à 400 simulations, pour les points de contrôle final et au seuil de chaque exécution) et les évaluations de suivi aux générations 5 et 8. Les trois cellules d'ablation ont ajouté neuf exécutions du bras graphe (temps mural d'entraînement sous la même comptabilité : A1 7.61–8.01 h chacune parce que les réseaux divergents jouaient des parties courtes ; A2 32.05–47.44 h ; A1′ 23.89–30.35 h). Tout le calcul s'est exécuté séquentiellement sur un seul ordinateur portable maintenu éveillé, sans ressources infonuagiques ni ressources payantes ; les campagnes s'étendent du 10 septembre au 6 octobre 2026.

13 Résultats : ablations par composant (RQ-H3)

Ce chapitre répond à la troisième question de recherche : lequel des deux composants distinctifs du bras graphe, ses six relations d'arêtes typées par direction et son biais de pooling global, porte son comportement. Chaque ablation retire un composant et rien d'autre, aux réglages de la méthode complète (mêmes budgets, mêmes graines, mêmes adversaires et ouvertures gelés, même évaluation), avec trois graines par cellule ; une troisième exécution, explicitement étiquetée comme un changement à deux composants, complète la première ablation. Tout du long, rien n'est attribué au-delà du seul composant qui a changé.

13.1 Conception des trois cellules

Tableau 33 : Les cellules d'ablation. Les comptes de paramètres sont mesurés ; les différences de paramètres sont inhérentes aux composants retirés et sont rapportées plutôt que compensées.

Cellule	Ce qui change par rapport au bras graphe complet	Paramètres	Graines	Statut
A1 : adjacence naïve	les six matrices de messages typées par direction sont remplacées par une seule matrice partagée	0.54 M	3	à un composant, à parité
A2 : sans pooling global	le biais de pooling global est retiré de chaque couche	1.37 M	3	à un composant, à parité
A1′ : adjacence naïve, stabilisée	comme A1, plus un écrêtage de la norme du gradient à 1.0	0.54 M	3	**à deux composants**, jamais attribué au seul typage
Référence : bras graphe complet	aucun	1.47 M	5	comparaison principale

13.2 A1 : retirer les relations typées retire l'entraînabilité

Avec une seule matrice de messages partagée à la place de six matrices typées, l'entraînement a divergé vers des valeurs non finies à la génération 0 pour les trois graines ; chaque génération ultérieure a joué en auto-jeu et s'est entraînée sur des sorties de réseau non finies. Le symptôme qui a déclenché le diagnostic était statistique plutôt que numérique. Les évaluations finales des trois graines étaient identiques jusque dans le nombre de parties (par exemple 0 victoire, 0 nulle et 100 défaites contre l'heuristique pour chaque graine), ce que des entraînements indépendants à graines distinctes ne peuvent produire. Les points de contrôle ont des hachages distincts, mais chaque passe avant renvoie une politique et une valeur non finies dès le premier point de contrôle.

Les tables d'évaluation de la cellule ne sont donc pas des mesures de force, puisqu'une politique non finie joue une partie déterministe et dégénérée, indépendante de ses poids ; elles sont exclues en tant que scores, et la cellule H-T3 indique « entraînement

divergé (3/3 graines) ». Les exécutions ont pris 7.75, 8.01 et 7.61 heures de temps mural d'entraînement contre une moyenne de 36.70 heures pour le bras graphe, parce que des a priori non finis dégradent la recherche en parties courtes d'environ 42 plis, sans troncature ni abandon ; le budget configuré en parties et en simulations a été entièrement consommé.

Aux réglages de parité, la variante à adjacence naïve ne peut pas être entraînée du tout, de sorte que les relations typées apportent, au minimum, la stabilité d'optimisation du bras entier. Rien de leur contribution à la force de jeu ne peut être mesuré à parité, puisque la variante ne s'entraîne jamais. L'explication proposée ici, un argument raisonné plutôt qu'une décomposition mesurée, est qu'une seule matrice partagée reçoit le gradient sommé de six termes de voisinage, soit environ six fois l'échelle de gradient par matrice de la variante typée, à taux d'apprentissage et à momentum identiques. Le protocole avait énoncé, avant toute exécution, qu'un graphe à adjacence naïve « peut ne pas suffire » ; dans ces conditions, il n'optimise même pas.

Aucun balayage du taux d'apprentissage n'a été exécuté, parce que changer le taux d'apprentissage aurait fait de A1 une différence à deux composants. Le passage silencieux de valeurs non finies pendant dix générations était une lacune d'outillage : les boucles d'entraînement n'avaient aucune garde contre les valeurs non finies, et leur cadence d'impression de la perte (tous les 100 pas) ne se déclenchait jamais sur des époques d'environ 20 pas. La lacune a été comblée le 23 septembre 2026 par un arrêt explicite des deux boucles d'entraînement sur perte non finie ; le comportement des exécutions saines est inchangé, et le résultat de l'ablation tient.

13.3 A2 : retirer le pooling global ne change rien de mesurable

Tableau 34 : Ablation A2 : scores au point de contrôle final par graine (lecture à exemples égaux ; taux de troncature entre parenthèses lorsqu'il est non nul ; 100 parties par adversaire et par graine sur les 250 ouvertures gelées) et intervalle bootstrap sur les graines de la différence avec les trois graines originales du bras graphe complet (10 000 rééchantillonnages, ensembles de graines indépendants).

Adversaire	**A2 (sans pooling), graines 1/2/3**	**Bras graphe complet, graines 1/2/3**	**Différence A2 − complet [intervalle à 95%]**
B-RND	0.768 (59%) / 0.714 (65%) / 0.952 (17%)	0.733 (57%) / 0.981 (20%) / 0.722 (55%)	−0.001 [−0.170, +0.165]
B-HEU	0.050 / 0.070 / 0.025	0.025 / 0.050 / 0.090	−0.007 [−0.043, +0.030]
B-MCTS	0.070 / 0.125 / 0.105	0.115 / 0.125 / 0.100	−0.013 [−0.043, +0.013]

Les trois exécutions A2 se sont entraînées sainement (sorties finies, résultats distincts par graine) ; leurs temps muraux d'entraînement de 39.90, 47.44 et 32.05 heures se situent dans la bande du bras graphe complet (26.57–49.86 h).

Retirer le biais de pooling global a changé le score de −0.001 [−0.170, +0.165] contre l'adversaire aléatoire, de −0.007 [−0.043, +0.030] contre l'heuristique et de −0.013 [−0.043, +0.013] contre l'adversaire à recherche. Chaque intervalle chevauche zéro, et la cellule de l'adversaire aléatoire est dominée par la variance entre graines dans les deux variantes (taux de troncature entre 17% et 65% selon les graines).

L'hypothèse selon laquelle le pooling portait l'apprentissage de la valeur du bras graphe n'est pas soutenue. Huit couches de passage de messages directionnel reproduisent à elles seules le comportement du bras complet, y compris son mode de défaillance :

la pathologie de conversion contre l'adversaire aléatoire est présente dans les deux variantes. À cette échelle, le biais de pooling global est dispensable.

La cellule compte trois graines ; la différence de 0.10 M paramètres est inhérente au composant retiré ; et « aucun effet mesurable » est borné par des intervalles de largeur ±0.17 contre l'adversaire aléatoire, où un petit effet pourrait se cacher.

13.4 A1′ : la variante à adjacence naïve stabilisée (supplément à deux composants)

Tableau 35 : Supplément A1′, relations non typées plus écrêtage de gradient à 1.0 (deux composants diffèrent de la méthode complète). Scores au point de contrôle final par graine, moyennes sur trois graines, moyennes sur cinq graines du bras graphe complet, et intervalle bootstrap sur les graines de la différence sur les deux ensembles de graines de tailles inégales.

Adversaire	A1′ graines 1/2/3 (troncature)	Moyenne A1′ (3 graines)	Moyenne graphe complet (5 graines)	Différence [intervalle à 95%]
B-RND	0.566 (47%) / 0.671 (59%) / 0.995 (0%)	0.744	0.810	−0.066 [−0.254, +0.130]
B-HEU	0.035 / 0.135 / 0.110	0.093	0.061	+0.032 [−0.011, +0.078]
B-MCTS	0.090 / 0.195 / 0.060	0.115	0.107	+0.008 [−0.040, +0.063]

Avec un seul stabilisateur ajouté, les trois entraînements sont restés finis et la garde contre les valeurs non finies ne s'est jamais déclenchée, ce qui confirme le remède impliqué par le diagnostic de divergence. Les temps muraux d'entraînement de 23.89, 30.35 et 27.68 heures se situent à l'extrémité rapide de la bande du bras graphe.

La variante stabilisée obtient des scores dans la bande du bras graphe complet contre les trois adversaires : −0.066 [−0.254, +0.130], +0.032 [−0.011, +0.078] et +0.008 [−0.040, +0.063] ; chaque intervalle chevauche zéro.

Combiné avec A1, le plus que l'on puisse dire est qu'à cette échelle la contribution mesurable des relations typées par direction réside dans la stabilité d'optimisation ; aucune contribution à la force au-delà de celle-ci n'est détectable.

Chaque nombre de cette cellule est confondu par l'écrêtage de gradient par construction et n'est jamais attribué au seul typage des arêtes. La cellule compte trois graines et des intervalles larges, et la troncature à 0% de la graine 3 contre 47–59% pour les graines 1–2 montre que la pathologie de conversion reste volatile selon la graine dans cette variante aussi.

13.5 Table de synthèse H-T3

Tableau 36 : Attribution par composant pour le bras graphe (H-T3). Les différences sont variante moins bras graphe complet, moyennes au niveau des graines, trois graines par cellule d'ablation.

Ablation	Composant retiré	Résultat
A1 adjacence naïve	matrices de relations typées par direction → une seule matrice partagée	entraînement divergé (non fini, génération 0, 3/3 graines) ; inentraînable à parité ; relations typées ⇒ au minimum la stabilité d'optimisation

Ablation	**Composant retiré**	**Résultat**
A2 sans pooling global	biais de pooling global dans toutes les couches	effet nul : −0.001 / −0.007 / −0.013 contre les trois adversaires, tous les intervalles chevauchant zéro ; modes de défaillance inchangés ; pooling dispensable
Supplément A1′ (à deux composants)	relations non typées + écrêtage de gradient 1.0	s'entraîne ; scores dans la bande du bras complet (−0.066 / +0.032 / +0.008) ; jamais attribué au seul typage

Dans le périmètre testé, la réponse à RQ-H3 est que, des deux composants distinctifs du bras graphe, l'un est nécessaire à l'entraînabilité elle-même et l'autre est dispensable. La question de savoir si les relations typées contribuent aussi à la force de jeu au-delà de la stabilité ne peut être tranchée à partir de ces cellules.

14 Résultats : positions d'échec et comportement d'entraînement

Les scores agrégés disent que le bras graphe a moins appris ; ils ne disent pas comment il échoue. Ce chapitre examine trois parties individuelles sélectionnées par des critères mécaniques, le canal de troncature qui porte le mécanisme, et ce que montrent les métriques d'entraînement des dix exécutions principales.

14.1 Trois positions d'échec, sélectionnées mécaniquement

Le plan de recherche demande trois positions d'échec commentées avec des critères de sélection explicites. Les critères sont mécaniques et ont été appliqués aux enregistrements bruts par partie : F1 est la plus longue partie tronquée du bras graphe contre l'adversaire aléatoire légal ; F2 est la plus courte défaite décidée du bras grille contre l'adversaire heuristique ; F3 est la plus longue partie nulle du bras graphe contre l'adversaire à recherche. Chaque partie sélectionnée a été reproduite de façon déterministe à partir de son ouverture enregistrée et de ses graines par partie, et vérifiée coup par coup contre sa ligne de résultat enregistrée avant d'être rendue ; les trois reproductions ont concordé. Cette étape de vérification a révélé un défaut latent du lanceur d'affrontements (un chemin qui omettait d'écrire les enregistrements lorsque chaque partie d'un affrontement était tronquée) ; le défaut a été corrigé avant les évaluations à temps égal, et un audit a établi qu'il n'avait affecté aucune donnée de campagne (Chapitre 10).

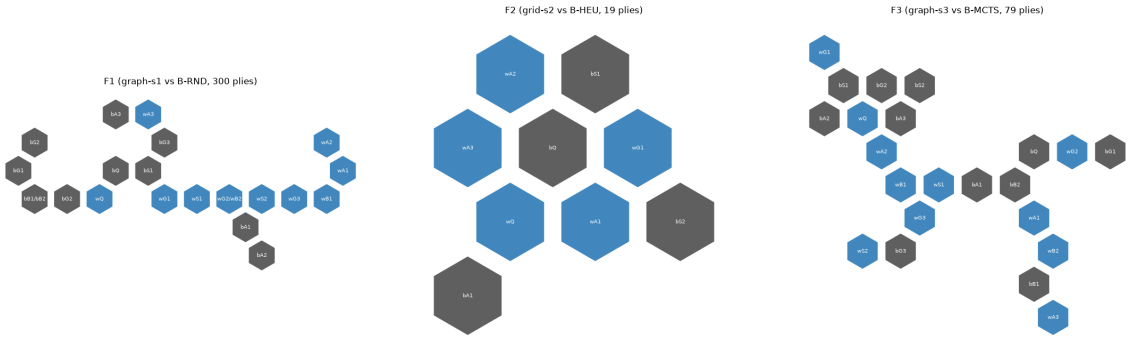


Fig. 6 : Trois positions d'échec commentées, reproduites et vérifiées contre leurs lignes de résultat enregistrées. F1 (à gauche) : bras graphe, graine 1, contre l'aléatoire légal, ouverture 2, blancs ; la partie atteint le plafond de 300 plis avec le bras graphe matériellement en avance mais incapable d'achever l'encerclement. F2 (au centre) : bras grille, graine 2, contre l'heuristique, ouverture 2, noirs ; une défaite décidée en 19 plis face à la tactique de ciblage de la reine de l'heuristique. F3 (à droite) : bras graphe, graine 3, contre l'adversaire à recherche, ouverture 19, blancs ; une nulle en 79 plis dans laquelle le bras graphe ne génère jamais de menace gagnante.

F1 : une position gagnée que le bras graphe ne parvient pas à conclure. Le bras graphe gagne du matériel tôt, puis déplace ses pièces autour d'une reine ennemie partiellement encerclée jusqu'au plafond. L'échec est tactique plutôt qu'évaluatif : le bras est en avance et le reste, mais il ne trouve pas, ou ne préfère pas, la séquence forçante qui achève l'encerclement.

C'est le motif derrière le taux de troncature de 20–57% du bras contre l'adversaire aléatoire, et il est invisible dans toute évaluation qui compte les parties plafonnées comme des nulles.

F2 : le prix des politiques de régime précoce face à une tactique tranchante. La plus courte défaite du bras grille contre l'heuristique dure 19 plis. Le terme dominant de libertés de la reine de l'heuristique la dirige droit sur la reine avant que la politique du réseau n'ait consolidé une défense. La partie illustre pourquoi les deux bras obtiennent des scores bas contre les deux adversaires forts après dix générations, puisque le régime est encore précoce, et pourquoi la résolution de la comparaison est la plus élevée contre l'adversaire aléatoire.

F3 : éviter la défaite sans créer de menaces. Contre l'adversaire à recherche, la plus longue nulle du bras graphe (79 plis) montre un réseau qui se défend convenablement, en ce qu'il évite de perdre, mais ne construit jamais d'attaque gagnante. Contre cet adversaire, les deux bras sont statistiquement indiscernables (Chapitre 11) ; F3 rappelle qu'« indiscernables » signifie ici que les deux bras annulent ou perdent, et non que l'un ou l'autre joue bien.

14.2 Le canal de troncature, exécution par exécution

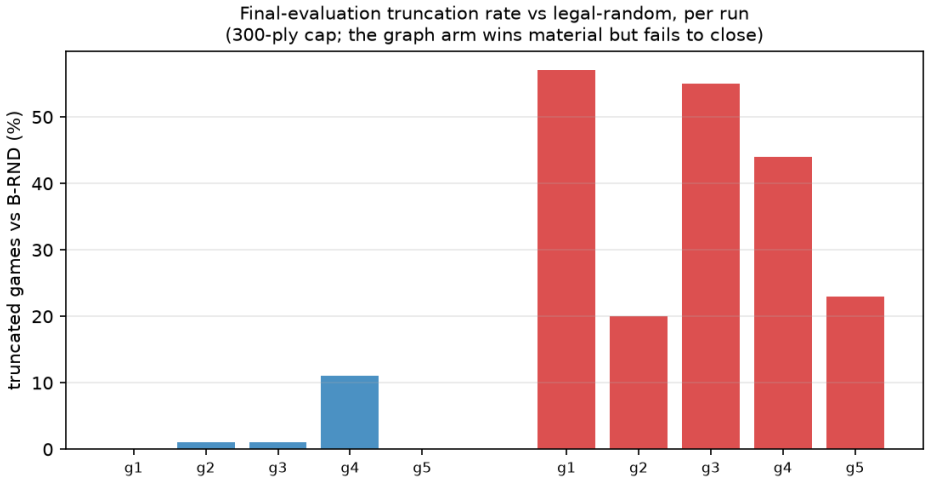


Fig. 7 : Taux de troncature à l'évaluation finale contre l'adversaire aléatoire légal pour chacune des dix exécutions principales (points de contrôle finaux ; 100 parties par exécution à 400 simulations sur les 250 ouvertures gelées ; plafond de 300 plis). Bleu : bras grille, graines 1–5 ; rouge : bras graphe, graines 1–5. La troncature est rapportée séparément des nulles tout au long de l'étude.

Le bras graphe a tronqué 57%, 20%, 55%, 44% et 23% de ses parties contre l'adversaire aléatoire au point de contrôle final pour les graines 1 à 5 ; le bras grille a tronqué 0%, 1%, 1%, 11% et 0%. Contre l'heuristique et l'adversaire à recherche, aucune partie de l'un ou l'autre bras n'a atteint le plafond.

L'écart de troncature est la plus grande différence comportementale entre les bras, plus grande que tout écart de score, et sa direction est cohérente entre graines : chaque graine graphe tronque plus que chaque graine grille. La graine grille 4 (11%) ne dépasse aucune graine graphe, mais elle montre que la pathologie n'est pas strictement exclusive à un bras. Parce que la troncature a été définie comme une issue à part entière avant la

première exécution, la pathologie est visible au lieu d'être absorbée dans les nulles ; et parce que le plafond ne peut pas sauver l'hypothèse (Chapitre 11), le canal est diagnostique plutôt que décisif.

Le score du bras graphe contre l'adversaire aléatoire est par conséquent mesuré sur moins de parties décidées (entre 43 et 80 par graine), ce qui élargit son intervalle ; l'analyse de sensibilité borne cet effet sans le supprimer.

14.3 Ce que montrent les métriques d'entraînement

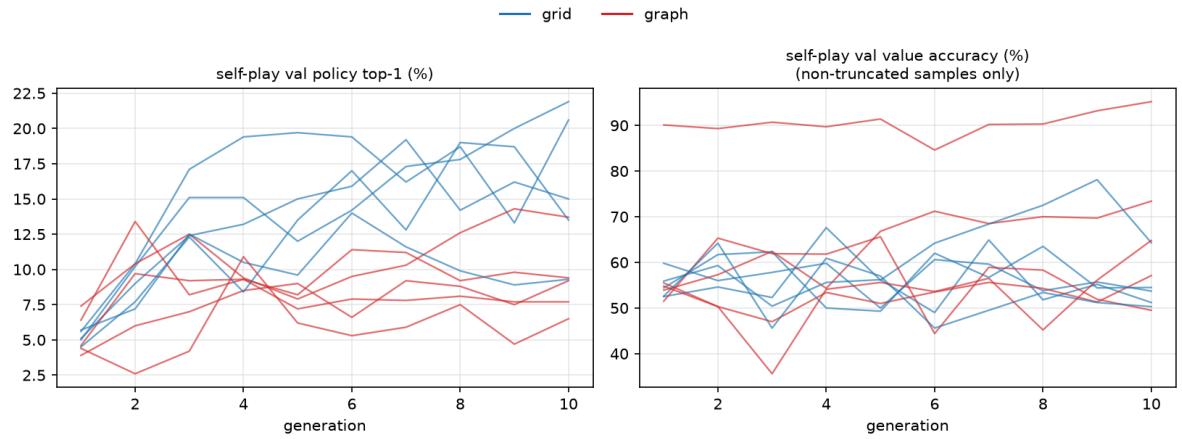


Fig. 8 : Métriques d'entraînement par génération pour les dix exécutions de la campagne principale (ligne de l'époque 1 de chaque génération) : à gauche, accord top-1 de la politique avec la distribution de visites MCTS sur la partition de validation d'auto-jeu ; à droite, exactitude de la valeur sur les échantillons non tronqués. Bleu : bras grille ; rouge : bras graphe ; une ligne par graine. Les exécutions d'ablation sont exclues.

Les deux bras ajustent leurs données d'auto-jeu tout au long des dix générations : l'argmax de politique (top-1) monte avec la génération dans chaque exécution, et l'exactitude de la valeur sur les échantillons non tronqués monte pour les deux bras. L'exactitude de la valeur du bras graphe est comparable à celle du bras grille, tandis que son argmax de politique (top-1) reste en retrait.

L'écart entre les bras n'est donc pas un simple échec d'optimisation du bras graphe complet (contrairement à la variante ablatée à adjacence naïve, qui n'a pas optimisé du tout). Le bras graphe apprend une évaluation utilisable des positions et une politique plus faible. Les positions d'échec donnent la même image, un jugement adéquat de qui est mieux et une génération inadéquate de coups forçants, et cela motive le premier prolongement proposé dans Chapitre 17.

Les métriques d'entraînement sont calculées sur la partition de validation d'auto-jeu propre à chaque exécution, dont la distribution se déplace avec la génération et diffère entre les bras ; elles constituent une preuve sur l'ajustement, non sur la force, et aucun nombre de cette figure n'entre dans aucun intervalle ni dans aucune affirmation de la comparaison principale.

14.4 Trajectoires par adversaire

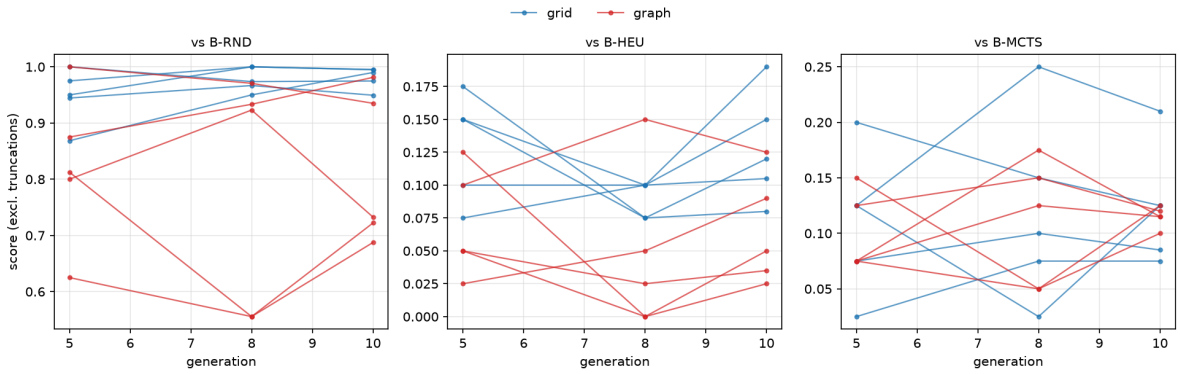


Fig. 9 : Score contre chaque adversaire gelé aux générations évaluées (5, 8 et 10 ; 20, 20 et 100 parties par adversaire respectivement) pour chaque exécution principale. Bleu : bras grille ; rouge : bras graphe ; une ligne par graine. Les scores excluent les parties tronquées.

Les trajectoires par adversaire décomposent les courbes agrégées de Chapitre 12 : contre l'adversaire aléatoire, les graines grille approchent tôt du plafond et y restent, tandis que les graines graphe se dispersent largement et bougent de façon non monotone ; contre l'heuristique et l'adversaire à recherche, les deux bras restent dans une bande basse dont le mouvement d'une génération à l'autre est de l'ordre du bruit lié au nombre de parties à 20 parties (les évaluations intermédiaires ont été dimensionnées pour le suivi et non pour l'inférence ; seules les évaluations finales à 100 parties entrent dans les tables).

15 Une seconde étude : la comparaison sous une boucle à démarrage à chaud avec fenêtre de rejeu

La première étude a répondu à sa question dans un pipeline qui entraînait le réseau de chaque génération depuis une initialisation nouvelle, sur les seules parties de cette génération (Chapitre 6, Chapitre 16). Ce chapitre rend compte d'une seconde étude, préenregistrée séparément, qui applique les deux remèdes usuels, poids transmis d'une génération à la suivante et entraînement sur une fenêtre de générations récentes, à l'identique aux deux bras, et demande si l'ordre des deux représentations y survit. Son protocole a été gelé le 13 octobre 2026, avant toute exécution ; sa campagne a démarré le même jour. Tout ce que la première étude avait gelé (adversaires, ouvertures, réglages d'évaluation, réseaux, décodeur) est réutilisé sans modification, de sorte que les deux études diffèrent par le régime d'entraînement et par rien d'autre.

15.1 Pourquoi une seconde étude

La réinitialisation à chaque génération de la première étude est neutre pour la comparaison, puisque les deux encodeurs ont vu exactement les mêmes enregistrements sous les mêmes conventions, mais elle borne le régime atteint : chaque réseau a appris sur les positions de 500 parties, et la force absolue après dix générations est restée proche du plafond contre l'adversaire aléatoire et autour de 0.1 contre les deux adversaires forts. La première objection d'un relecteur à la généralité d'un verdict négatif est donc que la représentation en graphe pourrait avoir besoin de plus de données par réseau, ou d'une trajectoire d'optimisation plus longue, avant que son a priori structurel ne paie. Les grands systèmes d'auto-jeu transmettent les poids de génération en génération et s'entraînent sur une fenêtre de rejeu de parties récentes ; une seconde étude sous ce régime est le test direct de l'objection. La question posée est celle de la première étude sous le nouveau régime (RQ-H1′), avec la même hypothèse (H1′) : le bras graphe atteint un score moyen plus élevé que le bras grille contre la population gelée, à budget égal. Une question secondaire, descriptive, demande si le contraste graphe–grille se rapproche de zéro ou change de signe par rapport à la première étude ; les deux études sont comparées côte à côte et jamais agrégées.

15.2 Ce qui change, et ce qui ne change pas

Exactement deux choses changent, dans les deux bras. D'abord, le réseau de la génération g est initialisé depuis le point de contrôle final de la génération $g-1$ (poids du modèle seulement ; l'optimiseur et le calendrier cosinus du taux d'apprentissage redémarrent à chaque génération, comme avant), la génération 0 partant de l'initialisation aléatoire ensemencée. Ensuite, la génération g s'entraîne sur les enregistrements des générations $\max(0, g-2)$ à g, une fenêtre de rejeu de trois générations : 500 parties à la génération 0, 1000 à la génération 1 et 1500 à partir de la génération 2, toujours pour deux époques, de sorte que le nombre de pas de gradient croît avec la fenêtre.

La politique d'hyperparamètres a été décidée par l'auteur au gel et enregistrée avec lui. Les deux bras conservent sans changement les réglages d'entraînement de la première étude (taux d'apprentissage 0.02, lot 256, décroissance des poids 10^{-4}, poids de la perte de valeur 0.6, deux époques par génération, perte de politique masquée par la légalité ; 96 canaux et 8 blocs pour le réseau grille, 152 unités cachées et 8 couches pour le réseau graphe). L'alternative examinée était un budget identique et pré-enregistré de trois taux d'apprentissage (0.02, 0.01 et 0.005) pour les deux bras sur la première graine seulement, le meilleur taux par bras servant ensuite aux graines restantes. Elle a été écartée parce qu'elle aurait triplé le coût d'une graine par bras (environ quatre jours de calcul de plus), sélectionné sur une seule graine, et parce que le pilote décrit plus bas n'a montré aucun signe d'instabilité d'optimisation sous le régime à démarrage à chaud. L'objection selon laquelle les deux bras tournent sous des réglages par défaut non ajustés reste donc ouverte pour les deux études et est énoncée comme une limite.

Tout le reste est celui de la première étude : le moteur et ses réglages de recherche (128 et 32 simulations avec randomisation du plafond de simulations, bruit d'exploration, 12 plis de température, abandon sous −0.92 avec l'audit à 10%, plafond de 300 plis), le format des enregistrements, la perte et le masquage, les deux encodeurs et réseaux à 1 443 168 et 1 465 452 paramètres, le décodeur partagé, les trois adversaires gelés, les 250 ouvertures gelées, l'évaluation épinglée (400 simulations, sans bruit, 100 parties appariées à couleurs échangées par adversaire), les statistiques (la graine comme unité, bootstrap percentile à 10 000 rééchantillonnages, ensembles de graines indépendants pour le contraste, la troncature comme issue à part entière) et la règle de rejet, qui est celle de la première étude mot pour mot dans sa forme : aucun avantage du graphe cohérent entre graines sous les deux lectures du budget, et des intervalles au niveau des graines sur la différence graphe−grille excluant un avantage graphe substantiel, la plus grande borne supérieure étant rapportée pour que le lecteur applique le seuil de son choix. Les évaluations intermédiaires sont descriptives et n'entrent dans aucun verdict. Les nouvelles exécutions utilisent les graines 11 à 15, de sorte qu'aucun flux aléatoire n'est partagé avec les graines 1 à 5 de la première étude.

Le plan est de nouveau deux bras, cinq graines, dix générations de 500 parties, lues deux fois. La lecture à exemples égaux compare les points de contrôle finaux. La lecture à temps mural égal compare, pour chaque exécution, le dernier point de contrôle achevé au plus tard à un seuil à temps égal défini, comme dans la première étude, par une règle indépendante des scores et écrite avant toute exécution : la médiane du temps mural total d'entraînement des cinq exécutions grille de cette étude, chaque total étant la somme des dix horloges par génération (auto-jeu, entraînement et export ; évaluations exclues), calculée une seule fois depuis les fichiers d'horloge quand les cinq exécutions grille existent et avant qu'aucun nombre inter-bras ne soit assemblé, et jamais recalculée. Le seuil de la seconde étude est sa propre valeur, mesurée sous le nouveau régime ; ce n'est pas les 18.77 h de la première.

Les pré-engagements reprennent ceux de la première étude : les cinq graines par bras entrent dans l'analyse finale quelle que soit leur direction ; une graine n'est exclue que pour un plantage, une corruption ou une erreur du moteur, chacun consigné ; aucun adversaire, ouverture, réglage d'évaluation, hyperparamètre ou architecture n'est touché ; le chapitre est écrit depuis les tables gelées. Les règles d'arrêt sont inchangées, avec un ajout rendu explicite : une campagne en cours n'est jamais interrompue ni relancée sans l'auteur.

15.3 Implémentation et vérification avant le gel

Les deux changements ont été implémentés comme deux options du pilote de campagne, identiques pour les deux bras et désactivées par défaut, de sorte que le pilote de campagne avec ses options par défaut produit toujours les commandes de la première étude. Cette propriété a été épinglée par un test de non-régression avant le gel du protocole : pour quatre exécutions de la première étude (deux par bras) et leurs dix générations chacune, la commande d'entraînement produite par défaut a été comparée à la configuration d'entraînement enregistrée de chaque génération (liste des segments, graine, nombre d'époques, absence de point de contrôle d'initialisation), et la commande d'auto-jeu au manifeste enregistré (graine, nombre de parties, budgets de simulation, indice de génération) ; les quarante générations concordent. Le même test vérifie qu'avec les deux options actives, seules la liste des segments et le point de contrôle d'initialisation changent, que la génération 0 ne démarre jamais à chaud, et que la génération g s'initialise depuis la génération $g-1$. Les exécutions de la seconde étude sont écrites dans leurs propres répertoires, et chaque exécution enregistre son régime dans un fichier de configuration, de sorte que les enregistrements de la première étude ne peuvent être ni touchés ni confondus avec les nouveaux.

Un pilote approuvé par l'auteur le 12 octobre 2026 a ensuite parcouru le chemin complet sur les deux bras, une graine hors des ensembles de graines des deux études, deux générations de 200 parties, démarrage à chaud et fenêtre de trois générations, avec une évaluation finale réduite à 20 parties par adversaire qui servait de contrôle du chemin et n'entre dans aucune analyse. Le Tableau 37 donne les phases mesurées. Sur les deux bras, la seconde génération s'est entraînée depuis les poids de la première sur les segments des deux générations, comme l'enregistrent les fichiers de configuration et les journaux d'entraînement ; le temps d'entraînement a crû en proportion du nombre d'enregistrements (facteur 2.15 sur le bras grille et 2.02 sur le bras graphe à l'entrée de la seconde génération dans la fenêtre) et est resté une petite part d'une génération, parce que l'auto-jeu domine : dans la première étude, l'entraînement occupait 3% d'une exécution grille (194 à 250 s par génération) et 5% d'une exécution graphe (516 à 833 s). La fenêtre atteignant trois générations à partir de la génération 2, le temps d'entraînement sur une exécution vaut environ 2.7 fois celui de la première étude, ce qui ajoute environ une heure à une exécution grille et trois heures à une exécution graphe. Le coût projeté de la campagne est donc celui de la première étude plus cet incrément : environ 18 à 20 h par exécution grille, 30 à 53 h par exécution graphe, environ 295 h soit 12 à 13 jours de calcul séquentiel sur portable pour les dix exécutions, plus environ un jour d'évaluations.

Tableau 37 : Pilote de la seconde étude (12 octobre 2026) : temps mural des phases d'auto-jeu et d'entraînement de chaque génération, une graine par bras, 200 parties par génération, démarrage à chaud et fenêtre de rejeu de trois générations, sur le même portable que les campagnes. La génération 1 s'entraîne sur les enregistrements des générations 0 et 1 ; le nombre d'enregistrements d'entraînement exclut la partie de validation. L'export a pris 2 s par génération sur les deux bras. Le pilote a mesuré des coûts et exercé le chemin ; son évaluation à 20 parties n'a aucune signification statistique et n'est pas rapportée.

Bras	Génération	Auto-jeu (s)	Entraînement (s)	Enregistrements d'entraînement (segments)
grille	0	2196	78	10188 (4)
grille	1	2584	168	22399 (8)
graphe	0	5821	303	10655 (4)

Bras	Génération	Auto-jeu (s)	Entraînement (s)	Enregistrements d'entraînement (segments)
graphe	1	5 800	613	21 386 (8)

Les sept vérifications automatiques du pipeline (Chapitre 6) ont ensuite été passées sur l'ensemble de segments fenêtré de chaque exécution du pilote. Une vérification a échoué comme prévu. La vérification d'estampille exigeait que chaque enregistrement d'un jeu d'entraînement porte l'estampille génération-réseau d'un seul manifeste, ce qui encode l'hypothèse d'une génération par entraînement de la première étude ; sur l'ensemble fenêtré, elle a signalé des enregistrements de deux générations face à un seul manifeste. La vérification a été généralisée plutôt qu'assouplie : elle prend désormais les manifestes de chaque génération de la fenêtre et exige que l'ensemble des estampilles des enregistrements soit égal à l'ensemble des estampilles listées, de sorte que chaque enregistrement provient d'une génération listée et que chaque génération listée est présente ; elle passe toujours sur un ensemble de segments de la première étude avec un seul manifeste et échoue toujours avec un mauvais manifeste. Avec cette généralisation, toutes les vérifications ont passé sur les deux bras (normalisation de la politique sous le masque de légalité sur une passe avant réelle, légalité de toutes les cibles stockées, domaine des issues avec 17 054 des 22 856 enregistrements grille et 11 298 des 21 822 enregistrements graphe tronqués et exclus de la perte de valeur, estampilles des enregistrements, sauvegarde et reprise identiques au bit près, et séparation des entrées d'entraînement d'avec les fichiers d'évaluation).

15.4 Exécution

Le protocole a été gelé le 13 octobre 2026 sous son empreinte de contenu, avec la politique d'hyperparamètres, la fenêtre de trois générations et la règle de seuil décidées par l'auteur, et la campagne a été approuvée et lancée le même jour à 15:02 CET, après vérification que la machine était inoccupée, que 161 GB de disque étaient libres pour une garde de 20 GB, et que l'empreinte du protocole gelé était inchangée. Les dix exécutions se déroulent séquentiellement sur le même portable que la première étude, en alternant grille puis graphe pour chaque graine de 11 à 15, de sorte qu'une interruption laisserait aux deux bras un nombre égal de graines ; chaque exécution est reprenable à la granularité de la génération depuis ses manifestes enregistrés, et des évaluations intermédiaires de 20 parties par adversaire aux générations 5 et 8 ne servent qu'aux figures de trajectoire. Aucun autre processus lourd ne tourne sur la machine pendant la campagne, puisque les horloges par génération sont la mesure de la lecture à temps mural égal. Le Tableau 38 enregistre chaque exécution à son achèvement.

Tableau 38 : Registre d'exécution de la campagne de la seconde étude : début et fin de chaque exécution (CET) et son temps mural total d'entraînement, somme des dix horloges par génération (auto-jeu, entraînement et export ; temps d'évaluation exclu). Dix générations de 500 parties par exécution, démarrage à chaud et fenêtre de rejeu de trois générations, en séquence sur un portable.

Exécution	Début	Fin	Horloge d'entraînement (h)
grille, graine 11	13 oct. 2026 15:02	14 oct. 2026 09:33	17.46
graphe, graine 11	14 oct. 2026 09:33	16 oct. 2026 02:39	39.28
grille, graine 12	16 oct. 2026 02:39	16 oct. 2026 21:46	17.95

Exécution	Début	Fin	Horloge d'entraînement (h)
graphe, graine 12	16 oct. 2026 21:46	18 oct. 2026 13:40	37.33
grille, graine 13	18 oct. 2026 13:40	19 oct. 2026 08:19	17.63
graphe, graine 13	19 oct. 2026 08:19	21 oct. 2026 17:55	52.85
grille, graine 14	21 oct. 2026 17:55	22 oct. 2026 12:18	17.46
graphe, graine 14	22 oct. 2026 12:18	24 oct. 2026 10:44	42.20
grille, graine 15	24 oct. 2026 10:44	25 oct. 2026 06:27	18.52
graphe, graine 15	25 oct. 2026 06:27	26 oct. 2026 23:00	36.27

15.5 Résultats

Chaque table de cette section montre chaque graine ; les scores excluent les parties tronquées, rapportées dans leur propre colonne ; les intervalles sont des intervalles bootstrap percentiles à 95% sur les graines (10 000 rééchantillonnages), les deux bras étant rééchantillonnés indépendamment pour les contrastes. Les tables brutes avec les colonnes de sensibilité se trouvent dans Annexe D.8 ; la provenance de chaque nombre dans Annexe E.

15.5.1 Le verdict en un paragraphe

Sous les deux lectures de budget, le bras graphe ne montre aucun avantage cohérent entre graines contre aucun adversaire, et chaque intervalle sur la différence de score graphe−grille inclut zéro : les six contrastes ponctuels se situent entre −0.052 et +0.013, et les plus grandes bornes supérieures sont +0.090 à exemples égaux, contre l'adversaire de recherche, et +0.055 à temps mural égal. La première condition de la règle gelée est donc remplie sous les deux lectures. Sa seconde condition parle, comme dans la première étude, d'un avantage « substantiel » sans en chiffrer le seuil ; les bornes sont donc énoncées plutôt qu'un seuil choisi après coup : la règle rejette H1′ pour tout seuil supérieur à +0.090 ; pour les seuils compris entre +0.055 et +0.090, la lecture à temps mural égal rejette tandis que la lecture à exemples égaux laisse non exclu un avantage du graphe contre l'adversaire de recherche ; la borne réalisée de la première étude était +0.028. **H1′ n'est pas étayée** : aucun avantage du graphe n'a été détecté sous l'une ou l'autre lecture, et les données ne peuvent pas en exclure un petit, inférieur à environ 0.09, contre l'adversaire de recherche à exemples égaux. Le reste de la section présente la preuve graine par graine.

15.5.2 Lecture 1 : nombre égal d'exemples d'entraînement

Les deux bras sont comparés à leurs points de contrôle de la génération 10, chaque exécution ayant engendré dix générations de 500 parties sous des budgets de simulation identiques et s'étant entraînée dessus ; sous le régime à démarrage à chaud, chaque point de contrôle a de plus hérité des poids des neuf qui le précèdent.

Tableau 39 : Seconde étude, lecture à exemples égaux. Score au point de contrôle final (génération 10) de chaque exécution contre chaque adversaire gelé, cinq exécutions d'entraînement indépendantes à démarrage à chaud par bras (graines 11 à 15, dix générations de 500 parties, fenêtre de rejeu de trois générations) ; 100 parties appariées à couleurs échangées par adversaire et par exécution sur les 250 ouvertures gelées, à 400 simulations par coup sans bruit d'exploration. Score = moyenne sur les parties décidées (victoire 1, nulle 0.5, défaite 0). Troncature = part des

100 parties arrêtées au plafond de 300 demi-coups ; aucune partie contre B-HEU ou B-MCTS n'a été tronquée par l'un ou l'autre bras.

Bras	Graine	Score B-RND	Troncature B-RND	Score B-HEU	Score B-MCTS
grille	11	0.984	5%	0.040	0.100
grille	12	0.984	4%	0.055	0.075
grille	13	0.995	1%	0.105	0.175
grille	14	0.990	2%	0.120	0.260
grille	15	0.989	7%	0.065	0.045
graphe	11	1.000	0%	0.125	0.245
graphe	12	0.995	4%	0.030	0.070
graphe	13	0.940	42%	0.080	0.145
graphe	14	0.928	24%	0.030	0.100
graphe	15	0.992	41%	0.105	0.110

Contre l'adversaire aléatoire légal, les graines grille obtiennent de 0.984 à 0.995 et les graines graphe de 0.928 à 1.000 ; contre l'heuristique, les graines grille de 0.040 à 0.120 et les graines graphe de 0.030 à 0.125 ; contre l'adversaire de recherche, les graines grille de 0.045 à 0.260 et les graines graphe de 0.070 à 0.245. Le bras graphe tronque de 0 à 42% de ses parties contre l'adversaire aléatoire, trois graines entre 24 et 42% et deux entre 0 et 4% ; le bras grille de 1 à 7%. Les plages par graine des deux bras se recouvrent contre chaque adversaire, et la dispersion entre les graines d'un même bras contre l'adversaire de recherche (0.045 à 0.260 pour le bras grille) dépasse toute différence entre les bras. Cinq graines par bras ne peuvent pas résoudre des différences à l'intérieur d'une telle dispersion.

Tableau 40 : Seconde étude, lecture à exemples égaux. Score moyen au niveau des graines par bras et par adversaire, avec l'intervalle bootstrap à 95% sur les cinq graines (10 000 rééchantillonnages) ; mêmes exécutions, adversaires et volume d'évaluation que le Tableau 39.

Bras	vs B-RND	vs B-HEU	vs B-MCTS
grille	0.989 [0.985, 0.992]	0.077 [0.051, 0.104]	0.131 [0.068, 0.200]
graphe	0.971 [0.943, 0.996]	0.074 [0.040, 0.108]	0.134 [0.090, 0.196]

15.5.3 Lecture 2 : temps mural égal sur la même machine

Chaque exécution est prise au dernier point de contrôle qu'elle avait achevé dans le seuil à temps mural égal propre à la seconde étude, 17.63 heures, la médiane du temps mural total d'entraînement des cinq exécutions grille (17.46, 17.95, 17.63, 17.46 et 18.52 heures), calculée le 25 octobre 2026 à partir des seuls fichiers d'horloge, avant qu'aucun score de l'étude n'ait été lu, et jamais recalculée. Le seuil sélectionne les générations 10, 9, 10, 10 et 9 pour les graines grille 11 à 15, et les générations 4, 5, 3, 3 et 4 pour les graines graphe 11 à 15 (en comptant le réseau initial comme génération 1 ; dans les journaux d'exécution, ces points de contrôle sont étiquetés gen009/gen008/gen009/gen009/gen008 et gen003/gen004/gen002/gen002/gen003). Lorsque le point de contrôle au seuil est le dernier, l'évaluation finale est réutilisée ; les sept autres ensembles ont été joués après la campagne, au même volume et avec les graines de seuil épinglées.

Tableau 41 : Seconde étude, lecture à temps mural égal. Score de chaque exécution à son dernier point de contrôle achevé dans le seuil de 17.63 heures (graines grille aux générations 10, 9, 10, 10, 9 ; graines graphe aux générations 4, 5, 3, 3, 4), contre chaque adversaire gelé ; même évaluation que le Tableau 39 (100 parties appariées par adversaire, 250 ouvertures gelées, 400 simulations,

sans bruit). Troncature = part des parties au plafond de 300 demi-coups ; aucune contre B-HEU ou B-MCTS.

Bras	Graine	Score B-RND	Troncature B-RND	Score B-HEU	Score B-MCTS
grille	11	0.984	5%	0.040	0.100
grille	12	0.984	5%	0.060	0.110
grille	13	0.995	1%	0.105	0.175
grille	14	0.990	2%	0.120	0.260
grille	15	0.973	7%	0.065	0.075
graphe	11	0.995	1%	0.110	0.160
graphe	12	0.984	4%	0.115	0.095
graphe	13	0.762	58%	0.140	0.140
graphe	14	0.965	28%	0.065	0.120
graphe	15	0.958	40%	0.025	0.080

Tableau 42 : Seconde étude, lecture à temps mural égal. Score moyen au niveau des graines par bras et par adversaire, avec l'intervalle bootstrap à 95% sur les cinq graines (10 000 rééchantillonnages) ; mêmes exécutions et volume d'évaluation que le Tableau 41.

Bras	vs B-RND	vs B-HEU	vs B-MCTS
grille	0.985 [0.979, 0.992]	0.078 [0.053, 0.105]	0.144 [0.092, 0.206]
graphe	0.933 [0.846, 0.986]	0.091 [0.056, 0.124]	0.119 [0.094, 0.144]

À temps égal, le bras graphe est lu à ses troisième à cinquième points de contrôle et le bras grille à son neuvième ou dixième. La moyenne du bras graphe contre l'adversaire aléatoire tombe à 0.933 [0.846, 0.986] contre 0.985 [0.979, 0.992], un écart porté par la graine 13 (0.762 avec 58% de troncature) et par les graines 14 et 15 (28% et 40% de troncature) ; contre l'heuristique, sa moyenne est de 0.091 [0.056, 0.124] contre 0.078 [0.053, 0.105], et contre l'adversaire de recherche de 0.119 [0.094, 0.144] contre 0.144 [0.092, 0.206], chaque intervalle recouvrant son homologue. Trois à cinq générations à démarrage à chaud du bras graphe atteignent donc, contre les deux adversaires forts, le niveau qu'atteignent neuf ou dix générations grille, et l'écart contre l'adversaire aléatoire est de nouveau le canal de troncature. La lecture impute au bras graphe son coût mesuré sur cette machine, 2.34 fois celui du bras grille par exécution, ce qui est son rôle ; ce qu'elle ne peut pas faire avec cinq graines, c'est séparer des moyennes dont les intervalles se recouvrent sur plus de la moitié de leur largeur.

15.5.4 Le contraste entre bras

Tableau 43 : Seconde étude, le contraste pré-enregistré : différence des scores moyens au niveau des graines (bras graphe moins bras grille), avec l'intervalle bootstrap à 95% sur les graines (cinq graines indépendantes à démarrage à chaud par bras, rééchantillonnées indépendamment, 10 000 rééchantillonnages), par adversaire gelé et par lecture de budget. Une valeur négative favorise le bras grille.

Adversaire	Exemples égaux : graphe – grille	Temps mural égal : graphe – grille
B-RND (aléatoire légal)	−0.018 [−0.045, +0.008]	−0.052 [−0.140, +0.002]
B-HEU (heuristique)	−0.003 [−0.046, +0.039]	+0.013 [−0.032, +0.055]
B-MCTS (recherche, 6 400 simulations)	+0.003 [−0.082, +0.090]	−0.025 [−0.091, +0.034]

Aucun des six contrastes n'exclut zéro. Contre l'adversaire aléatoire légal, la différence graphe–grille est de −0.018 [−0.045, +0.008] à exemples égaux et de −0.052 [−0.140, +0.002] à temps égal ; contre l'heuristique, −0.003 [−0.046, +0.039] et +0.013 [−0.032, +0.055] ; contre l'adversaire de recherche, +0.003 [−0.082, +0.090] et −0.025 [−0.091, +0.034]. Les contrastes ponctuels sont à moins de 0.05 de zéro dans cinq cellules sur six, et la sixième touche zéro à sa borne supérieure. Les intervalles sont plus larges que ceux de la première étude dans quatre des six cellules, le plus large s'étendant sur 0.17 contre l'adversaire de recherche, où les graines des deux bras s'étalent de 0.045 à 0.260.

La première condition de la règle gelée demande l'absence d'un ordre cohérent entre graines en faveur du bras graphe. Les graines graphe surpassent les graines grille dans 3, 2 et 2 des 5 paires de même indice contre les trois adversaires à exemples égaux (13, 11 et 13 des 25 paires croisées), et dans 2, 3 et 2 sur 5 (7, 15 et 11 sur 25) à temps égal : aucun adversaire n'ordonne les graines en faveur du bras graphe sous l'une ou l'autre lecture. La seconde condition demande des intervalles excluant un avantage graphe substantiel, sans nombre attaché : la plus grande borne supérieure est +0.090 à exemples égaux et +0.055 à temps égal. Le verdict est celui énoncé en tête de la section, dans ces termes ; l'étude ne revendique aucun seuil qu'elle n'a pas pré-enregistré, et le lecteur peut appliquer celui de son choix.

15.5.5 Troncature et lecture de sensibilité

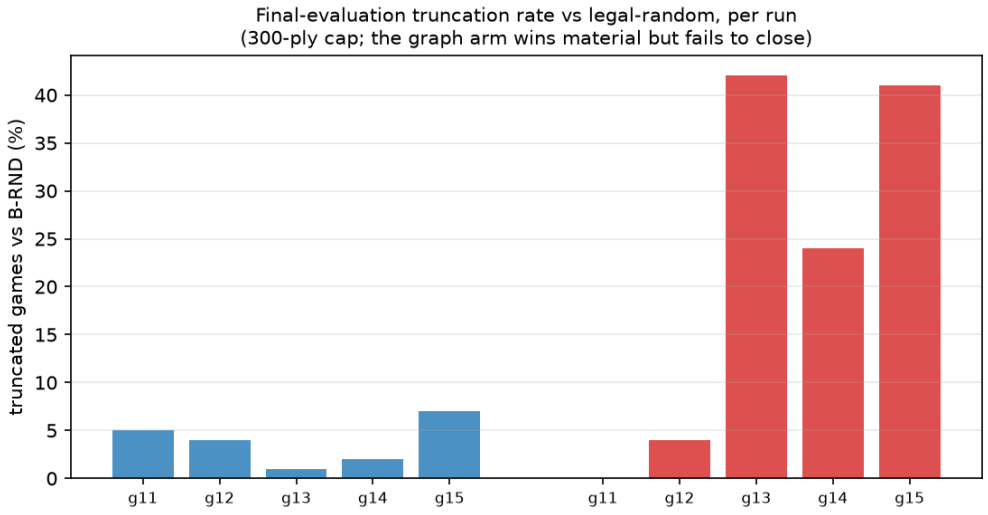


Fig. 10 : Seconde étude : part des 100 parties d'évaluation finale contre l'adversaire aléatoire légal (B-RND) arrêtées au plafond de 300 demi-coups, par exécution ; en bleu, les cinq graines grille (11 à 15, de gauche à droite) ; en rouge, les cinq graines graphe. Points de contrôle finaux (génération 10) de dix exécutions à démarrage à chaud, 100 parties appariées à couleurs échangées chacune sur les 250 ouvertures gelées à 400 simulations.

La troncature ne survient de nouveau que contre l'adversaire aléatoire, dans aucune des 20 cellules contre l'heuristique et l'adversaire de recherche. Aux points de contrôle finaux, le bras graphe tronque 0%, 4%, 42%, 24% et 41% de ses parties contre l'adversaire aléatoire pour les graines 11 à 15, le bras grille 5%, 4%, 1%, 2% et 7% (Fig. 10) ; aux points de contrôle au seuil, 1%, 4%, 58%, 28% et 40% contre 5%, 5%, 1%, 2% et 7%. La pathologie de conversion de la première étude, présente là dans chaque graine graphe (20 à 57%), persiste dans trois graines graphe sur cinq et est absente dans deux, dont

les taux se situent dans la bande du bras grille. Le score qui exclut les troncatures est proche entre les bras, mais sous le traitement de sensibilité, chaque partie tronquée comptée 0.5, la différence graphe−grille contre l'adversaire aléatoire est de −0.101 [−0.187, −0.012] à exemples égaux et de −0.131 [−0.256, −0.015] à temps égal, et les deux intervalles excluent zéro ; les quatre autres contrastes sont inchangés sous tout traitement, n'ayant aucune troncature. Sous le régime à démarrage à chaud, le déficit du bras graphe contre l'adversaire le plus faible n'est donc plus un déficit dans les parties décidées ; c'est un déficit à les décider, concentré dans trois graines.

15.5.6 Trajectoires et coût

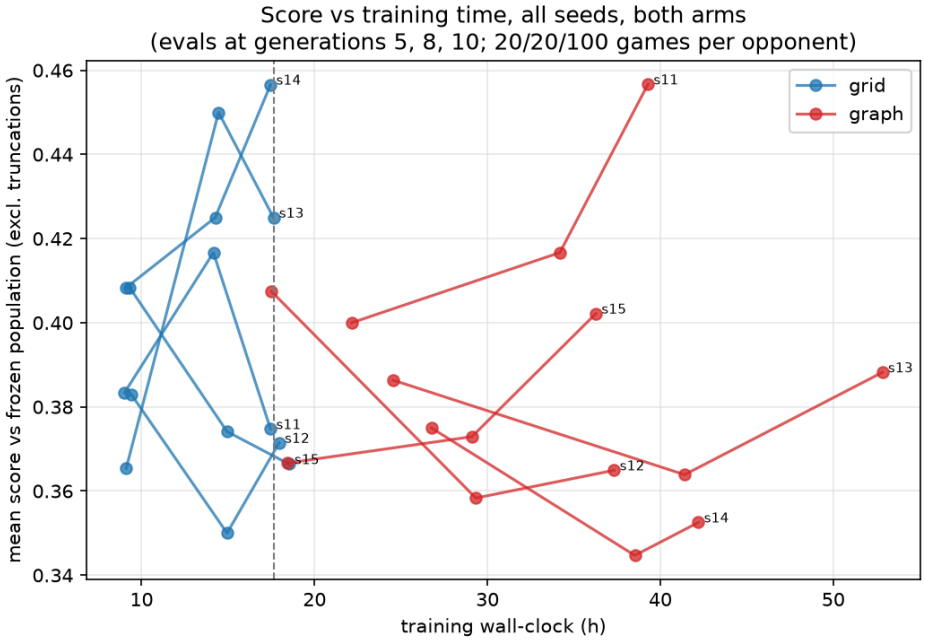


Fig. 11 : Seconde étude : score moyen contre la population gelée (moyenne des trois adversaires, troncatures exclues) en fonction du temps mural d'entraînement cumulé, pour les dix exécutions à démarrage à chaud ; évaluations aux générations 5, 8 et 10 (20, 20 et 100 parties par adversaire). Bleu : bras grille ; rouge : bras graphe ; une ligne par graine, étiquetée à son extrémité. La ligne verticale en pointillés marque le seuil à temps mural égal de la seconde étude, 17.63 heures.

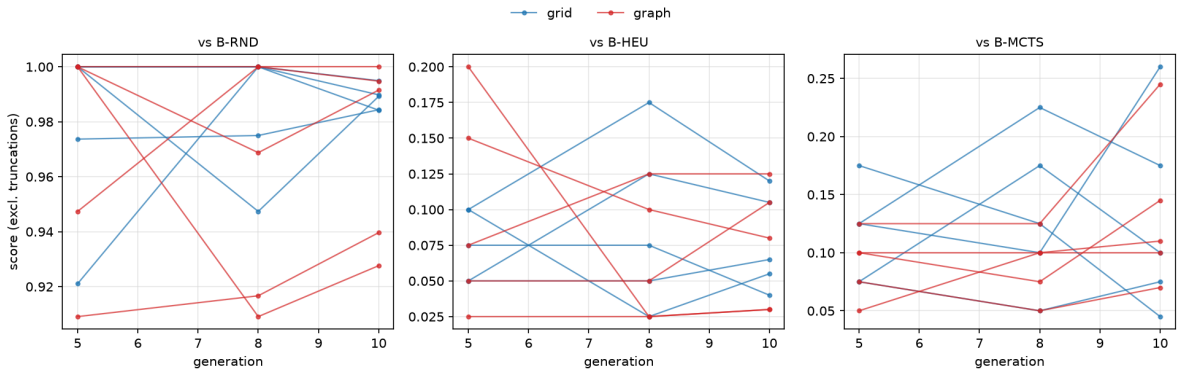


Fig. 12 : Seconde étude : score contre chaque adversaire gelé aux générations 5, 8 et 10 (20, 20 et 100 parties appariées par adversaire), une ligne par exécution, cinq graines à démarrage à chaud par bras ; en bleu, le bras grille ; en rouge, le bras graphe ; troncatures exclues. Les points intermédiaires reposent sur 20 parties et ne servent qu'à la trajectoire.

La Fig. 11 place les dix exécutions sur un axe de temps commun : les exécutions grille s'achèvent entre 17.46 et 18.52 heures de temps mural d'entraînement, les exécutions graphe entre 36.27 et 52.85 heures, et au seuil les deux nuages se recouvrent en score. La Fig. 12 montre les trajectoires par adversaire ; les points intermédiaires reposent sur 20 parties et ne se lisent que pour leur forme. Les profils d'horloge d'entraînement des exécutions graphe ont pris deux formes, une montée jusqu'à un pic vers la génération 4 suivie d'une descente vers des durées proches de celles de la grille à la génération 10 pour les graines 11 et 14, et un profil plat pour les graines 12, 13 et 15 ; l'horloge par génération de la graine 13 a crû de façon monotone de 14 456 s à 21 112 s et en a fait l'exécution la plus lente de la campagne, à 52.85 heures.

Tableau 44 : Seconde étude, résumé des coûts et des scores par bras. Le temps mural d'entraînement couvre la génération en auto-jeu, l'entraînement et l'export pour les dix générations d'une exécution, parties d'évaluation exclues, sur la machine d'étude (Apple M1 Pro, quatre threads de travail, exécutions séquentielles) ; coût d'auto-jeu = ce temps mural divisé par 5 000 parties. Score contre la population = moyenne, sur les trois adversaires gelés, du score moyen au niveau des graines, troncatures exclues. Les paramètres, l'inférence et les mesures de référence du débit sont inchangés par rapport à la première étude.

Métrique	**Bras grille**	**Bras graphe**
Temps mural d'entraînement par exécution, graines 11 à 15	17.46 / 17.95 / 17.63 / 17.46 / 18.52 h	39.28 / 37.33 / 52.85 / 42.20 / 36.27 h
Temps mural d'entraînement par exécution, moyenne de 5 graines	17.80 h	41.59 h (2.34×)
Coût moyen d'auto-jeu	12.8 s/partie	29.9 s/partie
Score contre la population, lecture à exemples égaux	0.399	0.393
Score contre la population, lecture à temps mural égal (seuil 17.63 h)	0.402	0.381

Les dix exécutions ont consommé 296.9 heures de temps mural d'entraînement (grille 89.0 h, graphe 207.9 h), contre une projection d'environ 295 heures issue du pilote, sur un seul ordinateur portable, sans calcul payant. Le bras graphe a coûté 2.34 fois le bras grille par exécution, contre 2.04 dans la première étude : la fenêtre de rejeu ajoute du temps d'entraînement en proportion du nombre d'enregistrements sur les deux bras, et l'entraînement du bras graphe est le plus coûteux des deux. Les scores contre la population diffèrent de 0.006 à exemples égaux et de 0.021 à temps égal en faveur du bras grille, contre 0.081 et 0.079 dans la première étude.

15.6 Les deux études côte à côte

La comparaison entre les études est descriptive. Les deux études utilisent des ensembles de graines différents, deux changements de régime ont été appliqués à la fois, et aucun intervalle n'est calculé d'une étude à l'autre ; les deux jeux d'intervalles sont placés l'un à côté de l'autre et lus comme tels.

Tableau 45 : Le contraste graphe–grille des deux études, par adversaire gelé et par lecture de budget : différence des scores moyens au niveau des graines avec l'intervalle bootstrap à 95% sur les graines (cinq graines indépendantes par bras dans chaque étude, rééchantillonnées indépendamment, 10 000 rééchantillonnages). Première étude : graines 1 à 5, le réseau de chaque génération entraîné depuis une initialisation nouvelle sur les parties de cette génération (seuil 18.77 h). Seconde étude : graines 11 à 15, démarrage à chaud et fenêtre de rejeu de trois générations (seuil 17.63 h). Les études ne sont pas agrégées.

Adversaire	Lecture	Première étude : graphe − grille	Seconde étude : graphe − grille
B-RND	exemples égaux	−0.169 [−0.272, −0.062]	−0.018 [−0.045, +0.008]
B-HEU	exemples égaux	−0.064 [−0.111, −0.017]	−0.003 [−0.046, +0.039]
B-MCTS	exemples égaux	−0.009 [−0.055, +0.028]	+0.003 [−0.082, +0.090]
B-RND	temps mural égal	−0.161 [−0.278, −0.048]	−0.052 [−0.140, +0.002]
B-HEU	temps mural égal	−0.059 [−0.087, −0.029]	+0.013 [−0.032, +0.055]
B-MCTS	temps mural égal	−0.018 [−0.066, +0.025]	−0.025 [−0.091, +0.034]

Le contraste s'est rapproché de zéro contre chaque adversaire sous les deux lectures. Contre l'adversaire aléatoire, il est passé de −0.169 à −0.018 à exemples égaux et de −0.161 à −0.052 à temps égal ; contre l'heuristique, de −0.064 à −0.003 et de −0.059 à +0.013 ; contre l'adversaire de recherche, de −0.009 à +0.003 et de −0.018 à −0.025. Quatre contrastes qui excluaient zéro dans la première étude l'incluent dans la seconde, et le signe a changé dans deux cellules, toutes deux à l'intérieur d'intervalles qui incluent zéro. Le déficit du bras graphe de la première étude n'est plus visible à exemples égaux et est réduit à temps mural égal, où il survit sous la forme d'un intervalle touchant zéro contre l'adversaire aléatoire. Ce que la seconde étude ne montre pas, c'est un avantage quelconque du bras graphe : ses deux contrastes ponctuels positifs (+0.003 et +0.013) sont les plus petits nombres de leurs lignes.

Les niveaux absolus des bras ont bougé dans des directions différentes. La moyenne du bras graphe contre l'adversaire aléatoire est montée de 0.812 à 0.971 à exemples égaux, là où logeait son déficit de la première étude ; ses moyennes contre l'heuristique et l'adversaire de recherche sont passées de 0.065 à 0.074 et de 0.115 à 0.134. La moyenne du bras grille contre l'heuristique est descendue de 0.129 [0.098, 0.165] à 0.077 [0.051, 0.104], deux intervalles qui ne se recouvrent pas, tandis que ses moyennes contre l'adversaire aléatoire et l'adversaire de recherche sont restées à moins de 0.01 (0.981 à 0.988 ; 0.124 à 0.131). Lu de façon descriptive, le régime à démarrage à chaud avec fenêtre de trois générations n'a pas élevé la force atteinte contre les adversaires forts en dix générations à ces réglages ; il a resserré l'écart entre les bras, et pour le bras grille contre l'heuristique il a coïncidé avec un niveau plus bas. Que ce niveau plus bas soit un effet du régime (un réseau transmis de génération en génération s'entraîne sur une fenêtre qui comprend ses propres parties antérieures) ou de l'ensemble de graines ne peut être tranché à partir de deux études de cinq graines, et l'étude ne le tranche pas.

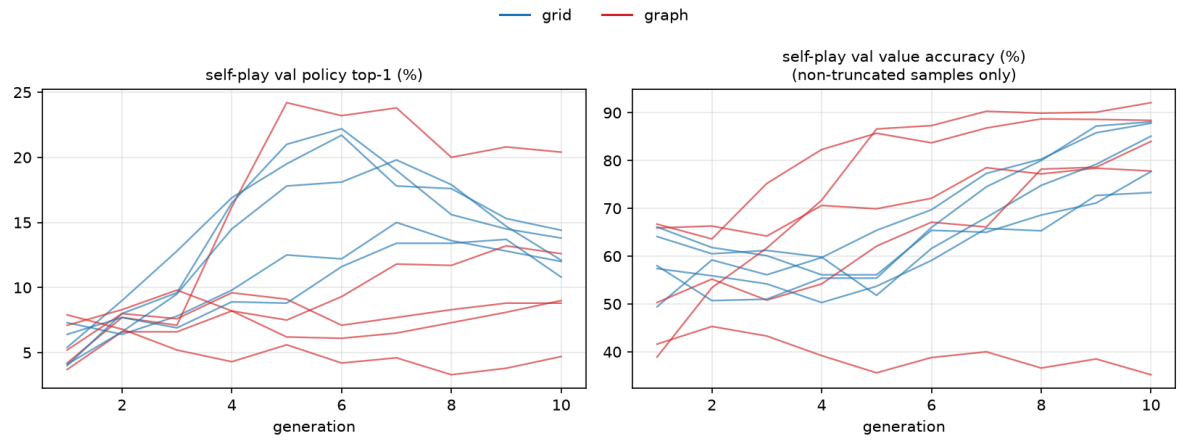


Fig. 13 : Seconde étude : exactitude top-1 de la politique en validation (à gauche) et exactitude de la valeur sur les échantillons non tronqués (à droite) après la première époque de chaque

génération, telles qu'imprimées par l'entraîneur sur la partition de validation de l'ensemble d'entraînement fenêtré de la génération ; une ligne par exécution, cinq graines à démarrage à chaud par bras ; en bleu, le bras grille ; en rouge, le bras graphe. Diagnostics d'entraînement, non mesures de force.

Les diagnostics d'entraînement (Fig. 13) séparent une exécution des neuf autres. Neuf exécutions terminent la dixième génération avec une exactitude de la valeur comprise entre 73.3% et 92.1% sur leur partition de validation ; la graine graphe 15 ne dépasse jamais 45% et termine à 35.2%, avec un top-1 de la politique qui descend de 7.9% à 4.7%, et pourtant son point de contrôle final obtient 0.992 contre l'adversaire aléatoire (41% de troncature), 0.105 contre l'heuristique et 0.110 contre l'adversaire de recherche, à l'intérieur des bandes des autres graines. Dans la première étude, l'exactitude de la valeur de chaque exécution graphe était montée dans la bande du bras grille. L'exécution est conservée, comme l'exigent les pré-engagements ; elle montre que sous ce régime un réseau peut jouer au niveau de ses pairs tout en ajustant mal ses propres données fenêtrées, et l'étude ne dispose d'aucune mesure qui l'explique.

Trois limites bornent cette section. Le mouvement d'une étude à l'autre n'est pas attribuable séparément au démarrage à chaud ou à la fenêtre, puisque les deux ont été appliqués à la fois, par construction. Les hyperparamètres sont les réglages par défaut non ajustés de la première étude dans les deux bras, un choix fait au gel et énoncé comme une limite des deux études ; un bras graphe ajusté, ou un bras grille ajusté, pourrait se situer ailleurs. Et dix générations restent un régime précoce : la seconde étude a rapproché le pipeline de la pratique courante des systèmes d'auto-jeu, et la force absolue contre les adversaires forts n'a pas bougé avec lui, ce qui dit que le budget, et non seulement le régime, borne ce que l'une ou l'autre représentation montre ici.

Dans ces limites, la seconde étude répond à l'objection qu'elle a été conçue pour tester. L'ordre de la première étude ne survit pas au régime à démarrage à chaud sous la forme d'un déficit : la représentation en graphe, transmise de génération en génération et entraînée sur une fenêtre de parties récentes, joue contre la population gelée aussi bien que la représentation en grille à exemples égaux, dans les limites d'intervalles à cinq graines, et à moins de 0.05 d'elle à temps égal, pour 2.34 fois le coût. Elle ne joue pas mieux. L'hypothèse d'origine, selon laquelle la représentation en graphe apprend une meilleure politique à budget égal, n'est étayée dans aucun des deux régimes ; ce qui a changé de l'un à l'autre est la taille de l'écart, d'un net avantage de la grille à aucun que cinq graines puissent détecter, avec une pathologie de conversion que le régime a fait disparaître dans deux graines graphe et laissée dans trois.

Partie V. Discussion et conclusion

16 Discussion et menaces à la validité

L'hypothèse a été rejetée exactement comme le protocole gelé définissait le rejet. Ce chapitre demande ce que le résultat signifie, quelles explications les données soutiennent et lesquelles elles ne font que suggérer, comment le résultat se rapporte aux travaux antérieurs qui l'ont motivé, et ce qui pourrait être erroné en lui. Les menaces sont organisées comme le plan de recherche le prescrit : internes, de mesure, statistiques et externes.

16.1 Lire le résultat

Trois observations déterminent la manière dont le résultat doit être lu.

Premièrement, le déficit est modelé par l'adversaire. Il est le plus grand contre l'adversaire aléatoire légal, net contre l'heuristique et absent contre l'adversaire de recherche (Tableau 31). Un bras graphe qui serait simplement plus faible partout montrerait un déficit uniforme ; un déficit concentré là où les parties se décident en achevant un encerclement auquel rien ne résiste désigne la *conversion tactique locale* plutôt que le jugement positionnel.

Deuxièmement, le canal de troncature porte le mécanisme. Le bras graphe gagne du matériel contre l'adversaire aléatoire puis échoue à conclure : 20–57% de ces parties se terminent au plafond de 300 demi-coups, contre 0–11% pour le bras grille (Fig. 7, Fig. 6 F1). Ce motif est invisible dans toute étude qui compte les parties plafonnées comme des nulles, et c'est pourquoi le protocole a exigé, avant la première exécution, que la troncature soit une issue à part entière.

Troisièmement, les ablations affinent la lecture. Aux réglages de parité, retirer les relations typées par direction a fait plus qu'affaiblir le bras graphe ; cela a détruit d'emblée l'entraînabilité (divergence non finie à la génération 0 pour les trois graines), de sorte que les relations typées portent au minimum la stabilité d'optimisation du bras entier. Retirer le biais de pooling global a modifié les scores de $\approx 0.00 \pm 0.17$, -0.01 ± 0.04 et -0.01 ± 0.03 contre les trois adversaires, avec des modes de défaillance inchangés. La machinerie distinctive du bras réside donc dans ses relations directionnelles plutôt que dans son pooling. Une fois la variante naïve stabilisée par écrêtage de gradient, cependant, elle joue dans la bande du bras complet, de sorte que la contribution mesurable des relations typées à cette échelle se concentre dans l'optimisation plutôt que dans la représentation (Chapitre 13).

16.2 Quelles explications sont démontrées, et lesquelles sont plausibles

Le plan de recherche distingue une explication démontrée par ablation d'une explication simplement plausible. Quatre explications candidates de l'écart méritent ce traitement.

Géométrie (plausible, non démontrée). La tactique de Hive est dominée par la géométrie d'encerclement à courte portée, le régime dans lequel Keller et al. (2023) ont trouvé les réseaux convolutifs plus forts que les réseaux de graphes sur Hex. Une convolution 3×3 sur le cadre déplié voit tout le voisinage d'une cellule, et les blocs résiduels empilés voient

le voisinage du voisinage, sans coût d'apprentissage ; le réseau à passage de messages doit composer les mêmes motifs à partir de six relations typées, un saut à la fois. Cela est cohérent avec chacune des observations ci-dessus, mais aucune ablation de cette étude n'isole la composition du champ récepteur, de sorte que cela reste une hypothèse.

Capacité (contrôlée). Les deux réseaux diffèrent de +1.5% en nombre de paramètres (1.44 M contre 1.47 M), en faveur du bras graphe. La capacité n'explique pas un déficit du graphe.

Décodeur (contrôlé). Les deux bras scorent l'ensemble identique de paires légales (pièce, destination) avec un masquage, une normalisation, des cibles et un départage identiques ; le décodeur ne peut favoriser aucun bras. Ce qui diffère est la *paramétrisation* derrière l'interface partagée : un tenseur plat à 28 673 voies pour le bras grille, une fonction de scorage par candidat pour le bras graphe. Cette différence fait partie de ce que « représentation » signifie dans cette étude plutôt que d'en être un facteur de confusion.

Vitesse (démontrée pour la seconde lecture seulement). Les générations du bras graphe coûtent environ deux fois plus de temps mural sur cette machine, de sorte qu'à temps égal il achève moins de la moitié de ses générations (Chapitre 12). Cela élargit de façon démontrable l'écart dans la lecture à temps mural égal ; cela ne joue aucun rôle dans la lecture à exemples égaux, où le déficit existe déjà.

Optimisation (partiellement démontrée). Les ablations montrent que l'optimisation du bras graphe complet est fragile d'une manière précise (elle dépend des relations typées pour sa stabilité), mais les courbes d'entraînement du bras complet (Fig. 8) le montrent ajustant ses données pour chaque graine. Rien n'indique que le bras complet ait échoué à s'optimiser ; l'écart est un écart entre deux réseaux entraînés. La question de savoir si un taux d'apprentissage ou une normalisation différents auraient favorisé le bras graphe n'a pas été testée, parce qu'aucun des deux bras n'a reçu de réglage. La politique de réglage nul est symétrique, mais elle n'est pas neutre si une architecture est plus sensible à ses valeurs par défaut, un point repris sous la validité externe.

16.3 Rapport aux travaux antérieurs

Le résultat ne contredit ni les constats positifs de Rigaux et Kashima (2024) sur les graphes aux échecs, ni l'avantage à longue portée que Keller et al. (2023) ont mesuré sur Hex. Il borne là où leur optimisme se transfère. Le résultat aux échecs repose sur une seule exécution d'entraînement par modèle avec des intervalles couvrant la seule estimation Elo, un appariement de capacité lâche et des décodeurs propres à chaque bras ; le résultat sur Hex a été obtenu avec un apprenant fondé sur la valeur, avec une construction de graphe propre au jeu, et a trouvé le bras convolutif plus aigu sur les motifs locaux, c'est-à-dire le régime qui décide des parties de Hive aux petits budgets. Le présent rejet est cohérent entre graines sur cinq exécutions par bras sous deux lectures de budget pré-enregistrées, à capacité appariée et avec le décodeur partagé. Lues ensemble, les trois études suggèrent que le signe de la comparaison grille-contre-graphe dépend de la portée tactique du jeu et du standard de preuve, plutôt que du paradigme des graphes en tant que tel. L'unique étude antérieure de Hive de type AlphaZero (de Goede et al., 2022) avait déjà montré que le choix d'encodage change l'apprentissage au sein de la famille grille ; cette étude étend le constat d'une famille à l'autre et trouve le côté grille en avance.

16.4 Validité interne : bogues et comparabilité

Le moteur est validé indépendamment de l'apprentissage : égalité perft avec les tables publiées jusqu'à la profondeur 6 pour les huit types de jeu, conformité de protocole 21/21, accord différentiel des ensembles de coups légaux sur 27 829 positions avec deux moteurs de référence, un corpus de 30 cas dérivé des règles et annoté avant toute exécution, et 10.9 millions de transitions aléatoires sans violation d'invariant (Chapitre 5). Les deux encodeurs sont épinglés à l'octet près contre des tests dorés (golden tests) inter-langages exécutés chaque nuit (240 et 160 positions). Les bras partagent le décodeur, les enregistrements, les fonctions de perte, les budgets, la recherche et l'évaluation gelée ; la capacité diffère de +1.5% et est rapportée.

Des risques résiduels demeurent. L'architecture graphe est un point de l'espace de conception ; un réseau de graphes différent pourrait se comporter différemment, et rien n'est affirmé au-delà de celui-ci. Des défauts d'outillage ont été trouvés *pendant* l'étude : un chemin du lanceur de matchs qui perdait des enregistrements lorsque chaque partie d'un match était tronquée, une boucle d'entraînement qui laissait passer silencieusement des valeurs non finies pendant dix générations, et un générateur de résultats dont la moyenne de temps mural par exécution divisait encore par les trois exécutions originales après l'extension à cinq. Chacun a été attrapé par la discipline de vérification (Chapitre 10), corrigé et audité : le premier n'a touché aucune donnée de campagne ; le second n'a affecté que l'ablation qui l'a révélé et est devenu le résultat de cette ablation ; le troisième a affecté deux lignes de la table des coûts et aucun score, intervalle ni affirmation. Ils illustrent la leçon la plus pratique de l'étude : à cette échelle, le mode de défaillance dominant est l'erreur de harnais plutôt que le hasard, et seule la vérification mécanique l'attrape.

16.5 Validité de mesure : adversaires et troncature

Les scores sont relatifs à une population de trois adversaires fixes couvrant une force du plancher au niveau intermédiaire ; aucune affirmation de force universelle n'est faite. Tous les bras entraînés perdent encore lourdement contre les deux lignes de base fortes après dix générations, de sorte que la comparaison vit dans un régime de scores bas où les différences contre l'adversaire de recherche ne peuvent pas être résolues. La troncature est traitée comme une issue à part entière avec des bornes de sensibilité : le plafond ne peut pas renverser le verdict, mais les taux de troncature élevés du bras graphe signifient que son score contre l'adversaire aléatoire est mesuré sur moins de parties décidées (43–80 par graine), ce qui élargit son intervalle. Les ouvertures d'évaluation sont gelées et partagées, de sorte qu'un effet de l'ensemble d'ouvertures affecterait les deux bras de la même manière ; il pourrait néanmoins modeler les scores absolus.

16.6 Validité statistique : graines et dépendances

Cinq graines par bras bornent les statistiques. Le rejet repose sur plus qu'un seul intervalle : sur la cohérence entre graines à travers deux adversaires et deux lectures simultanément, sur quatre des six intervalles excluant zéro, et sur une analyse des bornes du plafond. L'extension pré-engagée de trois à cinq graines a resserré quatre des six intervalles et absorbé les graines les plus favorables au graphe observées sans changer le verdict ; le contraste contre l'adversaire de recherche reste indiscernable de zéro sous les

deux lectures et est rapporté comme tel. Le contraste entre bras applique le bootstrap aux graines plutôt qu'aux parties, de sorte qu'aucune pseudo-réplication au niveau des parties n'entre dans aucun intervalle, et les ensembles de graines des deux bras sont rééchantillonnés indépendamment, ce qui est conservateur pour un plan apparié dans lequel les deux bras jouent les mêmes ouvertures.

16.7 Validité externe : variante, matériel et budget

Une variante (Hive de base, sans extensions), une machine, un petit budget (dix générations de 500 parties ; entre 16.7 et 19.1 heures de temps mural d'entraînement par exécution grille et entre 26.6 et 49.9 heures par exécution graphe), auto-jeu en régime précoce tout du long. L'asymétrie de fournisseur d'inférence est une propriété authentique du matériel de déploiement : le réseau convolutif s'exécute le plus vite sur l'accélérateur neuronal de la machine, le réseau de graphes riche en opérations de collecte (gather) le plus vite sur le CPU. Elle est rapportée et imputée, et sur d'autres accélérateurs la lecture à temps mural pourrait se déplacer tandis que la lecture à exemples égaux ne le ferait pas. La politique de réglage nul est symétrique mais peut ne pas être neutre : les réseaux de graphes sont communément plus sensibles aux valeurs par défaut de l'optimiseur que les réseaux convolutifs résiduels, et la première ablation a montré que cette famille est fragile sous ces valeurs par défaut ; un bras graphe réglé pourrait réduire l'écart, au prix de la rupture de la symétrie de la comparaison. Rien ici ne se généralise à d'autres jeux, à de plus grands budgets ou à des architectures de graphes plus riches. L'étude répond à sa question pré-enregistrée à l'intérieur de son périmètre pré-enregistré, et la réponse négative est le constat. Le choix d'encodage importe, comme l'étude antérieure sur Hive l'avait montré, et pour Hive à petit budget il favorise la grille.

16.8 Une propriété du pipeline qui borne la force absolue atteinte

Une propriété du pipeline partagé mérite d'être énoncée aussi clairement que possible, parce qu'elle conditionne la manière dont les « dix générations » de cette étude doivent être lues. Le pilote de campagne entraîne le réseau de chaque génération à partir d'une initialisation fraîche issue de la graine, pendant deux époques, sur les seuls enregistrements de cette génération ; il ne transmet à l'entraîneur ni point de contrôle de démarrage à chaud ni fenêtre de rejeu (Chapitre 6). La boucle s'améliore néanmoins de génération en génération, parce que le réseau de la génération g est entraîné sur des parties jouées par la recherche guidée par le réseau de la génération $g-1$, et que cette recherche est plus forte que le réseau brut ; mais aucun poids n'est reporté et aucun enregistrement n'est réutilisé. Chaque réseau voit donc les positions de 500 parties, ce qui est un petit ensemble d'entraînement au regard des standards de l'apprentissage par auto-jeu, et la force absolue atteinte après dix générations (près du plafond de score contre l'adversaire aléatoire, autour de 0.1 contre les deux adversaires forts) doit être lue à la lumière de ce fait. Pour la *comparaison*, la propriété est neutre : elle est identique dans les deux bras, les deux encodeurs voient exactement les mêmes enregistrements, et le protocole gelé spécifiait des conventions d'entraînement identiques plutôt qu'une politique particulière de démarrage à chaud. Pour la *généralisation* du résultat, c'est une limite qui s'ajoute à celles déjà énumérées : un pipeline qui accumulerait les données

et reporterait les poids atteindrait un régime différent, dans lequel l'ordre des bras est une question ouverte. Cette étude successeur a été menée après la première rédaction de ce rapport, les deux changements étant appliqués à l'identique aux deux bras sous un second protocole gelé, et elle est rapportée dans Chapitre 15 : sous une boucle à démarrage à chaud avec une fenêtre de rejeu de trois générations, le déficit n'est plus visible à exemples égaux et est réduit à temps égal, aucun avantage du graphe n'apparaît, et la force atteinte contre les deux adversaires forts ne monte pas.

16.9 Ce que nous ferions différemment

Quatre changements renforceraient une étude successeur sans changer sa logique. Le premier, le démarrage à chaud et la fenêtre de rejeu qui viennent d'être décrits, a été réalisé sous la forme de la seconde étude ; il laisse ouverte la question du réglage évoquée plus bas, et ses intervalles plus larges ont montré que la règle de rejet a besoin d'un seuil chiffré pour ce qui compte comme un avantage substantiel (Chapitre 15.5). Les évaluations intermédiaires (20 parties par adversaire aux générations 5 et 8) ont été dimensionnées pour le suivi et sont trop bruitées pour porter une inférence ; une étude successeur devrait évaluer chaque génération au volume final si les courbes doivent être analysées. La lecture à temps égal devrait être accompagnée d'une lecture à *énergie* égale sur des accélérateurs hétérogènes, puisque le temps mural seul cache l'asymétrie de fournisseur. Et la symétrie du réglage nul devrait être complétée par un petit budget de réglage identique et pré-enregistré pour les deux bras, de sorte que la fragilité sous valeurs par défaut soit mesurée plutôt qu'héritée.

17 Conclusion et perspectives

17.1 La réponse à la question de recherche

Le périmètre testé est Hive en jeu de base, un réseau relationnel simple à passage de messages à capacité appariée contre un réseau convolutif en grille, dix générations d'auto-jeu à petit budget, cinq graines indépendantes par bras, une population gelée de trois adversaires et 250 ouvertures gelées. À l'intérieur de ce périmètre, **la réponse à la question de recherche principale est non** : la représentation en graphe n'a pas appris une meilleure politique que la représentation en grille, ni lorsque les deux bras ont consommé le même nombre d'exemples d'entraînement, ni lorsque les deux ont reçu le même temps mural sur la même machine, et la règle de rejet pré-enregistrée s'est déclenchée exactement telle qu'elle avait été gelée. Le déficit du bras graphe est le plus grand là où Hive est le plus tactique (contre l'adversaire aléatoire légal, −0.169 et −0.161 sous les deux lectures) et net contre l'adversaire heuristique (−0.064 et −0.059) ; contre l'adversaire de recherche, les deux bras sont indiscernables (−0.009 et −0.018, intervalles chevauchant zéro). Le bras graphe paie environ deux fois le temps mural par exécution. Son mode de défaillance caractéristique consiste à gagner du matériel sans convertir le gain ; il se manifeste en ce que 20–57% de ses parties contre l'adversaire aléatoire se terminent au plafond de coups, et il n'est observable que parce que la troncature n'a jamais été repliée en nulles.

La seconde étude, pré-enregistrée séparément et menée sous une boucle à démarrage à chaud avec une fenêtre de rejeu de trois générations sur cinq nouvelles graines par bras, a testé la principale objection à cette réponse. Elle n'a trouvé aucun avantage du graphe sous l'une ou l'autre lecture : chaque intervalle graphe−grille inclut zéro, les plus grandes bornes supérieures étant +0.090 et +0.055 ; le déficit de la première étude n'est plus visible à exemples égaux et est réduit à temps égal ; la pathologie de conversion persiste dans trois graines graphe sur cinq ; et le bras graphe coûte 2.34 fois le bras grille par exécution. La réponse à la question de recherche tient donc dans la forme que les deux études réunies autorisent : à ce budget, sur cette variante, la représentation en graphe n'a appris une meilleure politique que la représentation en grille dans aucun des deux régimes d'entraînement, et sous le régime standard elle en a appris une équivalente, dans les limites d'intervalles à cinq graines, pour plus du double du coût.

Le résultat borne, plutôt qu'il ne contredit, les constats positifs rapportés pour les graphes à Hex sous apprentissage fondé sur la valeur et aux échecs sous auto-jeu. Là où les tactiques d'encerclement à courte portée décident des parties et où les budgets sont petits, les artefacts de cadre d'un encodage en grille se révèlent moins coûteux que l'absence de cadre. L'analyse par composant ajoute une lecture mécaniste : des deux composants distinctifs du bras graphe, les relations typées par direction sont nécessaires à l'optimisation elle-même (les retirer à réglages identiques a fait diverger l'entraînement pour chaque graine), tandis que le biais de pooling global est dispensable à cette échelle.

17.2 Contributions, telles que démontrées

Quatre contributions ont été annoncées en Chapitre 1 ; chacune est démontrée dans les chapitres qui y sont nommés. (1) Une comparaison grille-contre-graphe contrôlée, à budget apparié et multi-graines pour Hive, les deux bras sous un même pipeline d'auto-jeu, avec une réponse négative pré-enregistrée (Chapitre 11, Chapitre 12). (2) Un harnais de comparaison reproductible pour les jeux sans cadre et à empilement, publié avec les enregistrements bruts : un moteur validé côté règles, deux encodeurs épinglés à l'octet près entre deux langages, un décodeur partagé à actions variables, une évaluation consciente de la troncature et une discipline d'artefacts gelés (Chapitre 5 à Chapitre 9). (3) Une attribution par composant pour le bras graphe, avec un supplément à deux composants honnêtement étiqueté (Chapitre 13). (4) Une seconde comparaison pré-enregistrée sous le régime standard à démarrage à chaud, dont le protocole, le seuil et les graines ont été fixés avant toute exécution, et dont le résultat nul borne le résultat négatif de la première étude (Chapitre 15).

17.3 Deux suites motivées par les données

La première suite vise la **pathologie de conversion**, que les données isolent comme un défaut de politique plutôt qu'un défaut de valeur : l'exactitude de la valeur du bras graphe en entraînement est comparable à celle du bras grille, tandis que sa politique échoue sur les séquences forcées (Chapitre 14). Une expérience naturelle est un remède au moment de la recherche, appliqué à l'identique aux deux bras sous la même évaluation gelée, en tant que changement à un seul composant du pipeline partagé : un budget d'évaluation plus profond dans les positions que la tête de valeur juge déjà gagnées, ou des cibles d'entraînement auxiliaires pour les coups forçants. La seconde suite concerne le **constat de stabilité** : l'ablation et son supplément ont montré que les relations typées par direction importent surtout pour l'optimisation à cette échelle, mais l'écrêtage de gradient du supplément confond l'attribution. Une étude contrôlée des choix de normalisation et d'échelle de gradient pour des couches de graphe à relations partagées découplerait l'entraînabilité du contenu représentationnel et dirait si l'adjacence naïve, correctement stabilisée, est suffisante pour Hive. L'une et l'autre suite ont désormais le régime à démarrage à chaud pour ligne de base, et l'une et l'autre devraient pré-enregistrer un seuil chiffré pour un avantage substantiel, dont les intervalles de la seconde étude ont montré la nécessité.

17.4 Une méthode qui tient indépendamment du signe

Le pré-enregistrement avec artefacts gelés, les deux lectures budgétaires, la troncature comme issue, l'inférence au niveau des graines et la rédaction au fil de l'avancement des travaux ont transformé une réponse négative en un objet scientifique utilisable sur un seul ordinateur portable. La même discipline, décrite en Chapitre 10 avec les incidents qu'elle a détectés, est la part de ce travail la plus directement transférable à d'autres études à petit budget de calcul.

Bibliographie

Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A., & Bellemare, M. G. (2021). Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems* (NeurIPS 2021) (peer-reviewed; Outstanding Paper Award). arXiv:2108.13264 (v4, 5 January 2022). https://doi.org/10.48550/arXiv.2108.13264

Ben-Assayag, S., & El-Yaniv, R. (2021). Train on small, play the large: Scaling up board games with AlphaZero and GNN. arXiv preprint arXiv:2107.08387v1 (18 July 2021) (no peer-reviewed version found as of 26 September 2026). https://doi.org/10.48550/arXiv.2107.08387

Cazenave, T., Chen, Y.-C., Chen, G.-W., Chen, S.-Y., Chiu, X.-D., Dehos, J., Elsa, M., Gong, Q., Hu, H., Khalidov, V., Li, C.-L., Lin, H.-I., Lin, Y.-J., Martinet, X., Mella, V., Rapin, J., Roziere, B., Synnaeve, G., Teytaud, F., Teytaud, O., Ye, S.-C., Ye, Y.-J., Yen, S.-J., & Zagoruyko, S. (2020). Polygames: Improved zero learning. *ICGA Journal*, 42(4), 244–256. https://doi.org/10.3233/ICG-200157. Preprint: arXiv:2001.09832 (January 2020).

de Goede, D., Kampert, D., & Varbanescu, A. L. (2022). The cost of reinforcement learning for game engines: The AZ-Hive case-study. In *Proceedings of the 13th ACM/SPEC International Conference on Performance Engineering* (ICPE 2022, Beijing), pp. 145–152. https://doi.org/10.1145/3489525.3511685

Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems* (NIPS 2017) (peer-reviewed). arXiv:1706.02216 (v4, 10 September 2018). https://doi.org/10.48550/arXiv.1706.02216

Jones, A. L. (2021). Scaling scaling laws with board games. arXiv preprint arXiv:2104.03113v2 (15 April 2021) (not peer-reviewed). https://doi.org/10.48550/arXiv.2104.03113

Kampert, D., Varbanescu, A.-L., Müller-Brockhausen, M., & Plaat, A. (2021). Mimicking the human approach in the game of Hive. In *2021 IEEE Symposium Series on Computational Intelligence* (SSCI 2021, Orlando). IEEE Xplore document 9659999. https://ieeexplore.ieee.org/document/9659999/. Preprint circulated as « Better AI for Hive: Mimicking human game-play strategies ».

Keller, Y., Blüml, J., Sudhakaran, G., & Kersting, K. (2023). From images to connections: Can DQN with GNNs learn the strategic game of Hex? arXiv preprint arXiv:2311.13414 (22 November 2023) (not peer-reviewed; OpenReview submission dYaeDrazj5). https://arxiv.org/abs/2311.13414

Rigaux, T., & Kashima, H. (2024). Enhancing chess reinforcement learning with graph representation. In *Advances in Neural Information Processing Systems 37* (NeurIPS 2024, main conference track). https://doi.org/10.52202/079017-0006. Preprint: arXiv:2410.23753v1 (31 October 2024), https://doi.org/10.48550/arXiv.2410.23753

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144 (peer-reviewed). https://doi.org/10.1126/science.aar6404. Preprint (2017): Mastering chess and shogi by self-play with a general reinforcement learning algorithm, arXiv:1712.01815v1 (5 December 2017), https://doi.org/10.48550/arXiv.1712.01815

van der Pol, E., Worrall, D. E., van Hoof, H., Oliehoek, F. A., & Welling, M. (2020). MDP homomorphic networks: Group symmetries in reinforcement learning. In *Advances in Neural Information Processing Systems* (NeurIPS 2020) (peer-reviewed). arXiv:2006.16908 (v2, 20 January 2021). https://arxiv.org/abs/2006.16908

Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. In *Advances in Neural Information Processing Systems 28* (NIPS 2015) (peer-reviewed). arXiv:1506.03134 (v2, 2 January 2017). https://arxiv.org/abs/1506.03134

Wu, D. J. (2020). Accelerating self-play learning in Go. arXiv preprint arXiv:1902.10565v5 (9 November 2020); presented at the AAAI-20 Workshop on Reinforcement Learning in Games (not a full peer-reviewed proceedings paper). https://doi.org/10.48550/arXiv.1902.10565

Webographie

edre (GitHub user). *nokamute* : Hive engine in Rust, with a Universal Hive Protocol conformance tester and a built-in match runner [source-code repository]. GitHub. https://github.com/edre/nokamute. Version 1.0.3 utilisée comme moteur de référence. Consulté en juillet 2026 (conception du moteur) et le 9 septembre 2026 (campagne de validation).

jonthysell (GitHub user). *Mzinga* : reference Hive engine and project wiki, including the Universal Hive Protocol specification and the perft tables [source-code repository and wiki]. GitHub. https://github.com/jonthysell/Mzinga perft tables: https://github.com/jonthysell/Mzinga/wiki/Perft. Release MzingaEngine v0.16.0 utilisé comme moteur de référence. Consulté en juillet 2026 (conception du moteur) et le 9 septembre 2026 (campagne de validation).

Pfeifer, J. (GitHub user janpfeifer). (2018–2026). *hiveGo — Go implementation of Hive game* [source-code repository]. GitHub. Created 24 August 2018; main branch, commit d6ff95418d2b (20 August 2026). https://github.com/janpfeifer/hiveGo. Consulté le 26 septembre 2026.

World Hive Tournaments. *Rules of Hive: Rules FAQ* [web page]. https://www.worldhivetournaments.com/rules-of-hive/. Consulté le 9 septembre 2026.

Yianni, J. (2010). *Hive rules* [publisher's rules sheet]. Gen42 Games. https://www.gen42.com/wp-content/uploads/Hive-rules.pdf. Consulté le 9 septembre 2026.

Yianni, J. *The Pillbug: Additional Hive Pieces* [publisher's rules sheet]. Gen42 Games. https://www.gen42.com/wp-content/uploads/Pillbug_Rules.pdf. Consulté le 9 septembre 2026.

Annexes

A Corpus de positions annotées

Chaque attendu ci-dessous a été écrit à la main à partir des règles de l'éditeur avant toute exécution du moteur, afin que le corpus puisse servir d'oracle non biaisé ; le moteur a ensuite été exécuté contre lui. L'unique désaccord constaté a été résolu contre le corpus (une erreur de transit One-Hive dans une séquence de mise en place), le moteur ayant eu raison. Ces rendus sont générés directement à partir des cas de test exécutables, de sorte que les tests et cette annexe ne peuvent pas diverger. Les séquences de mise en place sont données dans la notation de coups de l'Universal Hive Protocol.

A.1 Corpus de règles critiques (30 cas)

Cas de correction des règles : placement, glissement/liberté de mouvement, portes, empilement, One-Hive, états terminaux, passe forcée, et cas d'étourdissement (stun) du Pillbug comme garde du noyau.

A.1.1 C001 : La Reine ne peut pas être placée au premier tour (règle de tournoi)

- **Règle :** Règle d'ouverture de tournoi (*Tournament variant of the official rules (README source 4); Gen42 2010 rulesheet p. 3 alone would allow it*)
- **Mise en place :** ``
- **Attente :** move_illegal `{'move': 'wQ'}`
- **Justification manuscrite :** La feuille de règles de base 2010 dit que la Reine « peut être placée à tout moment de votre premier à votre quatrième tour » (p. 3), mais la variante de tournoi (adoptée par l'UHP et par les deux moteurs de référence, et la convention que cette étude fixe) interdit de placer la Reine au premier tour de l'un ou l'autre joueur. Le premier coup des Blancs « wQ » doit donc être rejeté.

A.1.2 C002 : La Reine doit être placée au quatrième tour si elle ne l'a pas été avant

- **Règle :** Placement de votre Reine (*Gen42 Hive rulesheet p. 3*)
- **Mise en place :** `wS1;bS1 wS1-;wG1 -wS1;bG1 bS1-;wA1 -wG1;bA1 bG1-`
- **Attente :** all_moves_place `{'piece': 'wQ'}`
- **Justification manuscrite :** « Vous devez placer votre Reine à votre quatrième tour si vous ne l'avez pas placée avant. » (p. 3). C'est le quatrième tour des Blancs et wQ est encore en réserve, donc chaque coup légal doit être un placement de wQ. (Les coups de déplacement sont de plus exclus par la règle de Déplacement, p. 3 : aucun déplacement avant que la reine soit placée.) Légalité de la mise en place : chaque placement blanc ne touche que des pièces blanches, chaque placement noir que des pièces noires (Placement, p. 2).

A.1.3 C003 : Aucune pièce ne peut se déplacer avant que la reine de ce joueur soit placée

- **Règle :** Déplacement (*Gen42 Hive rulesheet p. 3*)
- **Mise en place :** `wS1;bS1 wS1-`
- **Attente :** all_moves_are_placements

- **Justification manuscrite :** « Une fois votre Reine placée (mais pas avant), vous pouvez décider d'utiliser chaque tour suivant pour placer une autre tuile ou pour déplacer l'une des pièces déjà placées. » (p. 3). La reine des Blancs n'est pas placée au tour 2, donc wS1 ne doit avoir aucun coup de déplacement ; seuls des placements sont proposés.

A.1.4 C004 : Après les premières pièces, les placements ne peuvent pas toucher la couleur adverse

- **Règle :** Placement (*Gen42 Hive rulesheet p. 2*)
- **Mise en place :** `wG1;bS1 wG1/`
- **Attente :** moves_for_piece `{'piece': 'wB1', 'moves': ['wB1 wG1\\', 'wB1 /wG1', 'wB1 -wG1']}`
- **Justification manuscrite :** « ...à l'exception de la première pièce placée par chaque joueur, les pièces ne peuvent pas être placées à côté d'une pièce de la couleur de l'adversaire. » (p. 2). wG1 est à l'origine avec bS1 à son nord-est. Des cinq voisines vides de wG1 (E, SE, SO, O, NO), les cellules E et NO touchent chacune aussi bS1 (ce sont les deux cellules adjacentes à la fois à wG1 et à sa voisine NE), donc une nouvelle pièce blanche ne peut aller qu'au SE, au SO ou à l'O de wG1. Placements attendus pour wB1 : exactement ces trois cellules. Géométrie à la main : axial E=(1,0), NE=(1,-1) ; les voisines de la cellule NE (1,-1) comprennent (1,0)=E-de-l'origine et (0,-1)=NO-de-l'origine.

A.1.5 C005 : La première pièce du second joueur rejoint la première pièce (contact ennemi permis)

- **Règle :** Déroulement de la partie / Placement (*Gen42 Hive rulesheet pp. 2-3*)
- **Mise en place :** `wS1`
- **Attente :** moves_for_piece `{'piece': 'bG1', 'moves': ['bG1 wS1-', 'bG1 wS1/', 'bG1 wS1\\', 'bG1 -wS1', 'bG1 /wS1', 'bG1 \\wS1']}`
- **Justification manuscrite :** « La partie commence par un joueur qui place une pièce de sa main au centre de la table, puis le joueur suivant joint l'une de ses propres pièces à celle-ci, bord à bord. » (p. 2). C'est l'exception de la première pièce à la règle de placement sur sa propre couleur. La première pièce des Noirs doit rejoindre wS1 bord à bord, donc bG1 peut être placée sur n'importe laquelle des six cellules adjacentes à wS1, et nulle part ailleurs.

A.1.6 C006 : Reine à la pointe de la ruche : exactement les deux glissements qui gardent le contact

- **Règle :** Reine / Liberté de mouvement / One-Hive (contact) (*Gen42 Hive rulesheet pp. 4, 9, 10*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-`
- **Attente :** moves_for_piece `{'piece': 'wQ', 'moves': ['wQ \\wS1', 'wQ /wS1']}`
- **Justification manuscrite :** La Reine « ne peut se déplacer que d'un espace par tour » (p. 4) en un mouvement de glissement (p. 10), et « toutes les pièces doivent toujours toucher au moins une autre pièce » (p. 3 NB). wQ est à la pointe ouest d'une ligne droite de quatre. De ses cinq voisines vides, seules les deux cellules qui sont aussi adjacentes à sa voisine wS1 (les cellules au NO et au SO de wS1) gardent le contact avec la ruche après le glissement ; les trois cellules plus à l'ouest ne touchent plus rien une fois la reine partie. Aucune des deux destinations n'est derrière une porte (pour chaque

glissement, des deux cellules flanquantes, l'une est occupée, l'autre vide). Attendu : exactement ces deux coups.

A.1.7 C007 : Fourmi enfermée dans une poche : la seule sortie est une porte, donc elle ne peut pas bouger

- **Règle :** Liberté de mouvement (*Gen42 Hive rulesheet p. 10*)
- **Mise en place :** `wA1;bS1 wA1-;wQ \wA1;bQ bS1-;wG1 -wA1;bB1 bQ/;wS1 /wA1;bB1 bS1/;wB1 \wQ;bB1 wA1/`
- **Attente :** moves_for_piece `{'piece': 'wA1', 'moves': []}`
- **Justification manuscrite :** « Si une pièce est entourée au point de ne plus pouvoir physiquement glisser hors de sa position, elle ne peut pas être déplacée. » (p. 10). Cinq des six voisines de la fourmi sont occupées. La seule voisine vide (au SE de la fourmi) est flanquée par bS1 (à l'E de la fourmi) et wS1 (au SO de la fourmi), c'est-à-dire les deux cellules adjacentes à la fois à la fourmi et à cet espace ; la fourmi ne peut donc pas physiquement y glisser. Retirer la fourmi ne scinderait PAS la ruche (l'anneau bB1-wQ-wG1-wS1 plus bS1 reste connexe), donc le blocage relève purement de la liberté de mouvement, pas de One-Hive. La fourmi, normalement la pièce la plus mobile, a zéro coup légal.

A.1.8 C008 : L'araignée se déplace d'exactement trois espaces le long du bord de la ruche (deux destinations)

- **Règle :** Araignée (*Gen42 Hive rulesheet p. 7*)
- **Mise en place :** `wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wS1 \wQ;bG1 bQ-`
- **Attente :** moves_for_piece `{'piece': 'wS1', 'moves': ['wS1 bS1/', 'wS1 wQ\\']}`
- **Justification manuscrite :** « L'Araignée se déplace de trois espaces par tour : ni plus, ni moins. Elle doit suivre un chemin direct et ne peut pas revenir sur ses pas. Elle ne peut se déplacer qu'autour des pièces avec lesquelles elle est en contact direct à chaque pas. » (p. 7). La ruche moins l'araignée est une ligne droite de cinq pièces dont la frontière est un unique anneau de 14 cellules sans portes ; chaque cellule de l'anneau touche la ligne, et les cellules hors de l'anneau ne touchent rien (exclues par l'exigence de contact). Depuis sa position sur l'anneau, l'araignée a donc exactement deux marches de trois pas (trois cellules dans le sens horaire et trois cellules dans le sens antihoraire) : la cellule au NE de bS1, et la cellule au SE de wQ. Les arrêts après un ou deux pas sont exclus (« ni moins »), le retour en arrière est exclu.

A.1.9 C009 : Sauterelle : ne saute que le long de rangées occupées, pas de glissements d'un espace

- **Règle :** Sauterelle (*Gen42 Hive rulesheet p. 6*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 \wS1;bG1 bQ-`
- **Attente :** moves_for_piece `{'piece': 'wG1', 'moves': ['wG1 wS1\\', 'wG1 /wQ']}`
- **Justification manuscrite :** « Elle saute depuis son espace par-dessus un nombre quelconque de pièces (mais au moins une) jusqu'au premier espace inoccupé le long d'une rangée droite de pièces jointes. » (p. 6). La sauterelle touche des cellules occupées dans exactement deux de ses six directions : SE (par-dessus wS1, atterrissant dans l'espace suivant, au SE de wS1) et SO (par-dessus wQ, atterrissant au SO de wQ). Dans les quatre autres directions, la cellule adjacente est vide, et un saut « par-dessus au moins une »

pièce est impossible ; en particulier, les quatre cellules vides adjacentes ne sont PAS des destinations : la sauterelle « ne se déplace pas autour de l'extérieur de la Ruche comme les autres créatures ». Attendu : exactement les deux cellules d'atterrissage.

A.1.10 C010 : La sauterelle saute une rangée complète de cinq pièces jusqu'au premier espace vide

- **Règle :** Sauterelle (*Gen42 Hive rulesheet p. 6*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-`
- **Attente :** moves_for_piece `{'piece': 'wG1', 'moves': ['wG1 bG1-']}`
- **Justification manuscrite :** « …par-dessus un nombre quelconque de pièces (mais au moins une) jusqu'au premier espace inoccupé le long d'une rangée droite de pièces jointes. » (p. 6). Plein est, la sauterelle fait face à la rangée ininterrompue wQ, wS1, bS1, bQ, bG1 ; le premier espace inoccupé au-delà est la cellule à l'E de bG1, l'unique destination. Elle doit atterrir là, pas avant (chaque cellule plus proche dans la rangée est occupée). Dans les cinq autres directions, la cellule adjacente est vide, donc aucun saut n'existe. La sauterelle est une feuille de la ruche, donc One-Hive ne la restreint pas.

A.1.11 C011 : La seule voisine ouverte de la reine est derrière une porte : zéro coup

- **Règle :** Liberté de mouvement (*Gen42 Hive rulesheet p. 10*)
- **Mise en place :** `wS1;bS1 -wS1;wB1 wS1/;bQ -bS1;wQ wB1-;bG1 \bQ;wS2 wQ/;bA1 /bQ;wG1 -wS2;bS2 /bS1;wA1 wS2\;bB1 \bG1;wG2 wQ\;bG2 \bB1`
- **Attente :** moves_for_piece `{'piece': 'wQ', 'moves': []}`
- **Justification manuscrite :** « De même, aucune pièce ne peut se déplacer dans un espace où elle ne peut pas physiquement glisser. » (p. 10). Cinq des six voisines de la reine sont des pièces blanches ; la sixième (la cellule à l'O de wG2, également au SE de wB1) est vide, mais les deux cellules adjacentes à la fois à la reine et à cet espace sont wG2 et wB1, toutes deux occupées, donc la reine ne peut pas physiquement y glisser. Retirer la reine laisse le fer à cheval blanc wS1-wB1-wG1-wS2-wA1-wG2 connexe (et la chaîne noire pend de wS1 via bS1), donc One-Hive autoriserait le coup ; le blocage relève purement de la liberté de mouvement. Attendu : la reine n'a aucun coup légal.

A.1.12 C012 : La fourmi atteint chaque cellule du périmètre de la ruche (13 destinations)

- **Règle :** Fourmi soldat / Liberté de mouvement (*Gen42 Hive rulesheet pp. 8, 10*)
- **Mise en place :** `wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wA1 \wQ;bG1 bQ-`
- **Attente :** moves_for_piece `{'piece': 'wA1', 'moves': ['wA1 -wQ', 'wA1 \\wG1', 'wA1 \\bS1', 'wA1 \\bQ', 'wA1 \\bG1', 'wA1 bG1/', 'wA1 bG1-', 'wA1 bG1\\', 'wA1 bQ\\', 'wA1 bS1\\', 'wA1 wG1\\', 'wA1 wQ\\', 'wA1 /wQ']}`
- **Justification manuscrite :** « La Fourmi soldat peut se déplacer de sa position vers n'importe quelle autre position autour de la Ruche, pourvu que les restrictions soient respectées. » (p. 8). La ruche moins la fourmi est une ligne droite de cinq pièces ; sa frontière est un unique anneau de 14 cellules sans portes (chaque pas de glissement est flanqué d'une cellule de la ligne et d'une cellule vide), et chaque cellule de l'anneau touche la ligne. La fourmi part de l'anneau, sur la cellule au NO de wQ, donc elle peut s'arrêter sur n'importe laquelle des 13 autres cellules de l'anneau : le bout ouest (à l'O de wQ), les cinq cellules de l'épaule nord (au NO de chaque pièce de la ligne plus au NE de bG1), le bout est (à l'E de bG1), et les six cellules de l'épaule sud (au SE de chaque pièce

de la ligne plus au SO de wQ). Les cellules hors de l'anneau ne touchent aucune pièce et sont exclues (p. 3 NB : les pièces doivent toujours toucher au moins une autre pièce).

A.1.13 C013 : Scarabée au sol : deux glissements et deux escalades

- **Règle :** Scarabée (*Gen42 Hive rulesheet pp. 4-5, 10*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-`
- **Attente :** moves_for_piece `{'piece': 'wB1', 'moves': ['wB1 wS1', 'wB1 wQ', 'wB1 \ \bS1', 'wB1 \\wQ']}`
- **Justification manuscrite :** « Le Scarabée, comme la Reine, ne se déplace que d'un espace par tour. Contrairement à toute autre créature cependant, il peut aussi se déplacer sur le dessus de la Ruche. » (p. 4). Depuis (au NO de wS1), le scarabée peut grimper sur l'une ou l'autre des pièces adjacentes (wS1 ou wQ) ou glisser au sol vers les deux cellules vides qui gardent le contact avec la ruche : au NO de bS1 (touchant wS1 et bS1) et au NO de wQ (touchant wQ). Les deux voisines vides restantes ne touchent aucune pièce après que le scarabée se soulève, donc elles sont exclues (p. 3 NB). Aucune porte ne bloque aucun des quatre coups (chacun est flanqué d'au plus une cellule occupée, et pour les escalades les piles flanquantes ne sont pas plus hautes que la destination). Exactement quatre coups, ce qui correspond au compte de l'exemple de scarabée de la feuille de règles elle-même.

A.1.14 C014 : Scarabée au sommet de la ruche : les six cellules voisines

- **Règle :** Scarabée (*Gen42 Hive rulesheet p. 5; beetle-gate ruling, World Hive Tournaments Rules FAQ*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-;wB1 wS1;bA1 bG1-`
- **Attente :** moves_for_piece `{'piece': 'wB1', 'moves': ['wB1 bS1', 'wB1 wQ', 'wB1 \ \wS1', 'wB1 \\bS1', 'wB1 wS1\\', 'wB1 wQ\\']}`
- **Justification manuscrite :** « Depuis sa position au sommet de la Ruche, le Scarabée peut se déplacer de tuile en tuile sur le dessus de la Ruche. Il peut aussi descendre dans des espaces entourés et donc inaccessibles à la plupart des autres créatures. » (p. 5). Posé sur wS1, le scarabée peut aller sur chacune des six cellules voisines : passer sur bS1 ou wQ (deux piles de hauteur 1), ou descendre sur n'importe laquelle des quatre cellules vides autour de wS1, chacune touchant encore wS1 lui-même, donc le contact tient. Aucune paire de piles flanquantes n'est plus haute à la fois que l'origine (hauteur 1 sous le scarabée) et la destination, donc aucune porte du scarabée ne s'applique (FAQ). One-Hive ne peut pas être violée : wS1 reste où il est. Exactement six destinations.

A.1.15 C015 : Porte du scarabée : la descente entre deux piles de hauteur 2 est bloquée

- **Règle :** Liberté de mouvement au-dessus du sol (porte du scarabée) (*World Hive Tournaments Rules FAQ; Gen42 Hive rulesheet p. 10*)
- **Mise en place :** `wS1;bG1 wS1/;wQ /wS1;bQ bG1/;wG1 wS1\;bB1 bQ/;wB1 -wS1;bB1 bQ;wB2 / wQ;bB1 bG1;wB1 wS1;bQ bB1-;wB2 wQ;bQ bB1/;wB2 wG1;bA1 bQ/`
- **Attente :** move_illegal `{'move': 'wB1 bB1\\'}`
- **Justification manuscrite :** « Quand une pièce monte ou descend la ruche, ou se déplace en restant au sommet de la ruche, elle doit pouvoir glisser selon la règle de liberté de mouvement, qui s'applique aux niveaux supérieurs au sol. Si deux piles forment une porte au-dessus du niveau du sol (nous l'appelons porte du scarabée), les

pièces ne peuvent pas s'y glisser. » (WHT Rules FAQ). wB1 est posé sur wS1 (son propre niveau : au sommet d'une pièce de hauteur 1) ; la cellule cible au SE de la pile bB1 est vide (hauteur 0). Les deux cellules adjacentes à la fois à l'origine et à la cible portent les piles bG1+bB1 et wG1+wB2, toutes deux de hauteur 2, strictement plus hautes à la fois que l'origine sans le scarabée (1) et que la destination (0) ; le scarabée ne peut donc pas glisser vers le bas entre elles. La descente doit être rejetée. (One-Hive l'autoriserait : wS1 reste en place ; le contact tient via les piles flanquantes.)

A.1.16 C016 : Une pièce surmontée d'un scarabée ne peut pas bouger

- **Règle :** Scarabée (immobilité sous pile) (*Gen42 Hive rulesheet p. 5*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-;wB1 wS1;bA1 bG1-`
- **Attente :** move_illegal `{'move': 'wS1 bS1\\'}`
- **Justification manuscrite :** « Une pièce avec un scarabée sur elle est incapable de bouger » (p. 5). wS1 est sous wB1, donc toute tentative de déplacer wS1 (ici un coup d'araignée vers la cellule au SE de bS1) doit être rejetée, que le chemin soit par ailleurs légal ou non pour une araignée.

A.1.17 C017 : La pile prend la couleur du scarabée : les Blancs peuvent placer à côté d'une reine noire recouverte

- **Règle :** Scarabée (couleur de la pile) / Placement (*Gen42 Hive rulesheet pp. 2, 5*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bA1 bS1\;wB1 \bS1;bG1 bA1\;wB1 \bQ;bG2 bG1\;wB1 bQ;bB1 bG2\`
- **Attente :** move_legal `{'move': 'wG1 wB1-'}`
- **Justification manuscrite :** « …aux fins des règles de placement de la p. 2, la pile prend la couleur du Scarabée. » (p. 5). Le scarabée blanc est posé sur la reine noire à l'extrémité est de la ruche. La cellule à l'E de cette pile ne touche aucune autre pièce, donc un placement blanc à cet endroit n'est adjacent qu'à une pile dont la couleur est, par la règle, blanche. Le placement de wG1 à cet endroit doit être accepté. (Sans la règle de couleur de pile, la cellule serait adjacente à une pièce noire et le placement serait illégal, p. 2.)

A.1.18 C018 : La pile prend la couleur du scarabée : les Noirs ne peuvent PAS placer à côté de leur propre reine recouverte

- **Règle :** Scarabée (couleur de la pile) / Placement (*Gen42 Hive rulesheet pp. 2, 5*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bA1 bS1\;wB1 \bS1;bG1 bA1\;wB1 \bQ;bG2 bG1\;wB1 bQ;bB1 bG2\;wG1 -wQ`
- **Attente :** move_illegal `{'move': 'bB2 wB1-'}`
- **Justification manuscrite :** Miroir de C017 : la pile bQ+wB1 compte comme BLANCHE (« la pile prend la couleur du Scarabée », p. 5). La cellule à l'E de la pile ne touche que cette pile, donc pour les Noirs elle est adjacente à une pièce blanche et « les pièces ne peuvent pas être placées à côté d'une pièce de la couleur de l'adversaire » (p. 2). La tentative des Noirs de placer bB2 à cet endroit doit être rejetée, même si la pièce enfouie est la propre reine des Noirs.

A.1.19 C019 : One-Hive : la seule connexion entre deux parties ne peut pas bouger

- **Règle :** Règle One-Hive (*Gen42 Hive rulesheet pp. 3, 9*)

- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-`
- **Attente :** moves_for_piece `{'piece': 'wS1', 'moves': []}`
- **Justification manuscrite :** « Toutes les pièces doivent toujours toucher au moins une autre pièce. Si une pièce est la seule connexion entre deux parties de la Ruche, elle ne peut pas être déplacée. » (p. 3 NB) ; « Les pièces en jeu doivent être reliées à tout moment. À aucun moment vous ne pouvez laisser une pièce isolée (non reliée à la Ruche) ni séparer la Ruche en deux. » (p. 9). wS1 est le lien intérieur entre wQ d'un côté et bS1-bQ de l'autre : la soulever scinde la ruche, donc l'araignée n'a aucun coup légal ; chaque destination, si valide soit-elle comme déplacement d'araignée, est exclue par One-Hive.

A.1.20 C020 : Anneau : une pièce sur une boucle fermée peut bouger (pas un point d'articulation) ; l'œil de l'anneau est derrière une porte

- **Règle :** Règle One-Hive / Liberté de mouvement (*Gen42 Hive rulesheet pp. 9, 10*)
- **Mise en place :** `wS1;bS1 -wS1;wG1 wS1/;bQ -bS1;wQ wS1\;bG1 -bQ;wG2 wG1-;bG2 -bG1;wA1 wQ-;bA1 -bG2;wS2 wG2\;bB1 -bA1`
- **Attente :** moves_for_piece `{'piece': 'wQ', 'moves': ['wQ /wS1', 'wQ /wA1']}`
- **Justification manuscrite :** Les six pièces blanches forment un anneau fermé, donc retirer wQ laisse les cinq autres connexes le long de la boucle (et la queue noire pend de wS1) : One-Hive permet à la reine de bouger. En glissant d'un espace (p. 4), la reine a trois voisines vides : l'œil de l'anneau et deux cellules extérieures. L'œil est flanqué par wS1 et wA1, toutes deux occupées, donc la reine « ne peut pas se déplacer dans un espace où elle ne peut pas physiquement glisser » (p. 10). Les deux cellules extérieures (au SO de wS1, qui touche wS1 et bS1 ; et au SO de wA1, qui touche wA1) sont des glissements sans obstruction qui gardent le contact. Attendu : exactement ces deux destinations.

A.1.21 C021 : La partie se termine quand une reine est complètement encerclée, même par sa propre couleur

- **Règle :** Le but de Hive / La fin de la partie (*Gen42 Hive rulesheet pp. 1, 11*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-;wA1 -wG1;bG2 bQ/;wA2 -wA1;bB1 \bQ;wA3 -wA2;bA1 bS1\;wS2 -wA3;bA2 bQ\`
- **Attente :** game_over `{'state': 'WhiteWins'}`
- **Justification manuscrite :** « Les pièces entourant la Reine peuvent être un mélange de vos pièces et de celles de votre adversaire. » (p. 1). « La partie se termine dès qu'une Reine est complètement encerclée par des pièces de n'importe quelle couleur. La personne dont la Reine est encerclée perd la partie. » (p. 11). Le dernier placement des Noirs (bA2, au SE de leur propre reine) remplit la sixième et dernière cellule autour de bQ. Les pièces qui l'entourent sont toutes noires (sans importance selon la p. 1) et c'est le propre coup des Noirs qui achève l'encerclement : les Noirs perdent, l'état de la GameString doit indiquer WhiteWins immédiatement après ce coup.

A.1.22 C022 : Un seul coup encercle les deux reines simultanément : nulle

- **Règle :** La fin de la partie (*Gen42 Hive rulesheet p. 11*)
- **Mise en place :** `wS1;bS1 wS1/;wQ wS1\;bB1 bS1-;wA1 -wQ;bQ bB1\;wS2 /wQ;bQ /bB1;wG1 -wS2;bQ wQ-;wB1 -wA1;bA1 bQ\;wG1 wS2-;bG1 \bS1;wB1 \wA1;bA2 bQ-;wB1 \wS1;bA3 bB1\;wG2 -wB1;bG1 bS1\`

- **Attente :** game_over `{'state': 'Draw'}`
- **Justification manuscrite :** « La personne dont la Reine est encerclée perd la partie, sauf si la dernière pièce à encercler sa Reine achève aussi l'encerclement de l'autre Reine. Dans ce cas, la partie est nulle. » (p. 11). Avant le dernier coup des Noirs, chaque reine a exactement une voisine vide : la même cellule (1,0), adjacente aux deux reines (les reines sont côte à côte, wQ à l'épaule NE de wS1, bQ à côté d'elle). La sauterelle noire en (1,-2) saute au SE par-dessus bS1 dans cette cellule, remplissant d'une seule pièce la sixième voisine des deux reines : la partie doit se terminer par une nulle (Draw), pas par une victoire de l'un ou l'autre camp.

A.1.23 C023 : Un joueur qui ne peut ni placer ni bouger doit passer

- **Règle :** Impossibilité de déplacer ou de placer (*Gen42 Hive rulesheet pp. 2, 5, 10, 11*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bQ bS1/;wA1 /wQ;bQ bS1-;wB1 \bS1;bQ bS1/;wB1 bS1;bQ wB1-;wA1 /bQ;bQ wB1/;wA2 /wQ;bQ wB1-;wA2 bQ\;bQ wB1/;wA3 / wQ;bQ wB1-;wA3 bQ-;bQ wB1/;wG1 /wB1;bQ wB1-;wG1 wB1/`
- **Attente :** must_pass
- **Justification manuscrite :** « Si un joueur ne peut ni placer une nouvelle pièce ni déplacer une pièce existante, le tour passe à son adversaire, qui joue alors de nouveau. » (p. 11). Après le dernier coup des Blancs, les Noirs ont : bS1 sous wB1 (« une pièce avec un scarabée sur elle est incapable de bouger », p. 5), et la pile compte comme blanche pour le placement (p. 5) ; et bQ, dont les cinq voisines sont la pile, wG1, wA1, wA2 et wA3, sa seule voisine vide n'étant atteignable qu'entre wG1 et wA3, une porte où la reine « ne peut pas physiquement glisser » (p. 10). Aucun placement n'est possible non plus : la seule cellule adjacente à une pièce à sommet noir est cette même cellule derrière la porte, qui touche aussi des pièces blanches (p. 2). Les Noirs ne sont pas perdus (bQ a une voisine vide, donc elle n'est pas encerclée), mais doivent passer.

A.1.24 C024 : Capacité spéciale du Pillbug : déplacer une pièce amie adjacente (garde du noyau)

- **Règle :** Capacité spéciale du Pillbug (*Gen42 Pillbug rulesheet (English section)*)
- **Mise en place :** `wP;bS1 wP-;wQ -wP;bQ bS1-`
- **Attente :** move_legal `{'move': 'wQ wP\\'}`
- **Justification manuscrite :** « La capacité spéciale permet au Pillbug de déplacer une pièce adjacente (amie ou ennemie) de deux espaces ; vers le haut sur lui-même, puis vers le bas dans un autre espace vide adjacent à lui-même. » (feuille Pillbug). wQ est adjacente à wP ; la cellule cible au SE de wP est vide et adjacente à wP. Aucune des quatre exceptions ne s'applique : wQ ne vient pas d'être déplacée par l'autre joueur (le dernier coup des Noirs était le placement de bQ), wQ n'est pas dans une pile, retirer wQ ne scinde pas la ruche (c'est une feuille), et aucune pièce empilée ne forme de porte sur le chemin de montée et de descente. NOTE : ceci est un cas de garde du noyau étiqueté variante (protocole §2). La variante de l'étude est le jeu de base ; les cas Pillbug ne protègent que le noyau de règles partagé.

A.1.25 C025 : Une pièce que le Pillbug ennemi vient de déplacer est étourdie pour un tour (garde du noyau)

- **Règle :** Capacité spéciale du Pillbug (immobilité de la pièce déplacée) (*Gen42 Pillbug rulesheet (English section); World Hive Tournaments Rules FAQ*)

- **Mise en place :** `wS1;bP wS1−;wQ −wS1;bQ bP−;wA1 \wQ;bG1 bQ−;wA1 \bP;bG1 −wQ;wG1 −wA1;wA1 bP\`
- **Attente :** move_illegal `{'move': 'wA1 bP/'}`
- **Justification manuscrite :** « De plus, toute pièce déplacée par le Pillbug ne peut pas du tout être déplacée (directement ou via une action du Pillbug) au tour du joueur suivant. » (feuille Pillbug) ; FAQ : « toute pièce qui vient de bouger, au tour de l'autre joueur immédiatement après, est incapable de : bouger, être déplacée ou utiliser la capacité du pillbug. » Le Pillbug noir vient de lancer wA1 par-dessus lui-même jusqu'à la cellule au SE de bP (un usage légal : wA1 avait bougé pour la dernière fois deux plis plus tôt, donc l'exception du dernier-déplacé ne bloquait pas le lancer ; son retrait gardait la ruche entière puisque wG1 touche aussi wS1 et wQ). Au tour immédiatement suivant des Blancs, la fourmi lancée est étourdie : la tentative de coup de fourmi vers le NE de bP doit être rejetée. Cas de garde du noyau étiqueté variante (protocole §2).

A.1.26 C026 : Le Pillbug ne peut pas déplacer la pièce que l'adversaire vient de déplacer (garde du noyau)

- **Règle :** Capacité spéciale du Pillbug (exceptions) (*Gen42 Pillbug rulesheet (English section)*)
- **Mise en place :** `wS1;bP wS1−;wQ −wS1;bQ bP−;wA1 \wQ;bG1 bQ−;wA1 \bP`
- **Attente :** move_illegal `{'move': 'wA1 bP\\'}`
- **Justification manuscrite :** « Le Pillbug ne peut pas déplacer la pièce qui vient d'être déplacée par l'autre joueur. » (feuille Pillbug, première exception). La fourmi des Blancs s'est déplacée vers la cellule au NO de bP au pli immédiatement précédent ; la tentative des Noirs d'utiliser la capacité du Pillbug sur cette même fourmi (la lancer au SE de bP) doit être rejetée. (Le même lancer devient légal deux plis plus tard, ce qui constitue la mise en place du cas C025.) Cas de garde du noyau étiqueté variante (protocole §2).

A.1.27 C027 : One-Hive lie même la sauterelle : un point d'articulation ne peut pas sauter

- **Règle :** Règle One-Hive / Sauterelle (*Gen42 Hive rulesheet pp. 3, 6, 9*)
- **Mise en place :** `wG1;bS1 wG1−;wQ −wG1;bQ bS1−`
- **Attente :** moves_for_piece `{'piece': 'wG1', 'moves': []}`
- **Justification manuscrite :** La sauterelle est exemptée de la restriction de glissement (p. 10 : elle « peut sauter dans un espace ou hors d'un espace »), mais pas de One-Hive : « Si une pièce est la seule connexion entre deux parties de la Ruche, elle ne peut pas être déplacée. » (p. 3 NB). wG1 est entre wQ et la paire noire ; la soulever pour n'importe quel saut scinde la ruche en deux, donc malgré des lignes de saut dans les directions E et O, la sauterelle n'a aucun coup légal.

A.1.28 C028 : Un scarabée ne peut pas être PLACÉ directement au sommet de la ruche

- **Règle :** Scarabée (NB de placement) (*Gen42 Hive rulesheet p. 5*)
- **Mise en place :** `wS1;bS1 wS1−;wQ −wS1;bQ bS1−`
- **Attente :** move_illegal `{'move': 'wB1 wS1'}`
- **Justification manuscrite :** « Lorsqu'il est placé pour la première fois, le Scarabée est placé de la même manière que toutes les autres pièces. Il ne peut pas être placé

directement sur le dessus de la Ruche, même s'il peut y être déplacé plus tard. » (p. 5 NB). wB1 est encore en réserve ; la tentative de l'introduire au sommet de wS1 doit être rejetée. (C013/C014 vérifient que le même scarabée peut y grimper par un coup une fois placé.)

A.1.29 C029 : L'araignée ne peut pas s'arrêter après un pas (« ni plus, ni moins »)

- **Règle :** Araignée (*Gen42 Hive rulesheet p. 7*)
- **Mise en place :** `wG1;bS1 wG1-;wQ -wG1;bQ bS1-;wS1 \wQ;bG1 bQ-`
- **Attente :** move_illegal `{'move': 'wS1 \\wG1'}`
- **Justification manuscrite :** « L'Araignée se déplace de trois espaces par tour : ni plus, ni moins. » (p. 7). La cellule au NO de wG1 est à exactement un pas de glissement de la position de l'araignée, et aucun chemin légal de trois pas sans retour en arrière ne s'y termine (les deux marches de trois pas se terminent au NE de bS1 et au SE de wQ, cas C008) ; un chemin passant par cette cellule la traverse au premier pas et ne peut pas s'y arrêter. Le coup d'un seul pas doit être rejeté.

A.1.30 C030 : Reine entre deux pièces : deux glissements le long de l'épaule

- **Règle :** Reine / One-Hive / Liberté de mouvement (*Gen42 Hive rulesheet pp. 4, 9, 10*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wB1 \wS1;bG1 bQ-`
- **Attente :** moves_for_piece `{'piece': 'wQ', 'moves': ['wQ -wB1', 'wQ /wS1']}`
- **Justification manuscrite :** La reine touche wS1 (E) et wB1 (NE). La retirer garde la ruche entière (wB1 touche encore wS1), donc One-Hive autorise un coup. Glissements d'un espace (p. 4) : de ses quatre voisines vides, seules la cellule à l'O de wB1 (gardant le contact avec wB1) et la cellule au SO de wS1 (gardant le contact avec wS1) touchent encore la ruche après qu'elle se soulève ; les deux cellules plus à l'ouest ne touchent rien et sont exclues (p. 3 NB). Aucun des deux glissements n'est derrière une porte (chacun est flanqué d'exactement une cellule occupée). Attendu : exactement ces deux destinations.

A.2 Jeu de vérification tactique (5 cas)

Cas de correction de la recherche : mat en 1 par marche et par saut, pour les deux couleurs, et évitement d'auto-encerclement ; résolus 5/5 par la base MCTS à 400, 1600 et 6400 simulations.

A.2.1 T001 : Les Blancs font mat en 1 : occuper la dernière liberté de la reine noire (SE de bQ)

- **Règle :** La fin de la partie (*Gen42 Hive rulesheet pp. 1, 11*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ-;wA1 -wG1;bG2 bQ/;wA2 -wA1;bB1 \bQ;wA3 -wA2;bA1 bS1\`
- **Attente :** bestmove_to_cell `{'target': 'bQ\\'}`
- **Justification manuscrite :** La reine noire a exactement une voisine vide, la cellule au SE de bQ. Toute pièce blanche qui s'y pose achève l'encerclement et gagne immédiatement (« la partie se termine dès qu'une Reine est complètement encerclée par des pièces de n'importe quelle couleur », p. 11 ; mélange de couleurs permis, p. 1). La cellule est atteignable : une fourmi blanche peut parcourir le périmètre sud en un coup

(l'entrée au-delà de bA1 n'est pas fermée par une porte), donc un coup gagnant existe. Aucun autre coup unique ne termine la partie. La recherche doit jouer sur cette cellule.

A.2.2 T002 : Les Noirs font mat en 1 : occuper la dernière liberté de la reine blanche (SE de wQ)

- **Règle :** La fin de la partie (*Gen42 Hive rulesheet pp. 1, 11*)
- **Mise en place :** `wS1;bS1 -wS1;wQ wS1-;bQ -bS1;wG1 wQ-;bG1 -bQ;wG2 wQ/;bG2 -bG1;wB1 \wQ;bA1 -bG2;wA1 wS1\;bA2 -bA1;wA2 wG1-`
- **Attente :** bestmove_to_cell `{'target': 'wQ\\'}`
- **Justification manuscrite :** Miroir de T001 avec les couleurs échangées et les Noirs au trait. Cette paire est la vérification d'alternance des joueurs au niveau du coup : le motif gagnant doit être trouvé des deux côtés. La seule voisine vide de la reine blanche est la cellule au SE de wQ ; une fourmi noire l'atteint le long du périmètre sud (route via le SE de la colonne de bS1 : le pas d'entrée dans la cellule est flanqué par la cellule occupée wA1, donc le contact tient et aucune porte ne bloque). S'y poser achève l'encerclement : les Noirs gagnent (p. 11).

A.2.3 T003 : Les Blancs font mat en 1 par saut de sauterelle par-dessus quatre pièces (E de bQ)

- **Règle :** Sauterelle / La fin de la partie (*Gen42 Hive rulesheet pp. 6, 11*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 -wQ;bG1 bQ/;wA1 \wQ;bB1 \bQ;wA2 \wA1;bA1 bS1\;wA3 \wA2;bA2 bQ\`
- **Attente :** bestmove_to_cell `{'target': 'bQ-'}`
- **Justification manuscrite :** La seule voisine vide de la reine noire est la cellule à l'E de bQ. Plein est depuis wG1, au bout ouest, court la rangée occupée ininterrompue wQ, wS1, bS1, bQ ; le premier espace inoccupé le long de cette rangée est exactement la cellule gagnante, donc la sauterelle saute par-dessus quatre pièces et achève l'encerclement (p. 6 : « par-dessus un nombre quelconque de pièces … jusqu'au premier espace inoccupé le long d'une rangée droite de pièces jointes » ; p. 11 : l'encerclement termine la partie). Les fourmis blanches peuvent aussi entrer en marchant autour du périmètre ; l'attendu est la cellule de destination, quelle que soit la pièce que la recherche envoie.

A.2.4 T004 : Les Noirs font mat en 1 par saut de sauterelle par-dessus quatre pièces (O de wQ)

- **Règle :** Sauterelle / La fin de la partie (*Gen42 Hive rulesheet pp. 6, 11*)
- **Mise en place :** `wS1;bS1 wS1-;wQ -wS1;bQ bS1-;wG1 \wQ;bG1 bQ-;wG2 \wS1;bA1 bQ\;wA1 wQ\;bA2 bA1\;wA2 /wQ;bA3 bA2\;wB1 \wG1`
- **Attente :** bestmove_to_cell `{'target': '-wQ'}`
- **Justification manuscrite :** Miroir d'alternance de T003 avec le saut noir. Les voisines de la reine blanche : E wS1, NO wG1 (placé au NO de wQ), NE wG2 (placé au NO de wS1 = NE de wQ), SE wA1, SO wA2. Cinq sont occupées ; seule la cellule à l'O de wQ est vide. Plein est de ce trou court la rangée ininterrompue wQ, wS1, bS1, bQ avec bG1 au bout est (3,0) : depuis bG1, le premier espace inoccupé vers l'ouest le long de la rangée est exactement le trou, donc la sauterelle saute par-dessus quatre pièces et achève l'encerclement (pp. 6, 11). La queue sud des Noirs (bA1..bA3 au SE de bG1) garde les placements noirs antérieurs légaux et à l'écart des Blancs.

A.2.5 T005 : Ne pas remplir la dernière liberté de sa propre reine

- **Règle :** La fin de la partie (*Gen42 Hive rulesheet p. 11*)
- **Mise en place :** `wS1;bS1 -wS1;wQ wS1-;bQ -bS1;wG1 wQ-;bG1 -bQ;wG2 wQ/;bG2 -bG1;wB1 \wQ;bA1 -bG2;wA1 wS1\;bA2 -bA1`
- **Attente :** bestmove_avoid_cell `{'target': 'wQ\\'}`
- **Justification manuscrite :** La cellule au SE de wQ est la dernière liberté de la reine blanche. Que les Blancs y placent ou y déplacent n'importe quelle pièce achève l'encerclement de leur propre reine : « la personne dont la Reine est encerclée perd la partie » (p. 11), quel que soit le camp qui a fourni la sixième pièce. Le placement est parfaitement légal (la cellule ne touche que des pièces blanches), donc seul le jugement de la recherche l'empêche. Tout coup sauf un coup atterrissant sur cette cellule convient ; la recherche ne doit pas y jouer. (Ceci n'affirme pas que les Blancs survivent à long terme, puisque les Noirs menacent la même cellule ; cela affirme seulement que l'auto-encerclement immédiat est évité.)

B Architectures, décodeur et formats de données en détail

Cette annexe complète Chapitre 7 par des tables couche par couche des deux réseaux et de leurs variantes d'ablation, l'arithmétique exacte du décodeur d'actions partagé, l'interface tensorielle du bras graphe, l'agencement en octets de l'enregistrement d'auto-jeu, et les réglages de recherche partagés par les deux bras. Les totaux de paramètres sont les valeurs mesurées par le compteur de paramètres du code d'entraînement et ont été revérifiés pour cette annexe ; les comptes par couche n'ont pas été enregistrés séparément, de sorte que les tables ne donnent que les formes et les largeurs.

B.1 HiveNet, le réseau grille

L'entrée est un tenseur de 77 plans × 32 × 32 à valeurs float32 dans [0, 1] (Tableau 13). Le corps et les têtes sont listés dans Tableau 46. L'adjacence hexagonale en coordonnées axiales est un sous-ensemble de 7 cellules du voisinage 3 × 3, de sorte que les noyaux 3 × 3 la couvrent avec deux coins morts apprenables par noyau.

Tableau 46 : HiveNet, le réseau du bras grille, couche par couche. Les blocs sont numérotés à partir de 0 comme dans l'implémentation ; les deux blocs à pooling global sont ceux situés au tiers et aux deux tiers de la profondeur. Les largeurs sont des nombres de canaux ; le total est le compte de paramètres mesuré.

Étape	Opération	Largeur	Notes
Entrée	77 plans × 32 × 32	77	float32 dans [0, 1]
Tronc	Conv 3 × 3 (sans biais), BatchNorm, ReLU	77 → 96	remplissage (padding) 1
Bloc 0	Conv 3 × 3, BatchNorm, ReLU ; Conv 3 × 3, BatchNorm ; addition résiduelle ; ReLU	96 → 96	convolutions sans biais
Bloc 1	comme le bloc 0	96 → 96	
Bloc 2	comme le bloc 0, plus un biais de pooling global après le second BatchNorm	96 → 96	moyenne ‖ max sur le cadre (2 × 96) → linéaire → 96, ajouté par canal
Bloc 3	comme le bloc 0	96 → 96	
Bloc 4	comme le bloc 0	96 → 96	
Bloc 5	comme le bloc 2	96 → 96	biais de pooling global
Bloc 6	comme le bloc 0	96 → 96	
Bloc 7	comme le bloc 0	96 → 96	
Pooling	moyenne sur le cadre 32 × 32	96	alimente le logit de passe et la tête de valeur
Politique, spatiale	Conv 1 × 1 (avec biais)	96 → 28	aplatie en 28 × 1024 = 28 672 logits, indice emplacement × 1024 + y × 32 + x
Politique, passe	Linéaire sur les caractéristiques agrégées	96 → 1	ajouté comme indice 28 672
Valeur	Linéaire, ReLU, Linéaire sur les caractéristiques agrégées	96 → 64 → 3	logits victoire / nulle / défaite du point de vue du camp au trait
Total			1.44 M paramètres

B.2 HiveGraphNet, le réseau graphe

Le réseau du bras graphe consomme des tenseurs à formes fixes (Tableau 47) construits à partir d'un enregistrement par le constructeur Python pour l'entraînement et par son miroir Rust pour l'inférence. Les agencements des caractéristiques de nœud et des caractéristiques globales sont donnés dans Tableau 14 et Tableau 15 ; le corps et les têtes sont listés dans Tableau 48, et les variantes d'ablation dans Tableau 49.

Tableau 47 : Interface tensorielle du bras graphe, par position. Capacités : 224 nœuds (cellules occupées plus l'anneau vide) et 321 lignes de coups (320 coups légaux, le plafond de l'enregistrement, plus une ligne de passe) ; dépasser l'une ou l'autre capacité lève une erreur plutôt que de tronquer. L'ordre des nœuds est déterministe : les cellules occupées d'abord, puis les cellules de l'anneau, chacune triée par (y, x) en coordonnées du cadre.

Tenseur	**Forme**	**Type**	**Contenu**
nœuds	224 × 56	float32	caractéristiques de nœud, complétées par des zéros au-delà des nœuds réels
voisins	224 × 6	int64	indice du voisin dans chaque direction hexagonale ; 224 = aucun
masque de nœuds	224	bool	1 pour un nœud réel
globaux	23	float32	caractéristiques globales
coups	321 × 3	int64	(emplacement, nœud de destination, nœud source) par coup légal ; source = 224 pour les placements et la passe ; la ligne de passe utilise l'emplacement 28
masque de coups	321	bool	1 pour une ligne légale
cible	321	float32	distribution de visites sur les lignes légales (entraînement seulement)
issue	scalaire	int64	0 défaite / 1 nulle / 2 victoire / 3 tronquée, du point de vue du camp au trait (entraînement seulement)

Tableau 48 : HiveGraphNet, le réseau du bras graphe, couche par couche. Les couches sont numérotées à partir de 0 comme dans l'implémentation ; les couches à pooling sont une couche sur trois. Les largeurs sont des nombres de canaux ; le total est le compte de paramètres mesuré.

Étape	**Opération**	**Largeur**	**Notes**
Entrée	caractéristiques de nœud ‖ globaux diffusés	56 + 23	par nœud
Linéaire d'entrée	Linéaire	56 + 23 ⟶ 152	masqué aux nœuds réels ; une ligne nulle à l'indice 224 tient lieu de voisin absent
Couche 0	couche relationnelle : W_{self} (avec biais) + six W_d (sans biais) ; résiduelle ; ReLU ; masque	152 ⟶ 152	Équation 4
Couche 1	comme la couche 0	152 ⟶ 152	
Couche 2	comme la couche 0, plus un biais de pooling global à l'intérieur de la non-linéarité	152 ⟶ 152	moyenne masquée ‖ max masqué (2 × 152) ⟶ linéaire ⟶ 152, ajouté à chaque nœud
Couche 3	comme la couche 0	152 ⟶ 152	
Couche 4	comme la couche 0	152 ⟶ 152	
Couche 5	comme la couche 2	152 ⟶ 152	biais de pooling global
Couche 6	comme la couche 0	152 ⟶ 152	
Couche 7	comme la couche 0	152 ⟶ 152	

Étape	Opération	Largeur	Notes
Lecture (readout)	moyenne masquée ‖ max masqué sur les nœuds réels	2×152	
Valeur	Linéaire, ReLU, Linéaire sur lecture ‖ globaux	$2 \times 152 + 23 \to 64 \to 3$	logits victoire / nulle / défaite du point de vue du camp au trait
Plongement d'emplacement	table de plongements	28 emplacements de pièces + passe $\to 32$	
Vecteur de réserve	vecteur appris	152	plongement de source pour les placements et pour la ligne de passe
Politique	Linéaire, ReLU, Linéaire par ligne de coup sur destination ‖ source ‖ emplacement	$2 \times 152 + 32 \to 128 \to 1$	lignes illégales remplies d'une grande constante négative avant la softmax
Total			1.47 M paramètres

Tableau 49 : Les variantes du réseau graphe utilisées dans les ablations, avec leurs comptes de paramètres mesurés en millions. L'encodeur, le décodeur, les réglages d'entraînement, les budgets et les réglages d'évaluation sont identiques d'une variante à l'autre ; seul le composant nommé change.

Variante	Différence par rapport au bras graphe complet	Paramètres
Bras graphe complet	aucune	1.47 M
Arêtes non typées	une seule matrice partagée pour les six directions, dans chaque couche	0.54 M
Sans pooling global	aucun biais de pooling dans aucune couche	1.37 M
Arêtes non typées + écrêtage de gradient (supplément)	une seule matrice partagée pour les six directions, plus un écrêtage global de la norme du gradient à 1.0 pendant l'entraînement (une différence à deux composants)	0.54 M

B.3 Le décodeur d'actions partagé en détail

Emplacements. Une pièce est adressée par un emplacement relatif au camp au trait (Tableau 50) : les pièces du joueur au trait dans l'ordre de l'effectif, puis celles de l'adversaire. Dans le jeu de base, qui compte 11 pièces par camp, seuls les 11 premiers emplacements du joueur au trait peuvent jamais être légaux ; les emplacements des extensions et tous les emplacements adverses existent pour l'uniformité du noyau (l'extension Pillbug déplace des pièces ennemies) et restent inertes. Les types d'insectes sont codés 0–7 dans l'ordre Q, S, B, G, A, M, L, P ; ce code indexe les plans de pièces de l'encodage grille (type d'insecte × 4) et le bloc one-hot de l'encodage graphe.

Tableau 50 : Emplacements de pièces du décodeur d'actions partagé. L'emplacement 28 est l'emplacement de passe dans les lignes de coups du bras graphe ; la passe du bras grille est l'indice plat 28 672.

Emplacement	Pièce du joueur au trait	Emplacement	Pièce de l'adversaire
0	Reine	14	Reine
1–2	Araignées	15–16	Araignées
3–4	Scarabées	17–18	Scarabées
5–7	Sauterelles	19–21	Sauterelles
8–10	Fourmis	22–24	Fourmis

Emplacement	Pièce du joueur au trait	Emplacement	Pièce de l'adversaire
11	Moustique	25	Moustique
12	Ladybug	26	Ladybug
13	Pillbug	27	Pillbug

Indice plat (bras grille). Avec (x, y) les coordonnées de la destination dans le cadre, l'indice de politique est $\text{emplacement} \times 1024 + y \times 32 + x$; la passe est l'indice 28 672 et le vecteur compte 28 673 entrées. C'est ce même indice que stockent les enregistrements, pour le coup joué et pour chaque entrée de la distribution de visites et de la liste légale.

Lignes par candidat (bras graphe). Chaque indice légal de l'enregistrement est décodé en une ligne de coup : emplacement = indice div 1024 ; reste = indice mod 1024 ; x = reste mod 32, y = reste div 32 ; le nœud de destination est l'indice de la cellule (x, y) dans l'ordre de l'ensemble candidat ; le nœud source est le nœud où se tient la pièce si elle est sur le plateau, et la sentinelle de réserve 224 sinon ; l'indice de passe devient la ligne (28, 224, 224). L'identité encodage → indice → décodage sur tous les coups légaux à travers les types de partie est la deuxième des sept vérifications automatisées pré-entraînement.

Propriétés partagées. Masquage : la softmax s'exécute sur exactement l'ensemble légal (plus la passe lorsqu'elle est légale), implémentée comme un remplissage des entrées illégales par une grande constante négative dans les deux boucles d'entraînement et comme une softmax sur les lignes légales dans les deux évaluateurs d'inférence ; la première vérification automatisée affirme une masse illégale nulle via une vraie passe avant. Indépendance à l'ordre : les scores s'attachent aux paires (emplacement, destination), jamais aux positions de liste. Départage : les égalités d'argmax se résolvent vers l'indice plat le plus bas dans les deux bras. Cibles : les 15 premières entrées (indice, poids de visite) de la distribution de visites à la racine, normalisées sur leur support ; lorsque chaque poids stocké est nul, la cible se replie sur le coup joué. Valeur : une tête victoire/nulle/défaite entraînée avec les enregistrements tronqués exclus ; la recherche consomme P(victoire) − P(défaite) du point de vue du camp au trait, avec des valeurs terminales de +1, −1 et 0.

B.4 Format d'enregistrement

Les positions d'auto-jeu sont écrites comme des enregistrements de taille fixe, petit-boutistes (little-endian), dans des shards qui commencent par un en-tête de 16 octets (8 octets magiques identifiant la version du format, 8 réservés). Le format a évolué en trois versions : 112 octets avec une cible de coup joué en one-hot ; 176 octets ajoutant la distribution de visites top-15 ; et la version 3 de l'étude, à 818 octets, ajoutant l'estampille du modèle générateur et la liste d'indices des coups légaux, qui est ce qui permet l'entraînement masqué sur l'ensemble légal dans les deux bras. Seuls les enregistrements de version 3 sont des entrées de l'étude. Tableau 51 en donne l'agencement.

Tableau 51 : Agencement en octets de l'enregistrement d'auto-jeu de version 3 (818 octets, petit-boutiste). « Joueur au trait » désigne le camp au trait à la position enregistrée.

Octets	Champ	Encodage
0–83	28 pièces × (x, y, niveau)	un octet chacun, coordonnées du cadre, ordre absolu des pièces ; x = 255 signifie en main
84	camp au trait	0 blanc, 1 noir
85	dernière pièce déplacée	identifiant de pièce ; 255 = aucune (état d'étourdissement)

Octets	Champ	Encodage
86	pli	borné à 255
87	bits de type de partie	1 = M, 2 = L, 4 = P
88, 89	libertés des reines (joueur au trait, adversaire)	255 = reine non placée
90, 91	comptes de réserves (joueur au trait, adversaire)	
92–95	masque de bits des clouages One-Hive	32 bits, un bit par identifiant de pièce
96–97	indice de politique du coup joué	indice plat 16 bits
98	issue du point de vue du joueur au trait	0 défaite / 1 nulle / 2 victoire / 3 tronquée (jamais une nulle)
99	version de l'enregistrement	3
100–107	estampille du modèle générateur	génération (32 bits) puis hachage du réseau (32 bits) ; 0 = non estampillé
jusqu'à 112	réservé	zéro
112–175	distribution de visites à la racine	15 × (indice de politique 16 bits, poids de visite 16 bits), puis total des visites stockées (32 bits)
176–177	compte de coups légaux	16 bits ; 0xFFFF = liste indisponible (débordement)
178–817	liste des indices de politique légaux	16 bits × 320 (plafond ; facteur de branchement maximal mesuré 213) ; emplacements inutilisés à zéro

Les deux bras lisent le même enregistrement. Le bras grille en décode les 77 plans ; le bras graphe en construit le graphe de cellules ; tous deux prennent leur cible de politique dans les octets 112–175, leur masque légal dans les octets 176–817, et leur issue dans l'octet 98. L'écrivain Rust et les lecteurs Python des plans et des tenseurs graphe sont épinglés identiques à l'octet près par les tests dorés inter-langages exécutés chaque nuit sur 240 et 160 positions respectivement.

B.5 Réglages de recherche

Tableau 52 : Réglages de recherche partagés par les deux bras. Les réglages d'auto-jeu proviennent du protocole gelé ; les réglages d'évaluation ont été épinglés par configuration le 9 septembre 2026, avant toute exécution d'entraînement, et tout changement ultérieur compterait comme une nouvelle étude.

Réglage	Auto-jeu (données d'entraînement)	Évaluation indépendante
Recherche	recherche arborescente Monte-Carlo PUCT, c = 1.4, évaluation des feuilles par lots, valeurs terminales remontées exactement	identique
Simulations par décision	128 sur 25% des décisions (enregistrées), 32 sur les 75% restants (non enregistrées) ; randomisation du plafond de simulations	400
Bruit de Dirichlet à la racine	ε = 0.25	0 (la valeur par défaut du code, affirmée par un test ; le binaire du moteur n'expose aucun drapeau de bruit)
Sélection du coup	échantillonnage en température pour les 12 premiers plis, puis argmax	argmax déterministe
Abandon	en dessous de −0.92, avec 10% des parties n'abandonnant jamais (audit)	ne fait pas partie des réglages épinglés
Plafond de plis	300 ; une partie plafonnée est enregistrée comme tronquée, jamais comme nulle	300 ; troncature rapportée séparément

Réglage	**Auto-jeu (données d'entraînement)**	**Évaluation indépendante**
Ouvertures	aucune	4 plis d'ouverture, appariés à couleurs échangées, identifiants d'ouverture partagés entre toutes les exécutions et tous les bras
Fournisseur d'inférence	le meilleur de chaque bras : CoreML pour le bras grille, CPU pour le bras graphe	identique

La machinerie d'exploration est structurellement confinée à l'auto-jeu : les valeurs par défaut du chemin d'évaluation portent $\varepsilon = 0$ et aucune température, un test affirme ces valeurs par défaut, et le binaire du moteur n'expose aucun drapeau permettant d'activer le bruit, de sorte que la voie d'évaluation ne peut pas y souscrire. L'auto-jeu active le bruit explicitement. Les deux chemins diffèrent donc par construction plutôt que par convention.

C Constantes gelées, hyperparamètres, graines et configurations

Chaque valeur de cette annexe est celle que les campagnes ont réellement exécutée ; les fichiers de configuration et de manifeste conservés avec chaque exécution font autorité, et rien ici n'est une recommandation. Les dates sont celles de l'approbation de l'auteur ou du calcul mécanique qui a fixé une valeur ; Annexe E indexe les enregistrements correspondants, et Chapitre 9 explique pourquoi chaque valeur a été fixée au moment où elle l'a été.

C.1 Constantes gelées

Tableau 53 : Constantes expérimentales gelées avec la date à laquelle chacune a été fixée et l'approbation sous laquelle elle l'a été. Aucune constante n'a été changée après sa date.

Constante	Valeur	Fixée le (2026)	Approuvée par
Variante de jeu	jeu de base (reine, araignée, scarabée, sauterelle, fourmi), règle d'ouverture de tournoi ; extensions hors du périmètre	9 sept. (périmètre) ; gelée dans le protocole le 10 sept.	l'auteur
Plafond de coups et troncature	300 demi-coups ; une partie tronquée est une issue en propre, jamais une nulle ; taux rapporté séparément	9 sept. (convention et valeur mesurée) ; gelés le 10 sept.	l'auteur
Budget de recherche de l'auto-jeu	128 simulations complètes / 32 économiques par décision ; fraction complète 0.25	10 sept. (protocole)	l'auteur
Exploration de l'auto-jeu	échantillonnage en température pendant 12 plis ; bruit de Dirichlet à la racine $\varepsilon = 0.25$; abandon à -0.92 avec un audit sans abandon de 10%	10 sept. (matrice pré-enregistrée)	l'auteur
Recherche d'évaluation	400 simulations par décision ; sans bruit à la racine ; argmax déterministe	9 sept.	l'auteur
Volume d'évaluation	100 parties appariées par adversaire aux points de contrôle finaux et au seuil ; 20 par adversaire aux générations 4 et 7	10 sept. ; confirmé inchangé le 16 sept.	l'auteur
Population d'adversaires	B-RND (aléatoire légal), B-HEU (heuristique, poids épinglés par hachage), B-MCTS (recherche, 6 400 simulations) ; point de contrôle antérieur exclu	9 sept.	l'auteur
Ouvertures	250 ouvertures uniques et légales de 4 demi-coups ; graine du générateur 20260910 ; la paire i joue la ligne i avec les deux couleurs	10 sept.	l'auteur
Seuil à temps mural égal	18.77 h = médiane du temps mural total des trois exécutions grille originales ; règle dans la matrice	règle le 10 sept. ; valeur le 16 sept.	règle : l'auteur ; valeur : calculée mécaniquement

Constante	Valeur	Fixée le (2026)	Approuvée par
Graines d'entraînement	1–3 par bras ; 4–5 ajoutées sous trois pré-engagements écrits	10 sept. ; extension le 26 sept.	l'auteur
Document de protocole	version 1.0 ; tout changement ultérieur est une nouvelle étude	10 sept.	l'auteur

Les artefacts gelés sont identifiés par hachage de contenu (Tableau 54). Le fichier du protocole portait son hachage au gel et le porte inchangé ; le hachage des ouvertures couvre les 250 lignes d'ouverture indépendamment de l'en-tête du fichier ; les poids de l'heuristique sont en outre épinglés par un test automatisé qui échoue si un poids quelconque change.

Tableau 54 : Hachages de contenu des artefacts gelés ; ce sont les identifiants scientifiques du protocole, des ouvertures et de la population d'adversaires.

Artefact	SHA-256
Protocole, version 1.0	f340a6b64db0f5f0bf126ffb251c3de339450bde192fd54b 719036a8a3aefeb5
Ouvertures, contenu (250 lignes)	63b318d071dfc3ecfae3585636c8e6f7327ddc08e7aed86a 466f915f8005af7b
Poids de l'heuristique (B-HEU)	d0602f1895fbed70b6f84ac2a3eb87bd68e811e53acf24d4 d7814a1495b0b97a
Configuration B-RND	f2fc4a06441d3c1a7922838a6693dbb48ec54543bc34fd81 4d41c9a742514cd7
Configuration B-HEU	7210a0a349c5bad5dcd2df099cc6865ee3cb5d8a2c30e106 ce58137804818999
Configuration B-MCTS	3fc8f75cf2b4f21012dd61e9924408fbfc32c8ea561aa96e b44f1091ba07364e

C.2 Hyperparamètres d'entraînement

La boucle d'entraînement et chaque hyperparamètre sont identiques pour les deux bras et pour toutes les variantes d'ablation, à la seule exception notée dans la dernière ligne.

Tableau 55 : Hyperparamètres d'entraînement, identiques pour le bras grille, le bras graphe et les variantes d'ablation ; l'écrêtage de gradient du supplément A1′ est le seul changement d'optimiseur de l'étude.

Hyperparamètre	Valeur (les deux bras)
Optimiseur	SGD, momentum 0.9, décroissance de poids 1e-4
Taux d'apprentissage	0.02, recuit en cosinus jusqu'à lr/100 sur les pas de chaque génération
Taille de lot	256
Époques par génération	2
Perte de politique	entropie croisée contre la distribution de visites MCTS, softmax exactement sur l'ensemble des coups légaux
Perte de valeur	victoire/nulle/défaite du point de vue du camp au trait, poids 0.6 ; enregistrements tronqués exclus
Données d'entraînement par génération	les 500 parties d'auto-jeu de la génération ; seules les décisions à plein budget (128 simulations) sont enregistrées
Écrêtage de gradient	aucun, sauf dans le supplément A1′ (norme globale 1.0)
Recherche d'hyperparamètres	aucune, pour l'un ou l'autre bras

C.3 Dérivation des graines

Chaque choix aléatoire de l'étude descend d'une graine enregistrée, de sorte qu'une exécution, un ensemble d'évaluation ou un intervalle bootstrap se régénère à l'identique.

Tableau 56 : Dérivation de chaque graine utilisée dans l'entraînement, l'évaluation et l'analyse.

Quantité	**Graine**
Exécution d'entraînement (bras, graine *s*)	graine de base = 100 000 × *s* ; espaces de graines disjoints entre exécutions
Génération *g* (0–9)	graine d'auto-jeu = base + *g* ; graine d'entraînement = base + *g*
Réseau de la génération 0	initialisation aléatoire à graine fixée avec la graine de base, exporté avant tout auto-jeu
Réseau en cours d'évaluation	9000 + *g* pour les évaluations en cours d'exécution (générations 4, 7 et 9) ; 9500 pour les ensembles de points de contrôle au seuil
Adversaires	B-RND 9101 ; B-MCTS 9201 ; B-HEU déterministe (sans graine)
Lanceur d'affrontements	777 000 + *g* pour les évaluations en cours d'exécution ; 888 000 pour les ensembles au seuil ; avec des ouvertures fixes, le calendrier d'ouvertures et de couleurs est indépendant de la graine (hachages du calendrier reproduits entre exécutions avec des agents et des graines d'affrontement différents : 8cd84b6564440666 pour les ensembles de 20 parties, 3bf5d3e51cea0c0d pour les ensembles de 100 parties, identiques dans les deux études)
Générateur d'ouvertures	20260910
Bootstrap	graine de rééchantillonnage fixe ; 10 000 rééchantillonnages

C.4 Configuration de chaque bras et variante

Les cinq configurations partagent les constantes gelées de Tableau 53 et les hyperparamètres de Tableau 55 ; elles ne diffèrent que comme indiqué dans la table. Les comptes de paramètres sont des comptes exacts obtenus sur les modèles instanciés ; la différence de capacité de +1.5% entre les bras est le rapport exact. Les différences de capacité des variantes d'ablation sont inhérentes au composant retiré et sont rapportées plutôt qu'égalisées.

Tableau 57 : Configuration des deux bras et des trois variantes d'ablation, avec les comptes exacts de paramètres et, entre parenthèses, la différence par rapport au bras grille. Tout ce qui n'est pas listé est identique sur les cinq lignes. Le fournisseur d'inférence est le meilleur disponible mesuré par bras sur la machine d'étude.

Variante	**Corps**	**Paramètres**	**Différence par rapport au bras graphe complet**	**Graines**	**Fournisseur d'inférence**
Bras grille	CNN résiduel, 96 canaux × 8 blocs, sur 77 plans × 32 × 32	1 443 168	sans objet (l'autre bras)	1–5	CoreML
Bras graphe (méthode complète)	passage de messages relationnel, 152 canaux cachés × 8 couches, plongement d'emplacement 32, capacité de nœuds 224, six relations typées par direction,	1 465 452 (+1.5%)	référence	1–5	CPU

Variante	Corps	Paramètres	Différence par rapport au bras graphe complet	Graines	Fournisseur d'inférence
	biais de pooling global				
A1 (adjacence naïve)	une seule matrice d'arêtes partagée à la place des six matrices typées par direction	541 292 (−62.5%)	typage des arêtes retiré	1–3	CPU
A2 (sans pooling global)	biais de pooling global retiré de chaque couche	1 372 732 (−4.9%)	pooling retiré	1–3	CPU
A1′ (supplément)	comme A1 plus écrêtage global de la norme du gradient à 1.0	comme A1	deux composants : typage des arêtes retiré et écrêtage ajouté	1–3	CPU

C.5 Profil machine mesuré

Tous les chiffres de temps mural de ce rapport ont été mesurés sur un seul Apple M1 Pro (10 cœurs, 16 Go, macOS 15.3.1), avec 4 threads de travail par exécution, des exécutions séquentielles et la machine maintenue éveillée. Le fournisseur d'inférence utilisé pour l'auto-jeu et l'évaluation de chaque bras est le meilleur disponible mesuré sur cette machine ; les opérations riches en collecte du réseau graphe ne sont que partiellement prises en charge par l'accélérateur (147 nœuds sur 287, 15 partitions), ce pour quoi son chemin CPU l'emporte.

Tableau 58 : Coût d'inférence par évaluation du réseau sur la machine d'étude, mesuré le 10 septembre 2026 ; le meilleur chemin disponible par bras est celui utilisé en auto-jeu et en évaluation.

Chemin (lot 1, par évaluation de décision)	Grille	Graphe
PyTorch, CPU	11.03 ms	10.51 ms
ONNX, fournisseur CPU	23.5 ms	3.67 ms
ONNX, fournisseur CoreML	2.62 ms	9.84 ms
Meilleur disponible (utilisé)	2.62 ms (CoreML)	3.67 ms (CPU)

Le débit d'entraînement a été mesuré le 10 septembre 2026 au lot 128, passes avant et arrière, sur le backend GPU de la machine : 274 positions/s pour le réseau grille et 138 positions/s pour le réseau graphe (passe avant seule sur le CPU : 138 et 478 positions/s ; le réseau graphe est plus rapide par position sur le CPU et plus lent sur le GPU). Les exécutions de campagne ont chargé leurs données en processus et se sont entraînées à ces chiffres ou en dessous ; la génération des parties en auto-jeu, et non l'entraînement, domine le temps mural d'une génération dans l'un et l'autre bras. Le temps mural d'entraînement par exécution (auto-jeu, entraînement et export sur 10 générations × 500 parties, parties d'évaluation exclues) était de 16.7–19.1 h pour les exécutions grille et de 26.6–49.9 h pour les exécutions graphe, avec des moyennes de 18.0 h et 36.7 h (valeurs par graine dans Tableau 68). Une partie d'évaluation à 400 simulations prenait ≈23–32 s ; l'adversaire de recherche à 6 400 simulations décide en ≈27 ms sur un seul thread ; un

aller-retour par sous-processus vers le moteur coûte 21.7 µs. L'usage disque a été estimé à ≈3–5 Go par campagne à trois graines, contre une garde de 20 Go d'espace libre.

C.6 La seconde étude : constantes gelées, régime et graines

La seconde étude (Chapitre 15) réutilise toutes les constantes du Tableau 53 sauf là où le Tableau 59 en dispose autrement, et tous les hyperparamètres du Tableau 55 sauf les données d'entraînement par génération. Son protocole a été gelé en version 2.0 le 13 octobre 2026 sous l'empreinte de contenu donnée au Tableau 60 ; les deux bras sont les configurations des deux premières lignes du Tableau 57, exécutées avec les graines 11 à 15, et aucune variante d'ablation n'a été exécutée.

Tableau 59 : Constantes gelées de la seconde étude, avec la date à laquelle chacune a été fixée et l'approbation sous laquelle elle l'a été ; tout ce qui n'est pas listé est celui de la première étude.

Constante	**Valeur**	**Fixée le (2026)**	**Approuvée par**
Régime d'entraînement	démarrage à chaud : le réseau de la génération g est initialisé depuis le point de contrôle final de la génération $g-1$ (poids du modèle seulement ; l'optimiseur et le calendrier cosinus redémarrent à chaque génération) ; la génération 0 part de l'initialisation aléatoire ensemencée	13 oct. (protocole v2)	l'auteur
Fenêtre de rejeu	3 générations : la génération g s'entraîne sur les enregistrements des générations $\max(0, g-2)$ à g, soit 500, 1 000, puis 1 500 parties à partir de la génération 2 ; 2 époques par génération	13 oct.	l'auteur
Politique d'hyperparamètres	option A : les réglages de la première étude dans les deux bras, sans ajustement ; option B (trois taux d'apprentissage sur une graine par bras) examinée et écartée	13 oct.	l'auteur
Graines d'entraînement	11–15 par bras ; graine de base 100 000 × graine ; aucun flux aléatoire partagé avec les graines 1–5	13 oct.	l'auteur
Seuil à temps mural égal de la seconde étude	17.63 h = médiane du temps mural total d'entraînement des cinq exécutions grille de la seconde étude (17.46, 17.95, 17.63, 17.46, 18.52 h ; somme des dix horloges par génération, évaluations exclues), calculée une seule fois depuis les fichiers d'horloge le 25 octobre 2026 avant tout nombre inter-bras, jamais recalculée	règle 13 oct. ; valeur 25 oct.	règle : l'auteur ; valeur : calcul mécanique
Calendrier d'évaluation	générations 5 et 8 à 20 parties appariées par adversaire (descriptif seulement) ; point de contrôle final à 100 ; points de contrôle au seuil à 100 avec graine de réseau 9500 et graine d'appariement 888 000	13 oct. (comme dans la première étude)	l'auteur
Actifs gelés réutilisés	population d'adversaires, 250 ouvertures, réglages d'évaluation épinglés, les deux réseaux et le décodeur partagé, intouchés	13 oct. (réutilisation déclarée)	l'auteur
Ordre d'exécution	dix exécutions séquentielles sur la machine de l'étude, en alternant grille puis	13 oct.	l'auteur

Constante	Valeur	Fixée le (2026)	Approuvée par
	graphe pour les graines 11 à 15 ; aucun autre processus lourd pendant la campagne		
Document de protocole	version 2.0 ; tout changement ultérieur est une nouvelle étude	13 oct.	l'auteur

Tableau 60 : Empreinte de contenu du protocole gelé de la seconde étude ; les empreintes des artefacts réutilisés sont celles du Tableau 54.

Artefact	SHA-256
Protocole, version 2.0	e296a12e8d1e3dee36c3b210ececced62314e64657aede1cb91b1ea452b461eb

La dérivation des graines du Tableau 56 s'applique sans changement avec s = 11 à 15 : la graine de base est 100 000 × s, la génération g utilise base + g pour l'auto-jeu et l'entraînement, le réseau de la génération 0 est initialisé depuis la graine de base, et les graines d'évaluation (9000 + g en cours d'exécution, 9500 pour les ensembles au seuil ; appariement 777 000 + g en cours d'exécution, 888 000 pour les ensembles au seuil) sont celles de la première étude. Le pilote de campagne enregistre le régime de chaque exécution (démarrage à chaud actif, fenêtre 3) dans un fichier de configuration conservé avec l'exécution, et la configuration d'entraînement de chaque génération liste le point de contrôle d'initialisation et les segments effectivement lus.

D Procédures statistiques et tables de résultats brutes

Cette annexe énonce les estimateurs de l'étude exactement tels qu'ils ont été calculés et reproduit, sans arrondi ni recalcul, chaque nombre par exécution qui sous-tend la comparaison principale et les ablations. Les valeurs sont copiées depuis les fichiers de résultats régénérés et depuis les relevés d'analyse datés ; lorsqu'un relevé n'énonce pas une quantité, la cellule le dit.

D.1 Score, taux de troncature et score de sensibilité

Un affrontement d'évaluation entre un point de contrôle et un adversaire compte 100 parties sur les lignes d'ouverture gelées, chaque ligne étant jouée une fois avec chaque couleur. Une partie se termine par une victoire, une nulle, une défaite ou une troncature au plafond de 300 demi-coups. Des lignes de parties d'une cellule (graine, adversaire) sont dérivées trois statistiques : le **score**, moyenne de victoire = 1, nulle = 0.5, défaite = 0 sur les parties *non tronquées* ; le **taux de troncature**, nombre de parties tronquées divisé par les 100 parties jouées ; et le **score de sensibilité**, moyenne sur *toutes* les 100 parties, chaque partie tronquée comptant 0.5, rapporté comme colonne et jamais utilisé comme métrique primaire. Les parties sont d'abord agrégées en un score par cellule ; la graine est l'unité de chaque étape ultérieure, et aucune partie n'entre dans aucun intervalle comme observation indépendante.

D.2 Le bootstrap au niveau des graines

Chaque intervalle du rapport est un bootstrap percentile dont l'unité de rééchantillonnage est la graine, c'est-à-dire une exécution d'entraînement indépendante. Pour une série de cellule $s_1, \ldots, s_n$ (un score par graine, avec n = 5 dans l'analyse finale et n = 3 dans l'analyse du 19 septembre 2026), la procédure tire n indices uniformément avec remise, moyenne les scores correspondants, répète cette opération B = 10 000 fois, trie les moyennes de rééchantillonnage et rapporte les éléments situés aux positions (indexées à partir de zéro) $\lfloor 0.025\,B \rfloor$ et $\lfloor 0.975\,B \rfloor$ comme intervalle à 95%. Aucune correction de biais ni accélération n'est appliquée ; l'estimation ponctuelle imprimée à côté de chaque intervalle est la moyenne simple des n scores par graine. Le générateur pseudo-aléatoire est le `random.Random` de Python, initialisé avec la constante 0 à chaque appel, de sorte que chaque intervalle est reproductible au dernier chiffre près à partir des mêmes entrées par graine.

Le contraste entre bras est la différence des moyennes au niveau des graines, graphe moins grille. Dans chacun des B = 10 000 rééchantillonnages, l'ensemble de graines du graphe et l'ensemble de graines de la grille sont rééchantillonnés *indépendamment*, chacun avec sa propre taille, et la différence des deux moyennes de rééchantillonnage est enregistrée ; les différences triées sont coupées aux deux mêmes positions. Les graines sont indépendantes entre bras par construction (la graine k d'un bras ne partage rien avec la graine k de l'autre au-delà des ouvertures et des adversaires gelés), de

sorte qu'aucun appariement entre bras n'est imposé. Le même estimateur sert pour des ensembles de graines de tailles inégales : le supplément A1′ (trois graines) contre le bras graphe complet (cinq graines) rééchantillonne respectivement trois et cinq scores.

```
procédure PERCENTILE-CI(x[1..n]; B = 10 000; seed = 0)
    rng <- Random(seed)
    for b in 1..B:
        m[b] <- mean of n draws x[rng.randrange(n)]
    sort m ascending
    retourner m[floor(0.025 * B)], m[floor(0.975 * B)]       # positions indexées
à partir de zéro

procédure DIFF-CI(a[1..p], c[1..q]; B = 10 000; seed = 0)   # a = graphe, c =
grille
    rng <- Random(seed)
    for b in 1..B:
        d[b] <- (mean of p draws a[rng.randrange(p)]) - (mean of q draws
c[rng.randrange(q)])
    sort d ascending
    retourner d[floor(0.025 * B)], d[floor(0.975 * B)]
```

D.3 Tables brutes par graine : lecture à exemples égaux

La lecture à exemples égaux évalue le point de contrôle de chaque exécution après la dixième et dernière génération (les identifiants de points de contrôle sont indexés à partir de zéro, il s'agit donc de gen009), chaque exécution ayant consommé le même budget d'auto-jeu de 10 générations × 500 parties. Le Tableau 61 donne les scores et le taux de troncature contre l'aléatoire légal ; le Tableau 63, placé après la seconde lecture, donne les scores de sensibilité contre l'aléatoire légal sous les deux lectures ; l'aléatoire légal est le seul adversaire contre lequel une partie a été tronquée.

Tableau 61 : Scores par exécution, lecture à exemples égaux : le point de contrôle de dixième génération de chacune des cinq exécutions d'entraînement indépendantes par bras (10 générations × 500 parties d'auto-jeu chacune), 100 parties appariées, à couleurs échangées, par adversaire sur les ouvertures gelées à 400 simulations par décision, contre la population gelée (B-RND aléatoire légal, B-HEU heuristique, B-MCTS recherche à 6 400 simulations). Score = moyenne de victoire 1 / nulle 0.5 / défaite 0 sur les parties non tronquées (une fraction) ; tronc. = part des 100 parties arrêtées au plafond de 300 demi-coups, 0 % contre B-HEU et B-MCTS dans chaque cellule et omise. Valeurs brutes, sans intervalle.

Bras	Graine	Score B-RND	Tronc. B-RND	Score B-HEU	Score B-MCTS
grille	1	0.995	0 %	0.150	0.125
grille	2	0.975	1 %	0.080	0.125
grille	3	0.990	1 %	0.190	0.075
grille	4	0.949	11 %	0.105	0.085
grille	5	0.995	0 %	0.120	0.210
graphe	1	0.733	57 %	0.025	0.115
graphe	2	0.981	20 %	0.050	0.125
graphe	3	0.722	55 %	0.090	0.100
graphe	4	0.688	44 %	0.125	0.120
graphe	5	0.935	23 %	0.035	0.115

D.4 Tables brutes par graine : lecture à temps mural égal

La lecture à temps mural égal évalue le dernier point de contrôle de chaque exécution achevé dans le seuil à temps mural égal (Annexe D.6). Lorsque ce point de contrôle est le dernier de l'exécution, l'ensemble d'évaluation final sert aux deux lectures, de sorte que les lignes grille des graines 1, 2, 4 et 5 sont identiques dans les deux lectures.

Tableau 62 : Scores par exécution, lecture à temps mural égal : pour chacune des cinq exécutions d'entraînement indépendantes par bras, le dernier point de contrôle achevé dans les 18.77 h de temps mural d'entraînement cumulé (identifiants indexés à partir de zéro ; gen009 est la dixième génération), 100 parties appariées, à couleurs échangées, par adversaire sur les ouvertures gelées à 400 simulations par décision, contre la population gelée (B-RND aléatoire légal, B-HEU heuristique, B-MCTS recherche à 6 400 simulations). Score = moyenne de victoire 1 / nulle 0.5 / défaite 0 sur les parties non tronquées (une fraction) ; tronc. = part des 100 parties arrêtées au plafond de 300 demi-coups, 0 % contre B-HEU et B-MCTS dans chaque cellule et omise. Valeurs brutes, sans intervalle.

Bras	Graine	Point de contrôle	Score B-RND	Tronc. B-RND	Score B-HEU	Score B-MCTS
grille	1	gen009	0.995	0 %	0.150	0.125
grille	2	gen009	0.975	1 %	0.080	0.125
grille	3	gen008	0.939	1 %	0.145	0.080
grille	4	gen009	0.949	11 %	0.105	0.085
grille	5	gen009	0.995	0 %	0.120	0.210
graphe	1	gen003	0.798	58 %	0.045	0.075
graphe	2	gen004	0.926	39 %	0.055	0.110
graphe	3	gen004	0.713	53 %	0.060	0.105
graphe	4	gen002	0.631	39 %	0.100	0.150
graphe	5	gen005	0.980	24 %	0.045	0.095

Tableau 63 : Scores de sensibilité contre l'aléatoire légal (B-RND) sous les deux lectures, pour les mêmes exécutions, points de contrôle et volume d'évaluation que les Tableau 61 et Tableau 62 : la moyenne sur toutes les 100 parties de la cellule, chaque partie tronquée comptant 0.5, à la précision de quatre décimales des fichiers de résultats à valeurs séparées par des virgules. Contre B-HEU et B-MCTS, aucune partie d'aucune exécution n'a été tronquée sous l'une ou l'autre lecture, de sorte que le score de sensibilité y est égal au score des tables principales dans chaque cellule. Valeurs brutes, sans intervalle.

Bras	Graine	Exemples égaux : sens. B-RND	Temps mural égal : sens. B-RND
grille	1	0.9950	0.9950
grille	2	0.9700	0.9700
grille	3	0.9850	0.9350
grille	4	0.9000	0.9000
grille	5	0.9950	0.9950
graphe	1	0.6000	0.6250
graphe	2	0.8850	0.7600
graphe	3	0.6000	0.6000
graphe	4	0.6050	0.5800
graphe	5	0.8350	0.8650

D.5 Moyennes au niveau des graines, intervalles et contrastes entre bras

Le Tableau 64 donne la moyenne des cinq scores par graine pour chaque bras, adversaire et lecture, avec son intervalle bootstrap sur les graines ; le Tableau 65 donne les différences graphe moins grille de ces moyennes. Ce sont les nombres finaux de l'étude.

Tableau 64 : Scores moyens au niveau des graines des deux bras contre chaque adversaire gelé sous les deux lectures de budget (exemples égaux : points de contrôle de dixième génération après 10 générations × 500 parties ; temps mural égal : dernier point de contrôle dans les 18.77 h), cinq exécutions d'entraînement indépendantes par bras, 100 parties appariées, à couleurs échangées, par adversaire et par exécution. Chaque entrée est la moyenne simple des cinq scores par exécution (fraction des parties décidées, troncatures exclues) avec son intervalle bootstrap percentile à 95% sur les graines (10 000 rééchantillonnages).

Lecture	Bras	vs B-RND	vs B-HEU	vs B-MCTS
exemples égaux	grille	0.981 [0.964, 0.994]	0.129 [0.098, 0.165]	0.124 [0.087, 0.168]
exemples égaux	graphe	0.812 [0.710, 0.920]	0.065 [0.034, 0.100]	0.115 [0.107, 0.121]
temps mural égal	grille	0.971 [0.950, 0.991]	0.120 [0.098, 0.142]	0.125 [0.090, 0.168]
temps mural égal	graphe	0.810 [0.697, 0.922]	0.061 [0.047, 0.081]	0.107 [0.087, 0.130]

Tableau 65 : Contraste entre bras : différence des moyennes au niveau des graines du Tableau 64, bras graphe moins bras grille, par adversaire gelé et par lecture de budget, cinq exécutions d'entraînement indépendantes par bras ; unité : différence de score (fraction des parties décidées). Entre crochets : intervalle bootstrap percentile à 95% obtenu à partir des cinq graines du graphe et des cinq graines de la grille rééchantillonnées indépendamment, 10 000 rééchantillonnages.

Adversaire	Exemples égaux : graphe − grille	Temps mural égal : graphe − grille
B-RND	−0.169 [−0.272, −0.062]	−0.161 [−0.278, −0.048]
B-HEU	−0.064 [−0.111, −0.017]	−0.059 [−0.087, −0.029]
B-MCTS	−0.009 [−0.055, +0.028]	−0.018 [−0.066, +0.025]

Les graines ont été collectées en deux étapes : les graines 1–3 des deux bras lors de la campagne du 10 au 17 septembre 2026, analysées le 19 septembre 2026 ; les graines 4–5 des deux bras entre le 27 septembre et le 2 octobre 2026, sous un engagement, pris avant leur exécution, d'utiliser les cinq graines dans l'analyse finale quelle que soit leur direction. Le Tableau 66 consigne l'analyse à trois graines afin que les deux étapes figurent au dossier ; l'extension a resserré quatre des six intervalles de contraste, et la plus grande borne supérieure de tout contraste est passée de +0.035 à +0.028.

Tableau 66 : L'analyse originale à trois graines du 19 septembre 2026 (graines 1–3 de chaque bras ; mêmes adversaires, lectures de budget et volume d'évaluation que le Tableau 64) : moyennes au niveau des graines et contrastes graphe moins grille avec intervalles bootstrap percentiles à 95% sur trois graines par bras (10 000 rééchantillonnages ; bras rééchantillonnés indépendamment). Remplacée par les tables à cinq graines ; conservée comme relevé de la première étape de collecte des graines.

Quantité	Lecture	vs B-RND	vs B-HEU	vs B-MCTS
moyenne grille	exemples égaux	0.987 [0.975, 0.995]	0.140 [0.080, 0.190]	0.108 [0.075, 0.125]
moyenne graphe	exemples égaux	0.812 [0.722, 0.981]	0.055 [0.025, 0.090]	0.113 [0.100, 0.125]
moyenne grille	temps mural égal	0.970 [0.939, 0.995]	0.125 [0.080, 0.150]	0.110 [0.080, 0.125]
moyenne graphe	temps mural égal	0.812 [0.713, 0.926]	0.053 [0.045, 0.060]	0.097 [0.075, 0.110]
graphe − grille	exemples égaux	−0.175 [−0.268, −0.007]	−0.085 [−0.143, −0.025]	+0.005 [−0.020, +0.035]

Quantité	Lecture	vs B-RND	vs B-HEU	vs B-MCTS
graphe − grille	temps mural égal	−0.158 [−0.254, −0.050]	−0.072 [−0.100, −0.028]	−0.013 [−0.040, +0.015]

D.6 Le seuil à temps mural égal et la carte des points de contrôle

Le seuil a été fixé par une règle énoncée avant la campagne : la médiane des temps muraux d'entraînement d'exécution complète des trois exécutions originales du bras grille. Ces totaux étaient de 18.77 h, 16.73 h et 19.07 h, de sorte que le seuil est de 18.77 h ; il a été calculé le 16 septembre 2026, avant l'existence de tout nombre inter-bras, et les deux graines grille ultérieures ne sont jamais entrées dans la médiane. Le calcul final prend la médiane exactement à partir des fichiers de chronométrage par génération plutôt qu'à partir de la constante arrondie, de sorte que le point de contrôle final de l'exécution qui définit le seuil se situe au seuil, inclusivement ; la carte résultante pour les graines 1–3 reproduit, point de contrôle pour point de contrôle, celle consignée le 16 septembre 2026. Pour chaque exécution, le point de contrôle retenu est le dernier dont le temps mural d'entraînement cumulé n'excède pas le seuil (Tableau 67).

Tableau 67 : Point de contrôle retenu pour la lecture à temps mural égal dans chacune des dix exécutions d'entraînement : le dernier point de contrôle achevé dans le seuil à temps mural égal de 18.77 h de temps mural d'entraînement cumulé (identifiants indexés à partir de zéro ; gen009 est la dixième et dernière génération). Les heures cumulées sont celles indiquées dans les relevés d'analyse ; « non indiqué » signifie que le relevé donne le point de contrôle mais pas les heures. Lorsque le point de contrôle retenu est le dernier, l'ensemble d'évaluation final (100 parties appariées par adversaire) sert aux deux lectures ; sinon, un ensemble à temps égal distinct, de même volume, a été évalué.

Bras	Graine	Point de contrôle au seuil	Temps mural cumulé	Ensemble d'évaluation
grille	1	gen009 (final)	18.77 h	ensemble final réutilisé
grille	2	gen009 (final)	16.73 h	ensemble final réutilisé
grille	3	gen008	17.22 h	ensemble à temps égal
grille	4	gen009 (final)	non indiqué	ensemble final réutilisé
grille	5	gen009 (final)	non indiqué	ensemble final réutilisé
graphe	1	gen003	16.50 h	ensemble à temps égal
graphe	2	gen004	16.06 h	ensemble à temps égal
graphe	3	gen004	non indiqué	ensemble à temps égal
graphe	4	gen002	non indiqué	ensemble à temps égal
graphe	5	gen005	non indiqué	ensemble à temps égal

La carte est le contenu mesuré de la seconde lecture : à temps mural égal, le bras graphe avait achevé 3–6 de ses dix générations (4–5 sur les trois graines originales), le bras grille neuf ou dix. Le Tableau 68 donne les temps muraux d'entraînement d'exécution complète qui sous-tendent la carte. Sur les cinq graines, les moyennes sont de 18.00 h (grille) et 36.70 h (graphe), soit un rapport de 2.04× ; sur les trois graines originales, elles étaient de 18.2 h et 35.7 h, soit un rapport de 2.0×. Les ensembles d'évaluation à temps égal des graines 1–3 ont été exécutés le 18 septembre 2026 (environ 11 h), ceux des graines 4 et 5 du bras graphe le 6 octobre 2026 (environ 6 h).

Tableau 68 : Temps mural d'entraînement de chacune des dix exécutions de la campagne principale, en heures : les secondes de génération en auto-jeu et d'entraînement par génération, sommées sur les 10 générations de l'exécution (10 × 500 parties), parties d'évaluation exclues, d'après le journal de chronométrage de chaque exécution ; une seule machine, quatre threads de travail, exécutions séquentielles. Les sommes sont de 90.0 h (grille) et 183.5 h (graphe), soit 273.5 h au total. Le seuil à temps mural égal est la médiane des trois valeurs grille des graines 1–3.

Bras	Graine 1	Graine 2	Graine 3	Graine 4	Graine 5	Moyenne
grille	18.77	16.73	19.07	18.23	17.19	18.00
graphe	43.07	32.71	31.28	49.86	26.57	36.70

D.7 Bornes de sensibilité au plafond

Le protocole encadre l'estimateur primaire (troncatures exclues) par trois traitements alternatifs de chaque partie tronquée : comptée 0.5 (les scores de sensibilité du Tableau 63), comptée comme défaite pour le bras testé, et comptée comme victoire pour lui ; ce dernier traitement est une borne supérieure de ce que tout plafond plus grand pourrait apporter à un bras qui tronque. Les bornes ont été calculées à partir des enregistrements par partie dans l'analyse du 19 septembre 2026 (trois graines par bras) : chaque partie tronquée étant comptée comme victoire pour le bras testé, le contraste graphe moins grille contre l'aléatoire légal reste de −0.072 sous la lecture à exemples égaux et de −0.058 sous la lecture à temps mural égal, et sa direction est inchangée sous chaque traitement (exclue, 0.5, défaite, victoire). Contre B-HEU et B-MCTS, aucune partie d'aucun des deux bras n'a été tronquée, de sorte que tous les traitements y coïncident. Le relevé d'analyse final ne réénonce pas les bornes à cinq graines. Parce que le bras graphe a tronqué 20–57% de ses parties contre l'aléatoire légal sur les graines originales et 23–44% sur les graines d'extension (grille : 0–1%, avec une graine d'extension à 11%), son score primaire contre cet adversaire est une moyenne sur moins de parties décidées (43 à 80 par graine) que celui du bras grille.

D.8 La seconde étude : scores de sensibilité, carte des seuils et temps mural

La seconde étude (Chapitre 15) utilise sans changement les estimateurs de cette annexe : le même score, le même taux de troncature et le même score de sensibilité par cellule (graine, adversaire), le même bootstrap percentile au niveau des graines avec $B = 10\,000$ et le même rééchantillonnage indépendant des deux bras pour le contraste. Ses tables de scores par graine sous les deux lectures, ses moyennes au niveau des graines et ses contrastes sont donnés dans le chapitre (Tableau 39 à Tableau 43) et ne sont pas répétés ; cette section ajoute les scores de sensibilité, la carte des seuils avec les horloges cumulées qui la sous-tendent, et le temps mural par exécution.

Tableau 69 : Seconde étude, scores de sensibilité contre l'aléatoire légal (B-RND) sous les deux lectures, pour les exécutions, points de contrôle et volume d'évaluation des Tableau 39 et Tableau 41 : la moyenne sur toutes les 100 parties de la cellule, chaque partie tronquée comptant 0.5, à la précision de quatre décimales des fichiers de résultats à valeurs séparées par des virgules. Contre B-HEU et B-MCTS, aucune partie d'aucune exécution n'a été tronquée sous l'une ou l'autre lecture, de sorte que le score de sensibilité y est égal au score des tables principales dans chaque cellule. Valeurs brutes, sans intervalle.

Bras	Graine	Exemples égaux : sens. B-RND	Temps mural égal : sens. B-RND
grille	11	0.9600	0.9600
grille	12	0.9650	0.9600
grille	13	0.9900	0.9900
grille	14	0.9800	0.9800
grille	15	0.9550	0.9400
graphe	11	1.0000	0.9900
graphe	12	0.9750	0.9650
graphe	13	0.7550	0.6100
graphe	14	0.8250	0.8350
graphe	15	0.7900	0.7750

Sous le traitement de sensibilité, le contraste graphe moins grille contre l'aléatoire légal est de −0.101 [−0.187, −0.012] sous la lecture à exemples égaux et de −0.131 [−0.256, −0.015] sous la lecture à temps mural égal ; les quatre autres contrastes sont ceux du Tableau 43, puisqu'il n'y a aucune troncature contre les deux autres adversaires. Les deux traitements de bornage (chaque partie tronquée comptée comme défaite ou comme victoire pour le bras testé) n'ont pas été calculés pour la seconde étude.

Le seuil de la seconde étude a été fixé par la règle de son protocole : la médiane des temps muraux d'entraînement d'exécution complète de ses cinq exécutions grille, chacun étant la somme des dix horloges par génération, calculée exactement à partir des fichiers d'horloge le 25 octobre 2026, quelques minutes après la fin de la cinquième exécution grille et alors que la dernière exécution graphe en était à sa première génération, avant qu'aucun score de l'étude n'ait été lu. Les cinq totaux étaient de 17.4567, 17.9540, 17.6263, 17.4592 et 18.5176 h ; la médiane, 17.6263 h (63 454.8 s), est le total de la graine grille 13, dont le point de contrôle final se situe donc au seuil, inclusivement. Pour chaque exécution, le point de contrôle retenu est le dernier dont le temps mural d'entraînement cumulé n'excède pas le seuil (Tableau 70).

Tableau 70 : Seconde étude, point de contrôle retenu pour la lecture à temps mural égal dans chacune des dix exécutions à démarrage à chaud : le dernier point de contrôle achevé dans le seuil à temps égal de 17.63 h de temps mural d'entraînement cumulé (identifiants indexés à partir de zéro ; gen009 est la dixième et dernière génération), avec les heures cumulées à ce point de contrôle d'après les fichiers d'horloge. Lorsque le point de contrôle retenu est le dernier, l'ensemble d'évaluation final (100 parties appariées par adversaire) sert aux deux lectures ; sinon, un ensemble à temps égal distinct, de même volume, a été évalué les 26–27 octobre 2026 avec les graines de seuil épinglées (réseau 9500, lanceur d'affrontements 888 000).

Bras	Graine	Point de contrôle au seuil	Temps mural cumulé	Ensemble d'évaluation
grille	11	gen009 (final)	17.46 h	ensemble final réutilisé
grille	12	gen008	16.54 h	ensemble à temps égal
grille	13	gen009 (final)	17.63 h	ensemble final réutilisé (exécution définissant le seuil)
grille	14	gen009 (final)	17.46 h	ensemble final réutilisé
grille	15	gen008	16.77 h	ensemble à temps égal
graphe	11	gen003	17.25 h	ensemble à temps égal
graphe	12	gen004	17.51 h	ensemble à temps égal
graphe	13	gen002	13.95 h	ensemble à temps égal
graphe	14	gen002	15.38 h	ensemble à temps égal
graphe	15	gen003	14.93 h	ensemble à temps égal

À temps mural égal, le bras graphe avait achevé 3–5 de ses dix générations, le bras grille neuf ou dix (première étude : 3–6 et neuf ou dix). Le Tableau 71 donne les temps muraux d'entraînement d'exécution complète qui sous-tendent la carte ; leurs moyennes sont de 17.80 h (grille) et 41.59 h (graphe), soit un rapport de 2.34× (première étude : 2.04×).

Tableau 71 : Seconde étude, temps mural d'entraînement de chacune des dix exécutions à démarrage à chaud, en heures : les secondes de génération en auto-jeu, d'entraînement et d'export par génération, sommées sur les 10 générations de l'exécution (10 × 500 parties, fenêtre de rejeu de trois générations), parties d'évaluation exclues, d'après le journal de chronométrage de chaque exécution ; une seule machine, quatre threads de travail, exécutions séquentielles. Les sommes sont de 89.0 h (grille) et 207.9 h (graphe), soit 296.9 h au total. Le seuil à temps égal est la médiane des cinq valeurs grille.

Bras	Graine 11	Graine 12	Graine 13	Graine 14	Graine 15	Moyenne
grille	17.46	17.95	17.63	17.46	18.52	17.80
graphe	39.28	37.33	52.85	42.20	36.27	41.59

Les figures de trajectoire de la seconde étude sont tracées comme celles de la première, directement à partir des enregistrements d'évaluation de chaque exécution aux points de contrôle des générations 5, 8 et 10 et de son journal de chronométrage ; la figure des métriques d'entraînement est extraite du journal de la seconde campagne, et les valeurs par exécution qu'elle montre à la première et à la dixième génération sont tabulées dans le relevé d'analyse.

D.9 Tables brutes des ablations

Trois variantes du bras graphe ont été entraînées avec trois graines chacune au budget complet (10 générations × 500 parties, réglages d'auto-jeu, d'entraînement et d'évaluation identiques) et évaluées à leur point de contrôle de dixième génération sous la lecture à exemples égaux. Le Tableau 72 résume les variantes ; les Tableau 73 et Tableau 74 donnent les scores par graine des deux variantes qui se sont entraînées ; le Tableau 75 donne les contrastes contre le bras graphe complet.

Tableau 72 : Les trois variantes d'ablation du bras graphe, chacune entraînée avec trois graines indépendantes au budget complet de la comparaison principale (10 générations × 500 parties d'auto-jeu ; mêmes adversaires, ouvertures et réglages d'évaluation gelés). Temps mural d'entraînement en heures par graine, selon la même comptabilité que le Tableau 68 (secondes d'auto-jeu et d'entraînement par génération sommées, parties d'évaluation exclues) ; les exécutions graphe de référence des graines 1–3 ont pris 43.07, 32.71 et 31.28 h. Les exécutions courtes de A1 sont un symptôme de sa divergence (une politique NaN joue des parties dégénérées courtes) plutôt qu'une économie.

Variante	Composant modifié	Graines	Temps mural d'entraînement	Issue
A1 (`graph-untyped`)	les six matrices d'arêtes typées par direction remplacées par une seule matrice partagée	1, 2, 3	7.75 / 8.01 / 7.61 h	entraînement divergé vers NaN à la génération 0, 3 graines sur 3
A2 (`graph-nogpool`)	biais de pooling global retiré de chaque couche (1.37M paramètres vs 1.47M)	1, 2, 3	39.90 / 47.44 / 32.05 h	entraînée ; aucun effet mesurable

Variante	Composant modifié	Graines	Temps mural d'entraînement	Issue
A1′ (`graph-untyped-clip`)	arêtes non typées *et* écrêtage de la norme du gradient à 1.0 (deux composants)	1, 2, 3	23.89 / 30.35 / 27.68 h	entraînée ; dans la bande du bras complet

Aucune table de scores n'est donnée pour A1 parce qu'il n'en existe aucune qui constitue une mesure de force : chaque passe avant des trois réseaux a renvoyé NaN pour la politique et la valeur dès le premier point de contrôle, et les évaluations finales des trois exécutions à graines indépendantes étaient identiques à la partie près (0 victoire, 0 nulle et 100 défaites contre B-HEU ; 2 victoires et 45 nulles avec 53% de troncature contre B-RND), ce que des entraînements indépendants ne peuvent pas produire. Les journaux d'entraînement portent la signature « policy top-1 100.0%, value acc 0.0% » dès la génération 1, et les parties dégénérées comptaient en moyenne environ 42 demi-coups, sans troncature ni abandon. Les fichiers d'évaluation bruts sont conservés mais exclus de toute table en tant que scores.

Tableau 73 : Scores finaux par graine de l'ablation A2 (biais de pooling global retiré) à côté des graines 1–3 du bras graphe complet, lecture à exemples égaux : points de contrôle de dixième génération, 100 parties appariées, à couleurs échangées, par adversaire sur les ouvertures gelées à 400 simulations contre la population gelée. Score = fraction des parties décidées gagnées (nulles 0.5) ; taux de troncature des 100 parties entre parenthèses. Trois exécutions d'entraînement indépendantes par variante ; valeurs brutes, sans intervalle.

Adversaire	A2 graine 1	A2 graine 2	A2 graine 3	complet graine 1	complet graine 2	complet graine 3
B-RND	0.768 (59 %)	0.714 (65 %)	0.952 (17 %)	0.733 (57 %)	0.981 (20 %)	0.722 (55 %)
B-HEU	0.050 (0 %)	0.070 (0 %)	0.025 (0 %)	0.025 (0 %)	0.050 (0 %)	0.090 (0 %)
B-MCTS	0.070 (0 %)	0.125 (0 %)	0.105 (0 %)	0.115 (0 %)	0.125 (0 %)	0.100 (0 %)

Tableau 74 : Scores finaux par graine du supplément A1′ (arêtes non typées avec écrêtage de gradient à 1.0, une variante explicitement à deux composants), lecture à exemples égaux : points de contrôle de dixième génération, 100 parties appariées, à couleurs échangées, par adversaire sur les ouvertures gelées à 400 simulations contre la population gelée. Score = fraction des parties décidées gagnées (nulles 0.5) ; le taux de troncature des 100 parties est donné entre parenthèses là où le relevé l'indique (aléatoire légal seulement). Trois exécutions d'entraînement indépendantes ; valeurs brutes, sans intervalle.

Adversaire	A1′ graine 1	A1′ graine 2	A1′ graine 3
B-RND	0.566 (47 %)	0.671 (59 %)	0.995 (0 %)
B-HEU	0.035	0.135	0.110
B-MCTS	0.090	0.195	0.060

Tableau 75 : Contrastes d'ablation contre le bras graphe complet, lecture à exemples égaux, population gelée, 100 parties appariées par adversaire et par exécution. A2 – complet : différence des moyennes au niveau des graines sur les graines 1–3 des deux variantes. A1′ – complet : différence entre la moyenne de A1′ sur ses trois graines et la moyenne du bras graphe complet sur ses cinq graines. Unité : différence de score (fraction des parties décidées). Entre crochets : intervalle bootstrap percentile à 95%, les deux ensembles de graines étant rééchantillonnés indépendamment avec leurs propres tailles, 10 000 rééchantillonnages. Le contraste A1′ est confondu par l'écrêtage et n'est jamais attribué au seul typage des arêtes.

Adversaire	A2 – complet (3 vs 3 graines)	Moyenne A1′ (3 graines)	Moyenne graphe complet (5 graines)	A1′ – complet (3 vs 5 graines)
B-RND	−0.001 [−0.170, +0.165]	0.744	0.810	−0.066 [−0.254, +0.130]
B-HEU	−0.007 [−0.043, +0.030]	0.093	0.061	+0.032 [−0.011, +0.078]
B-MCTS	−0.013 [−0.043, +0.013]	0.115	0.107	+0.008 [−0.040, +0.063]

D.10 Données d'évaluation par génération

Aucune table de scores par génération n'existe dans les fichiers de résultats ni dans les relevés d'analyse. Les figures de score en fonction du temps et de trajectoires par adversaire sont tracées directement à partir des enregistrements d'évaluation de chaque exécution : chaque exécution de la campagne principale a été évaluée, avec les réglages et le volume épinglés, aux points de contrôle des générations 5, 8 et 10 (identifiants gen004, gen007 et gen009) et, lorsqu'il diffère du point de contrôle final, à son point de contrôle à temps égal ; le score contre la population en un point est la moyenne, sur les trois adversaires, des scores de cellule définis ci-dessus, et l'axe du temps est la somme cumulée des durées d'entraînement par génération de l'exécution. Les courbes sont reproductibles à partir des enregistrements par partie et des fichiers de chronométrage publiés ; leurs nombres ne sont pas reproduits ici.

E Provenance de chaque résultat

Cette annexe est le seul endroit du rapport où les identifiants internes sont l'objet même du propos. Elle donne, pour chaque table et chaque figure, l'artefact généré dont elle est tirée, les enregistrements bruts à partir desquels cet artefact a été calculé, et l'entrée de journal qui a consigné la mesure ; elle indexe ensuite le journal des décisions, reproduit intégralement le registre des affirmations et des preuves, et indique où résident les données brutes ainsi que les hachages de chaque artefact gelé.

E.1 La chaîne de preuve

Chaque nombre de ce rapport est atteignable le long d'une seule chaîne. Les **enregistrements bruts par partie** sont écrits par l'arène d'évaluation sous la forme d'un CSV par appariement (point de contrôle, adversaire) dans le répertoire de l'exécution ; les lignes d'en-tête du fichier nomment verbatim les deux lignes de commande des moteurs (chemin du réseau, nombre de simulations, graines), et chaque partie porte son issue avec la troncature comme catégorie à part entière. Les **tables générées** sont produites par `scripts/make_results.py`, qui lit ces CSV et les journaux de temps mural par exécution, agrège par (graine, adversaire), applique le bootstrap sur les graines (10 000 rééchantillonnages, la graine comme unité, aucune partie agrégée comme i.i.d.), et écrit `results/comparison/*.{md,csv}` ; il ne ressaisit jamais un nombre. Les **figures** sont produites par `scripts/make_figures.py` à partir des mêmes CSV, journaux de temps mural et journaux de campagne. **Le rapport** est assemblé à partir de ces fichiers ; un vérificateur de sources vérifie que chaque jeton numérique de chaque chapitre apparaît dans la base de preuves, et un second vérificateur vérifie l'identité numérique entre les versions anglaise et française. Le script de reproduction minimale reconstruit le moteur dans un clone frais, rejoue une partie enregistrée jusqu'à une correspondance exacte avec sa ligne livrée, et régénère les tables à l'identique à l'octet près (vérifié le 2026-10-09).

Journaux. Chaque expérience ou mesure a une entrée sous `journal/`, nommée `YYYY-MM-DD-<slug>.md` et portant un identifiant de la forme `<phase>-<date>-<slug>-<nn>` (par exemple `H6-2026-10-09-5seed-final-01`). L'en-tête fixe l'identifiant, la date, l'hypothèse, le commit git, la configuration, les graines, la version des données, le matériel, la durée et le coût ; le corps a des sections fixes : Méthodes, Résultats bruts et incertitude, Échecs, Limites et facteurs de confusion, Interprétation, Décision, Artefacts. Les résultats négatifs et les défauts d'outillage sont des entrées comme les autres. **Journal des décisions.** Chaque décision non triviale est une entrée `D-nnn` dans le fichier d'espace de travail `state/decisions.md`, en ajout seul : une décision renversée n'est jamais modifiée, une nouvelle entrée la remplace et renvoie vers elle. Les franchissements de porte consignent l'approbation de l'auteur dans sa formulation originale. **Journal de méthodologie.** `docs/methodology-log.md` (espace de travail) consigne, en ajout seul, chaque incident où la méthode a attrapé ou manqué quelque chose.

E.2 De chaque résultat à son artefact

Tableau 76 couvre les tables et figures générées ; Tableau 77 couvre les preuves de validation, de lignes de base, de pipeline et d'encodeurs citées dans les chapitres de méthode, qui résident dans des suites de tests, des configurations et des journaux plutôt que dans des tables générées.

Tableau 76 : Provenance de chaque table et figure générée : l'artefact dont elle est tirée, les enregistrements bruts à partir desquels cet artefact est calculé, et l'entrée de journal qui a consigné le résultat. Les répertoires d'exécution sont abrégés en `eval/…` pour `data/runs/cmp-<arm>-s<seed>/eval/…`.

Résultat dans le rapport	**Artefact généré**	**Entrée brute**	**Entrée de journal**
Scores par graine et taux de troncature, lecture à exemples égaux (2 bras × 5 graines × 3 adversaires, 100 parties chacun)	`results/comparison/results-same-examples.{md,csv}`	`data/runs/cmp-{grid,graph}-s{1..5}/eval/gen009-vs-{B-RND,B-HEU,B-MCTS}.csv`	`H6-2026-09-19-comparison-01` (3 graines) ; `H6-2026-10-09-5seed-final-01` (finale)
Scores par graine et taux de troncature, lecture à temps mural égal	`results/comparison/results-same-wallclock.{md,csv}`	`eval/tstar-gen00N-vs-*.csv` de chaque exécution (les fichiers du point de contrôle final lorsque le point de contrôle au seuil est le dernier) et `wallclock.json`	les deux mêmes entrées
Contraste entre bras graphe – grille, les deux lectures, bootstrap sur les graines	`results/comparison/results-arm-difference.md`	les deux fichiers CSV ci-dessus	les deux mêmes entrées
Seuil à temps mural égal T* = 18.77 h et carte des points de contrôle à T*	nombres dans le journal ; carte re-dérivée par `make_results.py`	`wallclock.json` des exécutions grille s1–s3 (médiane des trois totaux), puis de chaque exécution	`H6-2026-09-16-progress-01` ; re-dérivation vérifiée dans `H6-2026-10-09-5seed-final-01`
Bornes de sensibilité au plafond (chaque troncature comptée comme une victoire pour le bras testé)	nombres dans le journal	enregistrements par partie des évaluations finales	`H6-2026-09-19-comparison-01`
Table des coûts : paramètres, inférence au meilleur fournisseur, temps mural d'exécution (génération en auto-jeu + entraînement, évaluation exclue, moyenne sur les	`paper/figures/fig2-score-cost.md` (régénéré le 2026-10-09 après la découverte d'un défaut du générateur ; voir Chapitre 10)	`wallclock.json` par exécution ; `results-*.csv` ; référence dans `docs/representations/comparison-controls.md`	`H5-2026-09-10-encoders-01` ; `H6-2026-09-19-comparison-01`

Résultat dans le rapport	Artefact généré	Entrée brute	Entrée de journal
cinq graines), coût d'auto-jeu (ce temps mural / 5 000 parties), référence de débit d'entraînement (lot 128), scores contre la population			
Score en fonction du temps mural d'entraînement, toutes graines, les deux bras, T* marqué	`paper/figures/fig1-score-vs-time.png`	`eval/*.csv` et `wallclock.json` par exécution	`H6-2026-09-19-comparison-01` ; `H6-2026-10-09-5seed-final-01`
Plans de grille contre graphe de cellules pour une position	`paper/figures/fig3-encodings.png`	schéma : une position synthétique à cinq pièces dessinée dans le script ; aucune quantité mesurée	`H5-2026-09-10-encoders-01` (définitions des encodages)
Trois positions d'échec F1–F3	`paper/figures/fig4-failures.{md,png}`	`eval/gen009-vs-*.csv` et `results/comparison/openings-v1.txt` ; chaque partie reproduite de façon déterministe et vérifiée contre sa ligne CSV	`H6-2026-09-19-comparison-01`
Trajectoires de score par adversaire selon la génération, 5 graines	`paper/figures/fig5-per-opponent.png`	`eval/{gen004,gen007,gen009}-vs-*.csv` et `wallclock.json` par exécution	`H8-2026-10-09-detailed-edition-01`
Argmax de politique (top-1) et exactitude de la valeur à l'époque 1 selon la génération, 10 exécutions	`paper/figures/fig6-training-metrics.png`	`data/runs/campaign.log`, `data/runs/extension.log` (journaux d'entraînement des 10 exécutions principales)	`H8-2026-10-09-detailed-edition-01`
Taux de troncature à l'évaluation finale contre l'aléatoire légal par exécution	`paper/figures/fig7-truncation.png`	`eval/gen009-vs-B-RND.csv` par exécution	`H8-2026-10-09-detailed-edition-01`
Table d'ablation : divergence A1, effet nul A2, supplément A1′	`results/ablations/README.md`	`data/runs/cmp-graph-{untyped,nogpool,untyped-clip}-s{1,2,3}/eval/` contre les finales du bras graphe complet	`H7-2026-09-23-a1-divergence-01` ; `H7-2026-10-02-a2-nogpool-01` ; `H7-2026-10-09-a1prime-01`
Schémas du pipeline, des architectures et de la chronologie	`paper/figures/fig8-pipeline.png`, `fig9-architectures.png`, `fig10-timeline.png` (`scripts/make_report_figures.py`)	la documentation du système (`docs/inventory.md`), les entêtes de journaux et le journal des décisions ;	aucune

Résultat dans le rapport	**Artefact généré**	**Entrée brute**	**Entrée de journal**
		aucune quantité mesurée	
Seconde étude : scores par graine et taux de troncature, lecture à exemples égaux (2 bras × 5 graines × 3 adversaires, 100 parties chacun)	`results/study2/results-same-examples.{md,csv}`	`data/runs/cmp2-{grid,graph}-s{11..15}/eval/gen009-vs-*.csv`	`H9-2026-10-27-analysis-01`
Seconde étude : scores par graine, lecture à temps mural égal	`results/study2/results-same-wallclock.{md,csv}`	`eval/tstar-gen00N-vs-*.csv` de chaque exécution `cmp2-*` (les fichiers du point de contrôle final lorsque le point de contrôle au seuil est le dernier) et `wallclock.json`	`H9-2026-10-27-analysis-01`
Seconde étude : contraste entre bras, les deux lectures ; table étude 1 contre étude 2	`results/study2/results-arm-difference.md` ; `results/study2/study1-vs-study2.md`	les deux fichiers CSV ci-dessus et `results/comparison/results-*.csv`	`H9-2026-10-27-analysis-01`
Seconde étude : seuil à temps égal T*′ = 17.6263 h et carte des points de contrôle	nombres dans le journal ; carte re-dérivée par `make_results.py --study 2`	`wallclock.json` de `cmp2-grid-s11..15` (médiane des cinq totaux), puis de chaque exécution `cmp2-*`	`H9-2026-10-25-tstar-01`
Seconde étude : table des coûts et temps mural par exécution	`paper/figures/fig2-score-cost-study2.md` ; `results/study2/wallclock-per-run.md`	`wallclock.json` par exécution `cmp2-*` ; `results/study2/results-*.csv`	`H9-2026-10-27-analysis-01` ; `H9-2026-10-13-campaign-01`
Seconde étude : score en fonction du temps mural, trajectoires par adversaire, troncature par exécution, métriques d'entraînement	`paper/figures/fig{1,5,7}-*-study2.png`, `fig6-training-metrics-study2.png` (`make_figures.py --study 2`)	`eval/*.csv` et `wallclock.json` par exécution `cmp2-*` ; `data/runs/campaign2.log`	`H9-2026-10-27-analysis-01` (métriques d'entraînement tabulées dans son addendum)
Seconde étude : durées des phases du pilote ; registre d'exécution des dix exécutions	nombres dans les journaux	`data/runs/pilot2-{grid,graph}-s10/` ; `data/runs/cmp2-*/wallclock.json`, `data/runs/campaign2.log`	`H9-2026-10-12-pilot-01` ; `H9-2026-10-13-campaign-01`

Tableau 77 : Provenance des preuves des chapitres de méthode (validation du moteur, lignes de base, pipeline, encodeurs, conception de la campagne) : les artefacts et l'entrée de journal consignant chaque mesure.

Preuve dans les chapitres de méthode	**Où elle réside**	**Entrée de journal (date)**
Tables de perft (8 types de partie), conformité UHP 21/21, fuzzing différentiel contre deux moteurs de référence (27 829 positions à la graine 20260909), vérification croisée de l'encodeur de plans (240 positions)	suites de tests sous `crates/`, `scripts/nightly.sh`, `scripts/crosscheck_planes.py`	`H2-2026-09-09-suite-rerun-01` (2026-09-09)
30 positions critiques annotées à la main, 30/30 après une correction côté mise en place	`tests/critical_positions/cases/*.toml` ; `scripts/run_critical_corpus.py`	`H2-2026-09-09-corpus-run-01` (2026-09-09)
Zéro violation d'invariant sur 10.9M transitions appliquées	`crates/hive-core/tests/random_invariants.rs`	`H2-2026-09-09-random-invariants-01` (2026-09-09)
Profil de débit : aller-retour UHP de 21.7 µs ; 2.62 ms (CoreML) contre 23.5 ms (CPU) par évaluation ; distribution des longueurs de partie derrière le plafond de 300 plis	commandes consignées en ligne dans l'entrée	`H2-2026-09-09-throughput-profile-01` (2026-09-09)
Inférence CoreML restaurée dans le pipeline Rust (format MLProgram)	`crates/hive-mcts/src/ort_eval.rs`	`H4-2026-09-09-coreml-fix-01` (2026-09-09)
Définitions des lignes de base, round-robin de caractérisation, cinq cas tactiques, épinglage des poids	`configs/baselines/*.toml`, `docs/baselines.md`, `tests/tactical_positions/`	`H3-2026-09-09-baselines-01` (2026-09-09)
Pilote du pipeline et les sept vérifications automatisées pré-entraînement	`scripts/run_pilot.py`, `scripts/run_h4_checks.sh`, `data/runs/pilot0/`	`H4-2026-09-10-pilot-01` (2026-09-10)
Encodeurs grille et graphe, appariement de capacité (1.44M vs 1.47M), asymétries de coût, batterie de propriétés	`docs/representations/{grid,graph,comparison-controls}.md`, `python/hivenet/{graph_dataset,graph_model}.py`	`H5-2026-09-10-encoders-01` (2026-09-10)
Miroir Rust de l'encodeur graphe, vérification croisée dorée (160 positions), bras graphe de bout en bout à travers la même recherche	`scripts/crosscheck_graph.py`, module graphe de `hive-nn`	`H5-2026-09-10-graph-wiring-01` (2026-09-10)
Matrice de comparaison, génération des ouvertures, test du harnais à calendrier fixe, dimensionnement de la campagne	`configs/comparison-matrix.yaml`, `results/comparison/openings-v1.txt`, `results/comparison/opponents-manifest.md`	`H6-2026-09-10-matrix-01` (2026-09-10)
Vérifications d'assemblage du rapport et le défaut d'obsolescence trouvé par la passe de rendu	`paper/final-control.md`	`H8-2026-10-09-detailed-edition-01` (2026-10-09)
Seconde étude : options du pilote de campagne pour le démarrage à chaud et la fenêtre de rejeu, test de non-régression contre les manifestes enregistrés de la première étude (4 exécutions × 10 générations)	`scripts/run_comparison.py`, `scripts/test_driver_equivalence.py`	`H9-2026-10-09-driver-01` (2026-10-09)
Seconde étude : pilote (2 générations × 200 parties par bras), batterie de vérifications sur shards fenêtrés, vérification d'estampille généralisée à plusieurs manifestes	`scripts/pilot_h9.sh`, `scripts/run_h4_checks.sh`, `python/hivenet/checks.py`, `data/runs/pilot2-h4-checks.log`	`H9-2026-10-12-pilot-01` (2026-10-12)
Seconde étude : protocole v2 gelé, script de campagne, pilote des évaluations au seuil, commutateur d'analyse	`docs/protocol-v2.md`, `scripts/campaign2.sh`, `scripts/tstar_evals_study2.py`, `scripts/make_results.py --study 2`	`H9-2026-10-13-campaign-01`, `H9-2026-10-25-tstar-01`, `H9-2026-10-27-analysis-01`

E.3 Index des décisions

Tableau 78 liste les entrées du journal des décisions pertinentes pour cette étude avec leurs dates ; les entrées D-002, D-013, D-018 et D-040 concernent les autres projets du programme de recherche et sont omises. Les franchissements de porte sont marqués de leur porte.

Tableau 78 : Index des entrées du journal des décisions pertinentes pour cette étude, dans l'ordre de leur consignation.

Entrée	Date	Décision
D-001	2026-09-09	Règles de fonctionnement orientées objectif adoptées : fichier de routage, un document exécutable par phase, portes réservées à l'humain
D-003	2026-09-09	La boucle d'auto-jeu antérieure de 19 générations classée comme démonstration, non comme preuve ; son point de contrôle préservé
D-004	2026-09-09	Ordre protocole/validation fixé (rédiger, puis mesurer, puis geler) ; règles « rédiger au fil de l'eau » et « aucun argent réel » ajoutées ; les portes parquent une phase, non la session
D-005	2026-09-09	G-PUBLIC / G-SPEND : licence MIT ; boucle antérieure arrêtée définitivement ; pistes séquentielles
D-006	2026-09-09	Dépôts distants privés ; méthodologie documentée ; copie française intégrale de chaque rapport
D-007	2026-09-09	Variante de l'étude : jeu de base seulement
D-008	2026-09-09	Convention de troncature : plafond ≠ nulle, rapport séparé, procédure de sensibilité
D-009	2026-09-09	Positionnement de la nouveauté : pas de pivot ; affirmation de contribution circonscrite par rapport aux trois travaux les plus proches
D-010	2026-09-09	Liaison Python–Rust par sous-processus/UHP (aller-retour de 21.7 µs), pas de liaison en processus
D-011	2026-09-09	Propositions mesurées : 128/32 simulations, plafond de 300 plis, deux correctifs de suivi
D-012	2026-09-09	Revue G-FREEZE : gel du protocole différé jusqu'après les pilotes
D-014	2026-09-09	Définitions et budgets des lignes de base (aléatoire légal, heuristique à profondeur 1, recherche à 6 400 simulations)
D-015	2026-09-09	Décodeur d'actions partagé : le contrat (emplacement de pièce, destination) réutilisé par les deux bras
D-016	2026-09-09	Proposition d'exclure le point de contrôle antérieur de la population gelée
D-017	2026-09-09	**G-FREEZE** : population d'adversaires gelée, adversaire de recherche à 6 400 simulations, point de contrôle antérieur exclu ; hachages et commit moteur b94e7c1
D-019	2026-09-09	Réglages d'évaluation épinglés : 400 simulations, aucun bruit d'exploration, argmax déterministe
D-020	2026-09-10	**G-FREEZE** : protocole gelé en v1.0 sur les valeurs mesurées du pilote ; sha256 f340a6b6…aefeb5, commit 44a74ff
D-021	2026-09-10	Représentation grille réutilisée telle quelle ; vérification des bornes du cadre transformée en assertion toujours active
D-022	2026-09-10	Bras graphe : graphe de cellules, tenseurs à capacité fixe (plafond de 224 nœuds), Python d'abord avec un miroir Rust
D-023	2026-09-10	Augmentation par symétries exclue de la méthode complète, pour les deux bras
D-024	2026-09-10	**G-DESTRUCTIVE** (par l'auteur directement) : archive de l'espace de travail antérieur à l'étude supprimée ; la copie de recherche est la seule copie du travail antérieur
D-025	2026-09-10	**G-FREEZE** : 250 ouvertures partagées gelées, générées à l'aveugle ; sha256 du contenu 63b318d0…5af7b

Entrée	Date	Décision
D-026	2026-09-10	**G-SPEND** : campagne principale approuvée à pleine taille et lancée
D-027	2026-09-16	Volume d'évaluation maintenu à 100 parties par appariement ; la variance entre graines domine le bruit des parties
D-028	2026-09-19	Ablation (a) substituée par A2 (biais de pooling global) ; A1 adjacence naïve conservée
D-029	2026-09-20	**G-SPEND** : campagne d'ablation approuvée, les deux ablations à 3 graines chacune
D-030	2026-09-26	**G-SPEND** : supplément A1′ à deux composants approuvé à son dimensionnement corrigé
D-031	2026-09-26	**G-SPEND** : extension à 5 graines par bras approuvée avec trois pré-engagements énoncés avant toute nouvelle exécution
D-032	2026-10-09	Portée du rapport : édition détaillée, aucun artefact gelé, nombre ou affirmation modifié
D-033	2026-10-09	Forme du rapport : rapport académique autonome expliquant quoi, comment, quand et pourquoi, sans référence à des fichiers internes dans le corps
D-034	2026-10-09	Droits : les moteurs de référence tiers (Mzinga, nokamute) sont sous licence MIT ; résultats de validation publiables ; G-RIGHTS close
D-035	2026-10-09	Plan de diffusion (publication du dépôt, prépublication, billet public), chaque étape étant une décision G-PUBLIC distincte ; étude complémentaire mise en file d'attente avec un projet de protocole
D-036	2026-10-09	**G-PUBLIC** : dépôt rendu public à l'étiquette v2.0-report avec le rapport, l'article court et l'archive des enregistrements ; prépublication et billet approuvés
D-037	2026-10-12	**G-SPEND** : pilote de la seconde étude (2 générations × 200 parties par bras, démarrage à chaud et fenêtre de trois générations, graine hors des ensembles des deux études)
D-038	2026-10-13	**G-FREEZE** : protocole v2 gelé (option A, fenêtre 3, graines 11–15, règle du seuil) ; sha256 e296a12e…b461eb, commit 4a22a24
D-039	2026-10-13	**G-SPEND** : seconde campagne lancée (2 bras × 5 graines × 10 générations × 500 parties, évaluations au seuil comprises)

E.4 Registre des affirmations et des preuves

Tableau 79 reproduit `paper/claims.md` sans sa colonne de section ; la formulation des affirmations est condensée là où c'est nécessaire, les nombres sont inchangés. La règle est : pas de ligne, pas d'affirmation.

Tableau 79 : Le registre des affirmations et des preuves de ce rapport, reproduit depuis le fichier du dépôt sans sa colonne de section. Chaque affirmation faite dans le corps correspond à une ligne ; la colonne des limites énonce ce qui empêche une lecture plus large.

Affirmation	Preuve	Limite
Le moteur de règles reproduit les tables de perft publiées de Mzinga pour les 8 types de partie jusqu'à la profondeur 6 (profondeur ≤5 dans la suite standard, profondeur 7 dans les exécutions nocturnes).	`H2-2026-09-09-suite-rerun-01`	égalité des comptes de nœuds bornée en profondeur ; la ré-exécution d7 n'a pas été répétée le 2026-09-09 (d≤6 confirmé)
Le moteur passe le harnais de conformité UHP de la référence nokamute (21/21).	`H2-2026-09-09-suite-rerun-01`	conformité = comportement du protocole, non preuve complète des règles
Les ensembles de coups légaux par pli sont identiques à ceux des deux moteurs de ré-	`H2-2026-09-09-suite-rerun-01`	accord avec les références, non directement avec la feuille de règles ; couverture par marche aléatoire

Affirmation	Preuve	Limite
férence (MzingaEngine v0.16.0, nokamute 1.0.3) sur des parties aléatoires à graine fixée (27 829 positions à la graine 20260909 ; 200/100 parties/type en nocturne).		
Le moteur concorde avec un corpus de 30 cas de positions critiques annotées à la main dont les attentes ont été validées (commit) avant toute exécution du moteur (30/30 après une correction côté mise en place).	`H2-2026-09-09-corpus-run-01` ; `tests/critical_positions/`	annotations du corpus non revues à l'externe (limite déclarée) ; 30 cas, dans le bas de la fourchette proposée de 30–50
Des sessions de parties aléatoires à graine fixée sur les 8 types de partie montrent zéro violation d'invariant sur 10.9M transitions appliquées.	`H2-2026-09-09-random-invariants-01`	largeur pseudo-aléatoire, non profondeur adversariale ; sérialisation via UHP GameString seulement
Les encodeurs de plans Rust et Python concordent exactement (240 positions, plans identiques à l'octet près).	`H2-2026-09-09-suite-rerun-01`	encodeur du bras grille seulement à cette date
Un aller-retour UHP par sous-processus coûte ~22 µs (négligeable face aux coûts par décision), de sorte que la liaison Python↔Rust utilise sous-processus/UHP (pas de PyO3).	`H2-2026-09-09-throughput-profile-01` ; D-010	une seule machine (M1 Pro) ; à revoir si Python entre un jour dans une boucle par coup
Aux coûts d'inférence mesurés (2.62 ms/eval CoreML, 23.5 ms CPU, réseau gen-19 comme charge de travail), 128/32 simulations avec randomisation du plafond de simulations et un plafond de 300 plis sont réalisables dans l'enveloppe de calcul de cette étude.	`H2-2026-09-09-throughput-profile-01` ; D-011	historique : propositions confirmées ensuite par le pilote et gelées (D-020) ; les coûts sont propres au réseau
Inférence CoreML restaurée dans le pipeline Rust (correctif MLProgram) : l'auto-jeu à 128/32 simulations coûte ≈12 thread-s/partie (≈3 s/partie en temps mural à 4 threads), contre ≈320 thread-s/partie sur le repli CPU à 600/150.	`H4-2026-09-09-coreml-fix-01`	réseau gen-19 comme charge de travail ; petits nombres de sondes ; re-mesuré au pilote
La ligne de base heuristique bat l'aléatoire légal 100–0 (100 parties appariées, 0 troncature) ; le MCTS sans réseau à 6400 simulations obtient 99.5% contre l'aléatoire et 37.5% contre l'heuristique (−89 Elo [−150, −32]) ; une sonde à budget 4× atteint 56.2%.	`H3-2026-09-09-baselines-01`	volume de pilote (100 parties/appariement) ; IC d'arène en approximation non appariée ; constat d'ordre diagnostiqué, non réajusté ; population gelée depuis (D-017)
La ligne de base MCTS résout les 5 cas tactiques annotés à la main à 400, 1600 et 6400 simulations, et les signes de la	`H3-2026-09-09-baselines-01` ; `tests/tactical_positions/` ; tests de signe dans `crates/*/tests`	5 cas, non revus à l'externe (limite déclarée)

Affirmation	Preuve	Limite
valeur sont épinglés sous alternance des joueurs.		
Les sept vérifications pré-entraînement passent comme tests automatisés sur des shards réels d'auto-jeu : masse de politique illégale nulle, identité id↔coup, perspective d'issue correcte avec troncature distincte, sur-apprentissage d'un petit lot (KL 0.09, argmax 15/15, valeur 15/15), sauvegarde/reprise au bit près, bruit d'évaluation structurellement désactivé, séparation évaluation/entraînement par audit.	`H4-2026-09-10-pilot-01` ; `scripts/run_h4_checks.sh`	le critère de sur-apprentissage est la KL rapportée au plancher d'entropie des cibles douces, non perte→0
Une génération d'auto-jeu depuis une initialisation aléatoire (300 parties, 128/32 simulations, réseau grille de 1.44M paramètres) produit un réseau qui bat l'aléatoire légal 100–0 (28 victoires, 2 troncatures) tout en obtenant 3.3% contre l'heuristique et 3.3% contre le MCTS à 6400 simulations : apprentissage non dégénéré avec un diagnostic d'écart de données et d'itérations.	`H4-2026-09-10-pilot-01` ; `data/runs/pilot0/eval/`	une seule graine, 30 parties/adversaire, génération 0 seulement ; pas un résultat de l'étude
L'auto-jeu de la génération 0 tronque 56.7% des parties au plafond de 300 plis (170/300) ; le coût de génération est ≈12 s/partie en temps mural (4 threads) à la génération 0, retombant vers ≈3 s/partie avec un réseau entraîné au même budget.	`H4-2026-09-10-pilot-01` ; `data/runs/pilot0/selfplay/gen000-manifest.json`	une machine, une graine ; taux propres au jeu à initialisation aléatoire
Les deux bras sont à capacité appariée à +1.5% (grille 1.44M, graphe 1.47M paramètres) derrière le décodeur identique, et l'encodeur graphe ne perd aucune information d'état par rapport aux enregistrements générés par le moteur (batterie de propriétés P1–P5, 300 positions réelles, masse de politique illégale nulle de bout en bout).	`H5-2026-09-10-encoders-01` ; `docs/representations/`	historique : le miroir Rust et la vérification croisée dorée ont été livrés au vert le même jour (`H5-2026-09-10-graph-wiring-01`)
Les coûts d'inférence au meilleur fournisseur disponible diffèrent de ≈1.4× au détriment du bras graphe (grille 2.62 ms/eval CoreML contre graphe 3.67 ms ORT-CPU), tandis que le débit d'entraînement sur CPU favorise le bras graphe 3.5× et que MPS favorise le bras grille 2×.	`H5-2026-09-10-encoders-01` ; table dans `comparison-controls.md`	une seule machine ; rapporté, non égalisé ; alimente la lecture à temps mural égal

Affirmation	Preuve	Limite
Les encodeurs graphe Rust et Python concordent exactement (160 positions sur les 8 types de partie, tenseurs identiques à l'octet près, épinglés chaque nuit), et le bras graphe s'exécute de bout en bout à travers le MCTS Rust identique.	`H5-2026-09-10-graph-wiring-01` ; `scripts/crosscheck_graph.py`	entraînement à l'échelle d'un test de fumée ; les résultats de force relèvent de la comparaison
H1 est rejetée sous la règle pré-enregistrée : sous les deux lectures de budget, le bras graphe ne montre aucun avantage cohérent entre graines contre la population gelée, et chaque intervalle graphe−grille exclut un avantage graphe substantiel (plus grande borne supérieure +0.035).	`H6-2026-09-19-comparison-01` ; `results/comparison/results-arm-difference.md`	3 graines/bras ; une seule architecture graphe à une seule capacité et un seul budget ; auto-jeu en régime précoce (10 générations)
Contraste entre bras (moyenne par graine, bootstrap95 sur les graines) : vs B-RND −0.175 [−0.268, −0.007] (exemples égaux) et −0.158 [−0.254, −0.050] (temps mural égal) ; vs B-HEU −0.085 [−0.143, −0.025] et −0.072 [−0.100, −0.028] ; vs B-MCTS +0.005 [−0.020, +0.035] et −0.013 [−0.040, +0.015].	`results/comparison/results-*.csv` ; `H6-2026-09-19-comparison-01`	intervalles sur 3 graines indépendantes par bras
Le bras graphe tronque 20–57% de ses parties contre l'aléatoire légal au plafond de 300 plis (grille 0–1%) ; le rejet est robuste au plafond : compter toutes les troncatures comme des victoires du graphe laisse graphe−grille à −0.072/−0.058 contre B-RND.	`H6-2026-09-19-comparison-01` (bornes de sensibilité au plafond) ; fig4-F1	l'argument de borne se substitue à une ré-exécution à plafond plus grand (énoncé)
Coûts de campagne mesurés : les exécutions graphe ont pris en moyenne 2.0× le temps mural d'entraînement de la grille (36.7 vs 18.0 h par exécution de 10×500 parties, moyenne sur 5 graines, auto-jeu + entraînement, évaluation exclue ; les chiffres à 3 graines étaient 35.7 vs 18.2 h) ; à T* = 18.77 h le bras graphe achève 3–6 des 10 générations (4–5 sur les trois graines originales).	`data/runs/cmp-*/wallclock.json` ; `results/comparison/wallclock-per-run.md` ; fig2	une seule machine ; meilleur fournisseur disponible par bras (rapporté) ; le défaut de diviseur du générateur de la table des coûts du 2026-10-09 n'a affecté que les moyennes précédemment imprimées (30.0/61.2 h), jamais ce rapport
Tous les artefacts de comparaison ont été gelés avant toute exécution de comparaison (population D-017, protocole D-020, ouvertures D-025), T* a été calculé à partir des temps muraux de la grille avant l'existence de tout	D-017/D-020/D-025/D-026 ; `H6-2026-09-16-progress-01`, `H6-2026-09-19-comparison-01`	aucune

Affirmation	Preuve	Limite
nombre inter-bras, et aucun artefact gelé n'a été touché.		
Ablation A1 (parité) : retirer le typage géométrique des arêtes détruit l'entraînabilité : divergence NaN à la génération 0 pour 3/3 graines ; les arêtes typées contribuent au minimum à la stabilité d'optimisation ; les tables d'évaluation de A1 sont des artefacts d'une politique NaN et sont exclues en tant que scores.	`H7-2026-09-23-a1-divergence-01` ; `results/ablations/README.md`	le mécanisme (échelle de gradient 6× sur la matrice partagée) est une hypothèse fondée, non une décomposition mesurée
Ablation A2 : retirer le biais de pooling global n'a aucun effet mesurable : nogpool−complet = −0.001 [−0.170, +0.165] (B-RND), −0.007 [−0.043, +0.030] (B-HEU), −0.013 [−0.043, +0.013] (B-MCTS) ; modes de défaillance inchangés.	`H7-2026-10-02-a2-nogpool-01` ; `results/ablations/README.md`	3 graines/cellule ; différence de 0.10M paramètres inhérente au composant (rapportée)
Analyse finale à 5 graines (pré-engagée, D-031) : H1 reste rejetée. Contrastes graphe−grille : B-RND −0.169 [−0.272, −0.062] / −0.161 [−0.278, −0.048] ; B-HEU −0.064 [−0.111, −0.017] / −0.059 [−0.087, −0.029] ; B-MCTS −0.009 [−0.055, +0.028] / −0.018 [−0.066, +0.025] (exemples égaux / temps mural égal) ; plus grande borne supérieure +0.028. Remplace les tables à 3 graines comme nombres finaux de l'étude.	`H6-2026-10-09-5seed-final-01` ; `results/comparison/`	graines 4–5 collectées après l'analyse à 3 graines (divulgué) ; T* fixé à la valeur pré-enregistrée
Supplément A1′ (à deux composants : arêtes non typées + écrêtage de gradient) : s'entraîne de façon finie et obtient des scores dans la bande du bras graphe complet (tous les IC de différence chevauchent 0) ; avec A1, la contribution mesurable des relations typées à cette échelle se concentre dans la stabilité d'optimisation ; elle n'est jamais attribuée au seul typage (facteur de confusion de l'écrêtage).	`H7-2026-10-09-a1prime-01` ; `results/ablations/README.md`	3 vs 5 graines ; intervalles larges ; confondu par construction
La méthodologie de travail (délégation à l'IA au niveau des objectifs sous six portes réservées à l'humain, état dans des fichiers, vérification mécanique) a attrapé au moins cinq défauts de harnais ou de processus avant qu'ils ne puissent contaminer les résultats (interblocage du pipe-	`docs/methodology-log.md` (espace de travail) ; journaux cités par incident dans Chapitre 10	observations de processus issues d'une seule étude ; aucun contrôle contrefactuel

Affirmation	Preuve	Limite
line, erreur de mise en place du corpus, défaut d'adversaire du harnais d'évaluation, chemin silencieux de perte d'enregistrements, divergence NaN silencieuse), chacun journalisé au moment de sa détection.		
Seconde étude (protocole v2, démarrage à chaud + fenêtre de rejeu 3, graines 11–15) : H1′ n'est pas étayée. Graphe–grille : B-RND −0.018 [−0.045, +0.008] / −0.052 [−0.140, +0.002] ; B-HEU −0.003 [−0.046, +0.039] / +0.013 [−0.032, +0.055] ; B-MCTS +0.003 [−0.082, +0.090] / −0.025 [−0.091, +0.034] (exemples égaux / temps mural égal à 17.63 h) ; aucun avantage graphe cohérent entre graines ; plus grandes bornes supérieures +0.090 et +0.055.	`H9-2026-10-27-analysis-01` ; `results/study2/results-arm-difference.md`	5 graines par bras ; intervalles plus larges que ceux de la première étude ; un avantage graphe inférieur à ≈ 0.09 contre B-MCTS non exclu à exemples égaux ; aucun seuil chiffré dans la règle gelée
Seconde étude contre première (descriptif, sans agrégation) : le contraste s'est rapproché de zéro contre chaque adversaire sous les deux lectures (B-RND −0.169 → −0.018 et −0.161 → −0.052 ; B-HEU −0.064 → −0.003 et −0.059 → +0.013 ; B-MCTS −0.009 → +0.003 et −0.018 → −0.025).	`results/study2/study1-vs-study2.md` ; `H9-2026-10-27-analysis-01`	ensembles de graines différents ; deux changements de régime à la fois ; aucun intervalle inter-études
Seconde étude : la pathologie de conversion persiste dans trois graines graphe sur cinq (24–42% de troncature contre B-RND aux points de contrôle finaux, 28–58% aux seuils) et est absente dans deux (0–4%) ; grille 1–7% ; avec les troncatures comptées 0.5, le contraste B-RND est de −0.101 [−0.187, −0.012] et −0.131 [−0.256, −0.015].	`results/study2/results-*.csv` ; `H9-2026-10-27-analysis-01`	troncature contre B-RND seulement ; plafond de 300 plis ; lecture de sensibilité, non le score primaire
Coût de la seconde étude : horloge d'entraînement grille 17.46–18.52 h (moyenne 17.80 h), graphe 36.27–52.85 h (moyenne 41.59 h), rapport 2.34× (première étude 2.04×) ; la fenêtre ajoute du temps d'entraînement en proportion du nombre d'enregistrements, 3–5% d'une exécution ; au seuil, le bras grille avait achevé 9–10 générations, le bras graphe 3–5.	`results/study2/wallclock-per-run.md` ; `H9-2026-10-13-campaign-01`, `H9-2026-10-12-pilot-01`, `H9-2026-10-25-tstar-01`	une seule machine ; temps mural sur les fournisseurs de cette machine

Affirmation	Preuve	Limite
Seconde étude : la force absolue contre les adversaires forts n'a pas monté sous le régime à démarrage à chaud en dix générations (grille vs B-HEU 0.077 contre 0.129 dans la première étude ; vs B-MCTS 0.131 contre 0.124) ; le score du bras graphe contre B-RND est monté de 0.812 à 0.971.	`results/study2/study1-vs-study2.md`	descriptif ; graines différentes ; réglages par défaut non ajustés dans les deux bras
La seconde étude a été pré-enregistrée : protocole v2 gelé le 13 octobre 2026 (sha256 e296a12e…b461eb) avant toute exécution ; son seuil calculé à partir des horloges grille le 25 octobre 2026 avant qu'aucun score n'ait été lu ; les cinq graines par bras analysées ; aucune relance ni exclusion.	`docs/protocol-v2.md` ; `H9-2026-10-25-tstar-01`, `H9-2026-10-13-campaign-01` ; D-038, D-039	hyperparamètres non ajustés par choix (option A), énoncé comme une limite

E.5 Données brutes, identifiants et hachages gelés

Emplacement et agencement. Les enregistrements bruts résident sur disque sous `data/runs/` dans le dépôt de l'étude ; ils ne sont pas sous contrôle de version. Il y a un répertoire par exécution d'entraînement, `cmp-<arm>-s<seed>`, avec `<arm>` dans {`grid`, `graph`} pour la comparaison principale (graines 1–5) et dans {`graph-untyped`, `graph-nogpool`, `graph-untyped-clip`} pour les ablations A1, A2 et A1′ (graines 1–3), plus `pilot0/` pour le pilote du pipeline. Chaque répertoire d'exécution contient : `selfplay/gen00N-*.bin` (shards binaires d'enregistrements de la génération N, format v3, estampillés par le modèle) avec `gen00N-manifest.json` (réseau et son empreinte, génération du modèle, version des enregistrements, type de partie, graine, nombre de parties, nombres de simulations complètes et économiques, fraction de coups complets, plis en température, réglages d'abandon, nombres de parties abandonnées et tronquées, nombre de positions, liste des shards) ; `checkpoints/gen00N/` (`hivenet-e0.pt`, `hivenet-e1.pt`, `train-config.json`) et le `gen00N-b1.onnx` exporté utilisé pour le jeu ; `eval/<checkpoint>-vs-<opponent>.csv` avec son `.log` pour les évaluations intermédiaires (gen004, gen007) et finale (gen009), et `eval/tstar-gen00N-vs-<opponent>.{csv,log}` pour le point de contrôle au seuil à temps égal ; et `wallclock.json`, le temps mural par génération en secondes dont dérivent chaque chiffre de temps et le seuil. Les journaux au niveau de la campagne sont `data/runs/campaign.log` (exécutions principales, graines 1–3), `extension.log` (graines 4–5), `ablations.log` et `tstar-evals.log`. Les exécutions de la seconde étude suivent le même agencement sous `cmp2-<arm>-s<seed>` (graines 11–15), chacune avec un `driver-config.json` consignant le régime (démarrage à chaud actif, fenêtre 3) ; son journal de campagne est `campaign2.log`, son journal d'évaluations au seuil `tstar2.log`, et les exécutions du pilote sont `pilot2-{grid,graph}-s10`. Dérivation des graines : graine de base = 100 000 × graine ; la génération g utilise graine de base + g ; graine du réseau d'évaluation 9000 + gen (finales) ou 9500 (ensembles au seuil), graines des adversaires 9101 (aléatoire légal) et 9201 (recherche).

Volumes. La campagne principale a compris 10 générations × 500 parties d'auto-jeu par exécution (30 000 parties d'auto-jeu par bras à trois graines au stade à 3 graines) et 100 parties appariées par évaluation (point de contrôle, adversaire) sur les ouvertures

gelées ; la campagne s'est déroulée du 2026-09-10 12:42 au 2026-09-17 22:31 pour les graines 1–3 (≈163 h de temps machine, évaluations au seuil comprises), les graines 4–5 ayant été collectées du 2026-09-27 au 2026-10-02. L'empreinte disque n'est consignée dans la base de preuves que sous la forme des estimations au lancement (≈3–5 Go pour la campagne principale, ≈3–4 Go pour les ablations) contre 164 Go libres au lancement avec une garde de 20 Go ; les tailles exactes par exécution n'ont pas été journalisées. La seconde campagne s'est déroulée du 2026-10-13 15:02 au 2026-10-26 23:00 (296.9 h de temps mural d'entraînement sur les dix exécutions), suivie d'environ 14 h d'évaluations au seuil les 26–27 octobre ; ses dix répertoires d'exécution totalisent 4.8 Go. Depuis la suppression consignée dans D-024, le dépôt de l'étude est la seule copie du matériel antérieur à l'étude dont il a hérité (le point de contrôle de la boucle antérieure et ses données, 412 Mo et 75 Mo), ce qui explique que ces fichiers soient protégés par la porte des actions destructrices.

Artefacts gelés et leurs hachages. Tableau 80 liste chaque artefact gelé ou épinglé avec l'identifiant consigné lors de son gel. Le hachage du protocole est consigné dans le journal des décisions sous forme abrégée ; la valeur complète a été recalculée à partir du commit gelé lors de la préparation de cette annexe et est identique au hachage du fichier actuel, ce qui confirme que le protocole n'a pas changé depuis son gel.

Tableau 80 : Artefacts gelés et épinglés de l'étude avec les identifiants consignés lors de leur gel (hachages, commits, graines) et l'entrée du journal des décisions ou du journal qui les consigne.

Artefact	Gelé / épinglé le	Identifiant	Enregistrement
Protocole v1.0 (`docs/protocol.md`)	2026-09-10	sha256 f340a6b6…aefeb5 au commit 44a74ff (complet : f340a6b64db0f5f0bf126ffb251c3de339450bde192fd54b719036a8a3aefeb5)	D-020
Adversaire B-RND (`configs/baselines/random.toml`)	2026-09-09	sha256 f2fc4a06441d3c1a7922838a6693dbb48ec54543bc34fd814d41c9a742514cd7	D-017
Adversaire B-HEU (`configs/baselines/heuristic.toml`)	2026-09-09	sha256 7210a0a349c5bad5dcd2df099cc6865ee3cb5d8a2c30e106ce58137804818999	D-017
Adversaire B-MCTS (`configs/baselines/mcts-nonet.toml`)	2026-09-09	sha256 3fc8f75cf2b4f21012dd61e9924408fbfc32c8ea561aa96eb44f1091ba07364e	D-017
Poids de l'heuristique (`configs/baselines/heuristic-weights.toml`)	2026-09-09	sha256 d0602f1895fbed70b6f84ac2a3eb87bd68e811e53acf24d4d7814a1495b0b97a ; épinglé par le test `weights_pinned_for_h3_baselines`	D-014, D-017
Code du moteur au gel de la population	2026-09-09	commit b94e7c1	D-017
Ouvertures partagées (`results/comparison/openings-v1.txt`, 250 lignes)	2026-09-10	sha256 du contenu 63b318d071dfc3ecfae3585636c8e6f7327ddc08e7aed86a466f915f8005af7b ; sha256 du fichier gelé	D-025 ; `H9-2026-10-27-analysis-01`

Artefact	Gelé / épinglé le	Identifiant	Enregistrement
		538497390a3787299c67c3ca138b8feacb369d55dc562881d1aee45200cbccb2 ; graine du générateur 20260910 ; hachages du calendrier 8cd84b6564440666 (ensembles de 20 parties) et 3bf5d3e51cea0c0d (ensembles de 100 parties, les deux études)	
Réglages d'évaluation (`configs/eval_settings.toml`)	2026-09-09	400 simulations, aucun bruit d'exploration, argmax déterministe, plafond de 300 plis	D-019
Matrice de comparaison (`configs/comparison_matrix.yaml`)	2026-09-10	tailles pré-enregistrées et règle du seuil	D-026
Seuil à temps mural égal T*	2026-09-16	18.77 h = médiane des totaux des trois exécutions grille originales (18.77, 16.73, 19.07 h)	`H6_2026_09_16_progress_01` ; D-031
Code de campagne et d'analyse	2026-09-10 → 2026-10-09	campagne ac58773 ; analyse 1c65269 (3 graines), 7db074d (5 graines) ; ablations 216fded → bffeea9 (garde contre les valeurs non finies) ; file d'attente A1′ 1fdd7ec	journaux `H6_…`, `H7_…`
Protocole v2.0 (`docs/protocol_v2.md`), seconde étude	2026-10-13	sha256 e296a12e…b461eb au commit 4a22a24 (complet : e296a12e8d1e3dee36c3b210ececced62314e64657aede1cb91b1ea452b461eb)	D-038
Seuil à temps mural égal T*′, seconde étude	2026-10-25	17.6263 h = médiane des totaux des cinq exécutions grille (17.4567, 17.9540, 17.6263, 17.4592, 18.5176 h)	`H9_2026_10_25_tstar_01`
Code de la seconde étude	2026-10-09 → 2026-10-27	options du pilote de campagne 008aaee ; batterie de vérifications 99be4ef ; script de campagne dc0b6c2 ; commutateur d'analyse b0f1598 ; figures 32297fe	journaux `H9_…`

F Glossaire

Les tables ci-dessous fixent les termes français utilisés dans cette édition, leur sens dans le périmètre de l'étude, et le terme anglais correspondant du master anglais. La colonne française reproduit la formulation établie dans les chapitres français ; un astérisque marque une forme composée, construite à partir d'éléments établis, que l'édition française utilise ici pour la première fois.

Tableau 81 : Termes du plan d'étude.

Terme (français)	**Définition**	**Équivalent anglais**
bras grille / bras graphe	Les deux systèmes comparés, identiques à l'exception de l'encodage d'état et du corps de réseau qui le lit.	grid arm / graph arm
encodage d'état*	La fonction qui transforme une position en les tenseurs qu'un réseau lit ; épinglée à l'octet près entre le moteur et le code d'entraînement.	state encoding
décodeur d'actions partagé	Le contrat unique par lequel les deux bras émettent une politique sur les mêmes actions, à savoir (emplacement de pièce, destination) plus la passe, avec un masquage de l'ensemble légal et une normalisation identiques.	shared action decoder
population d'adversaires	Les trois adversaires fixes contre lesquels chaque point de contrôle est noté, gelés le 9 septembre 2026.	opponent population
aléatoire légal	Tirage uniforme parmi les coups légaux ; le plancher de la population.	B-RND, legal-random
heuristique documentée à poids fixes	Argmax glouton à un demi-coup d'une évaluation documentée à poids fixes, avec quiescence ciblant la reine.	B-HEU, heuristic
MCTS sans réseau à 6400 simulations	Recherche arborescente PUCT sans réseau, a priori uniformes, l'évaluation construite à la main comme valeur aux feuilles.	B-MCTS, search at 6 400 simulations
gelé / gelée	Fixé par une approbation datée et jamais retouché ensuite ; toute retouche ultérieure constituerait une nouvelle étude.	frozen
règle de rejet pré-enregistrée*	La condition inscrite dans le protocole gelé sous laquelle l'hypothèse est rejetée : aucun avantage graphe cohérent entre graines sous les deux lectures, avec des intervalles excluant un avantage substantiel.	pre-registered rejection rule
graine (exécution d'entraînement indépendante)	Une exécution d'entraînement complète à partir d'une initialisation à graine fixée ; l'unité de réplication et de rééchantillonnage.	seed (independent training run)
génération	Un cycle d'auto-jeu, d'entraînement et d'export de point de contrôle ; dix par exécution.	generation
point de contrôle	Le réseau exporté à la fin d'une génération ; identifiants indexés à partir de zéro (gen000 à gen009).	checkpoint
parties appariées, à couleurs échangées	Chaque ligne d'ouverture jouée une fois avec chaque couleur, de sorte que tous les bras, graines et adversaires font face à des calendriers identiques.	paired colour-swapped games
réglages d'évaluation épinglés	400 simulations par décision, aucun bruit d'exploration, argmax déterministe ; fixés par fichier et par test.	pinned evaluation settings

Terme (français)	Définition	Équivalent anglais
démarrage à chaud	Initialisation du réseau de la génération g depuis les poids du point de contrôle final de la génération $g-1$, l'optimiseur et le calendrier redémarrant ; utilisé dans la seconde étude seulement, la première réinitialisant chaque génération.	warm start
fenêtre de rejeu	Les générations récentes dont les enregistrements servent à l'entraînement d'une génération ; 3 générations dans la seconde étude, la génération courante seule dans la première.	replay window
seuil à temps mural égal de la seconde étude*	La médiane du temps mural total d'entraînement des cinq exécutions grille de la seconde étude, calculée une seule fois depuis les fichiers d'horloge avant tout nombre inter-bras ; une valeur propre, distincte des 18.77 h de la première étude.	equal-time cutoff of the second study
seconde étude	La comparaison pré-enregistrée séparément sous démarrage à chaud et fenêtre de rejeu (protocole version 2.0, gelé le 13 octobre 2026, graines 11 à 15) ; rapportée à côté de la première étude, jamais agrégée avec elle.	second study

Tableau 82 : Termes de Hive.

Terme (français)	Définition	Équivalent anglais
One-Hive (conservé en anglais)	Aucun coup ne peut scinder la ruche, y compris en transit ; une pièce dont le retrait la scinderait est clouée.	One-Hive rule
liberté de mouvement	Une pièce qui glisse ne peut passer que par un interstice dont les deux cellules adjacentes ne sont pas toutes deux occupées.	freedom to move
porte (porte du scarabée)	Deux cellules occupées encadrant un pas, bloquant un glissement ; au-dessus du niveau du sol, la porte du scarabée.	gate
empilement	Les scarabées grimpent sur les pièces, formant des piles ; seule la pièce du dessus agit.	stacking
étourdissement	Le marqueur porté par une pièce que vient de lancer un Pillbug, et qui ne peut pas bouger au pli suivant ; inerte dans le jeu de base.	stun
passe forcée	Sans coup légal, un camp doit passer ; la passe est légale exactement lorsqu'aucun coup n'existe.	forced pass
pli / demi-coup	Un coup d'un camp (demi-coup) ; le plafond est de 300 plis.	ply
ouverture	Un préfixe de quatre plis joué avant que les agents ne prennent la main ; 250 lignes gelées partagées par chaque affrontement.	opening (frozen line)

Tableau 83 : Termes de recherche et d'auto-jeu.

Terme (français)	Définition	Équivalent anglais
auto-jeu	Les parties que le réseau courant joue contre lui-même, à travers la recherche, pour produire des enregistrements d'entraînement.	self-play
recherche arborescente Monte-Carlo (MCTS)	La recherche des deux bras et de B-MCTS ; le réseau fournit les a priori et une valeur aux feuilles.	Monte-Carlo tree search (MCTS)
PUCT	La règle de sélection : maximiser Q + c·prior·$\sqrt{N}$/(1+n), c = 1.4.	PUCT
politique	La distribution du réseau sur les actions légales, entraînée vers la distribution de visites enregistrée.	policy

Terme (français)	Définition	Équivalent anglais
valeur	L'estimation par le réseau, dans [−1, 1], de l'issue pour le camp au trait ; une tête à trois classes, enregistrements tronqués exclus.	value
distribution de visites	Les comptes de visites à la racine, normalisés, après une recherche ; la cible de politique.	visit distribution
randomisation du plafond de simulations	Une fraction 0.25 des décisions d'auto-jeu reçoit 128 simulations et est enregistrée ; le reste en reçoit 32 et ne l'est pas.	playout-cap randomization
plis en température	Les 12 premiers plis d'une partie d'auto-jeu, échantillonnés plutôt que pris par argmax.	temperature plies
abandon	Une partie d'auto-jeu est abandonnée lorsque la valeur tombe sous −0.92.	resignation
audit sans abandon	Une fraction de 10% des parties d'auto-jeu, jouées jusqu'au bout quoi qu'il arrive, pour vérifier que l'abandon ne cache aucune victoire.	resignation audit
argmax de politique (top-1)*	Métrique d'ajustement d'entraînement : part des enregistrements dont l'argmax de la politique coïncide avec l'argmax de la cible.	policy top-1
exactitude de la valeur*	Métrique d'ajustement d'entraînement : part des enregistrements dont la classe d'issue prédite est la classe enregistrée.	value accuracy

Tableau 84 : Termes de représentation et de réseau.

Terme (français)	Définition	Équivalent anglais
cadre 32×32 ; plans de caractéristiques	La grille fixe dans laquelle le bras grille plonge une position, centrée sur la boîte englobante ; 77 plans de caractéristiques.	frame (32×32)
passage de messages	Le calcul du bras graphe : chaque nœud se met à jour à partir de ses voisins, couche par couche.	message passing
relations d'arêtes typées par direction	Les six directions hexagonales comme types d'arêtes, chacune avec ses propres poids.	direction-typed relations
biais de pooling global (biais d'agrégation globale)	Un résumé agrégé de tous les nœuds, réinjecté dans chaque nœud ; retiré dans l'ablation A2.	global-pooling bias
à capacité appariée	Des nombres de paramètres quasi égaux : 1.44 M (grille) et 1.47 M (graphe), +1.5%.	capacity-matched
adjacence naïve	Ablation A1 : les six matrices d'arêtes typées remplacées par une seule matrice partagée.	naive adjacency
écrêtage de gradient	L'écrêtage de la norme à 1.0 ajouté dans le supplément A1′, qui en fait une variante à deux composants.	gradient clipping

Tableau 85 : Termes d'issue et de statistique.

Terme (français)	Définition	Équivalent anglais
troncature (partie tronquée)	Une partie arrêtée au plafond de 300 plis ; une quatrième issue, jamais repliée dans les nulles.	truncation (never « draw »)
plafond de coups (plafond de 300 demi-coups)	La limite de 300 plis à laquelle une partie est tronquée.	move cap
score	Moyenne de victoire 1 / nulle 0.5 / défaite 0 sur les parties non tronquées d'une cellule (graine, adversaire).	score
taux de troncature*	Part des 100 parties d'une cellule qui ont été tronquées ; toujours rapporté à côté du score.	truncation rate

Terme (français)	Définition	Équivalent anglais
colonne de sensibilité « troncatures à 0.5 »	La moyenne de la cellule sur toutes les parties, les troncatures comptant 0.5.	sensitivity score
bornes de traitement du plafond	Le contraste recalculé avec les troncatures comptées comme défaites et comme victoires, encadrant tout autre plafond.	cap-treatment bounds
moyenne au niveau des graines	La moyenne simple des scores de cellule par graine d'un bras contre un adversaire.	seed-level mean
intervalle bootstrap sur les graines	Un intervalle bootstrap percentile à 95% obtenu en rééchantillonnant les graines (jamais les parties) 10 000 fois.	seed-level bootstrap interval
bootstrap percentile	L'intervalle dont les extrémités sont les quantiles 0.025 et 0.975 des statistiques de rééchantillonnage triées ; sans correction de biais.	percentile interval
contraste entre bras	La différence graphe moins grille des moyennes au niveau des graines, avec son intervalle issu d'ensembles de graines rééchantillonnés indépendamment.	arm contrast

Tableau 86 : Termes de budget.

Terme (français)	Définition	Équivalent anglais
budget (temps mural, matériel, états d'entraînement, simulations)	Les ressources d'une exécution en quatre dénominations : temps mural, matériel, états d'entraînement, simulations.	budget
temps mural	Le temps réel écoulé sur la machine d'étude, journalisé par génération ; le temps mural d'entraînement d'une exécution exclut ses parties d'évaluation.	wall-clock
lecture à exemples égaux	La comparaison à budget d'auto-jeu égal : les points de contrôle de dixième génération des deux bras après 10 générations × 500 parties.	same-examples reading
lecture à temps mural égal	La comparaison à temps d'entraînement égal : le dernier point de contrôle de chaque exécution dans le seuil à temps mural égal.	same-wall-clock reading
seuil à temps mural égal	18.77 h, la médiane des temps muraux d'exécution complète des trois exécutions originales du bras grille, fixée par une règle pré-enregistrée le 16 septembre 2026.	equal-time cutoff
simulations par décision	Le budget de recherche : 128 complètes / 32 économiques en auto-jeu, 400 en évaluation, 6 400 pour B-MCTS.	simulations per decision

G Guide de reproduction

C'est le seul endroit du rapport où commandes, noms de fichiers et identifiants apparaissent tels quels, parce que la reproduction de l'étude les exige : ce qui est publié, le scénario minimal exécuté dans un environnement vierge le 9 octobre 2026, l'ensemble complet des commandes, les durées à prévoir et les identifiants qu'une reproduction doit retrouver.

G.1 Ce qui est publié

L'ensemble publié (release) est le dépôt `hive-graph-selfplay` et les enregistrements qu'il contient. La première étude a été publiée le 9 octobre 2026 à l'étiquette `v2.0-report` du dépôt public https://github.com/mohamedbechirkefi/hive-graph-selfplay, avec le rapport dans les deux langues, l'article court et une archive des enregistrements jointe à la publication ; les enregistrements de la seconde étude et la présente révision accompagnent la publication étiquetée `v2.1-report`. Les moteurs de référence tiers utilisés uniquement pour la validation des règles sont sous licence MIT et ne sont pas redistribués.

- **Code.** Le moteur Rust (noyau de règles, serveur de protocole, recherche, arène, travailleurs d'auto-jeu), le code Python d'entraînement et d'export sous `python/hivenet/`, les scripts sous `scripts/`.
- **Configurations gelées.** `configs/comparison-matrix.yaml`, `configs/eval-settings.toml`, les configurations des adversaires et le fichier de poids de l'heuristique sous `configs/baselines/`, les spécifications d'ablation sous `configs/ablations/`.
- **Ouvertures gelées et manifeste de la population.** `results/comparison/openings-v1.txt` (250 lignes de quatre plis) et `opponents-manifest.md`, qui consigne les hachages du Tableau 88.
- **Tables de résultats.** `results/comparison/results-same-examples.{md,csv}`, `results-same-wallclock.{md,csv}`, `results-arm-difference.md`, `wallclock-per-run.md` ; `results/ablations/README.md` ; pour la seconde étude, les quatre mêmes tables plus `study1-vs-study2.md` sous `results/study2/`.
- **Enregistrements par exécution**, sous `data/runs/cmp-<arm>-s<seed>/` (première étude, graines 1–5) et `data/runs/cmp2-<arm>-s<seed>/` (seconde étude, graines 11–15, chacune avec un `driver-config.json` consignant le régime) : `eval/` (un fichier à valeurs séparées par des virgules par point de contrôle et par adversaire, une ligne par partie : `opening_id, a_is_white, score_a, truncated, plies, outcome`, métadonnées dans les lignes d'en-tête `#`) ; `wallclock.json` (secondes par génération) ; les manifestes `selfplay/` liant chaque shard à son réseau générateur, et les shards ; `checkpoints/gen000-b1.onnx` à `gen009-b1.onnx`. Les tables et les figures n'ont besoin que de `eval/` et de `wallclock.json` ; le rejeu n'a besoin que d'un point de contrôle.

Les moteurs tiers utilisés dans la validation des règles (Mzinga, nokamute) ne sont pas livrés et ne sont pas nécessaires : aucun nombre de l'étude n'en dérive.

G.2 Le scénario minimal en environnement vierge

`scripts/reproduce_minimal.sh <workdir>` effectue, à partir d'un clone propre et des enregistrements livrés, la plus petite vérification de bout en bout qui touche chaque maillon de la chaîne : construire, jouer, enregistrer, agréger. Elle a réussi le 9 octobre 2026, puis de nouveau le même jour après une correction de prose du générateur de tables, avec une différence numérique vide. Ses quatre étapes :

1. **Cloner et construire.** `git clone` dans `<workdir>/clone`, puis `cargo build --release -p hive-engine -p hive-arena` ; le binaire du moteur doit exister ensuite. Le crate d'inférence télécharge le binaire ONNX Runtime à la première construction, de sorte que la première construction nécessite un accès réseau.
2. **Livrer les enregistrements.** Copier `results/comparison/`, le `eval/` et le `wallclock.json` de chaque exécution, et l'unique point de contrôle `cmp-grid-s2/checkpoints/gen009-b1.onnx` dans le clone, comme le ferait l'agencement de publication.
3. **Rejouer une partie enregistrée de manière déterministe.** La partie est la position d'échec F2 des résultats qualitatifs : le point de contrôle final de la graine 2 du bras grille contre B-HEU sur la ligne d'ouverture 2, le bras jouant les Noirs, perdue en 19 plis. L'arène joue la paire de couleurs de cette ouverture (`--games 2 --depth 1 --seed 1 --threads 1`), le réseau à 400 simulations avec la graine 9009 (la règle de l'évaluation finale, 9000 + indice de génération), B-HEU à la profondeur 1 sur un seul thread ; le script vérifie que les champs `score_a`, `truncated` et `plies` de la ligne du côté noir sont égaux à ceux de la ligne livrée dans `cmp-grid-s2/eval/gen009-vs-B-HEU.csv` et imprime `replay matches shipped row: plies 19, score 0`.
4. **Régénérer et comparer.** `python3 scripts/make_results.py` (bibliothèque standard de Python uniquement) reconstruit les tables des deux lectures à partir des enregistrements livrés ; `cmp` contre les fichiers livrés `results-same-examples.md` et `results-same-wallclock.md` doit les déclarer identiques à l'octet près. Le script se termine par `MINIMAL REPRODUCTION: PASS`.

Le scénario vérifie que le code publié se construit à partir d'une copie propre du dépôt, qu'une partie enregistrée est rejouée exactement par le point de contrôle publié contre l'adversaire publié sous les réglages épinglés, et que les tables publiées sont une fonction pure des enregistrements publiés. Il ne vérifie pas l'entraînement ; c'est l'objet de la reproduction complète ci-dessous.

G.3 Commandes de reproduction complète

```
# toolchain: Rust (cargo) and Python 3; the project virtual environment is
python/.venv
cargo build --release                       # engine, arena, self-play and fuzz
binaries
cargo test                                  # rules kernel: unit tests, perft to
depth 5 for all 8 game types, UHP server tests
cargo test --release -p hive-core --test perft_tables -- --ignored   # perft to
depth 6

# one training run of the matrix (resumable per generation); repeat for --seed 2
… 5
python3 scripts/run_comparison.py --arm grid  --seed 1 --gens 10 --games 500
python3 scripts/run_comparison.py --arm graph --seed 1 --gens 10 --games 500
```

```
# ablations (seeds 1–3): --arm graph-untyped | graph-nogpool | graph-untyped-clip

# equal-time checkpoint evaluations for runs whose cutoff checkpoint is not the
final one
bash scripts/tstar_evals.sh               # seeds 1–3
bash scripts/tstar_evals2.sh              # graph seeds 4–5

# regenerate every table and figure from raw per-game records
python3 scripts/make_results.py
python/.venv/bin/python scripts/make_figures.py

# second study (protocol v2): driver defaults reproduce the first study
(regression test),
# then the ten warm-started runs, the cutoff evaluations and the analysis
python3 scripts/test_driver_equivalence.py
bash scripts/campaign2.sh                 # seeds 11–15, --warm-start --window 3
--run-prefix cmp2
python3 scripts/tstar_evals_study2.py     # T*' from the five grid clocks, then
the seven cutoff sets
python3 scripts/make_results.py --study 2
python/.venv/bin/python scripts/make_figures.py --study 2

# pre-training check battery on any shard set
bash scripts/run_h4_checks.sh '<abs>/gen000-*.bin' '<abs>/gen000-manifest.json'

# fresh-environment minimal reproduction (clone, build, replay, compare)
bash scripts/reproduce_minimal.sh /tmp/repro

# French/English numeric-identity check; report build
python3 scripts/check_fr_numbers.py
python/.venv/bin/python scripts/build_report.py all
```

Le pilote d'entraînement est reprenable par génération : une ré-exécution saute les générations achevées et poursuit avec le même calendrier de taux d'apprentissage, et chaque exécution écrit ses réglages de générateur, ses graines et ses empreintes de modèle dans ses manifestes. Les graines dérivent du numéro de graine de l'exécution : graine de base = 100 000 × graine, la génération g utilisant la graine de base + g ; l'évaluation utilise la graine de réseau 9000 + génération (ensembles finaux) ou 9500 (ensembles à temps égal), et les graines d'adversaire 9101 (B-RND) et 9201 (B-MCTS).

G.4 Durées attendues et disque

Toutes les durées du Tableau 87 ont été mesurées sur la machine d'étude (Apple M1 Pro, 10 cœurs, 16 Go, macOS 15.3.1) avec quatre threads de travail par exécution et des exécutions séquentielles ; l'inférence du bras grille s'exécute sur CoreML (2.62 ms par évaluation), celle du bras graphe sur CPU (3.67 ms), chacun étant le fournisseur le plus rapide mesuré pour ce réseau.

Tableau 87 : Durées mesurées des calculs de l'étude sur la machine d'étude (Apple M1 Pro, quatre threads de travail par exécution, exécutions séquentielles), telles que consignées dans les relevés d'exécution et d'analyse ; les plages couvrent les exécutions de l'étape. Le temps mural d'entraînement est la somme des secondes d'auto-jeu et d'entraînement par génération d'une exécution, parties d'évaluation exclues ; les chiffres de campagne, d'extension et d'ablation sont des temps écoulés tels que consignés. Les exécutions courtes de A1 reflètent sa divergence (une

politique NaN joue des parties dégénérées courtes) plutôt qu'un coût moindre. La construction du moteur et le scénario minimal n'ont pas été chronométrés.

Étape	**Durée mesurée**
Une exécution d'entraînement grille (10 générations × 500 parties) : temps mural d'entraînement, parties d'évaluation exclues	16.7–19.1 h
Une exécution d'entraînement graphe (même budget) : temps mural d'entraînement, parties d'évaluation exclues	26.6–49.9 h
Les dix exécutions de la campagne principale, temps mural d'entraînement sommé	273.5 h (grille 90.0 h, graphe 183.5 h)
Campagne originale de six exécutions (graines 1–3, les deux bras, séquentielles), temps écoulé	10–17 septembre 2026, ≈7.4 jours ; ≈163 h de temps machine
Exécutions d'extension (graines 4–5, les deux bras), temps écoulé par exécution tel que consigné	18.0–54.9 h, du 27 septembre au 2 octobre 2026
Une partie d'évaluation à 400 simulations	≈23–32 s
Ensembles d'évaluation à temps égal, graines 1–3 (quatre points de contrôle × trois adversaires)	≈11 h, 18 septembre 2026
Ensembles d'évaluation à temps égal, graines 4–5 du graphe (deux points de contrôle × trois adversaires)	≈6 h, 6 octobre 2026
Exécutions d'ablation A1 / A2 / A1′ (par graine), temps mural d'entraînement	7.61–8.01 h (un symptôme de divergence) / 32.05–47.44 h / 23.89–30.35 h
Seconde étude, une exécution grille (démarrage à chaud, fenêtre 3) : temps mural d'entraînement	17.46–18.52 h
Seconde étude, une exécution graphe (même régime) : temps mural d'entraînement	36.27–52.85 h
Seconde étude, les dix exécutions, temps mural d'entraînement sommé	296.9 h (grille 89.0 h, graphe 207.9 h)
Seconde campagne (graines 11–15, les deux bras, séquentielles), temps écoulé	13–26 octobre 2026, 13 jours 8 h
Seconde étude, ensembles d'évaluation à temps égal (sept points de contrôle × trois adversaires)	≈ 14 h, 26–27 octobre 2026
Seconde étude, pilote (2 générations × 200 parties, évaluation à 20 parties par adversaire)	grille 1 h 35 min ; graphe 4 h 17 min
Construction du moteur ; scénario minimal	non mesuré

Disque : avant la campagne, la matrice de six exécutions a été dimensionnée à environ 3–5 Go de shards, de points de contrôle et d'enregistrements, pour 164 Go libres avec une garde de 20 Go ; à mi-campagne, la machine indiquait 200 Go libres. L'empreinte finale des répertoires d'exécution de la première étude n'a pas été consignée ; les dix répertoires d'exécution de la seconde étude totalisent 4.8 Go.

G.5 Identifiants qu'une reproduction doit retrouver

Une reproduction est fidèle lorsqu'elle utilise les artefacts gelés identifiés ci-dessous et, pour le scénario minimal, reproduit la ligne de partie livrée et les tables identiques à l'octet près. Les constantes expérimentales gelées que chaque commande doit laisser intactes sont tabulées dans l'Annexe C : jeu de base avec la règle d'ouverture de tournoi, plafond de 300 plis, 128/32 simulations d'auto-jeu avec une fraction complète de 0.25, 12 plis en température, abandon à −0.92 avec un audit de 10%, 400 simulations d'évaluation sans bruit, 100 parties par adversaire, 250 ouvertures, le seuil de 18.77 h, graines 1–5 par bras ; pour la seconde étude, les mêmes constantes avec le démarrage à chaud, la fenêtre de trois générations, les graines 11–15 et le seuil de 17.63 h du protocole version 2.0 (Annexe C.6).

Tableau 88 : Identifiants des artefacts gelés et des états du code de l'étude. Les hachages sont des condensés hexadécimaux SHA-256 du contenu des fichiers tels que consignés au gel ; les hachages du calendrier sont les préfixes des condensés imprimés par l'outil d'analyse pour le calendrier (ouverture, couleur), identiques pour chaque affrontement d'un volume donné dans les deux études (la première édition de ce rapport attribuait le hachage des 20 parties aux affrontements de 100 parties) ; les commits sont des identifiants de commit abrégés du dépôt.

Artefact	**Identifiant**
Fichier de poids de l'heuristique (`configs/baselines/heuristic_weights.toml`), SHA-256	`d0602f1895fbed70b6f84ac2a3eb87bd68e811e53acf24d4d7814a1495b0b97a`
Configuration de B-RND (`configs/baselines/random.toml`), SHA-256	`f2fc4a06441d3c1a7922838a6693dbb48ec54543bc34fd814d41c9a742514cd7`
Configuration de B-HEU (`configs/baselines/heuristic.toml`), SHA-256	`7210a0a349c5bad5dcd2df099cc6865ee3cb5d8a2c30e106ce58137804818999`
Configuration de B-MCTS (`configs/baselines/mcts_nonet.toml`), SHA-256	`3fc8f75cf2b4f21012dd61e9924408fbfc32c8ea561aa96eb44f1091ba07364e`
Ouvertures gelées, contenu des 250 lignes, SHA-256	`63b318d071dfc3ecfae3585636c8e6f7327ddc08e7aed86a466f915f8005af7b`
Ouvertures gelées, fichier tel que gelé, SHA-256	`538497390a3787299c67c3ca138b8feacb369d55dc562881d1aee45200cbccb2`
Calendrier d'ouvertures et de couleurs de chaque affrontement de 100 parties (ensembles finaux et au seuil, les deux études)	`3bf5d3e51cea0c0d`
Calendrier d'ouvertures et de couleurs de chaque affrontement intermédiaire de 20 parties (les deux études)	`8cd84b6564440666`
Document de protocole gelé	`f340a6b6…aefeb5` (consigné sous forme abrégée), commit `44a74ff`
Document de protocole gelé, version 2.0 (seconde étude)	`e296a12e8d1e3dee36c3b210ececced62314e64657aede1cb91b1ea452b461eb`, commit `4a22a24`
Code du moteur au gel de la population	commit `b94e7c1`
Code de campagne (comparaison principale)	commit `ac58773`
Code d'ablation ; garde contre les pertes non finies	commits `216fded` ; `bffeea9`
Code d'analyse (tables finales à cinq graines)	commit `7db074d`
Seconde étude : options du pilote de campagne ; script de campagne ; commutateur d'analyse ; commutateur de figures	commits `008aaee` ; `dc0b6c2` ; `b0f1598` ; `32297fe`